

Forecast — Manuale Completo

Indice

Elenco delle figure	3
Elenco delle tabelle principali	4
Parte 0 — Come usare questo libro	5
0.1 Percorsi di lettura e a chi serve questo libro	5
0.1.1 Se vieni da Java e non conosci Python	5
0.1.2 Se conosci già Python e vuoi solo il progetto	5
0.1.3 Se stai preparando la tesi	6
0.2 Convenzioni tipografiche e riquadri ricorrenti	6
0.2.1 Le etichette Fatto / Inferenza / Da verificare	6
0.2.2 I sei riquadri ricorrenti	6
0.2.3 Come leggere i riferimenti file:riga	7
0.3 Mappa manuale → tesi	7
0.3.1 Struttura tipica di una tesi triennale in informatica	7
0.3.2 Tabella di corrispondenza capitoli-sezioni (anteprima)	8
0.3.3 Cosa non troverai in questo libro	8
Riepilogo	8
Domande di autoverifica	8
Capitolo 1 — Perché questo progetto esiste	11
1.1 Il problema clinico reale: diagnosi vs. rischio di riammissione	11
1.2 Chi decide, e con quali conseguenze	12
1.3 Cosa si faceva prima: classificazione diretta su dati tabellari	12
Riepilogo	13
Domande di autoverifica	13
Capitolo 2 — Il quadro del machine learning supervisionato	15
2.1 Apprendimento supervisionato, non supervisionato, per rinforzo: una mappa minima	15
2.2 Il ciclo addestramento → validazione → inferenza	16
2.3 Overfitting, underfitting, generalizzazione	16
Riepilogo	17
Domande di autoverifica	17
Capitolo 3 — Dalla tabella al testo	19
3.1 Limiti di una rappresentazione tabellare per i modelli linguistici	19
3.2 L'idea del progetto: tabellare → testo → embedding → classificazione	19
3.3 Approcci alternativi esistenti	20
Riepilogo	21
Domande di autoverifica	21
Capitolo 4 — Classificazione binaria: l'essenziale per seguire il libro	23
4.1 Due classi, una probabilità, una soglia	23

4.2 Sbilanciamento delle classi	23
4.3 Vero/falso positivo/negativo	24
Riepilogo	25
Domande di autoverifica	25
Capitolo 5 — Rappresentazioni testuali ed embedding	27
5.1 Cos'è un embedding, in una frase, prima di ogni formula	27
5.2 Da testo a vettore: tokenizzazione, encoder transformer, pooling	27
5.3 Perché esistono modelli generalisti e modelli biomedici	28
Riepilogo	29
Domande di autoverifica	29
Capitolo 6 — Il gergo del codice: dizionario variabili → concetti	31
6.1 Tabella di corrispondenza completa	31
6.2 I nomi che ingannano	33
6.3 Le due domande di ricerca del progetto, ora leggibili nel codice	34
Riepilogo	34
Domande di autoverifica	35
Capitolo 7 — Sintassi minima e idiomi core	37
7.1 Indentazione come sintassi, niente <code>{}</code> né <code>;</code>	37
7.2 Tipizzazione dinamica e duck typing vs. tipizzazione statica di Java	37
7.3 <code>self</code> esplicito, niente <code>this</code> implicito	38
Riepilogo	39
Domande di autoverifica	39
Capitolo 8 — Strutture dati e comprehension	41
8.1 Liste, dizionari, tuple vs. <code>List/Map</code> /nessun equivalente diretto per le tuple	41
8.2 List comprehension, dict comprehension ed espressioni generatore	42
8.3 Il dizionario globale mutabile di <code>function.py</code>	42
Riepilogo	43
Domande di autoverifica	44
Capitolo 9 — Funzioni, argomenti, trappole	45
9.1 Argomenti posizionali, keyword, default — e il pericolo dei default mutabili	45
9.2 Funzioni come cittadini di prima classe	46
9.3 Assenza di overloading: come Python distingue le firme	46
Riepilogo	47
Domande di autoverifica	47
Capitolo 10 — Moduli, package e l'assenza di incapsulamento reale	49
10.1 Un file <code>.py</code> è un modulo: cosa importa rispetto a un file <code>.java</code>	49
10.2 <code>import</code> vs. <code>import</code> Java: risoluzione, side-effect al caricamento	49
10.3 Niente <code>private</code> reale: la convenzione dell'underscore	50
Riepilogo	51
Domande di autoverifica	51
Capitolo 11 — Eccezioni, context manager, decoratori	53
11.1 <code>try/except/raise</code> ... <code>from</code> : differenze da <code>throws/catch</code>	53
11.2 Context manager (<code>with</code>): il “try-with-resources” di Python	54
11.3 Decoratori: cosa sono, dove NON compaiono in questo progetto	55
Riepilogo	55
Domande di autoverifica	56

Capitolo 12 — Concorrenza e memoria: GIL, thread, processi	57
12.1 Il Global Interpreter Lock: cosa cambia rispetto ai thread Java	57
12.2 ThreadPoolExecutor e Semaphore nel progetto, confrontati con ExecutorService	57
12.3 Gestione della memoria: reference counting + garbage collector, niente finalize	59
Riepilogo	59
Domande di autoverifica	59
Capitolo 13 — Tabella di traduzione Java→Python e trappole finali	61
13.1 Tabella sistematica	61
13.2 Le cinque trappole più insidiose per chi arriva da un linguaggio tipizzato staticamente	62
13.3 Prova tu: leggere un traceback	63
Riepilogo	63
Domande di autoverifica	63
Capitolo 14 — L’interprete e gli ambienti virtuali	65
14.1 Perché Python ha bisogno di un “classpath” alternativo	65
14.2 env/ in questo progetto: cosa contiene, perché non è versionato	66
14.3 pip e requirements.txt confrontati con Maven/Gradle	66
Riepilogo	67
Domande di autoverifica	67
Capitolo 15 — Installazione passo passo su macOS	69
15.1 Prerequisiti: Python 3.14, Ollama, token Hugging Face	69
15.2 Procedura completa comando per comando	69
15.3 Prima esecuzione: <code>python main.py --dataset heart_disease</code>	71
Riepilogo	71
Domande di autoverifica	71
Capitolo 16 — Troubleshooting e verifica dell’ambiente	73
16.1 Gli errori più probabili e le loro cause reali	73
16.2 Verificare che l’installazione sia corretta prima di lanciare l’intera pipeline	74
16.3 Cosa NON esiste in Python: niente bytecode portabile tra versioni, nessun .jar .	74
Riepilogo	75
Domande di autoverifica	75
Capitolo 17 — Vista d’insieme: componenti e confini	77
17.1 Diagramma architetturale commentato	77
17.2 Cosa entra, cosa esce, da ogni fase	78
17.3 Confronto con un’architettura Java a livelli	79
Riepilogo	80
Domande di autoverifica	80
Capitolo 18 — Il ciclo di vita di un’esecuzione	81
18.1 Diagramma di sequenza da <code>main.py</code> al report	81
18.2 Isolamento per dataset	81
18.3 Fallimento a metà pipeline	82
Riepilogo	82
Domande di autoverifica	83
Capitolo 19 — Stato, configurazione, punti di estensione	85
19.1 Dove vive la configurazione: <code>function.py</code> come “file unico di config”	85
19.2 Stato globale mutabile: dove, perché, rischi	85
19.3 Dove aggiungeresti un ottavo modello di embedding, se dovessi farlo tu	86

Riepilogo	87
Domande di autoverifica	87
Capitolo 20 — <code>function.py</code>: la spina dorsale silenziosa	89
20.1 Config modelli e famiglie	89
20.2 Funzioni di I/O e pulizia cartelle	89
20.3 Le nove funzioni di plotting	90
Interfaccia pubblica (sintesi)	91
Errori tipici	92
Riepilogo	92
Domande di autoverifica	92
Capitolo 21 — <code>preprocessing.py</code>: dalla tabella grezza al training set bilanciato	95
21.1 Caricamento e unione sorgenti	95
21.2 Imputazione, SMOTENC, codifica: tre trasformazioni, tre scopi diversi	96
21.3 Cosa viene salvato e perché	97
Interfaccia pubblica	97
Errori tipici	98
Riepilogo	98
Domande di autoverifica	98
Capitolo 22 — <code>embedding.py</code>: tabellare → testo → vettore	101
22.1 <code>record_to_text_*</code> () riga per riga	101
22.2 Generazione batch verso Ollama: retry, semaforo	102
22.3 Generazione verso Hugging Face: autenticazione, modalità offline	103
Interfaccia pubblica	103
Errori tipici	104
Riepilogo	104
Domande di autoverifica	104
Capitolo 23 — <code>classification.py</code>: addestrare e validare	107
23.1 <code>StratifiedKFold</code> e perché “stratified” conta	107
23.2 La soglia F1-ottima per fold	108
23.3 Cosa viene salvato e a cosa serve dopo	109
Interfaccia pubblica	109
Errori tipici	110
Riepilogo	110
Domande di autoverifica	110
Capitolo 24 — <code>evaluation.py</code>: bootstrap e grafici	111
24.1 Il bootstrap in poche righe	111
24.2 Intervalli di confidenza: come si leggono i grafici	112
24.3 Dal dizionario Python al PNG/PDF	112
Interfaccia pubblica	113
Errori tipici	113
Riepilogo	113
Domande di autoverifica	113
Capitolo 25 — <code>error_analysis.py</code>: dall’errore statistico al caso clinico	115
25.1 Ricostruire il record dall’indice di validazione	115
25.2 “Hardest cases”: cosa significa essere sbagliato da tutti i modelli	116
25.3 Deviazione di feature: una standardizzazione con un’interpretazione clinica	116
Interfaccia pubblica	117

Errori tipici	117
Riepilogo	117
Domande di autoverifica	118
Capitolo 26 — <code>statisticaltest.py</code>: tre test, tre garanzie diverse	119
26.1 Wilcoxon e t-test appaiato sulle distribuzioni bootstrap	119
26.2 Il test di DeLong e la libreria <code>MLstatkit</code>	120
26.3 Perché tre test invece di uno	120
Interfaccia pubblica	121
Errori tipici	121
Riepilogo	121
Domande di autoverifica	121
Capitolo 27 — <code>generatereport.py</code>: l'ultimo miglio, e il suo limite più istruttivo	123
27.1 Come si assembla un report Markdown da CSV e PNG	123
27.2 Il caso del testo narrativo statico: quando il report “mente” per omissione	124
27.3 Come lo riscriveresti tu	124
Interfaccia pubblica	125
Errori tipici	125
Riepilogo	125
Domande di autoverifica	125
Capitolo 28 — <code>main.py</code>: l'orchestratore letto per ultimo apposta	127
28.1 Le sette fasi in ordine, con un occhio a ciò che già conosci	127
28.2 Il flag <code>--dataset</code> e il comportamento di default	128
28.3 Cosa succederebbe ad aggiungere una fase 8	128
Interfaccia pubblica	129
Errori tipici	129
Riepilogo	129
Domande di autoverifica	129
Capitolo 29 — Il dataset UCI Heart Disease	131
29.1 Origine e centri clinici	131
29.2 Schema delle 14 feature e valori mancanti mascherati da zero	131
29.3 Limiti noti	133
Riepilogo	133
Domande di autoverifica	133
Capitolo 30 — Il dataset Diabetes 130-US Hospitals	135
30.1 Scala, 101.766 righe, e campionamento a 20.000	135
30.2 Le 19 feature scelte tra ~50	135
30.3 La ridefinizione del target	136
Riepilogo	137
Domande di autoverifica	137
Capitolo 31 — Qualità dei dati e casi limite	139
31.1 Valori mancanti espliciti e impliciti	139
31.2 SMOTENC su feature miste: cosa genera un record sintetico	139
31.3 Casi limite nella conversione a testo	140
Riepilogo	141
Domande di autoverifica	141
Capitolo 32 — La regressione logistica, dalle basi alla riga di codice	143

32.1 Funzione logistica come probabilità	143
32.2 Funzione di costo e ottimizzazione	143
32.3 Perché lineare su embedding, non una rete profonda	144
Riepilogo	145
Domande di autoverifica	145
Capitolo 33 — Come si valida correttamente un modello di classificazione	147
33.1 Train/validation/test: a cosa serve ciascuno	147
33.2 K-fold e k-fold stratificato: formula e procedura	148
33.3 Data leakage: tassonomia generale e perché è la minaccia più insidiosa	148
Riepilogo	149
Domande di autoverifica	149
Capitolo 34 — Le tre metriche del progetto	151
34.1 Accuracy	151
34.2 Macro-F1	151
34.3 ROC-AUC	152
Riepilogo	153
Domande di autoverifica	153
Capitolo 35 — Soglie di decisione	155
35.1 Perché 0.5 non basta sempre	155
35.2 La soglia F1-ottima: formula e codice	155
35.3 Il costo nascosto della sua ottimizzazione	156
Riepilogo	156
Domande di autoverifica	157
Capitolo 36 — Bootstrap e intervalli di confidenza	159
36.1 Ricampionamento con reinserimento	159
36.2 Intervalli percentile al 95%	160
36.3 10.000 iterazioni: perché questo numero, cosa cambierebbe con 100 o 1.000.000	160
Riepilogo	160
Domande di autoverifica	161
Capitolo 37 — I tre test di significatività	163
37.1 Wilcoxon signed-rank	163
37.2 t-test appaiato	163
37.3 Il test di DeLong	164
Riepilogo	165
Domande di autoverifica	165
Capitolo 38 — Riduzione di dimensionalità e visualizzazione: UMAP	167
38.1 Perché non si può “vedere” uno spazio a 768/1024 dimensioni	167
38.2 UMAP in breve: vicinanza locale, ottimizzazione di un layout 2D	167
38.3 Cosa può e non può dirti un grafico UMAP	168
Riepilogo	168
Domande di autoverifica	169
Capitolo 39 — Iperparametri del progetto: valori e rationale	171
39.1 Tabella completa	171
39.2 Effetto di valori alternativi	172
39.3 Riproducibilità: la mappa completa dei semi casuali nel progetto	173
Riepilogo	173

Domande di autoverifica	173
Capitolo 40 — Caso d’uso 1: <code>python main.py --dataset heart_disease</code> dall’inizio alla fine	175
40.1 Cosa succede fase per fase	175
40.2 Come cambiano i dati	175
40.3 Cosa trovi su disco	176
Riepilogo	177
Domande di autoverifica	177
Capitolo 41 — Caso d’uso 2: <code>python main.py --dataset diabetes130</code> e le differenze che contano	179
41.1 Stesse fasi, numeri diversi	179
41.2 Perché i punteggi assoluti sono più bassi	180
41.3 Isolamento tra run	180
Riepilogo	180
Domande di autoverifica	181
Capitolo 42 — Caso d’uso 3: aggiungere un ottavo encoder passo per passo	183
42.1 Dove si registra in <code>function.py</code>	183
42.2 Cosa fa la pipeline da sola	184
42.3 Verifica finale	184
Riepilogo	185
Domande di autoverifica	185
Capitolo 43 — Protocollo sperimentale	187
43.1 Cosa è stato eseguito da chi, e quando	187
43.2 Comandi eseguiti in questa sessione (con output reale) vs. comandi da eseguire tu	187
43.3 Limiti dichiarati del protocollo	188
Riepilogo	189
Domande di autoverifica	189
Capitolo 44 — Risultati — UCI Heart Disease	191
44.1 Tabella completa delle metriche con intervalli di confidenza	191
44.2 Confronto tra famiglie di modelli	192
44.3 Cosa dicono davvero i test statistici	192
Riepilogo	193
Domande di autoverifica	193
Capitolo 45 — Risultati — Diabetes 130-US Hospitals	195
45.1 Tabella completa delle metriche con intervalli di confidenza	195
45.2 Confronto tra famiglie di modelli	196
45.3 Perché questo dataset è metodologicamente più interessante	196
Riepilogo	197
Domande di autoverifica	197
Capitolo 46 — Analisi degli errori sui due dataset	199
46.1 Tassi di falsi positivi/negativi per modello	199
46.2 I casi più difficili: cosa hanno in comune	199
46.3 Deviazione di feature: quali variabili “tradiscono” un errore	200
Riepilogo	200
Domande di autoverifica	201

Capitolo 47 — Confronto con un modello di riferimento banale	203
47.1 Cos'è un baseline e perché la tesi ne ha bisogno	203
47.2 Costruzione di un classificatore banale sugli stessi fold	203
47.3 Il confronto numerico: quanto valore aggiunge davvero l'embedding	204
Riepilogo	205
Domande di autoverifica	205
Capitolo 48 — Lo stato attuale: nessun test automatico	207
48.1 Cosa significa, onestamente, per l'affidabilità del progetto	207
48.2 Cosa testeresti per primo, e perché	207
48.3 pytest confrontato con JUnit	208
Riepilogo	208
Domande di autoverifica	208
Capitolo 49 — Scrivere il primo test per questo progetto	211
49.1 Un test per <code>record_to_text_heart_disease</code>	211
49.2 Un test per la logica di soglia F1-ottima con dati sintetici	212
49.3 Cosa serve per testare le funzioni che toccano il disco o la rete	213
Riepilogo	214
Domande di autoverifica	214
Capitolo 50 — Debugging e log in pratica	217
50.1 Il debugger di un IDE Python confrontato con quello di un IDE Java	217
50.2 Leggere i <code>print()</code> del progetto come se fossero log strutturati	217
50.3 Isolare un bug reale: percorso guidato su un modulo del progetto	218
Riepilogo	218
Domande di autoverifica	218
Capitolo 51 — Limiti metodologici del protocollo di valutazione	221
51.1 Assenza di test set finale indipendente	221
51.2 Soglia ottimizzata sullo stesso fold di validazione	221
51.3 Il campionamento a 20.000 record di Diabetes130	222
Una tabella di sintesi	222
Riepilogo	222
Domande di autoverifica	222
Capitolo 52 — Il caso del reporting automatico non aggiornato	225
52.1 Confronto riga per riga dei due report reali	225
52.2 Perché è pericoloso in contesto clinico	225
52.3 Come si preverrebbe	226
Riepilogo	226
Domande di autoverifica	226
Capitolo 53 — Posizionamento rispetto allo stato dell'arte	229
53.1 Tabellare→testo→embedding vs. classificazione tabellare diretta, "linear probing" su embedding congelati	229
53.2 Cosa NON prova questo progetto	229
53.3 Minacce alla validità: interna, esterna, di costruito, statistica	230
Riepilogo	231
Domande di autoverifica	231
Capitolo 54 — Il chatbot clinico: cosa esiste già, non integrato	233
54.1 Architettura di <code>chatbot_core.py</code> , <code>app_streamlit.py</code> , <code>bot_telegram.py</code>	233

54.2 Cosa manca per l'integrazione pulita	234
54.3 Rischi di un chatbot di rischio clinico non validato	234
Riepilogo	235
Domande di autoverifica	235
Capitolo 55 — Direzioni di sviluppo difendibili in sede di discussione	237
55.1 Aggiungere un test set finale indipendente	237
55.2 Un classificatore non lineare come confronto	237
55.3 Calibrazione delle probabilità e reportistica generata dai dati	238
Riepilogo	238
Domande di autoverifica	238
Capitolo 56 — Mappa capitolo per capitolo verso le sezioni tipiche di una tesi triennale	241
56.1 Introduzione e motivazione	241
56.2 Stato dell'arte	241
56.3 Materiali e metodi	241
56.4 Risultati	242
56.5 Discussione e limiti	242
56.6 Conclusioni e lavori futuri	242
Tabella di sintesi	242
Riepilogo	242
Domande di autoverifica	242
Capitolo 57 — Cosa è già pronto e cosa manca ancora	245
57.1 Figure e tabelle riutilizzabili, capitolo per capitolo	245
57.2 Misure ancora da produrre, con procedura	245
57.3 Riferimenti bibliografici da recuperare e verificare	246
Riepilogo	246
Domande di autoverifica	246
Capitolo 58 — Venti domande della commissione, con traccia di risposta	249
58.1 Dominio e motivazione (5 domande)	249
58.2 Metodo e implementazione (8 domande)	249
58.3 Limiti e validità (7 domande)	250
Riepilogo	251
Domande di autoverifica	251
Appendice A — Glossario generale	253
Appendice B — Riferimento completo delle funzioni pubbliche	255
main.py	255
function.py	255
preprocessing.py	256
embedding.py	256
classification.py	256
evaluation.py	256
error_analysis.py	257
statisticaltest.py	257
generatereport.py	257
Appendice C — Formulario	259

Appendice D — Bibliografia annotata	261
Dataset (citati direttamente nel progetto)	261
Metodo statistico (verificati con WebSearch in questa sessione)	261
Modelli di embedding (verificati con WebSearch in questa sessione)	262
Riepilogo affidabilità	262
Appendice E — Zone d'ombra	265
Come usare questa lista	266
Appendice F — Indice analitico	267

Forecast

Manuale Completo

Una pipeline locale di embedding testuale clinico
per la classificazione di malattie

Guida completa al progetto per chi programma in Java

Versione 1.0 — 6 settembre 2026

Materiale tecnico verificato per lo studio del progetto
e come base per una tesi triennale in Informatica

Forecast — Manuale Completo

Versione 1.0, 6 settembre 2026

Questo manuale documenta un progetto software reale (nome storico della cartella: **Forecast**), una pipeline locale che confronta modelli di embedding testuale generalisti e biomedici per la classificazione clinica binaria a partire da due dataset pubblici UCI (Heart Disease, Diabetes 130-US Hospitals), convertiti da tabella a testo prima dell'embedding.

Ogni affermazione tecnica è marcata **Fatto** (verificata leggendo il codice sorgente o eseguendo un comando reale, con riferimento `file:rigo` o comando), **Inferenza** (deduzione motivata) o **Da verificare** (ipotesi aperta, elencata per intero in Appendice E).

Scritto per uno sviluppatore software professionista con esperienza in Java, senza conoscenza pregressa di Python, dell'ecosistema di data science, o del dominio applicativo.

Bibliografia esportata separatamente in `bibliografia.bib` (12 riferimenti, ciascuno verificato con identificativo concreto – DOI, arXiv ID o ISBN).

Colophon. Scritto in Markdown, capitolo per capitolo, con formule in $\text{L}_\text{A}\text{T}_\text{E}_\text{X}$ e diagrammi in Mermaid. Composto con Pandoc e Tectonic (motore compatibile XeLaTeX). Font di testo serif, font di codice monospazio. Font Latin Modern.

Elenco delle figure

Figura 2.1 — Il ciclo addestramento/validazione/inferenza, con i riferimenti esatti a dove ciascun momento avviene nel codice del progetto (capitolo 2.2)

Figura 3.1 — La catena di trasformazioni centrale del progetto: tabellare, testo, embedding, classificazione (capitolo 3.2)

Figura 12.1 — Tre thread attivi, ma una sola chiamata a Ollama alla volta (capitolo 12.2)

Figura 17.1 — Architettura completa della pipeline (capitolo 17.1)

Figura 18.1 — Sequenza completa di un'esecuzione, dalla riga di comando al report finale (capitolo 18.1)

Figura 54.1 — Flusso di inferenza del chatbot, con il punto esatto di riutilizzo del codice della pipeline offline (capitolo 54.1)

Elenco delle tabelle principali

Tabella di traduzione Java, $\rightarrow$, Python (capitolo 13.1)

Schema delle 14 feature di Heart Disease (capitolo 29.2)

Schema delle 19 feature di Diabetes130 (capitolo 30.2)

Tabella completa degli iperparametri del progetto (capitolo 39.1)

Tabella 44.1 — Risultati completi con intervalli di confidenza, Heart Disease (capitolo 44.1)

Tabella 45.1 — Risultati completi con intervalli di confidenza, Diabetes130 (capitolo 45.1)

Tabella 46.1 — Tassi di errore reali, entrambi i dataset (capitolo 46.1)

Tabella dei limiti metodologici con gravità e correggibilità (capitolo 51)

Tabella di sintesi manuale, $\rightarrow$, tesi (capitolo 56)

Nota: questo elenco è compilato manualmente, non generato automaticamente da Pandoc, perché le tabelle e le figure del libro usano didascalie numerate nel testo (es. "Figura 17.1", "Tabella 44.1") anziché il meccanismo di didascalia nativo di Pandoc/LaTeX. Il formulario completo (Appendice C) e il riferimento alle funzioni pubbliche (Appendice B) elencano ulteriori tabelle non riportate qui per brevità.

Parte 0 — Come usare questo libro

Obiettivi del capitolo

- Sapere quale percorso di lettura seguire, in base a cosa già sai e a cosa ti serve.
- Riconoscere le convenzioni tipografiche usate in ogni capitolo, prima di incontrarle nel merito.
- Capire come questo manuale si collega alla tesi che dovrai scrivere, e cosa questo libro deliberatamente non fa.

Questo libro documenta un progetto reale: una pipeline Python che confronta modelli di embedding testuale per la classificazione clinica. Non è la tua tesi. È il materiale — verificato, citato, misurato — da cui la tesi si scrive. Le due cose hanno regole diverse, e questo capitolo te le mostra prima di cominciare.

0.1 Percorsi di lettura e a chi serve questo libro

Il libro presuppone una cosa sola con certezza: sai già programmare, in un linguaggio a oggetti staticamente tipizzato, e conosci il mestiere — HTTP, SQL, test automatici, gestione delle dipendenze, concorrenza in astratto. Il libro è scritto assumendo che tu venga da Java in particolare, perché è il riferimento più utile per marcare le differenze che contano. Non presuppone nulla sul dominio clinico, su Python, o su questo progetto specifico. Tre profili di lettura, con percorsi diversi:

0.1.1 Se vieni da Java e non conosci Python

Leggi in ordine dalla Parte I alla Parte VIII senza saltare la Parte II. È tentante saltarla — “conosco già la programmazione” — ma la Parte II non insegna a programmare: insegna dove Python *si comporta diversamente* da Java pur sembrando uguale, che è esattamente il tipo di differenza che morde a metà di un debugging, non mentre leggi con calma. Un esempio su tutti, anticipato qui e spiegato per intero nel capitolo 8: in Python un valore di default mutabile in una firma di funzione viene creato *una sola volta*, non ad ogni chiamata. In Java questo problema semplicemente non esiste nella stessa forma. Se lo scopri leggendo un capitolo, costa un paragrafo. Se lo scopri in produzione, costa una sera.

0.1.2 Se conosci già Python e vuoi solo il progetto

Salta la Parte II. Puoi anche scorrere rapidamente la Parte III (ambiente) se il tuo ambiente locale è già configurato. Comincia dalla Parte IV (architettura) e prosegui linearmente: Parte V (codice modulo per modulo), Parte VI (dati), Parte VII (matematica del modello), Parte VIII (casi d’uso end-to-end). Le Parti I, IX, X, XI, XII, XIII restano utili quanto per chiunque altro: coprono il dominio, i risultati, la qualità e la messa in prospettiva critica, argomenti indipendenti dalla lingua di programmazione.

0.1.3 Se stai preparando la tesi

Leggi il libro nell'ordine in cui è scritto almeno una prima volta — la Parte XIII da sola, senza il resto, è materiale senza fondamenta. Poi usa la Parte XIII come indice di ritorno: ogni sua sezione ti rimanda ai capitoli con il materiale grezzo (dati, formule, tabelle, figure, criticità) per quella parte specifica della tesi. La sezione 0.3.2 di questo capitolo ti dà un'anteprima immediata di quella mappa.

ATTENZIONE — questo libro non usa prosa da incollare in una tesi. È scritto come manuale tecnico: schemi, tabelle, definizioni, dati verificati. La tesi la scrivi tu, con le tue parole, usando questo materiale come fondamenta verificate. Copiare paragrafi da qui alla tesi senza riscriverli è un rischio che riguarda te, non un problema di questo libro — ma è bene dirlo chiaramente da subito.

0.2 Convenzioni tipografiche e riquadri ricorrenti

0.2.1 Le etichette **Fatto** / **Inferenza** / **Da verificare**

Ogni affermazione tecnica in questo libro porta una di tre etichette, in grassetto, all'inizio della frase o del paragrafo a cui si applica.

[Fatto] significa: verificato leggendo il codice sorgente reale del progetto, o eseguendo un comando reale e osservandone l'output. Ogni “Fatto” porta con sé un riferimento nella forma `percorso/file.py:riga` (spiegata in 0.2.3) oppure il comando esatto eseguito. Esempio reale, che rincontrerai nel capitolo 28: **[Fatto]** `main.py` accetta un solo argomento da riga di comando, `--dataset`, con default `heart_disease` (`main.py:14-18`).

[Inferenza] significa: una deduzione ragionevole, non direttamente scritta nel codice ma costruita a partire da ciò che il codice fa, con la motivazione resa esplicita subito dopo. Esempio reale, dal capitolo 35: **[Inferenza]** scegliere la soglia di decisione che massimizza F1 sullo stesso fold di validazione su cui poi si misura la metrica introduce un ottimismo statistico lieve — motivazione: il modello non vede le etichette di validazione per allenarsi, ma la soglia sì, quindi il numero riportato è calibrato sui dati su cui viene anche giudicato.

[Da verificare] significa: una domanda aperta, un'ipotesi non controllata, qualcosa che richiederebbe una fonte esterna, un esperimento, o una conversazione con chi ha scritto il codice per essere chiusa. Esempio reale, dal capitolo 6: **[Da verificare]** la libreria `ucimlrepo` compare in `requirements.txt` ma non risulta importata in nessuno dei nove file Python del progetto letti per questo libro — probabilmente un residuo di una fase esplorativa precedente, ma non verificato.

Tutte le voci “Da verificare” del libro confluiscono anche nell'Appendice E, in un elenco unico, pensato per essere portato così com'è a chi supervisiona la tesi.

ATTENZIONE — le tre etichette non sono intercambiabili, e la differenza conta più in una tesi che in questo manuale. Presentare in tesi un “Da verificare” come se fosse un “Fatto” significa dichiarare come risultato verificato qualcosa che non lo è. Chi la legge — un relatore, un membro di commissione — verifica le fonti. Trovarne una che non regge mina la credibilità di tutto il resto, anche della parte solida.

0.2.2 I sei riquadri ricorrenti

Oltre alle etichette puntuali, il libro usa sei riquadri più estesi, sempre nella stessa forma tipografica — una citazione (blockquote) con un'etichetta in maiuscolo in grassetto, senza icone o decorazioni:

SE VIENI DA JAVA — una differenza specifica tra Python e Java, sul punto preciso che il capitolo sta trattando. Non spiega cos'è una variabile: spiega, per esempio, perché in Python una funzione può restituire tipi diversi a seconda del ramo eseguito, e cosa significa per te che vieni da un linguaggio dove la firma del metodo lo vieta a priori.

ATTENZIONE — un punto dove un'assunzione ragionevole ti porterebbe fuori strada. Il libro ne apre uno, per esempio, sul fatto che eseguire `python main.py` senza argomenti esegue silenziosamente il dataset Heart Disease, non entrambi i dataset e non un dataset scelto a caso: un flag con un default è comodo finché non te ne accorgi mentre confronti risultati che pensavi appartenessero a un altro dataset.

APPROFONDIMENTO FACOLTATIVO — un contenuto che arricchisce ma non è necessario per seguire il filo del capitolo. Puoi saltarlo alla prima lettura e tornarci se ti serve per la tesi. Il libro lo usa, per esempio, per il funzionamento interno di UMAP nel capitolo 38: capire cosa succede nella pipeline non richiede sapere come UMAP costruisce il suo grafo di vicinanza, ma la tesi potrebbe trarne beneficio.

PROVA TU — un piccolo esercizio pratico, quasi sempre eseguibile in pochi minuti sull'ambiente che avrai configurato nella Parte III. Non ha una soluzione nascosta altrove nel libro: è pensato perché tu lo faccia, non perché tu legga cosa succederebbe.

RIFERIMENTO AL CODICE — un rimando puntuale a un file e a una riga (o intervallo di righe) del progetto, quando serve isolarlo dal flusso del paragrafo — per esempio quando la stessa riga è rilevante per due argomenti diversi in due capitoli diversi, e conviene poterla ritrovare rapidamente in entrambi.

MATERIALE PER LA TESI — presente alla fine di ogni capitolo, indica tre elementi di quel capitolo — una tabella, una formula, un dato, un grafico, un'osservazione critica — pensati per essere ripresi, con le tue parole, in una sezione specifica della tesi.

0.2.3 Come leggere i riferimenti file:riga

Ogni volta che il libro scrive qualcosa come `classification.py:16`, significa: riga 16 del file `classification.py`, che si trova nella radice del repository del progetto. Un intervallo si scrive `preprocessing.py:43-70` e significa dalla riga 43 alla riga 70 comprese. Questi riferimenti sono verificabili da chiunque abbia il repository aperto: aprilo, vai a quella riga, e troverai esattamente ciò che il libro descrive. Se un giorno il codice cambia e la riga non corrisponde più, il riferimento è invecchiato — non inventato: era corretto al momento della scrittura, verificato leggendo il commit 41bf90e del branch `master` (l'ultimo al momento in cui questo libro è stato scritto).

RIFERIMENTO AL CODICE — quando un riferimento riguarda un file su un branch diverso da `master` — capita una sola volta in questo libro, per il chatbot del capitolo 54 — il libro lo dice esplicitamente: `chatbot_core.py:12` (branch `chatbot`). In assenza di questa precisazione, il riferimento è sempre su `master`.

0.3 Mappa manuale → tesi

0.3.1 Struttura tipica di una tesi triennale in informatica

Una tesi triennale in informatica, nell'ambito di ingegneria del software, segue tipicamente uno schema semplice: un'introduzione che motiva il problema, una rassegna dello stato dell'arte che lo colloca rispetto a ciò che già esiste, una descrizione di materiali e metodi che spiega cosa hai

costruito e come, una presentazione dei risultati, una discussione che ne pesa la portata e i limiti, e delle conclusioni che indicano cosa si potrebbe fare dopo. Questo libro non ha quella struttura — ha la struttura di un manuale tecnico, pensata per l'apprendimento e la consultazione, non per la difesa di una tesi — ma ogni sua parte alimenta uno o più di quei momenti.

0.3.2 Tabella di corrispondenza capitoli-sezioni (anteprima)

La Parte XIII (capitoli 56-58) tratta questa corrispondenza capitolo per capitolo, con il dettaglio di quali figure, tabelle e misure sono già pronte e quali vanno ancora prodotte. Questa è solo l'anteprima, utile da subito per orientarti.

Sezione tipica di tesi	Parti del manuale a cui attingere
Introduzione e motivazione	Parte I
Stato dell'arte	Parte I, Parte XI (cap. 53), Appendice D
Materiali e metodi	Parte IV, Parte V, Parte VI, Parte VII
Risultati	Parte IX
Discussione e limiti	Parte XI
Conclusioni e lavori futuri	Parte XII

0.3.3 Cosa non troverai in questo libro

Non troverai paragrafi già scritti nello stile di una tesi, pronti per un copia-incolla. Non troverai una bibliografia che accetta un riferimento perché “suona giusto”: ogni voce in Appendice D è verificabile con un identificativo reale (DOI, ISBN o URL), oppure è marcata esplicitamente come da controllare prima di essere citata. Non troverai concetti di programmazione generale spiegati da zero — se compare una spiegazione di cos'è un ciclo o un'eccezione, è un errore di calibrazione di questo libro, non una scelta. E non troverai, nella parte di dominio, alcun accenno a serie storiche, stagionalità o trend: nonostante il nome storico del progetto, il codice non fa previsioni temporali — fa classificazione clinica binaria a partire da embedding testuali, come stabilito leggendo `main.py`, `preprocessing.py` ed `embedding.py`, non deducendolo dal nome della cartella. Il capitolo 1 lo spiega per intero.

Riepilogo

Questo libro ha tre percorsi di lettura a seconda di cosa sai già; usa tre etichette (Fatto, Inferenza, Da verificare) per dichiarare quanto è solida ogni affermazione tecnica, e sei riquadri ricorrenti per isolare contenuti di natura diversa dal corpo del testo. Non è la tesi: è il materiale verificato da cui la tesi si scrive, e la Parte XIII è la mappa esplicita che collega l'uno all'altra.

Domande di autoverifica

1. Perché un'affermazione marcata “Da verificare” non va citata in tesi come se fosse un fatto assodato? Perché “Da verificare” segnala un'ipotesi non ancora controllata. Presentarla come “Fatto” significa dichiarare come risultato verificato qualcosa che non lo è — un errore che chiunque verifichi la fonte scopre subito, con un costo di credibilità che si estende al resto del lavoro.

2. In quale parte del manuale trovi le formule matematiche del progetto, con il rimando alla riga di codice che le implementa? Parte VII — Il modello: matematica e implementazione (capitoli 32-39).

3. Il tuo relatore ti chiede quali sono i limiti di questo lavoro. Dove guardi per primo? Parte XI — Analisi critica (capitoli 51-53), con supporto dal capitolo 47 (confronto con un baseline) e dall'Appendice E (zone d'ombra), che raccoglie in un solo elenco tutte le domande aperte del libro.

MATERIALE PER LA TESI 1. La tabella di corrispondenza sezione-di-tesi → parte-del-manuale (§0.3.2): riusabile per motivare, nell'introduzione della tesi, come è organizzato il lavoro. 2. La convenzione Fatto/Inferenza/Da verificare (§0.2.1): riusabile come nota metodologica esplicita nel capitolo "Materiali e metodi", per dichiarare il grado di certezza di ogni affermazione empirica. 3. L'elenco dei sei riquadri ricorrenti (§0.2.2): riusabile come schema per organizzare, in un'appendice della tesi, le osservazioni critiche raccolte durante l'analisi del progetto.

Capitolo 1 — Perché questo progetto esiste

Obiettivi del capitolo

- Capire quali sono, in concreto, i due problemi clinici che il progetto affronta.
- Capire chi userebbe una previsione di questo tipo, e cosa cambia per un paziente reale a seconda che il sistema sbaglia in un verso o nell'altro.
- Sapere cosa si intende, in questo campo, per approccio “tradizionale”, e in cosa il progetto se ne discosta.

1.1 Il problema clinico reale: diagnosi vs. rischio di riammissione

[**Fatto**] Il progetto affronta due problemi distinti, uno per dataset, entrambi di classificazione binaria su dati clinici strutturati:

Il primo problema — dataset **UCI Heart Disease** — è un problema di supporto alla diagnosi: dato il profilo clinico di un paziente (età, pressione a riposo, colesterolo, risultati di un elettrocardiogramma, e altre nove misure), il sistema deve produrre una stima della presenza di malattia coronarica. Non sostituisce una diagnosi medica — restituisce una probabilità, calcolata su dati storici, che può informare una decisione umana successiva. È il problema classico di *classificazione diagnostica*: la variabile che il progetto chiama `num` nel codice (`function.py:12`) diventa, dopo la binarizzazione (`num > 0`) (`preprocessing.py:41`), la risposta a una domanda semplice: malattia presente, sì o no.

Il secondo problema — dataset **Diabetes 130-US Hospitals** — è un problema diverso nella sostanza, anche se identico nella forma matematica: dato un ricovero ospedaliero già avvenuto per un paziente diabetico (quanti giorni è rimasto ricoverato, quanti farmaci ha ricevuto, quante visite di pronto soccorso ha avuto nell'anno precedente, e altre quindici misure), il sistema deve stimare se quel paziente tornerà ricoverato entro 30 giorni dalla dimissione. Non è una diagnosi: è una previsione di utilizzo futuro del sistema sanitario, il tipo di previsione che serve a decidere l'intensità del follow-up post-dimissione. [**Fatto**] La colonna `readmitted` viene binarizzata in `load_diabetes130()` con la regola (`readmitted == "<30"`) (`function.py:124`): 1 se il paziente è tornato entro 30 giorni, 0 altrimenti (anche se non è mai tornato, o è tornato dopo più di 30 giorni — una scelta di framing di cui riparlamo al capitolo 30.3).

Due problemi diversi, la stessa struttura: una tabella di feature cliniche in ingresso, un'etichetta binaria in uscita. È esattamente questa uniformità di struttura, al di là del contenuto clinico specifico, che rende sensato trattarli con la stessa pipeline — il tema di cui parla il capitolo 3.

1.2 Chi decide, e con quali conseguenze

Un sistema come questo non prende decisioni cliniche: le informa. Ma è utile, fin da subito, chiedersi chi guarderebbe questo numero e cosa ne farebbe, perché la risposta anticipa un concetto tecnico — l’asimmetria fra i due tipi di errore — che il capitolo 4.3 formalizza.

Per Heart Disease, il numero informerebbe un clinico durante una valutazione di rischio cardiovascolare. Se il sistema dice “malattia probabile” per un paziente sano (un **falso positivo**), la conseguenza tipica è un esame diagnostico in più — invasivo o costoso, ma raramente pericoloso di per sé. Se il sistema dice “nessuna malattia” per un paziente che invece ce l’ha (un **falso negativo**), la conseguenza è la mancata individuazione di una condizione che può aggravarsi. I due errori non pesano allo stesso modo, e nessuna delle metriche che il progetto calcola — le vedremo al capitolo 34 — lo sa già da sola: lo sai tu, quando le interpreti.

Per Diabetes130, il ragionamento è simmetrico ma le conseguenze pratiche sono diverse. Un falso positivo (il sistema segnala rischio di riammissione per un paziente che non tornerà) costa una telefonata di follow-up in più, forse non necessaria. Un falso negativo (il sistema non segnala un paziente che invece tornerà entro 30 giorni) costa l’assenza di un intervento — educazione del paziente, controllo ravvicinato, aggiustamento della terapia — che avrebbe potuto prevenire il ricovero.

ATTENZIONE — in nessuno dei due casi il progetto stesso discute esplicitamente questa asimmetria nel codice: le metriche calcolate (accuratezza, F1, AUC — capitolo 34) trattano i due tipi di errore in modo sostanzialmente simmetrico. È un limite reale, non un dettaglio implementativo: lo riprendiamo con la dovuta attenzione critica nel capitolo 51.

1.3 Cosa si faceva prima: classificazione diretta su dati tabellari

[Livello: teoria consolidata del settore] L’approccio più diffuso per problemi di questo tipo — dati clinici strutturati, etichetta binaria — è addestrare un classificatore direttamente sulle feature tabellari originali: un modello lineare (come la regressione logistica che il progetto stesso usa più avanti nella pipeline, capitolo 32), oppure un modello ad albero o un insieme di alberi (random forest, gradient boosting). Questi approcci non richiedono alcuna conversione a testo: la tabella con le sue colonne numeriche e categoriali entra direttamente nel modello. È un approccio maturo, ben studiato, spesso difficile da battere su dati genuinamente tabellari con poche migliaia di righe — esattamente la scala dei due dataset di questo progetto.

[Fatto] docs/DATASET.md:59–64 e docs/DATASET.md:109–110 sono espliciti su questo punto: il contributo di questo progetto **non** è un classificatore tabellare diretto. È qualcos’altro — una trasformazione del problema, prima ancora che una soluzione — che il capitolo 3 introduce da zero.

[Interpretazione] Vale la pena notare, con lo sguardo critico che questo libro adotta ovunque, che scegliere di non fare ciò che la letteratura consolidata farebbe per primo non è né giusto né sbagliato in astratto: è una scelta di ricerca che sposta la domanda da “qual è il modello più accurato” a “cosa succede se rappresenti i dati in un modo diverso”. È una domanda legittima. Ma significa anche che il confronto onesto richiederebbe, idealmente, anche un classificatore tabellare diretto come termine di paragone — cosa che il progetto non include, e che il capitolo 47 costruisce da zero come esercizio di validazione.

Riepilogo

Il progetto affronta due problemi di classificazione binaria su dati clinici strutturati — diagnosi di malattia coronarica e rischio di riammissione ospedaliera — con la stessa struttura tecnica ma conseguenze pratiche diverse per i due tipi di errore possibile. L’approccio consolidato in letteratura per questo genere di problema è la classificazione tabellare diretta; il progetto sceglie invece una strada diversa, che introduciamo dal capitolo 3 in poi.

Domande di autoverifica

1. Le due variabili target, `num` per Heart Disease e `readmitted` per Diabetes130, come diventano etichette binarie nel codice? `num` diventa binaria con la regola (`num > 0`) in `preprocessing.py:41` (qualunque grado di malattia rilevato conta come “presente”). `readmitted` diventa binaria con la regola (`readmitted == "<30"`) in `function.py:124` (solo la riammissione entro 30 giorni conta come “positiva”; una riammissione più tardiva conta come “negativa”, esattamente come nessuna riammissione).

2. Perché l’asimmetria tra falsi positivi e falsi negativi non è già incorporata nelle metriche che il progetto calcola? Perché accuratezza, F1 e AUC (capitolo 34) sono metriche generiche, pensate per riassumere le prestazioni di un classificatore senza assumere nulla sul costo relativo dei due tipi di errore in un dominio specifico. Incorporare quell’asimmetria richiederebbe una scelta esplicita — per esempio una soglia di decisione calibrata sul costo clinico, non solo sull’F1 — che il progetto, come vedremo al capitolo 35, non fa.

3. In che senso il progetto “non fa” ciò che la letteratura consolidata farebbe per primo? Non addestra un classificatore direttamente sulle feature tabellari originali. Trasforma prima ogni record in una frase di linguaggio naturale, poi calcola un embedding di quella frase, e allena il classificatore sull’embedding — una scelta metodologica di per sé, discussa dal capitolo 3 in poi, non un’omissione.

MATERIALE PER LA TESI 1. La formulazione precisa dei due task (diagnosi binaria; riammissione a 30 giorni binarizzata da una variabile originariamente non binaria) con i relativi riferimenti `file:riga` — riusabile parola per parola nell’introduzione della tesi per definire il problema. 2. Il ragionamento sull’asimmetria costo-FP/costo-FN, differenziato per i due dataset — riusabile nella discussione (capitolo 51) come base per una critica motivata alla scelta delle metriche. 3. L’osservazione che il progetto omette il confronto con un classificatore tabellare diretto — riusabile come motivazione esplicita per l’esperimento di baseline che costruiamo al capitolo 47, e che rafforza la sezione “materiali e metodi” della tesi.

Capitolo 2 — Il quadro del machine learning supervisionato

Obiettivi del capitolo

- Collocare quello che fa il progetto in una mappa più ampia: cosa distingue apprendimento supervisionato, non supervisionato e per rinforzo.
- Avere un modello mentale del ciclo addestramento → validazione → inferenza, prima di vederlo nel codice.
- Sapere cosa significano overfitting, underfitting e generalizzazione, e perché sono il motivo per cui esiste la validazione.

Questo capitolo non parla ancora del progetto in dettaglio. Parla del quadro concettuale dentro cui il progetto si colloca — necessario perché il tuo punto di partenza su questo, per calibrazione esplicita di questo libro, è zero, indipendentemente da quanto tu sappia già di ingegneria del software.

2.1 Apprendimento supervisionato, non supervisionato, per rinforzo: una mappa minima

[**Livello: teoria consolidata del settore**] Il machine learning si divide, a grandi linee, in tre famiglie di problemi, distinte da *cosa hai a disposizione durante l'addestramento*.

Nell'**apprendimento supervisionato**, hai esempi già etichettati: coppie (input, output corretto). Il compito è imparare una funzione che, dato un nuovo input mai visto, produca un output plausibile, generalizzando da quegli esempi. Un embedding di un record clinico come input, l'etichetta “malattia presente/assente” come output atteso: questo è supervisionato, ed è tutto ciò che fa questo progetto nella sua parte di classificazione.

Nell'**apprendimento non supervisionato**, hai solo input, senza alcun output di riferimento. Il compito è trovare struttura nei dati stessi: raggruppamenti, direzioni di massima variazione, vicinanze. [**Fatto**] Il progetto usa esattamente una tecnica di questa famiglia — UMAP, in `function.py:223 plot_umap` — ma solo per produrre un grafico bidimensionale a scopo di ispezione visiva (capitolo 38), mai per decidere l'etichetta di un record. È un dettaglio che vale la pena notare subito: la pipeline è supervisionata dall'inizio alla fine tranne che in un punto puramente illustrativo.

Nell'**apprendimento per rinforzo**, un agente interagisce con un ambiente, riceve una ricompensa numerica per le sue azioni, e impara una strategia che la massimizzi nel tempo. [**Fatto**] Questa famiglia non compare da nessuna parte nel codice del progetto: la citiamo solo perché la mappa sia completa, non perché sia rilevante qui.

SE VIENI DA JAVA — non c'è un'analogia diretta e onesta con qualcosa che conosci già dal mondo enterprise. La cosa più vicina, concettualmente, è la differenza

fra scrivere una regola esplicita (`if pressione > 140: rischio_alto = true`) e *dedurre* quella soglia da migliaia di casi passati invece di scriverla tu. Il “programma” nell’apprendimento supervisionato non è il codice che hai scritto: sono i parametri numerici che l’addestramento ha trovato, e che il codice si limita ad applicare.

2.2 Il ciclo addestramento → validazione → inferenza

[Livello: teoria consolidata del settore] Ogni pipeline supervisionata attraversa tre momenti concettualmente distinti, anche quando il codice li esegue uno via l’altro senza soluzione di continuità.

Addestramento (training): al modello vengono mostrati input ed etichette corrette insieme, e un algoritmo di ottimizzazione regola i parametri interni del modello per minimizzare l’errore su quegli esempi. **[Fatto]** Nel progetto, questo è esattamente `logisticReg.fit(X_train, y_train)` (`classification.py:26`): `X_train` sono gli embedding, `y_train` le etichette vere, e `fit` è il verbo che in scikit-learn significa sempre “esegui l’addestramento”.

Validazione: il modello addestrato viene applicato a dati che *non ha usato per allenarsi*, di cui però conosci comunque l’etichetta vera — così puoi confrontare previsione e realtà e stimare quanto bene il modello si comporterà su dati futuri, genuinamente sconosciuti. **[Fatto]** Nel progetto, questo è `logisticReg.predict_proba(X_val)` (`classification.py:28`), dove `X_val` è la parte di dati tenuta fuori dall’addestramento in quel fold — il meccanismo esatto è il tema del capitolo 33.

Inferenza: il modello, già addestrato e già validato, viene applicato a un input completamente nuovo, di cui *non conosci* l’etichetta vera — è il momento in cui il sistema produce effettivamente valore, perché prima di allora stavi solo misurando quanto ti puoi fidare di lui. **[Fatto, ma non in master]** Il progetto, sul branch `master`, si ferma alla validazione: non esiste, nel codice che orchestra `main.py`, un percorso che prenda un record clinico nuovo e restituisca una previsione. Quel percorso esiste solo sul branch `chatbot`, non unito (`chatbot_core.py`, capitolo 54): è lì che il ciclo si chiude davvero, e non a caso è anche il punto in cui le implicazioni etiche di un errore diventano più concrete.

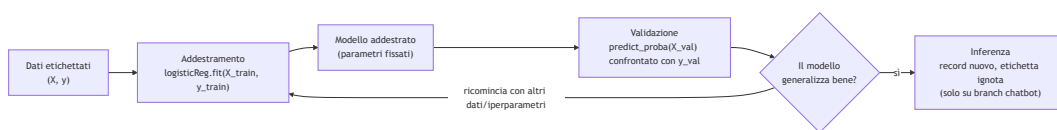


Figura 2.1 — Il ciclo addestramento/validazione/inferenza, con i riferimenti esatti a dove ciascun momento avviene nel codice del progetto.

2.3 Overfitting, underfitting, generalizzazione

[Livello: teoria consolidata del settore] Un modello che si comporta perfettamente sui dati di addestramento non è necessariamente un buon modello. Può darsi che abbia imparato a memoria le particolarità di *quei* dati — incluso il rumore, le coincidenze, gli errori di misura — invece di catturare il pattern reale che si ripete anche su dati nuovi. Questo fenomeno si chiama **overfitting** (sovradattamento): l’errore sui dati di addestramento è basso, l’errore su dati nuovi è alto. Il fenomeno opposto, l’**underfitting** (sottoadattamento), succede quando il modello è troppo semplice per catturare anche il pattern reale, e sbaglia sia sui dati di addestramento sia su quelli nuovi.

La **generalizzazione** è la capacità di un modello di comportarsi bene su dati che non ha mai visto durante l'addestramento. È l'unica cosa che conta davvero in un sistema che dovrà essere usato su pazienti futuri, non sui pazienti già registrati nel dataset. Ed è precisamente per stimare la generalizzazione, senza aspettare che il sistema sia già in uso per scoprire che sbaglia, che esiste la validazione del paragrafo precedente: tieni da parte una porzione di dati con etichetta nota, fingi di non conoscerne l'etichetta, e misura quanto il modello ci va vicino.

ATTENZIONE — la stima di generalizzazione vale solo quanto è onesta la separazione fra dati di addestramento e dati di validazione. Se anche un solo bit di informazione sui dati di validazione trapela nel processo di addestramento o di scelta del modello — un fenomeno chiamato **data leakage**, che il capitolo 33 tratta per esteso — la stima diventa artificialmente ottimistica, e lo scopri solo quando il sistema è già in produzione e i numeri reali sono peggiori di quelli misurati. Anticipiamo qui perché è rilevante da subito: questo progetto, come vedremo nel dettaglio ai capitoli 33, 35 e 51, ha almeno due punti in cui questa separazione è meno netta di quanto sembri a prima lettura.

[**Approfondimento facoltativo**] Il compromesso fra overfitting e underfitting è spesso descritto in letteratura come *bias-variance tradeoff*: un modello troppo semplice ha alto bias (sbaglia sistematicamente, in modo simile su ogni campione di dati), un modello troppo complesso ha alta varianza (il suo comportamento cambia molto a seconda del campione specifico di dati di addestramento che riceve). Una trattazione rigorosa di questo compromesso è materia da manuale di machine learning generale — per esempio Hastie, Tibshirani e Friedman, *The Elements of Statistical Learning* [DA VERIFICARE - non citare prima del controllo: edizione, anno ed editore esatti da confermare] — e non è necessaria per seguire il resto di questo libro: la citiamo per chi volesse approfondirla nella tesi.

Riepilogo

Il progetto si colloca interamente nell'apprendimento supervisionato, con una singola incursione non supervisionata (UMAP) usata solo a scopo illustrativo. Il ciclo addestramento→validazione→inferenza è il telaio concettuale su cui si legge ogni riga di `classification.py`; il progetto, sul branch `master`, copre addestramento e validazione ma non l'inferenza su casi nuovi. Overfitting, underfitting e generalizzazione sono il motivo per cui la validazione esiste, e la loro affidabilità dipende interamente dal non lasciare trapelare informazione dai dati di validazione verso l'addestramento — un punto su cui questo progetto merita attenzione, come i capitoli successivi mostreranno.

Domande di autoverifica

1. Perché UMAP, che è una tecnica non supervisionata, non “rompe” la natura supervisionata della pipeline? Perché non partecipa mai alla decisione di classificazione: `plot_umap` (`function.py:223`) produce solo una proiezione 2D usata per un grafico ispezionabile a occhio. Il classificatore vero e proprio non riceve mai l'output di UMAP come input — riceve gli embedding originali a piena dimensione.

2. Qual è la differenza pratica, non solo definitoria, fra “validazione” e “inferenza”? In validazione conosci già l'etichetta vera e la usi per misurare l'errore del modello; in inferenza non la conosci affatto, ed è per quello che ti serve il modello. Una pipeline che confonde le due cose — per esempio scegliendo un parametro guardando l'etichetta vera del dato su cui poi riporta il punteggio — produce una stima di generalizzazione fin troppo ottimistica.

3. Cosa significa, in una frase, che un modello “generalizza bene”? Che l’errore misurato su dati mai visti durante l’addestramento è vicino all’errore misurato sui dati di addestramento stesso — cioè che il modello ha imparato un pattern reale, non le particolarità del campione specifico che gli è stato mostrato.

MATERIALE PER LA TESI 1. La mappa supervisionato/non supervisionato/per rinforzo, con la collocazione esplicita di ogni tecnica del progetto in una delle tre categorie — riusabile come paragrafo di inquadramento metodologico in “Materiali e metodi”. 2. Il diagramma Mermaid del ciclo addestramento/validazione/inferenza con i riferimenti di codice (Figura 2.1) — riusabile direttamente come figura nella tesi, con didascalia originale. 3. La definizione operativa di data leakage e l’anticipazione dei due punti del progetto in cui la separazione training/validazione è meno netta del previsto — riusabile come apertura della sezione “Discussione e limiti”, da sviluppare con il dettaglio dei capitoli 33, 35 e 51.

Capitolo 3 — Dalla tabella al testo

Obiettivi del capitolo

- Capire perché una tabella di numeri e categorie non è, di per sé, qualcosa che un modello linguistico può usare direttamente.
- Vedere l’idea centrale del progetto — tabellare→testo→embedding→classificazione — prima di incontrarne il codice.
- Sapere quali strade alternative esistono per lo stesso problema, e in cosa questa se ne distingue.

3.1 Limiti di una rappresentazione tabellare per i modelli linguistici

Immagina di dover descrivere un paziente a un collega al telefono, senza poter mostrargli una tabella. Non gli detteresti “sesso: 1, età: 63, cp: 4, trestbps: 145” — gli diresti “un paziente di 63 anni, uomo, con dolore toracico di tipo asintomatico, pressione a riposo di 145”. La differenza non è cosmetica: la seconda forma porta con sé un contesto — “dolore toracico asintomatico” è un’espressione che compare in migliaia di testi medici, con relazioni semantiche note (è associata a un certo tipo di rischio, ricorre vicino ad altre espressioni cliniche specifiche) — mentre il numero “4” da solo non porta nessuna di queste relazioni. Un modello linguistico pre-addestrato su enormi quantità di testo ha imparato quelle relazioni semantiche osservandole nel testo. Non le ha mai osservate nei codici numerici di una tabella, perché in una tabella quelle relazioni semplicemente non compaiono nella stessa forma.

[Livello: teoria consolidata del settore] Questo è il motivo tecnico per cui un modello di embedding testuale — costruito e addestrato per rappresentare frasi, non righe di tabella — non può essere applicato direttamente ai dati tabellari originali. Il dato tabellare deve prima diventare testo, se vuoi che un modello di questo tipo lo elabori.

3.2 L’idea del progetto: tabellare → testo → embedding → classificazione

[Fatto] Il diagramma che apre `README.md:5-16` descrive esattamente questa catena di trasformazioni, e corrisponde punto per punto a ciò che il codice fa (lo verifichiamo modulo per modulo nella Parte V):

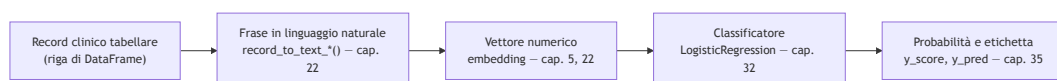


Figura 3.1 — La catena di trasformazioni centrale del progetto: ogni passaggio ha un capitolo dedicato più avanti nel libro.

Il primo passaggio — da riga di tabella a frase — è realizzato da funzioni come `record_to_text_heart_disease()` (`embedding.py:43-59`), che il capitolo 22 legge riga per riga. Il secondo — da frase a vettore — usa modelli di embedding pre-addestrati, sia generalisti sia specializzati in ambito biomedico (capitolo 5 per l'intuizione, capitolo 22 per il codice). Il terzo — dal vettore a una previsione — è una regressione logistica ordinaria (capitolo 32), lo stesso tipo di modello che avresti potuto addestrare direttamente sulla tabella originale, se avessi scelto quella strada.

Questa è l'osservazione più importante del capitolo, ed è bene renderla esplicita: **il progetto non inventa un nuovo tipo di classificatore**. Usa un classificatore lineare ordinario. Quello che cambia è *da cosa* quel classificatore impara — non dalle 14 o 19 colonne originali, ma da centinaia di numeri prodotti da un modello linguistico che ha “letto” la versione testuale di quelle colonne. La domanda di ricerca del progetto (capitolo 6.3 la riprende nel dettaglio) è se questa trasformazione aiuti o no.

3.3 Approcci alternativi esistenti

[Livello: teoria consolidata del settore] Per lo stesso tipo di problema — classificazione binaria su dati clinici tabellari — la letteratura offre principalmente tre famiglie di approccio, oltre a quella scelta da questo progetto:

- **Classificazione tabellare diretta con modelli lineari o ad albero.** Regressione logistica, random forest, gradient boosting (per esempio XGBoost o LightGBM) addestrati direttamente sulle colonne originali, senza alcuna conversione. È l'approccio più comune e spesso il più difficile da battere su dataset di poche migliaia di righe con feature già pulite e clinicamente significative — esattamente la situazione di entrambi i dataset di questo progetto. [Fatto] È anche l'approccio che lo stesso `generatereport.py:221` cita come miglioramento suggerito (“Introduce a non-linear classifier (XGBoost, LightGBM)”), pur non implementandolo mai nella pipeline.
- **Reti neurali specializzate per dati tabellari.** Architetture che imparano rappresentazioni delle colonne categoriali (embedding di categoria, non di testo) insieme a una rete che le combina con le colonne numeriche. Sono un'area di ricerca attiva proprio perché, a differenza di immagini o testo, i dati tabellari non hanno una struttura spaziale o sequenziale ovvia da cui le reti neurali tradizionalmente traggono vantaggio.
- **Classificazione diretta su testo clinico non strutturato**, quando la fonte non è una tabella ma una nota clinica scritta in linguaggio naturale da un medico. Questo è, in un certo senso, lo scenario “naturale” per cui i modelli di embedding testuale usati in questo progetto sono stati pensati — e qui emerge la particolarità metodologica di questo lavoro: i dati di partenza *sono* tabellari, e vengono trasformati in testo apposta per poter usare quei modelli, invertendo il percorso più comune.

[Interpretazione] Il progetto si colloca quindi in una zona intermedia: non usa testo clinico autentico (note mediche scritte da un professionista), ma testo *sintetico*, generato meccanicamente da una tabella con un template fisso (lo vedi al capitolo 22.1: sempre la stessa struttura di frase, cambiano solo i valori). Vale la pena tenerlo a mente quando, al capitolo 53, mettiamo il progetto in prospettiva rispetto allo stato dell'arte: il vantaggio “linguistico” di un modello pre-addestrato su testo reale è testato qui su un testo che non assomiglia granché a quello su cui quel modello si aspetterebbe di essere usato.

PROVA TU — prendi una riga qualunque di `datas/heart_disease/preprocessing/X_train_raw.csv` (la incontri per la prima volta al capitolo 21) e prova a scriverne tu una descrizione in linguaggio naturale, senza guardare il codice di `embedding.py`. Poi confronta la tua frase con quella che produce `record_to_text_heart_disease()` (capitolo

22.1). Le differenze che trovi — cosa hai scelto di enfatizzare, cosa hai omesso — sono esattamente il tipo di scelta progettuale che un template fisso, come quello del codice, non può fare caso per caso.

Riepilogo

Un modello di embedding testuale non può essere applicato a dati puramente numerici perché non ha mai osservato, durante il proprio addestramento, le relazioni che quei numeri codificano — le ha osservate nel testo. Il progetto risolve il problema convertendo ogni record in una frase con un template fisso, poi calcolando un embedding di quella frase, poi allenando un classificatore lineare ordinario su quell'embedding. È un approccio distinto sia dalla classificazione tabellare diretta sia dall'uso “naturale” di questi modelli su testo clinico autentico.

Domande di autoverifica

1. Perché non si può semplicemente dare in pasto a un modello di embedding testuale la tabella originale, colonna per colonna? Perché il modello è stato addestrato a rappresentare il significato del testo, non i codici numerici di una tabella: le relazioni semantiche che sa cogliere esistono nella forma in cui il testo le presenta, non nella forma in cui una tabella codifica gli stessi fatti.

2. Cosa resta invariato, e cosa cambia, rispetto a un classificatore tabellare diretto? Resta invariato il classificatore finale: è una regressione logistica in entrambi i casi. Cambia l'input che riceve — colonne originali nella classificazione diretta, un vettore di embedding calcolato da una frase generata automaticamente in questo progetto.

3. In che senso il testo generato da `record_to_text_*` non è testo clinico “naturale”? Perché non è scritto da un medico che descrive un paziente con le proprie parole: è generato meccanicamente da un template fisso, applicato riga per riga a una tabella. È testo sintetico con una struttura grammaticale identica per ogni record, non prosa clinica autentica.

MATERIALE PER LA TESI 1. Il diagramma Mermaid della catena tabella→testo→embedding→classificazione (Figura 3.1), con i rimandi di capitolo — riusabile come figura di apertura della sezione “Materiali e metodi”. 2. La tassonomia delle famiglie di approccio alternative (tabellare diretto, reti neurali per dati tabellari, testo clinico autentico) — riusabile per la sezione “Stato dell'arte”, con la collocazione esplicita di questo progetto rispetto a ciascuna. 3. L'osservazione che il testo generato è sintetico e non clinico-autentico — riusabile come premessa esplicita a una qualunque affermazione, nella tesi, sul “vantaggio” dei modelli biomedici: è un vantaggio misurato su un tipo di testo diverso da quello per cui quei modelli sono stati pensati.

Capitolo 4 — Classificazione binaria: l'essenziale per seguire il libro

Obiettivi del capitolo

- Sapere cosa significa, con precisione, che un modello “classifica” qualcosa in due categorie.
- Riconoscere lo sbilanciamento delle classi come problema concreto, con i numeri reali di questo progetto.
- Padroneggiare il vocabolario minimo — vero/falso positivo/negativo — su cui si appoggia ogni metrica del capitolo 34.

4.1 Due classi, una probabilità, una soglia

Un classificatore binario, nella sua forma più comune — quella usata in questo progetto — non produce direttamente un’etichetta “sì” o “no”. Produce un numero fra 0 e 1, interpretabile come una probabilità stimata che l’esempio appartenga alla classe positiva. **[Fatto]** Nel codice, questo numero è `y_score`, prodotto da `logisticReg.predict_proba(X_val)[: , 1]` (`classification.py:28`) — la seconda colonna dell’output di `predict_proba`, quella associata alla classe 1.

Per ottenere un’etichetta vera e propria — “malattia presente” oppure “malattia assente” — serve un secondo passaggio: confrontare quel numero con una **soglia di decisione**. Se il punteggio supera la soglia, l’etichetta prevista è 1; altrimenti è 0. La soglia più ovvia è 0.5, ma non è affatto obbligata, e il capitolo 35 mostra perché questo progetto ne sceglie una diversa, calcolata per ogni fold di validazione.

SE VIENI DA JAVA — non c’è un `enum` a due valori da qualche parte nel codice. La classe prevista è sempre calcolata al volo con un confronto numerico — `(y_score >= tau).astype(int)` (`classification.py:37`) — e non esiste, in nessun punto della pipeline, un tipo che rappresenti esplicitamente “la classe”: è sempre e solo un intero 0 o 1, con il significato che gli attribuisce tu leggendo il codice, non il compilatore.

4.2 Sbilanciamento delle classi

[Livello: teoria consolidata del settore] Uno sbilanciamento delle classi si ha quando le due categorie da distinguere non compaiono con la stessa frequenza nei dati. È un problema pratico, non solo statistico: un classificatore che si limitasse a rispondere sempre “classe maggioritaria” otterrebbe già un’accuratezza numericamente alta, senza aver imparato nulla di utile — un punto a cui torniamo con un calcolo preciso al capitolo 47, quando costruiamo un modello di riferimento banale.

[Fatto] I due dataset di questo progetto hanno un grado di sbilanciamento molto diverso, entrambi verificati caricando i file grezzi con lo stesso procedimento usato dal codice (comando Python eseguito in questa sessione):

Dataset	Righe totali (prima di split e SMOTE)	Classe positiva	Classe negativa	% positiva
Heart Disease	920	509	411	55.3%
Diabetes130 (intero file, pre- campionamento)	101.766	11.357	90.409	11.16%

Heart Disease è, sorprendentemente, quasi bilanciato: la classe “malattia presente” è addirittura leggermente maggioritaria. Diabetes130 è invece marcatamente sbilanciato: solo un ricovero su nove circa produce una riammissione entro 30 giorni. Sono due situazioni realistiche e diverse, ed è un bene che il libro le tratti entrambe: il grado di sbilanciamento cambia quali metriche sono informative (capitolo 34) e quanto lavoro deve fare il bilanciamento sintetico delle classi (capitolo 31.2).

ATTENZIONE — la tabella riporta lo sbilanciamento **prima** della suddivisione training/test e prima di SMOTENC (capitolo 31.2), che poi riequilibra artificialmente il set di addestramento. Il numero informativo su “quanto è difficile il problema nella realtà” è quello di questa tabella, non quello — reso paritario ad arte — degli embedding effettivamente usati per l’addestramento.

[Fatto] La cifra di 920 righe per Heart Disease merita una precisazione, perché contraddice la documentazione del progetto: sia `README.md:78` sia `docs/DATASET.md:17` dichiarano “297” record per questo dataset. Il codice reale (`load_heart_disease()`, `function.py:96-113`) concatena però tutti e quattro i file grezzi senza scartare nessuna riga, per un totale verificabile di 920.

[Inferenza] La cifra “297” è quella storicamente citata in letteratura per il solo sottoinsieme Cleveland dopo aver scartato le righe con dati mancanti nelle feature più critiche — un dataset diverso da quello che il codice di questo progetto carica davvero. Ne riparlamo, con tutto il dettaglio, al capitolo 29.3 e di nuovo, in chiave critica, al capitolo 51.

4.3 Vero/falso positivo/negativo

[Livello: teoria consolidata del settore] Per qualunque coppia (etichetta prevista, etichetta vera), esistono esattamente quattro esiti possibili, riassunti nella matrice di confusione che il capitolo 24 mostra per ogni modello del progetto:

	Etichetta vera: positiva	Etichetta vera: negativa
Prevista: positiva	Vero positivo (VP)	Falso positivo (FP)
Prevista: negativa	Falso negativo (FN)	Vero negativo (VN)

Un **vero positivo** è un caso positivo correttamente riconosciuto come tale; un **vero negativo**, un caso negativo correttamente riconosciuto come tale. Un **falso positivo** è un caso negativo scambiato per positivo; un **falso negativo**, un caso positivo scambiato per negativo. Nel linguaggio del capitolo 1.2: per Heart Disease, un falso negativo è una malattia coronarica non individuata; per Diabetes130, un falso negativo è un paziente ad alto rischio di riammissione lasciato senza follow-up rinforzato.

[Fatto] Il codice calcola esplicitamente questi quattro esiti in `error_analysis.py:26-27`:

```
\NormalTok{fp\_mask }\OperatorTok{=}\NormalTok{(y\_true }\OperatorTok{==}
↪ \DecValTok{0}\NormalTok{) }\OperatorTok{\&}\NormalTok{(y\_pred }\OperatorTok{==}
↪ \DecValTok{1}\NormalTok{)}
\NormalTok{fn\_mask }\OperatorTok{=}\NormalTok{(y\_true }\OperatorTok{==}
↪ \DecValTok{1}\NormalTok{) }\OperatorTok{\&}\NormalTok{(y\_pred }\OperatorTok{==}
↪ \DecValTok{0}\NormalTok{)}
```

Nota, per chi viene da Java, l'operatore `&` fra due array booleani NumPy: non è l'AND logico di Python (`and`), che qui darebbe un errore — è l'AND *elemento per elemento* fra due vettori di verità, un'operazione che in Java richiederebbe un ciclo esplicito o uno stream. Ne parliamo con tutto il dettaglio necessario al capitolo 8.

Riepilogo

Un classificatore binario in questo progetto produce sempre una probabilità, non direttamente un'etichetta: l'etichetta nasce dal confronto con una soglia. Heart Disease è quasi bilanciato (55.3%/44.7%, su 920 righe reali — non 297 come dichiarato nella documentazione del progetto), mentre Diabetes130 è marcatamente sbilanciato (11.16% di classe positiva su oltre 100.000 righe). I quattro esiti possibili di una previsione — vero/falso positivo/negativo — sono il vocabolario su cui si costruisce ogni metrica dei capitoli successivi.

Domande di autoverifica

- 1. Perché `y_score` non è già una previsione utilizzabile da sola?** Perché è un numero continuo fra 0 e 1 (una probabilità stimata), non un'etichetta binaria. Serve un secondo passaggio — il confronto con una soglia — per ottenere `y_pred`, l'etichetta vera e propria.
- 2. Quale dei due dataset del progetto è più sbilanciato, e di quanto?** Diabetes130, nettamente: 11.16% di classe positiva contro l'88.84% negativa, calcolato sull'intero file grezzo. Heart Disease è invece quasi bilanciato, 55.3%/44.7%.
- 3. Un modello segnala “riammissione probabile” per un paziente che poi non torna in ospedale entro 30 giorni. Che tipo di esito è, e qual è il suo “gemello” nella matrice di confusione?** È un falso positivo. Il suo gemello concettuale — l'altro tipo di errore — è il falso negativo: un paziente che il modello giudica a basso rischio, ma che invece torna in ospedale entro 30 giorni.

MATERIALE PER LA TESI 1. La tabella con i numeri reali di sbilanciamento per entrambi i dataset (§4.2), con il comando Python usato per ottenerli — riusabile direttamente nella sezione “Materiali e metodi” o in una tabella descrittiva dei dati. 2. La discrepanza numerica 297 vs. 920 per Heart Disease, con la spiegazione della causa più probabile — riusabile come osservazione critica autonoma, con rimando al capitolo 51 per il trattamento esteso. 3. La matrice di confusione generica con la traduzione clinica di ciascun esito per entrambi i dataset — riusabile come figura o tabella nella sezione “Discussione”, per motivare perché una metrica aggregata da sola non basta a giudicare un sistema di supporto clinico.

Capitolo 5 — Rappresentazioni testuali ed embedding

Obiettivi del capitolo

- Avere un'intuizione di cos'è un embedding prima di vedere una sola formula o una sola riga di codice.
- Capire, a livello concettuale, come un modello linguistico trasforma una frase in un vettore di numeri.
- Sapere perché il progetto distingue esplicitamente modelli “generalisti” da modelli “biomedici”, e da dove viene questa distinzione nel codice.

5.1 Cos'è un embedding, in una frase, prima di ogni formula

Un **embedding** è la rappresentazione di un testo — una parola, una frase, un intero documento — come un elenco fisso di numeri, un vettore. Non è una codifica arbitraria: due frasi che un lettore umano giudicherebbe simili nel significato producono, con un buon modello di embedding, vettori numericamente vicini fra loro; due frasi di significato lontano producono vettori numericamente lontani. La “vicinanza” fra vettori si misura di solito con una distanza geometrica ordinaria (o con una quantità strettamente imparentata, la similarità coseno), lo stesso tipo di distanza che useresti fra due punti su una mappa.

[Fatto] In questo progetto, ogni frase generata da `record_to_text_*` (capitolo 22.1) diventa, dopo l'embedding, un vettore di 768 o 1024 numeri in virgola mobile a seconda del modello (verificato caricando i file `.npy` con NumPy — la tabella completa è al capitolo 39.1). Quel vettore, non la frase originale, è ciò che il classificatore vede.

SE VIENI DA JAVA — non c'è una `class Embedding` da qualche parte con campi nominati. Un embedding, in questo progetto come nella maggior parte del codice di questo tipo, è semplicemente un array NumPy di numeri in virgola mobile — un `float[]` di lunghezza fissa, se preferisci pensarlo in termini Java, ma senza alcun nome per le singole posizioni: la posizione 347 del vettore non “significa” nulla di dichiarabile a parole, è solo una delle centinaia di coordinate che insieme codificano il significato della frase.

5.2 Da testo a vettore: tokenizzazione, encoder transformer, pooling

[Livello: teoria consolidata del settore] Tutti i sette modelli usati in questo progetto — sia quelli generalisti serviti da Ollama sia quelli biomedici serviti da Hugging Face — condividono la stessa famiglia architettuale di fondo, quella dei modelli **transformer** basati su encoder (la

stessa famiglia architetturale di BERT). Il percorso da frase a vettore attraversa tre passaggi concettuali:

1. **Tokenizzazione.** La frase viene spezzata in unità più piccole di una parola intera — sotto-parole, spesso — ciascuna delle quali corrisponde a un identificativo numerico in un vocabolario fisso che il modello ha imparato durante il proprio pre-addestramento. “Asymptomatic” potrebbe diventare due o tre token, non uno solo.
2. **Passaggio attraverso l’encoder.** Ogni token, inizialmente rappresentato da un vettore che dipende solo da se stesso, attraversa più strati della rete neurale che ricalcolano la sua rappresentazione tenendo conto di *tutti gli altri token della frase* — è questo il meccanismo che permette al modello di rappresentare, per esempio, che “non” prima di “diabetico” ne inverte il significato clinico.
3. **Pooling.** A questo punto hai un vettore per ogni token della frase, non un vettore per l’intera frase. Il pooling li combina in un unico vettore di lunghezza fissa — tipicamente calcolandone la media (*mean pooling*), talvolta prendendo solo il vettore associato a un token speciale di inizio-frase.

[Fatto] Il codice del progetto conferma esplicitamente che questo è il meccanismo in gioco per i modelli biomedici: un commento in `embedding.py:170` dichiara che “SentenceTransformer applicherà automaticamente il ‘mean pooling’ ai token di Bio_ClinicalBERT” — la libreria `sentence-transformers` (capitolo 22.3) si occupa dei tre passaggi per te, e la riga di codice che li innesca è una sola: `model.encode(texts, ...)` (`embedding.py:178`).

APPROFONDIMENTO FACOLTATIVO — i modelli generalisti di questo progetto (E5, GTE) sono addestrati con un obiettivo specifico chiamato apprendimento contrastivo: durante il pre-addestramento, il modello vede coppie di testi che dovrebbero essere semanticamente vicini (per esempio una domanda e la sua risposta corretta) insieme a testi che non dovrebbero esserlo, e viene aggiustato in modo che le prime coppie producano vettori vicini e le seconde vettori lontani. I paper originali di questi modelli — Wang et al. per E5, Li et al. per GTE [DA VERIFICARE - non citare prima del controllo: titoli, sedi e anni esatti da confermare] — dettagliano la procedura; non è necessaria per seguire il resto del libro.

5.3 Perché esistono modelli generalisti e modelli biomedici

[Fatto] Il progetto distingue esplicitamente tre famiglie di modelli, con un campo `family` nella configurazione di ciascuno (`function.py:39-51`):

Famiglia	Modelli nel progetto	Pre-addestrati principalmente su
<code>general-purpose</code>	<code>e5-base</code> , <code>e5-large</code> , <code>gte-base</code> , <code>gte-large</code>	Testo generico, multilingue, di larga scala
<code>biomedical</code>	<code>bioclinicalbert</code> , <code>pubmedbert</code>	Letteratura biomedica e/o note cliniche
<code>biomedical-st</code>	<code>sentence-biobert</code>	Variante biomedica adattata specificamente per produrre embedding di frase

[Livello: teoria consolidata del settore] Un modello pre-addestrato su un corpus generico apprende relazioni semantiche valide per il linguaggio comune, ma può non aver mai incontrato abbastanza testo specialistico da rappresentare bene termini come “discharge disposition” o

“A1C” con la stessa finezza con cui un modello addestrato specificamente su letteratura medica li rappresenterebbe. È esattamente l’ipotesi che il progetto vuole mettere alla prova, non darla per scontata: da qui la seconda domanda di ricerca dichiarata in `README.md:60-61` (“i modelli specializzati nel dominio biomedico producono rappresentazioni semantiche più efficaci di quelli generalisti, per un compito di classificazione clinica basato su dati tabellari trasformati in testo?”). Il capitolo 6.3 mostra esattamente come questa domanda si traduce in codice eseguibile; i capitoli 44 e 45 ne riportano la risposta empirica sui due dataset.

ATTENZIONE — “biomedico” qui non significa “addestrato sugli stessi due dataset di questo progetto”, né su dati simili a quelli in `record_to_text_*`(`*`). Significa addestrato su corpora biomedici in generale — letteratura scientifica, per esempio, per PubMedBERT. È un dominio linguistico affine a quello del progetto, non identico: un punto da non dare per scontato quando si interpretano i risultati del capitolo 44.

Riepilogo

Un embedding rappresenta un testo come un vettore numerico di lunghezza fissa, costruito in modo che testi semanticamente simili producano vettori vicini. Il percorso da frase a vettore, per tutti i modelli di questo progetto, attraversa tokenizzazione, un encoder transformer che contestualizza ogni token rispetto agli altri, e un pooling finale che produce un unico vettore per l’intera frase. Il progetto distingue esplicitamente tre famiglie di modelli — generalisti, biomedici, biomedici per frasi — proprio per poter confrontare, in modo controllato, se la specializzazione di dominio migliora la rappresentazione di testo clinico sintetico.

Domande di autoverifica

- 1. Perché due frasi “simili” dovrebbero produrre vettori “vicini”, e come si misura questa vicinanza?** Perché un buon modello di embedding è costruito e addestrato apposta perché la vicinanza geometrica fra vettori rispecchi la vicinanza semantica fra i testi corrispondenti. Si misura tipicamente con una distanza geometrica o con la similarità coseno fra i due vettori.
- 2. A cosa serve il passaggio di pooling, e perché non basta l’output dell’encoder da solo?** L’encoder produce un vettore per ciascun token della frase, non un vettore per l’intera frase. Il pooling (per esempio la media dei vettori di tutti i token) li riduce a un unico vettore di lunghezza fissa, indipendente da quante parole aveva la frase originale — necessario perché il classificatore successivo si aspetta un input di dimensione costante.
- 3. In che senso la distinzione “generalista/biomedico” non è solo un’etichetta descrittiva, ma il cuore della domanda di ricerca del progetto?** Perché il progetto è costruito apposta per confrontare, a parità di tutto il resto della pipeline, le prestazioni dei modelli delle due famiglie sullo stesso compito — non per usarne uno solo. La domanda “il dominio specialistico aiuta?” ha una risposta solo se il confronto è esplicito e controllato, come qui.

MATERIALE PER LA TESI 1. La tabella delle tre famiglie di modello con i relativi membri e la fonte del pre-addestramento (§5.3) — riusabile come tabella descrittiva in “Materiali e metodi”. 2. Lo schema in tre passaggi tokenizzazione→encoder→pooling, con il riferimento al commento originale del codice (`embedding.py:170`) — riusabile come spiegazione tecnica sintetica nello stato dell’arte. 3. La citazione letterale della seconda domanda di ricerca da `README.md:60-61`, con l’avvertenza sul significato di “biomedico” (§5.3, riquadro Attenzione) — riusabile nell’introduzione della tesi per formulare l’ipotesi in modo preciso, evitando affermazioni più forti di quelle che i dati possono sostenere.

Capitolo 6 — Il gergo del codice: dizionario variabili → concetti

Obiettivi del capitolo

- Avere, prima ancora di leggere una riga di implementazione, un dizionario di riferimento fra i nomi che il codice usa e i concetti della Parte I.
- Riconoscere alcuni nomi del progetto che, letti senza contesto, portano fuori strada.
- Vedere dove, esattamente, le due domande di ricerca del progetto diventano scelte concrete nel codice.

Questo capitolo è pensato per essere consultato, non solo letto una volta. Da qui in avanti, ogni volta che il libro cita una variabile o una costante del codice, il significato è quello di questa tabella — non lo ripetiamo capitolo per capitolo.

6.1 Tabella di corrispondenza completa

Nome nel codice	Dove compare	Cosa significa in linguaggio comune
<code>tau</code>	<code>classification.py:35</code>	La soglia di decisione: il punteggio minimo oltre il quale un caso viene classificato come positivo, scelta per massimizzare F1 su quel fold (capitolo 35)
<code>y_score</code>	ovunque nel progetto	Il punteggio di probabilità stimata della classe positiva, prima di applicare la soglia
<code>y_pred</code>	ovunque nel progetto	L'etichetta binaria prevista, dopo aver applicato <code>tau</code> a <code>y_score</code>
<code>y_true</code>	ovunque nel progetto	L'etichetta vera, nota dal dataset originale
<code>val_idx</code>	<code>classification.py:46</code> , <code>error_analysis.py:24</code>	Gli indici di riga, nel training pool bilanciato, dei record usati come validazione in quel fold — il ponte che permette di tornare dal punteggio statistico al record clinico originale (capitolo 25.1)

Nome nel codice	Dove compare	Cosa significa in linguaggio comune
<code>X_train_bal</code> , <code>y_train_bal</code>	<code>preprocessing.py:53,70</code>	Le feature e le etichette del training set dopo il bilanciamento sintetico delle classi con SMOTENC (capitolo 31.2)
<code>X_train_raw.csv</code>	<code>preprocessing.py:66-68</code>	La versione leggibile (non codificata numericamente) delle feature bilanciate, salvata su disco riga-per-riga allineata agli embedding, così un embedding può sempre essere ricondotto al record clinico che lo ha generato
<code>boot_acc</code> , <code>boot_f1</code> , <code>boot_auc</code>	<code>evaluation.py:23-25</code>	Gli array di 10.000 valori della metrica corrispondente, uno per ogni ricampionamento bootstrap (capitolo 36)
<code>family</code>	<code>function.py:39-51</code>	La categoria del modello di embedding: <code>general-purpose</code> , <code>biomedical</code> o <code>biomedical-st</code> — non ha nulla a che fare con una gerarchia di classi
<code>model_name</code>	dizionari di configurazione in <code>function.py:38-51</code>	L'etichetta breve del modello (es. <code>"e5-base"</code>), usata nei nomi dei file e nei grafici
<code>name</code> (nello stesso dizionario)	idem	L'identificativo completo passato a Ollama o a Hugging Face per generare davvero l'embedding (es. <code>"jeffh/intfloat-e5-base-v2:q8_0"</code>) — non è lo stesso campo di <code>model_name</code> , anche se il nome della chiave non lo segnala
<code>dirs</code>	quasi ogni funzione della pipeline	Il dizionario delle cartelle di output specifiche per il dataset scelto: <code>preprocessing</code> , <code>embeddings</code> , <code>results</code> , <code>graphics</code> , <code>reports</code> (capitolo 20.2)

Nome nel codice	Dove compare	Cosa significa in linguaggio comune
<code>results</code>	<code>function.py:67-72</code> , <code>classification.py:51</code>	Un dizionario globale, a livello di modulo, che accumula le metriche medie per modello — non un risultato locale a una singola chiamata di funzione (capitolo 8.3 spiega perché questo è rilevante in Python)
<code>MODEL_FAMILY</code>	<code>function.py:54</code>	Il dizionario di lookup che, dato il nome breve di un modello, restituisce la sua famiglia
<code>FAMILY_COLORS</code>	<code>function.py:57-61</code>	La palette di colori assegnata a ciascuna famiglia, usata in modo coerente in tutti i grafici del progetto
<code>models_all</code>	<code>function.py:51</code>	La lista dei 7 modelli, unione di <code>models_ollama</code> (i 4 generalisti) e <code>models_medical</code> (i 3 biomedici)
<code>sample_size</code>	<code>function.py:116</code>	La dimensione del campione stratificato estratto da Diabetes130 (20.000 su 101.766 righe originali)
<code>hardest_cases.csv</code>	<code>error_analysis.py:58-63</code>	I record clinici classificati in modo errato dal maggior numero di modelli diversi
<code>feature_deviation.csv</code>	<code>error_analysis.py:65-75</code>	Per ogni feature numerica, la differenza standardizzata fra la sua media nei casi sbagliati e nei casi corretti, aggregata su tutti i modelli
<code>encoder_comparison_summary.csv</code>	<code>evaluation.py:56</code>	La tabella finale: media e intervallo di confidenza al 95% di ciascuna metrica, per ciascun modello
<code>HF_READ_TOKEN</code> , <code>OFFLINE_MODE</code>	<code>.env</code> , letto in <code>embedding.py:146,155</code>	Le due variabili d'ambiente che autenticano verso Hugging Face e attivano/disattivano la modalità offline (capitolo 15.1)

6.2 I nomi che ingannano

Tre nomi meritano un avvertimento esplicito, perché il significato più ovvio — quello che ti verrebbe naturale assumere — non è quello giusto.

tau. Se hai qualche familiarità con la statistica, **tau** potrebbe farti pensare al *tau di Kendall*, un coefficiente di correlazione per ranghi. Non c'è alcuna relazione: qui **tau** è solo il simbolo, per convenzione diffusa in letteratura sulle soglie di classificazione, di un numero fra 0 e 1 che non è una correlazione di nulla — è la soglia di decisione del capitolo 35, e basta.

family. Se vieni dal mondo enterprise, “family” potrebbe evocare una gerarchia di classi o un design pattern (una *abstract factory*, per esempio, spesso descritta in termini di “famiglie di oggetti correlati”). Qui è solo una stringa, un valore come un altro in un dizionario Python (`function.py:39` per esempio: `"family": "general-purpose"`), senza alcuna gerarchia di classi corrispondente nel codice.

num. Non è specifico del codice di questo progetto — viene dallo schema originale del dataset UCI Heart Disease — ma è ingannevole quanto gli altri due. Non significa “numero di qualcosa”: è il nome storico della colonna che codifica la diagnosi (0 = assente, 1-4 = presenza e gravità), poi binarizzata in `preprocessing.py:41`. La prima volta che lo vedi in una tabella, “num” sembra un contatore; è, letteralmente, l’etichetta che l’intero progetto cerca di prevedere.

RIFERIMENTO AL CODICE — questa non è una lista esaustiva di ogni nome del progetto — è una lista di ciò che ti servirà per non fermarti a chiederti “cosa significa questa variabile” mentre segui il filo di un capitolo più avanti. Se incontri un nome che non è in questa tabella, il capitolo che lo introduce lo spiega comunque al momento giusto.

6.3 Le due domande di ricerca del progetto, ora leggibili nel codice

[Fatto] `README.md:56-61` dichiara esplicitamente due domande di ricerca. La prima: gli embedding semantici generati localmente supportano efficacemente la classificazione clinica a partire da dati strutturati convertiti in linguaggio naturale? La seconda: i modelli specializzati nel dominio biomedico producono rappresentazioni più efficaci di quelli generalisti, per questo compito?

Con il vocabolario appena introdotto, puoi ora vedere esattamente dove queste due domande diventano codice eseguibile, non solo dichiarazioni nella documentazione:

- La **prima domanda** non ha una singola riga che la “risponde”: la risposta emerge dal confronto fra le metriche di *tutti* i modelli in `models_all` (capitolo 44-45) rispetto a un modello di riferimento banale che non usa affatto gli embedding (capitolo 47, costruito apposta perché il codice del progetto non lo include).
- La **seconda domanda** si materializza precisamente nel campo `family` di ogni voce di `models_all` (`function.py:39-51`) e nella funzione `plot_family_comparison()` (`function.py:338-357`, capitolo 24), che raggruppa i risultati bootstrap per famiglia invece che per singolo modello — l’unico punto della pipeline in cui la distinzione generalista/biomedico produce un output diverso da un semplice elenco di 7 righe.

ATTENZIONE — una domanda di ricerca dichiarata nella documentazione non garantisce, da sola, che la pipeline sia stata disegnata per rispondervi nel modo statisticamente più solido possibile. Il capitolo 53 mette in prospettiva critica quanto la seconda domanda, in particolare, riceva davvero una risposta netta dai dati — anticipazione utile da tenere a mente da qui in avanti.

Riepilogo

Questo capitolo è un dizionario di consultazione: traduce i nomi di variabili, costanti e file del progetto nei concetti della Parte I, segnala tre nomi (**tau**, **family**, **num**) il cui significato più

intuitivo è quello sbagliato, e mostra dove le due domande di ricerca dichiarate nel README diventano, letteralmente, righe di codice eseguibile.

Domande di autoverifica

1. Qual è la differenza fra `model_name` e `name` nei dizionari di configurazione di `function.py`? `model_name` è l'etichetta breve usata nei nomi di file e nei grafici (per esempio "e5-base"); `name` è l'identificativo completo che il codice passa davvero a Ollama o Hugging Face per generare l'embedding (per esempio "jeffh/intfloat-e5-base-v2:q8_0"). Sono due campi distinti dello stesso dizionario, con scopi diversi.

2. Perché `tau` non ha nulla a che fare con una correlazione statistica, nonostante il nome? Perché è solo la variabile che rappresenta la soglia di decisione ottimale per un fold — un valore fra 0 e 1 scelto massimizzando F1 — non un coefficiente di correlazione. La coincidenza è solo nel simbolo usato, non nel concetto.

3. In quale unica funzione del progetto la distinzione fra famiglie di modello produce un output visibilmente diverso da un confronto modello-per-modello? In `plot_family_comparison()` (`function.py:338-357`), che raggruppa i risultati bootstrap per famiglia (generalista, biomedico, biomedico per frasi) invece che per singolo modello.

MATERIALE PER LA TESI 1. La tabella di corrispondenza completa (§6.1) — riusabile come glossario tecnico in appendice alla tesi, o come tabella di riferimento nel capitolo “Materiali e metodi”. 2. I tre nomi ingannevoli con la spiegazione del perché (§6.2) — riusabile come nota metodologica su quanto la leggibilità del codice non garantisca, da sola, la correttezza dell'interpretazione. 3. La mappatura esplicita delle due domande di ricerca alle rispettive righe di codice (§6.3) — riusabile, quasi parola per parola, nella sezione che nella tesi formula le domande di ricerca, con la garanzia che ogni domanda sia ancorata a qualcosa di verificabile nel codice.

Capitolo 7 — Sintassi minima e idiomi core

Obiettivi del capitolo

- Leggere un blocco di codice Python senza cercare parentesi graffe o punti e virgola che non ci sono.
- Capire cosa significa “tipizzazione dinamica” concretamente, con un esempio reale del progetto.
- Sapere cosa fa `self` in Python, anche se — fatto notevole — non lo vedrai mai *definito* in questo progetto.

7.1 Indentazione come sintassi, niente `{}` né `;`

In Java, i blocchi di codice sono delimitati da `{` e `}`; l’indentazione è una convenzione stilistica, non richiesta dal compilatore. In Python, l’indentazione è la sintassi: due righe indentate allo stesso livello, subito dopo i due punti che chiudono un `if`, un `for`, o una definizione di funzione, appartengono allo stesso blocco. Non c’è modo di “sbagliare le graffe” in Python, perché non ce ne sono — ma c’è modo di sbagliare l’indentazione, ed è un errore che Java non ti lascerebbe mai commettere in questa forma, perché per lui l’indentazione è invisibile.

[Fatto] Un esempio qualunque del progetto, `main.py:20-24`:

```
\KeywordTok{def}\NormalTok{ main():}
\NormalTok{     args }\OperatorTok{=}\NormalTok{ parse\_args()}
\NormalTok{     dirs }\OperatorTok{=}\NormalTok{ get\_output\_dirs(args.dataset)}
\NormalTok{     delete\_files\_embeddings(dirs[0]\StringTok{"embeddings"}\NormalTok{[0])}
```

Le tre righe dopo `def main():` appartengono al corpo della funzione perché sono indentate di quattro spazi rispetto a `def`. Non c’è un `}` che segnali dove il corpo finisce: finisce quando l’indentazione torna al livello precedente.

SE VIENI DA JAVA — non esiste il punto e virgola come terminatore di istruzione: la fine della riga è la fine dell’istruzione (salvo pochi casi espliciti, come le espressioni fra parentesi che continuano su più righe, molto comuni in questo progetto — vedi la definizione multi-riga di `parse_args()` in `main.py:12-18`, tenuta insieme dalle parentesi tonde di `add_argument`, non da alcun carattere di continuazione).

7.2 Tipizzazione dinamica e duck typing vs. tipizzazione statica di Java

[Fatto] In tutti e nove i file Python di questo progetto, non compare **una sola** annotazione di tipo — né sui parametri di funzione (`def foo(x: int)`), né sui valori di ritorno (`->`

`bool`), verificato con una ricerca sistematica su tutti i file. Ogni parametro, ogni variabile, ogni valore di ritorno ha un tipo — Python è dinamicamente tipizzato, non “senza tipi” — ma quel tipo si scopre solo eseguendo il codice, non leggendo una firma dichiarata. `def preprocessing_data(dataset="heart_disease")` (`preprocessing.py:19`) ti dice che esiste un parametro `dataset` con un valore di default: non ti dice, e non può dirti staticamente, che deve essere una stringa fra `"heart_disease"` e `"diabetes130"` — lo scopri leggendo `get_output_dirs()` (`function.py:79-81`), che solleva un errore *a runtime* se il valore non è tra quelli attesi.

Questa non è un’assenza casuale: Python offre da tempo una sintassi di annotazioni di tipo opzionali (proprio come quella che ti aspetteresti, `def foo(x: int) -> bool:`), ma restano **decorative** senza uno strumento esterno che le controlli (come `mypy`) — l’interprete le ignora a runtime. Il fatto che questo progetto non le usi affatto è una scelta (o un’assenza di scelta) reale e verificabile, non un limite del linguaggio.

[Fatto] Il *duck typing* — “se si comporta come un’anatra, trattalo come un’anatra, senza chiederti se è davvero della classe `Anatra`” — compare esplicitamente in `preprocessing.py:109`:

```
\ControlFlowTok{if} \BuiltInTok{hasattr}\NormalTok{(X\_train\_emb,
↪ }\StringTok{"toarray"}\NormalTok{):}
\NormalTok{X\_train\_emb }\OperatorTok{=}\NormalTok{X\_train\_emb.toarray()}
```

Questo codice non controlla *il tipo* di `X_train_emb` con un `isinstance()`: controlla se l’oggetto *possiede il metodo* `toarray` — tipico di una matrice sparsa prodotta da `OneHotEncoder` quando ci sono molte categorie. Se l’oggetto ce l’ha, lo chiama; se non ce l’ha (perché `ColumnTransformer` ha già restituito un array denso), salta il passaggio. In Java, un controllo equivalente richiederebbe che `X_train_emb` implementasse un’interfaccia comune con un metodo dichiarato — qui basta che l’oggetto, qualunque cosa sia, risponda a quel nome di metodo.

ATTENZIONE — la stessa lettera, `X`, denota in questo progetto oggetti di tipo concreto diverso a seconda del file: in `preprocessing.py` è quasi sempre un `pandas.DataFrame`; in `classification.py:13`, dopo `np.load(...)`, è un `numpy.ndarray`. Nessuna riga di codice lo dichiara: lo capisci solo seguendo da dove viene il valore. È il prezzo della tipizzazione dinamica — flessibilità immediata, nessuna rete di sicurezza del compilatore.

7.3 `self` esplicito, niente `this` implicito

[Fatto] Nessuno dei nove file del progetto definisce una singola classe propria — zero occorrenze della parola chiave `class`, verificato con una ricerca sistematica. Il progetto è scritto interamente in stile procedurale: funzioni che si chiamano a vicenda, nessun oggetto con stato incapsulato definito dal progetto stesso. Questo significa che non troverai, nel codice di questo progetto, un solo posto dove `self` viene *dichiarato* in una firma di metodo.

Lo vedi però, indirettamente, ogni volta che il codice chiama un metodo su un oggetto di libreria. Quando scrivi `logisticReg.fit(X_train, y_train)` (`classification.py:26`), da qualche parte dentro `scikit-learn` esiste una definizione simile a `def fit(self, X, y):` — `self` è il parametro che riceve, esplicitamente, l’oggetto `logisticReg` su cui il metodo è stato chiamato. In Java, il riferimento all’oggetto corrente (`this`) è sempre disponibile implicitamente dentro un metodo di istanza: non lo dichiari nella firma, il compilatore lo aggiunge per te. In Python, quel primo parametro va scritto a mano in ogni definizione di metodo — non è un’eccezione stilistica, è una regola del linguaggio.

SE VIENI DA JAVA — la differenza pratica più immediata: se un giorno definisci tu una classe in Python e ti scordi `self` come primo parametro di un metodo, il

codice fallisce a runtime con un errore sul numero di argomenti — non a compile time, perché niente in Python controlla staticamente le firme dei metodi. Non è un problema che affronti leggendo questo progetto (non definisce classi), ma è tra i primi errori che chiunque incontra scrivendo Python per la prima volta.

[Interpretazione] Il fatto che l'intero progetto non usi mai una classe propria non è casuale rispetto al suo dominio: la pipeline è una sequenza di trasformazioni di dati (tabella → testo → embedding → previsione → grafico), non un sistema con entità che mantengono stato nel tempo. Uno stile procedurale, con funzioni pure per quanto possibile, è una scelta comune e ragionevole per questo tipo di codice — lo stesso lavoro, in un contesto enterprise Java, verrebbe probabilmente organizzato in classi di servizio con metodi statici, non troppo diverse concettualmente da queste funzioni a livello di modulo.

Riepilogo

L'indentazione in Python sostituisce le parentesi graffe come delimitatore di blocco, e il punto e virgola non serve. Il progetto non usa mai annotazioni di tipo: ogni tipo si scopre a runtime, e il duck typing (`hasattr` invece di `isinstance`, capitolo 7.2) è la norma, non l'eccezione. Nessuna classe è definita nel codice del progetto: `self` lo incontri solo quando chiami metodi di libreria, mai in una definizione propria.

Domande di autoverifica

- 1. Perché l'assenza di annotazioni di tipo in questo progetto non significa che le variabili non abbiano un tipo?** Perché Python è dinamicamente tipizzato, non privo di tipi: ogni valore ha un tipo concreto in ogni momento, ma quel tipo si stabilisce solo eseguendo il codice, non dichiarandolo in anticipo in una firma che il compilatore controlla.
- 2. Cosa controlla `hasattr(X_train_emb, "toarray")` in `preprocessing.py:109`, e cosa NON controlla?** Controlla se l'oggetto possiede un metodo chiamato `toarray`, indipendentemente dalla sua classe esatta. Non controlla se l'oggetto è un'istanza di un tipo specifico (non usa `isinstance`) — è duck typing puro.
- 3. Perché non troverai mai, in questo progetto, una definizione di metodo con `self` come primo parametro?** Perché il progetto non definisce alcuna classe propria — è scritto interamente in stile procedurale, con funzioni a livello di modulo che si richiamano a vicenda.

MATERIALE PER LA TESI 1. Il dato verificato “zero annotazioni di tipo in nove file, zero classi definite” — riusabile come caratterizzazione oggettiva dello stile di programmazione del progetto in “Materiali e metodi”. 2. L'esempio di duck typing con `hasattr` (`preprocessing.py:109`), confrontato esplicitamente con l'equivalente concettuale in un linguaggio a tipizzazione statica — riusabile come illustrazione tecnica in un capitolo che discuta le scelte implementative. 3. L'osservazione che lo stile puramente procedurale è coerente con la natura del progetto (una pipeline di trasformazioni, non un sistema stateful) — riusabile come argomento a favore di una scelta progettuale, utile in una sezione di giustificazione metodologica.

Capitolo 8 — Strutture dati e comprehension

Obiettivi del capitolo

- Sapere quali strutture dati Python il progetto usa davvero, e a cosa corrispondono nel mondo Java.
- Leggere una list comprehension o una dict comprehension come faresti con uno stream Java, riconoscendone la forma.
- Capire perché il dizionario globale `results` di `function.py` è un punto di attenzione, in termini di stato condiviso.

8.1 Liste, dizionari, tuple vs. List/Map/nessun equivalente diretto per le tuple

Python ha quattro strutture dati integrate nel linguaggio stesso — non in una libreria a parte — che ricorrono in ogni file di questo progetto: la **lista** (`[1, 2, 3]`, mutabile, ordinata, equivalente più vicino a `ArrayList<T>`), il **dizionario** (`{"a": 1}`, mutabile, chiavi-valori, equivalente più vicino a `HashMap<K,V>`), la **tupla** (`(1, 2)`, ordinata come una lista ma immutabile), e l'**insieme** (`{1, 2, 3}`, non ordinato, senza duplicati, equivalente più vicino a `HashSet<T>` — usato una sola volta nel progetto, implicitamente, da `.unique()` di pandas, non come `set` letterale).

La tupla merita attenzione perché non ha un equivalente diretto e altrettanto naturale in Java. **[Fatto]** `preprocessing.py:43`:

```
\NormalTok{X\_train, X\_test, y\_train, y\_test } \OperatorTok{=} \NormalTok{\n
```

```
↪ train\_test\_split(X, y, test\_size) \OperatorTok{=} \FloatTok{0.2} \NormalTok{\n
```

```
↪ random\_state} \OperatorTok{=} \DecValTok{42} \NormalTok{\n
```

```
↪ stratify} \OperatorTok{=} \NormalTok{y}} \NormalTok{\n
```

`train_test_split` restituisce **quattro valori in una volta sola**, impacchettati in una tupla, e la riga li assegna a quattro variabili in un solo passaggio — una forma di destrutturazione posizionale. In Java, un metodo può restituire un solo valore: per ottenerne quattro dovresti restituire un array, un oggetto dedicato con quattro campi, oppure quattro chiamate separate. Qui non serve dichiarare nulla in anticipo: la tupla di ritorno esiste solo per la durata di quella riga, e le quattro variabili a sinistra la “spacchettano” in base alla posizione, non al nome.

SE VIENI DA JAVA — questa destrutturazione posizionale ricorre in tutto il progetto ogni volta che una funzione ha più di un risultato logico da restituire: `X_train_bal, y_train_bal = balance_classes(...)` (`preprocessing.py:53`), `acc_mean, acc_ci = ci(bootstrap_metrics_dict['acc'])` (`evaluation.py:29`). Non esiste, in nessuno di questi casi, una classe che rappresenti “il risultato di questa funzione”: è sempre e solo una tupla anonima, letta per posizione.

8.2 List comprehension, dict comprehension ed espressioni generatore

Una **list comprehension** costruisce una nuova lista applicando un'espressione a ogni elemento di una sequenza esistente, in un'unica riga: `[espressione for elemento in sequenza]`, opzionalmente con un filtro (`if condizione`) alla fine. È l'equivalente concettuale di uno stream Java con `.map()` (ed eventualmente `.filter()`) seguito da `.collect(Collectors.toList())` — ma senza bisogno di alcuna chiamata esplicita a `collect`: la lista è il risultato diretto dell'espressione fra parentesi quadre.

[Fatto] Il progetto ne fa un uso pervasivo. Alcuni esempi reali, in ordine di complessità crescente:

```
\NormalTok{model\_order }\OperatorTok{=}\NormalTok{ [name
↪ }\ControlFlowTok{for}\NormalTok{ name, \_, \_ }\KeywordTok{in}\NormalTok{
↪ roc\_data] }\CommentTok{\# function.py:258}
\NormalTok{dfs }\OperatorTok{=}\NormalTok{ [pd.read\_csv(f,
↪ header)\OperatorTok{=}\VariableTok{None}\NormalTok{,
↪ na\_values}\OperatorTok{=}\StringTok{"?"}\NormalTok{
↪ }\ControlFlowTok{for}\NormalTok{ f }\KeywordTok{in}\NormalTok{ files]
↪ }\CommentTok{\# function.py:103}
\NormalTok{colors }\OperatorTok{=}\NormalTok{ [\StringTok{"\#e34948"}
↪ \ControlFlowTok{if}\NormalTok{ v }\OperatorTok{\textgreater}\DecValTok{0}
↪ \ControlFlowTok{else} \StringTok{"\#2a78d6"} \ControlFlowTok{for}\NormalTok{ v
↪ }\KeywordTok{in}\NormalTok{ df\_sorted}\StringTok{"deviation"}\NormalTok{
↪ }\CommentTok{\# function.py:378}
\NormalTok{texts }\OperatorTok{=}\NormalTok{ [record\_to\_text(r)
↪ }\ControlFlowTok{for}\NormalTok{ \_, r }\KeywordTok{in}\NormalTok{ X.iterrows()}
↪ }\CommentTok{\# embedding.py:17}
```

L'ultimo esempio merita un momento: `X.iterrows()` restituisce, per ogni riga di un DataFrame pandas, una coppia (indice, riga) — la comprehension usa `_` per l'indice, segnalando con la convenzione del trattino basso “questo valore esiste ma non mi interessa”, e applica `record_to_text` (che sia la versione per Heart Disease o per Diabetes130, scelta alla riga precedente) a ogni riga, producendo una lista di frasi.

[Fatto] Esiste anche l'equivalente per i dizionari, usato una sola volta nel progetto: `MODEL_FAMILY = {m["model_name"]: m["family"] for m in models_all} (function.py:54)` costruisce un dizionario che associa a ogni nome breve di modello la sua famiglia, iterando sulla lista di configurazione e producendo, per ogni elemento, una coppia chiave-valore.

APPROFONDIMENTO FACOLTATIVO — cambiando le parentesi quadre [...] in parentesi tonde (...), la stessa sintassi produce non una lista ma un **generatore**: un oggetto che produce i valori uno alla volta, su richiesta, invece di costruire subito l'intera lista in memoria. Questo progetto non usa mai questa forma (né la forma più esplicita con la parola chiave `yield` in una funzione, anch'essa assente — verificato con una ricerca sistematica su tutti i file). Vale comunque la pena saperlo riconoscere, perché la differenza sintattica è minima — una parentesi al posto di un'altra — mentre la differenza di comportamento (tutto subito in memoria contro un valore alla volta) può essere rilevante su dataset molto più grandi di quelli usati qui.

8.3 Il dizionario globale mutabile di `function.py`

[Fatto] `function.py:67-72` definisce, a livello di modulo — non dentro nessuna funzione — questo dizionario:

```

\NormalTok{results }\OperatorTok{=}\NormalTok{\{\}
  \StringTok{"e5{-}base"}\NormalTok{: \{\}\StringTok{"acc"}\NormalTok{: [],
    ↪ }\StringTok{"f1"}\NormalTok{: [], }\StringTok{"auc"}\NormalTok{: [],
    ↪ }\StringTok{"tau"}\NormalTok{: []\},}
  \StringTok{"e5{-}large"}\NormalTok{: \{\}\StringTok{"acc"}\NormalTok{: [],
    ↪ }\StringTok{"f1"}\NormalTok{: [], }\StringTok{"auc"}\NormalTok{: [],
    ↪ }\StringTok{"tau"}\NormalTok{: []\},}
  \StringTok{"gte{-}large"}\NormalTok{: \{\}\StringTok{"acc"}\NormalTok{: [],
    ↪ }\StringTok{"f1"}\NormalTok{: [], }\StringTok{"auc"}\NormalTok{: [],
    ↪ }\StringTok{"tau"}\NormalTok{: []\},}
  \StringTok{"gte{-}base"}\NormalTok{: \{\}\StringTok{"acc"}\NormalTok{: [],
    ↪ }\StringTok{"f1"}\NormalTok{: [], }\StringTok{"auc"}\NormalTok{: [],
    ↪ }\StringTok{"tau"}\NormalTok{: []\}}
\NormalTok{\}}

```

Nota subito una cosa: contiene solo 4 delle 7 chiavi che il progetto userà davvero (manca `bioclinicalbert`, `pubmedbert`, `sentence-biobert`). [Fatto] `classification.py:51` scrive `results[model['model_name']] = {...}` per ognuno dei 7 modelli in `models_all`: per le prime 4 chiavi questo *sovrascrive* il valore iniziale, per le altre 3 lo *crea da zero*. In Python un dizionario non ha uno schema fisso: assegnare a una chiave che non esiste ancora la crea silenziosamente, senza errori, che tu l'avessi prevista o no in fase di inizializzazione. In Java, l'equivalente più vicino — una `Map<String, Metriche>` — si comporterebbe allo stesso modo con `.put()`, quindi qui la differenza non è nel comportamento della mappa in sé, ma nel fatto che l'inizializzazione di `results` sembra dichiarare uno schema (4 chiavi specifiche) che poi il codice ignora completamente.

Il punto più delicato non è questo, però: `results` è un dizionario **a livello di modulo**, importato con `from function import ... results` in `classification.py:7` e mutato lì con `results[...] = ....` Nessuna riga usa la parola chiave `global` — verificato, non ce n'è traccia in tutto il progetto — e non ne serve una: `global` servirebbe solo per *riassegnare* interamente il nome `results` a un nuovo oggetto dentro una funzione (`results = {}`), non per modificare il contenuto dell'oggetto a cui `results` punta già. Scrivere `results[chiave] = valore` è una mutazione dell'oggetto esistente, permessa da qualunque funzione che abbia un riferimento a quell'oggetto, senza bisogno di dichiarazioni particolari.

ATTENZIONE — è precisamente questo comportamento — mutare senza riassegnare — a rendere `results` uno stato globale mutabile silenzioso: qualunque funzione che importi `results` da `function` può modificarlo, e ogni altra parte del programma che lo importi vede quella modifica, perché tutte condividono lo stesso oggetto dizionario in memoria, non una copia. Nell'uso attuale del progetto (un processo per dataset, `classification.py` chiamato una sola volta da `main.py`) questo non causa un bug osservabile. Se qualcuno riusasse queste funzioni in un contesto diverso — un notebook, un test che chiama `training_classifier()` due volte nello stesso processo — troverebbe `results` già popolato dalla chiamata precedente, un comportamento sorprendente per chi si aspetta che ogni chiamata di funzione parta “pulita”. Ne riparlamo, con l'inquadramento architetturale completo, al capitolo 19.2.

Riepilogo

Liste, dizionari e insiemi in Python corrispondono abbastanza direttamente a `List`, `Map` e `Set` in Java; la tupla, usata ovunque nel progetto per restituire più valori da una funzione in un colpo solo, non ha un equivalente altrettanto naturale. Le comprehension — usate in modo pervasivo nel progetto per liste, una sola volta per un dizionario — costruiscono una nuova collezione in un'unica espressione, senza bisogno di un ciclo esplicito o di `.collect()`. Il dizionario `results`

di `function.py` è uno stato globale mutabile: funziona nell'uso attuale del progetto, ma è un punto di attenzione se il codice venisse riusato in un contesto diverso da un singolo processo per dataset.

Domande di autoverifica

1. Perché `train_test_split(...)` può restituire quattro valori con un'unica istruzione, mentre un metodo Java tipicamente ne restituisce uno solo? Perché in Python i quattro valori vengono impacchettati in una tupla anonima, e l'istruzione di assegnazione la spaccetta per posizione in quattro variabili nello stesso momento. Java richiederebbe un array, un oggetto con quattro campi dichiarato appositamente, o quattro chiamate separate.

2. Cosa succede, in Python, se assegni a una chiave di dizionario che non esiste ancora? Il dizionario la crea silenziosamente, senza sollevare un errore — è esattamente ciò che fa `classification.py:51` per le tre chiavi di `results` non presenti nell'inizializzazione di `function.py:67-72`.

3. Perché mutare `results` da `classification.py` non richiede la parola chiave `global`? Perché `global` serve solo quando una funzione deve *riassegnare* interamente un nome di variabile di modulo a un nuovo oggetto. Qui il codice modifica il contenuto dell'oggetto dizionario esistente (`results[chiave] = valore`), un'operazione permessa a chiunque abbia un riferimento a quell'oggetto, senza bisogno di dichiarazioni aggiuntive.

MATERIALE PER LA TESI 1. L'esempio della destrutturazione di tupla in `preprocessing.py:43`, con il confronto esplicito rispetto alla necessità di un oggetto multi-campo in Java — riusabile come illustrazione di una differenza idiomatrica concreta in “Materiali e metodi”. 2. La sequenza di comprehension reali con relativa spiegazione (§8.2) — riusabile come esempio di stile idiomatrico Python, utile per motivare la leggibilità (o meno) del codice analizzato. 3. L'analisi del dizionario globale `results` come stato mutabile condiviso, con la precisazione sul perché non serve `global` — riusabile nella discussione critica (capitolo 19.2) sulla robustezza del codice a un riuso diverso da quello attuale.

Capitolo 9 — Funzioni, argomenti, trappole

Obiettivi del capitolo

- Distinguere argomenti posizionali e keyword, e riconoscere i valori di default nel codice del progetto.
- Conoscere la trappola degli argomenti mutabili di default — non presente in questo progetto, ma fra le più citate del linguaggio, e per questo da sapere comunque.
- Capire come Python gestisce funzioni con lo stesso nome ma scopi diversi, in assenza di overloading.

9.1 Argomenti posizionali, keyword, default — e il pericolo dei default mutabili

Una funzione Python può ricevere argomenti in due modi, spesso mescolati nella stessa chiamata: **posizionali** (l'ordine conta, come in Java) e **keyword** (il nome del parametro è scritto esplicitamente nella chiamata, l'ordine non conta più). **[Fatto]** `function.py:103`:

```
\NormalTok{dfs }\OperatorTok{=}\NormalTok{ [pd.read\_csv(f,  
↪ header)\OperatorTok{=}\VariableTok{None}\NormalTok{,  
↪ na\_values}\OperatorTok{=}\StringTok{"?"}\NormalTok{)}  
↪ }\ControlFlowTok{for}\NormalTok{ f }\KeywordTok{in}\NormalTok{ files}]}
```

`f` è posizionale (il primo argomento di `read_csv`); `header=None` e `na_values="?"` sono keyword. In Java, l'equivalente più vicino sarebbe un overload di metodo con quella combinazione specifica di parametri, oppure un builder — qui è semplicemente la stessa funzione, chiamata specificando solo i parametri che vuoi cambiare rispetto al loro default.

[Fatto] I default stessi compaiono ovunque nelle firme del progetto: `def preprocessing_data(dataset="heart", preprocessing.py:19)`, `def bootstrap_metrics(y_true, y_score, y_pred, n_iter=10000, seed=42)` (`evaluation.py:62`), `def load_diabetes130(sample_size=20000, random_state=42)` (`function.py:116`). In tutti questi casi il default è un valore immutabile — una stringa, un intero — e non c'è alcun problema.

ATTENZIONE — se il default fosse invece una lista o un dizionario, ci sarebbe un problema, e non un problema ipotetico: è una delle trappole più conosciute e più citate dell'intero linguaggio, e vale la pena saperla riconoscere anche se — verificato — non compare in nessuna firma di questo progetto. Considera una funzione scritta così, in un progetto diverso da questo:

```
\KeywordTok{def}\NormalTok{  
↪ aggiungi\_errore(lista\_errori)\OperatorTok{=}\NormalTok{ [] }:}  
\NormalTok{ lista\_errori.append()\StringTok{"errore"}\NormalTok{ }  
↪ }\ControlFlowTok{return}\NormalTok{ lista\_errori}
```

Ti aspetteresti che ogni chiamata a `aggiungi_errore()` senza argomenti parta da una lista vuota. Non è così: il valore di default `[]` viene creato **una sola volta**, nel momento in cui Python legge la definizione della funzione — non a ogni chiamata. Tutte le chiamate che non passano `lista_errori` esplicitamente condividono *lo stesso oggetto lista*, che quindi cresce a ogni chiamata: la seconda chiamata restituisce `["errore", "errore"]`, non `["errore"]`. In Java questo problema non si presenta nella stessa forma, perché ogni invocazione di metodo che crea un nuovo `ArrayList<>()` nel corpo del metodo ne crea davvero uno nuovo ogni volta — l’inizializzazione è parte del corpo eseguito a ogni chiamata, non della firma valutata una sola volta.

9.2 Funzioni come cittadini di prima classe

In Python, una funzione è un valore come un altro: puoi assegnarla a una variabile, passarla come argomento, restituirla da un’altra funzione, senza alcuna sintassi speciale o interfaccia da implementare. **[Fatto]** `function.py:246`:

```
\NormalTok{model\_order }\OperatorTok{=}
↪ \BuiltInTok{sorted}\NormalTok{(results\_dict.keys(),
↪ key)\OperatorTok{=}\KeywordTok{lambda}\NormalTok{ m: (MODEL\_FAMILY.get(m,
↪ )\StringTok{""}\NormalTok{), m))}
```

`lambda m: (MODEL_FAMILY.get(m, ""), m)` è una funzione anonima di una riga, passata a `sorted()` tramite il parametro keyword `key`: per ogni elemento `m` da ordinare, restituisce una tupla (famiglia, nome), e `sorted()` ordina usando quella tupla come chiave di confronto — prima per famiglia, poi per nome, entrambi in ordine alfabetico. La stessa identica riga ricorre altre due volte nel progetto (`function.py:292,363`), sempre per ordinare i modelli in modo coerente nei grafici.

[Fatto] Un esempio più esplicito ancora, `embedding.py:206`:

```
\NormalTok{futures.append(executor.submit(process\_model, m, texts, labels,
↪ embeddings\_dir))}
```

`process_model` — il *nome della funzione stessa*, senza parentesi che la invochino — è passato come primo argomento a `executor.submit()`, che la eseguirà più tardi, in un thread del pool, con gli argomenti successivi. Non è invocata qui: è passata come valore, per essere invocata altrove.

SE VIENI DA JAVA — da Java 8 in poi, `lambda` e riferimenti a metodo (`ClassName::methodName`) rendono questo pattern familiare, quindi la differenza è meno estranea di quanto sembri a prima vista. Ciò che resta diverso: in Python qualunque funzione è già un valore di prima classe, sempre, senza bisogno che il tipo del parametro che la riceve dichiari un’interfaccia funzionale compatibile (l’equivalente di `Runnable`, `Function<T,R>`, o un’interfaccia personalizzata con `@FunctionalInterface`). `executor.submit()` accetta letteralmente “una funzione, qualunque essa sia, più i suoi argomenti” — non esiste un tipo dichiarato a cui `process_model` deve conformarsi.

9.3 Assenza di overloading: come Python distingue le firme

In Java puoi definire due metodi con lo stesso nome e parametri di tipo diverso, e il compilatore sceglie quale invocare in base ai tipi degli argomenti nella chiamata. Python non ha questo meccanismo: due funzioni con lo stesso nome nello stesso modulo, la seconda sovrascrive sempli-

cemente la prima — non c'è “risoluzione di overload” perché non ci sono tipi dichiarati su cui basarla.

[Fatto] Il progetto risolve lo stesso tipo di problema — comportamento diverso a seconda dei dati — con funzioni dal **nome esplicitamente diverso**, scelte a runtime con un `if/else` ordinario: `record_to_text_heart_disease()` e `record_to_text_diabetes130()` (`embedding.py:43,64`), selezionate in `embedding.py:16`:

```
\NormalTok{record\_to\_text }\OperatorTok{=}\NormalTok{ record\_to\_text\_diabetes130
↪ }\ControlFlowTok{if}\NormalTok{ dataset }\OperatorTok{==}
↪ \StringTok{"diabetes130"} }\ControlFlowTok{else}\NormalTok{
↪ record\_to\_text\_heart\_disease}
```

Nota la seconda cosa interessante di questa riga, oltre alla scelta fra le due funzioni: `record_to_text` diventa qui una variabile che *contiene una funzione* (di nuovo, funzioni come valori — capitolo 9.2), poi chiamata più sotto come `record_to_text(r)` senza sapere, in quel punto del codice, quale delle due implementazioni sia stata effettivamente scelta.

ATTENZIONE — questo pattern — un `if/else` che sceglie fra due funzioni con nomi diversi — è l'equivalente funzionale di un overload, ma è responsabilità di chi scrive il codice mantenerlo coerente: non c'è alcun controllo che garantisca che `record_to_text_heart_disease` e `record_to_text_diabetes130` abbiano una firma compatibile (nel senso di “prendono entrambe una riga e restituiscono entrambe una stringa”). Lo sono, verificato leggendo entrambe (`embedding.py:43-59` e `embedding.py:64-85`), ma nessuna riga di codice lo impone: è una convenzione, rispettata da chi ha scritto le due funzioni, non verificata da nessuno strumento in questo progetto.

Riepilogo

Il progetto usa ampiamente argomenti keyword e default immutabili, ma mai un default mutabile — una trappola reale del linguaggio che vale la pena conoscere anche senza un esempio diretto nel codice. Le funzioni sono valori di prima classe: vengono passate come dati a `sorted()` e a `ThreadPoolExecutor.submit()`, senza bisogno di implementare un'interfaccia. In assenza di overloading, la stessa esigenza — comportamento diverso per dataset diverso — è risolta con funzioni dal nome esplicitamente diverso, scelte a runtime con un `if/else` ordinario, senza alcuna garanzia automatica di compatibilità fra le firme.

Domande di autoverifica

1. Perché una funzione con un parametro di default `lista=[]` è pericolosa, mentre `n_iter=10000` non lo è? Perché il valore di default viene creato una sola volta, quando Python legge la definizione della funzione. Un intero è immutabile, quindi non importa se è “condiviso” fra le chiamate. Una lista è mutabile: se una chiamata la modifica, la modifica persiste per tutte le chiamate successive che non forniscono l'argomento esplicitamente.
2. In che senso `process_model`, passato a `executor.submit()` in `embedding.py:206`, non viene “chiamato” in quel punto del codice? Perché compare senza parentesi: è il valore-funzione stesso, non il risultato della sua esecuzione. Verrà effettivamente invocato più tardi, dentro un thread del pool, con gli argomenti passati subito dopo.
3. Come risolve il progetto l'esigenza di un comportamento diverso per Heart Disease e per Diabetes130, in assenza di overloading di funzione? Definendo due funzioni

con nomi diversi (`record_to_text_heart_disease`, `record_to_text_diabetes130`) e scegliendo quale delle due usare con un'espressione condizionale a runtime (`embedding.py:16`), non con una risoluzione di tipo a compile time.

MATERIALE PER LA TESI 1. L'esempio codificato della trappola degli argomenti mutabili di default, chiaramente distinto dal codice reale del progetto (che non la presenta) — riusabile come nota metodologica su un rischio evitato, utile in una sezione che discuta la qualità del codice. 2. L'esempio di `executor.submit(process_model, ...)` con la spiegazione di “funzione come valore” — riusabile per illustrare, in “Materiali e metodi”, il meccanismo di concorrenza della generazione degli embedding. 3. Il pattern “funzioni con nome diverso invece di overload”, con l'osservazione sulla mancanza di garanzie automatiche di compatibilità di firma — riusabile come punto di discussione sulla manutenibilità del codice, in una sezione critica.

Capitolo 10 — Moduli, package e l'assenza di incapsulamento reale

Obiettivi del capitolo

- Capire perché un file `.py` da solo è già un modulo importabile, senza nulla di equivalente a una dichiarazione di package.
- Riconoscere come funziona `import` in questo progetto, e da cosa dipende perché funzioni.
- Sapere cosa significa davvero, in Python, un nome di funzione o variabile che comincia con `-`.

10.1 Un file `.py` è un modulo: cosa importa rispetto a un file `.java`

In Java, un file sorgente dichiara esplicitamente il proprio package nella prima riga, e quella dichiarazione deve corrispondere alla posizione del file nell'albero delle cartelle. In Python, un file `.py` è già, per il solo fatto di esistere, un **modulo** importabile con il proprio nome di file (senza estensione) — nessuna dichiarazione richiesta al suo interno.

[Fatto] Questo progetto non ha nessuna struttura a package: i nove file (`main.py`, `function.py`, `preprocessing.py`, `embedding.py`, `classification.py`, `evaluation.py`, `error_analysis.py`, `statisticaltest.py`, `generatereport.py`) vivono tutti nella cartella radice del repository, senza sottocartelle, e non esiste in nessun punto del progetto un file `__init__.py` (verificato cercandolo in tutto l'albero del repository). Un `__init__.py`, quando presente, è il segnale che una cartella deve essere trattata come un **package** — un contenitore organizzato di moduli, l'equivalente più vicino a un package Java con più classi. Qui non ce n'è motivo: un solo livello, nove file, tutti allo stesso livello.

10.2 `import` vs. `import` Java: risoluzione, side-effect al caricamento

[Fatto] `main.py:1-10` importa da sei moduli diversi del progetto:

```
\ImportTok{from}\NormalTok{ evaluation }\ImportTok{import}\NormalTok{
↪ evaluate\_results}
\ImportTok{from}\NormalTok{ generatereport }\ImportTok{import}\NormalTok{
↪ generate\_report}
\ImportTok{from}\NormalTok{ preprocessing }\ImportTok{import}\NormalTok{
↪ preprocessing\_data}
\ImportTok{from}\NormalTok{ embedding }\ImportTok{import}\NormalTok{ embeddings}
\ImportTok{from}\NormalTok{ classification }\ImportTok{import}\NormalTok{
↪ training\_classifier}
\ImportTok{from}\NormalTok{ function }\ImportTok{import}\NormalTok{
↪ delete\_files\_embeddings, delete\_files\_graphics, delete\_files\_preprocessing,
↪ delete\_files\_results, get\_output\_dirs}
```

Ogni `from <modulo> import <nome>` fa due cose insieme: prima trova ed esegue *l'intero file* `<modulo>.py` (una sola volta, anche se importato da più punti — Python tiene una cache dei moduli già caricati), poi rende disponibili nello spazio dei nomi corrente solo i nomi elencati dopo `import`. In Java, `import` è una pura dichiarazione: dice al compilatore dove trovare una classe usata più sotto, e non esegue nulla. In Python, importare un modulo *esegue* quel file da cima a fondo — comprese eventuali righe scritte a livello di modulo, non dentro nessuna funzione.

[Fatto] Questo ha una conseguenza concreta e verificabile in questo progetto: `function.py:154` contiene, a livello di modulo — non dentro una funzione — la chiamata `_configure_plot_style()`, che imposta lo stile globale di tutti i grafici di matplotlib/seaborn. Questa chiamata viene eseguita **nel momento in cui `function.py` viene importato per la prima volta** da qualunque altro file — `main.py`, `preprocessing.py`, o qualunque altro modulo del progetto — non quando qualcuno decide esplicitamente di “configurare lo stile”. È un side-effect al caricamento, un concetto che in Java non ha un equivalente diretto: la cosa più vicina sarebbe un blocco di inizializzazione statico (`static { ... }`) in una classe, eseguito al primo caricamento della classe da parte della JVM — meno comune e più esplicito di quanto non sia, in pratica, una riga scritta a livello di modulo in Python.

[Fatto] Perché questi `import` funzionino, un'altra condizione deve valere, implicita e mai dichiarata da nessuna parte nel codice: `python main.py` deve essere eseguito **dalla cartella radice del repository**. Python cerca i moduli importati in un elenco di percorsi (`sys.path`), che include automaticamente la cartella in cui si trova lo script avviato per primo. Se eseguiessi `python main.py` da una cartella diversa, o provassi a importare `preprocessing` da un progetto esterno, l'importazione fallirebbe con un `ModuleNotFoundError` — non perché il codice sia sbagliato, ma perché non esiste alcuna installazione del progetto come package (nessun `pyproject.toml`, nessun `setup.py`, verificato nella scheda tecnica del capitolo 14) che renda questi moduli risolvibili da un punto qualunque del filesystem.

SE VIENI DA JAVA — il “classpath” qui è, in pratica, una singola cartella — quella corrente al momento dell'esecuzione — non un elenco esplicito di `.jar` o cartelle compilate assemblato da un build tool. Il capitolo 14 mostra la differenza in modo più sistematico, confrontando `pip/requirements.txt` con Maven/Gradle.

10.3 Niente *private* reale: la convenzione dell'underscore

Java ha modificatori di accesso applicati e verificati dal compilatore: `private`, `protected`, `public`. Python non ha nulla di equivalente applicato dal linguaggio — ogni nome, in un modulo Python, è raggiungibile dall'esterno se lo importi esplicitamente, indipendentemente da come è scritto. Esiste però una convenzione, rispettata (non imposta) da chi scrive codice Python: un nome che comincia con un underscore (`_nome`) segnala “pensato per uso interno a questo modulo, non parte dell'interfaccia pubblica” — un'indicazione per chi legge, non una barriera per l'interprete.

[Fatto] Il progetto la usa con coerenza. In `embedding.py`, le funzioni di formattazione usate internamente da `record_to_text_*`() sono tutte prefissate: `_fmt_num`, `_fmt_cat`, `_fmt_bool`, `_fmt_raw` (`embedding.py:28,33,38,61`) — nessuna di queste è pensata per essere chiamata da un altro modulo, e nessun altro file del progetto le importa infatti. Lo stesso vale per `_ollama_semaphore` (`embedding.py:101`, il semaforo che serializza le chiamate a Ollama, capitolo 12.2) e per `_configure_plot_style` (`function.py:134`, appena vista al paragrafo precedente). Nessuna di queste è protetta da un compilatore: se scrivessi, in un altro file, `from embedding import _fmt_num`, funzionerebbe — l'underscore non blocca nulla, avverte soltanto.

ATTENZIONE — l'unico effetto realmente “applicato” dal linguaggio, non solo convenzionale, riguarda un'importazione con `from modulo import *` (mai usata in

questo progetto, che importa sempre nomi espliciti): in quel caso specifico, i nomi con underscore iniziale vengono esclusi automaticamente. Ma un'importazione esplicita per nome, come fa sempre questo progetto, ignora completamente la convenzione: l'underscore resta un messaggio a chi legge, non un cancello.

Riepilogo

Un file `.py` è già un modulo importabile, senza dichiarazioni di package; questo progetto non ha alcuna struttura a package (nessun `__init__.py`), solo nove file allo stesso livello. Importare un modulo in Python esegue l'intero file, incluse le righe a livello di modulo — un side-effect reale che questo progetto sfrutta per configurare lo stile dei grafici al primo import. L'assenza di `private` reale è colmata da una convenzione di nomenclatura (underscore iniziale), rispettata ovunque nel progetto ma non imposta dal linguaggio.

Domande di autoverifica

- 1. Perché l'assenza di `__init__.py` in questo progetto è coerente con la sua struttura, e non un'omissione?** Perché tutti i moduli vivono a un solo livello, nella cartella radice: `__init__.py` serve a segnalare che una cartella è un package con struttura interna da organizzare, cosa che qui non serve, dato che non ci sono sottocartelle di codice.
- 2. Cosa succede, esattamente, quando `main.py` importa `function`, e quando succede rispetto all'esecuzione di `main()`?** L'intero file `function.py` viene eseguito da cima a fondo, incluse le righe a livello di modulo come `_configure_plot_style()` (`function.py:154`) — e questo avviene nel momento dell'importazione, prima ancora che `main()` venga chiamata.
- 3. Se scrivessi `from embedding import _fmt_num` in un altro file, cosa succederebbe?** Funzionerebbe: l'underscore iniziale è una convenzione per chi legge il codice, non una restrizione imposta dall'interprete su un'importazione esplicita per nome. Solo `from embedding import *` escluderebbe automaticamente i nomi con underscore iniziale — una forma di importazione che questo progetto non usa mai.

MATERIALE PER LA TESI 1. L'osservazione verificata sull'assenza di `__init__.py` e di qualunque file di packaging (`pyproject.toml`, `setup.py`) — riusabile in “Materiali e metodi” per descrivere oggettivamente la struttura del progetto. 2. L'esempio del side-effect al caricamento (`_configure_plot_style()` eseguito all'import di `function.py`) — riusabile come illustrazione di una differenza semantica concreta fra `import` Python e Java. 3. La convenzione dell'underscore, con l'elenco reale dei nomi che la rispettano nel progetto — riusabile come nota di stile nella sezione che descrive la qualità e leggibilità del codice.

Capitolo 11 — Eccezioni, context manager, decoratori

Obiettivi del capitolo

- Leggere `try/except/raise ... from` riconoscendo le differenze rispetto a `try/catch/throws`.
- Riconoscere un context manager (`with`) come l'equivalente concettuale del `try-with-resources` di Java, con tre esempi reali diversi.
- Sapere cosa sono i decoratori, anche se — verificato — non ne troverai uno in questo progetto.

11.1 `try/except/raise ... from`: differenze da `throws/catch`

Python non distingue eccezioni “controllate” da eccezioni “non controllate”: ogni funzione può sollevare qualunque eccezione in qualunque momento, e non esiste una clausola equivalente a `throws` nella firma che dichiari quali. Chi chiama una funzione non ha modo di sapere, dalla sola firma, cosa può sollevare — solo leggendo il corpo, o la documentazione, se esiste.

[Fatto] `embedding.py:115-131` mostra un ciclo di `retry` con gestione dell'eccezione:

```
\ControlFlowTok{for}\NormalTok{ attempt }\KeywordTok{in}
↪ \BuiltInTok{range}\NormalTok{({}\DecValTok{1}\NormalTok{, max\_retries}
↪ )}\OperatorTok{+} \DecValTok{1}\NormalTok{)}:}
  \ControlFlowTok{try}\NormalTok{:}
    \ControlFlowTok{with}\NormalTok{ \_ollama\_semaphore:}
\NormalTok{
    result }\OperatorTok{=}\NormalTok{
↪ client.embed(model)\OperatorTok{=}\NormalTok{model\_name,}
↪ }\BuiltInTok{input}\OperatorTok{=}\NormalTok{batch)}
\NormalTok{
    all\_embeddings.extend(result.embeddings)}
  \ControlFlowTok{break}
\ControlFlowTok{except} \PreprocessorTok{Exception} \ImportTok{as}\NormalTok{ e:}
  \ControlFlowTok{if}\NormalTok{ attempt }\OperatorTok{==}\NormalTok{
↪ max\_retries:}
    \ControlFlowTok{raise} \PreprocessorTok{RuntimeError}\NormalTok{({}
      \SpecialStringTok{f"[Batch] }\SpecialCharTok{\}\NormalTok{model\_nam}
↪ e}\SpecialCharTok{\}\SpecialStringTok{: batch}
↪ }\SpecialCharTok{\}\NormalTok{batch\_idx }\OperatorTok{+} \DecVa}
↪ lTok{1}\SpecialCharTok{\}\SpecialStringTok{/}\SpecialCharTok{\}\}
↪ \NormalTok{num\_batches}\SpecialCharTok{\}\SpecialStringTok{
↪ "}
      \SpecialStringTok{f"failed after }\SpecialCharTok{\}\NormalTok{max\_}
↪ retries}\SpecialCharTok{\}\SpecialStringTok{ attempts:}
↪ }\SpecialCharTok{\e}\SpecialStringTok{"}
    ) }\ImportTok{from}\NormalTok{ e}
\NormalTok{
    wait }\OperatorTok{=}\NormalTok{ retry\_delay}
↪ }\OperatorTok{*}\NormalTok{ attempt}
```

```
\NormalTok{      ...}
\NormalTok{      time.sleep(wait))}
```

`except Exception as e` cattura qualunque eccezione (una scelta volutamente ampia: qualunque problema di rete o del server Ollama viene trattato allo stesso modo), la nomina `e`, e la usa per decidere cosa fare: se è l'ultimo tentativo, rilancia un errore più specifico; altrimenti, aspetta e riprova. `break` esce dal ciclo `for` solo se il blocco `try` è andato a buon fine senza sollevare eccezioni.

[Fatto] La clausola `from e` alla fine del `raise` (`embedding.py:199` in un punto analogo) è la forma esplicita di **concatenamento di eccezioni**: dichiara che questo nuovo errore (`RuntimeError`) è stato causato da quello originale (`e`), e Python preserva entrambi nel traceback finale — vedrai sia il messaggio del `RuntimeError` sia, più sotto, il messaggio dell'eccezione originale che lo ha scatenato. In Java, l'equivalente è passare la causa al costruttore di una nuova eccezione (`throw new RuntimeException("...", e)`); la differenza sintattica è che in Python questo si esprime con una parola chiave dedicata (`from`), non passando un parametro extra al costruttore.

ATTENZIONE — `except Exception as e` cattura *quasi* tutto — non le eccezioni che derivano da `BaseException` ma non da `Exception` (come l'interruzione da tastiera), una distinzione che in questo progetto non ha conseguenze pratiche, ma che vale la pena sapere che esiste: `Exception` non è, in Python, la radice assoluta della gerarchia di errori, a differenza di come spesso si pensa arrivando da Java dove `Throwable` gioca quel ruolo e `Exception` ne è già una sottoclasse specifica.

11.2 Context manager (with): il “try-with-resources” di Python

Un **context manager** è un oggetto che sa cosa fare all'inizio e alla fine di un blocco `with`, garantendo che la parte di “fine” (rilascio di una risorsa, ad esempio) avvenga sempre — anche se dentro il blocco viene sollevata un'eccezione. È l'equivalente concettuale del `try-with-resources` introdotto in Java 7, con una differenza pratica: in Python qualunque oggetto può diventare un context manager implementando due metodi (`__enter__` e `__exit__`), non serve un'interfaccia dichiarata come `AutoCloseable`.

[Fatto] Il progetto usa `with` in tre punti, con tre scopi diversi:

```
\ControlFlowTok{with}\NormalTok{ \_ollama\_semaphore:
↪     }\CommentTok{\# embedding.py:117 - un lock}
\NormalTok{     result }\OperatorTok{=}\NormalTok{
↪     client.embed(model)\OperatorTok{=}\NormalTok{model\_name,
↪     }\BuiltInTok{input}\OperatorTok{=}\NormalTok{batch}}
```

```
\ControlFlowTok{with}\NormalTok{
↪     ThreadPoolExecutor(max\_workers)\OperatorTok{=}\NormalTok{max\_workers)
↪     }\ImportTok{as}\NormalTok{ executor:     }\CommentTok{\# embedding.py:203 - un pool}
↪     di thread}
\NormalTok{     futures }\OperatorTok{=}\NormalTok{ [executor.submit(process\_model,
↪     m, texts, labels, embeddings\_dir) }\ControlFlowTok{for}\NormalTok{ m
↪     }\KeywordTok{in}\NormalTok{ models\_all}}
```

```
\ControlFlowTok{with} \BuiltInTok{open}\NormalTok{(md\_path,
↪     }\StringTok{"w"}\NormalTok{) }\ImportTok{as}\NormalTok{ f:
↪     }\CommentTok{\# generatereport.py:245 - un file}
\NormalTok{     f.write(md\_content)}
```

Nel primo caso, `_ollama_semaphore` (un `threading.Semaphore(1)`, capitolo 12.2) usato come context manager acquisisce il semaforo all'ingresso del blocco e lo rilascia all'uscita — anche se `client.embed(...)` sollevasse un'eccezione, il semaforo verrebbe comunque rilasciato, evitando un blocco permanente. Nel secondo, `ThreadPoolExecutor` come context manager garantisce che tutti i thread del pool vengano chiusi correttamente all'uscita dal blocco, qualunque cosa succeda dentro. Nel terzo — il più vicino a ciò che probabilmente hai già visto in Java — il file viene chiuso automaticamente all'uscita dal blocco, che il blocco sia andato a buon fine o abbia sollevato un'eccezione.

SE VIENI DA JAVA — la keyword `as` dopo `with` (`as executor`, `as f`) assegna a una variabile il valore restituito da `__enter__()` — non necessariamente l'oggetto originale: `open(...)` come context manager restituisce infatti l'oggetto file stesso, ma non è garantito in generale che sia così per ogni context manager. Non serve dichiarare il tipo di `f` da nessuna parte: lo scopri, di nuovo, solo leggendo cosa fa `__enter__()` di quell'oggetto specifico.

11.3 Decoratori: cosa sono, dove NON compaiono in questo progetto

Un **decoratore** è una sintassi (`@nome_decoratore` scritta subito sopra una definizione di funzione o di classe) che permette di avvolgere quella funzione con un comportamento aggiuntivo, senza modificarne il corpo. È uno degli idiomi più riconoscibili di Python — lo incontrerai quasi certamente in altri progetti, per esempio nelle route di un framework web come Flask (`@app.route("/utenti")`) o nei test parametrizzati di pytest (`@pytest.fixture`) — ma **[Fat-to]** non ne compare uno solo in questo progetto: una ricerca sistematica su tutti e nove i file non trova alcuna riga che cominci con `@` seguito da un nome, al di fuori di eventuali commenti.

[Interpretazione] Non è sorprendente, dato quanto visto al capitolo 7.3: i decoratori sono usati più spesso per aggiungere comportamento a metodi di classi o a funzioni che fanno da punto di ingresso per un framework esterno (una rotta HTTP, un test). Questo progetto non definisce classi proprie e non si appoggia a un framework che si aspetti funzioni decorate: è normale che l'idioma non compaia, non è un'omissione da segnalare come criticità.

APPROFONDIMENTO FACOLTATIVO — se un giorno scrivessi tu un decoratore per, per esempio, aggiungere automaticamente il retry-con-attesa che `embedding.py:103-137` implementa oggi a mano dentro `generate_embeddings_batch`, la forma sarebbe all'incirca `@retry(max_tentativi=5)` scritta sopra la definizione della funzione, con tutta la logica di ciclo, attesa e rilancio spostata una volta sola dentro il decoratore invece che ripetuta in ogni funzione che ne ha bisogno. Il progetto non lo fa: la logica di retry vive solo dentro `generate_embeddings_batch`, e non è riusata altrove — un punto a cui torniamo, in chiave di possibile miglioramento, al capitolo 55.

Riepilogo

Python non distingue eccezioni controllate da non controllate, e `raise ... from` concatena esplicitamente un nuovo errore alla sua causa originale, preservando entrambi nel traceback. Un context manager (`with`) garantisce che un'azione di chiusura avvenga sempre, e questo progetto ne usa tre tipi concettualmente diversi: un lock, un pool di thread, un file. I decoratori esistono come idiom del linguaggio ma non compaiono in questo progetto, verificato sistematicamente — un'assenza coerente con uno stile procedurale senza framework esterni che li richiedano.

Domande di autoverifica

1. Cosa aggiunge `from` e a un `raise`, rispetto a un `raise RuntimeError(...)` senza quella clausola? Concatena esplicitamente la nuova eccezione a quella originale che l’ha causata: il traceback finale mostra entrambe, rendendo visibile non solo cosa è fallito per ultimo ma anche cosa lo ha scatenato.

2. Perché i tre usi di `with` in questo progetto non fanno tutti la stessa cosa, pur usando la stessa sintassi? Perché ciascun oggetto usato come context manager (`_ollama_semaphore`, `ThreadPoolExecutor`, il file aperto con `open`) definisce autonomamente cosa succede all’ingresso e all’uscita del blocco: un semaforo acquisisce/rilascia un lock, un pool di thread garantisce la chiusura dei thread, un file garantisce la propria chiusura. La sintassi `with` è la stessa; il comportamento dipende dall’oggetto.

3. Perché l’assenza di decoratori in questo progetto non è, di per sé, un difetto? Perché i decoratori sono utili soprattutto per aggiungere comportamento a metodi di classi o a funzioni che si integrano con un framework esterno (rotte web, test parametrizzati) — nessuno dei due casi si applica a un progetto scritto in stile procedurale senza framework di questo tipo.

MATERIALE PER LA TESI 1. L’esempio del ciclo di retry con `try/except/raise ... from (embedding.py:115-131,199)` — riusabile come illustrazione della gestione degli errori nella sezione “Materiali e metodi”, in particolare per la fase di generazione degli embedding. 2. I tre usi distinti di `with` nel progetto, con la spiegazione di cosa garantiscono in ciascun caso — riusabile per motivare la robustezza (o i suoi limiti) della gestione delle risorse nella pipeline. 3. L’osservazione sulla logica di retry non incapsulata in un decoratore riusabile — riusabile come spunto concreto per la sezione “Lavori futuri” (capitolo 55), con una proposta di refactoring motivata.

Capitolo 12 — Concorrenza e memoria: GIL, thread, processi

Obiettivi del capitolo

- Sapere cos'è il Global Interpreter Lock e perché non rende inutile il threading in questo progetto.
- Leggere `ThreadPoolExecutor` e `Semaphore` con la stessa disinvoltura con cui leggeresti `ExecutorService` e un lock in Java.
- Capire perché la gestione della memoria non è un punto di attenzione critico in questo codice, a differenza di quasi tutto il resto di questo capitolo.

12.1 Il Global Interpreter Lock: cosa cambia rispetto ai thread Java

[Livello: teoria consolidata del settore] L'implementazione standard di Python (CPython, quella che stai usando se hai seguito l'installazione del capitolo 15) ha un **Global Interpreter Lock** (GIL): un unico lock globale che garantisce che, in ogni istante, un solo thread alla volta stia eseguendo bytecode Python, indipendentemente da quanti thread hai creato e da quanti core ha la CPU. In Java, più thread possono eseguire codice Java realmente in parallelo su core diversi; in Python (CPython), no — a livello di bytecode Python puro, il parallelismo reale fra thread non esiste.

Questo non significa che il threading in Python sia inutile — significa che è utile solo per un tipo specifico di lavoro. Il GIL viene **rilasciato temporaneamente** ogni volta che un thread è in attesa di qualcosa di esterno all'interprete: una risposta di rete, una lettura da disco, o l'esecuzione di codice nativo (scritto in C, come buona parte del motore di calcolo di NumPy o PyTorch) che dichiara esplicitamente di non aver bisogno del GIL per la durata di quell'operazione. Un thread in attesa di una risposta HTTP da Ollama non sta “usando la CPU”: sta aspettando, e mentre aspetta un altro thread può eseguire.

[Interpretazione] È esattamente questo il motivo per cui `ThreadPoolExecutor` ha senso nel prossimo paragrafo, nonostante il GIL: la generazione di embedding è dominata da tempo di attesa di rete (verso Ollama) o da calcolo dentro librerie native che rilasciano il GIL (PyTorch, per i modelli Hugging Face), non da bytecode Python puro che il GIL costringerebbe comunque a un solo thread alla volta.

12.2 `ThreadPoolExecutor` e `Semaphore` nel progetto, confrontati con `ExecutorService`

[Fatto] `embedding.py:201-209` orchestra la generazione di embedding per tutti i 7 modelli con un pool di thread:

```

\KeywordTok{def}\NormalTok{ generate\_all\_embeddings(texts, labels, embeddings\_dir,
↪ max\_workers)\OperatorTok{=}\DecValTok{3}\NormalTok{():}
  \ControlFlowTok{with}\NormalTok{
    ↪ ThreadPoolExecutor(max\_workers)\OperatorTok{=}\NormalTok{ {max\_workers}
    ↪ }\ImportTok{as}\NormalTok{ executor:}
\NormalTok{
    futures }\OperatorTok{=}\NormalTok{ []}
  \ControlFlowTok{for}\NormalTok{ m }\KeywordTok{in}\NormalTok{ models\_all:}
\NormalTok{
    futures.append(executor.submit(process\_model, m, texts,
↪ labels, embeddings\_dir))}
  \ControlFlowTok{for}\NormalTok{ f }\KeywordTok{in}\NormalTok{ futures:}
\NormalTok{
    f.result()}

```

`ThreadPoolExecutor(max_workers=3)` è concettualmente lo stesso oggetto di un `ExecutorService` Java creato con un pool fisso di 3 thread: `executor.submit(...)` accoda un lavoro (qui, la funzione `process_model` con i suoi argomenti) e restituisce immediatamente un oggetto `Future` (qui chiamato `f`), senza aspettare che il lavoro sia completato. Il ciclo finale, `f.result()` per ogni future, blocca finché ciascun lavoro non è terminato, propagando qualunque eccezione sollevata dentro il thread — il modo in cui un errore avvenuto in un thread secondario emerge nel thread principale, esattamente il ruolo di `Future.get()` in Java.

[Fatto] Con `max_workers=3` e 7 modelli in `models_all`, al più tre modelli vengono processati “in parallelo” in un dato momento — ma solo nel senso spiegato al paragrafo precedente: attesa di rete o calcolo nativo, non bytecode Python puro. Il codice restringe ulteriormente questo parallelismo per i modelli serviti da Ollama, con un semaforo dichiarato a livello di modulo:

```

\NormalTok{\_ollama\_semaphore }\OperatorTok{=}\NormalTok{
↪ threading.Semaphore()\DecValTok{1}\NormalTok{ } \CommentTok{\#}
↪ embedding.py:101}

```

usato in `generate_embeddings_batch()` (`embedding.py:117`) per avvolgere ogni singola chiamata `client.embed(...)`. Un `Semaphore(1)` con un solo permesso disponibile equivale, in pratica, a un lock esclusivo: **una sola chiamata a Ollama alla volta**, anche se più thread del pool ci arrivano nello stesso momento — gli altri aspettano in coda finché il permesso non si libera. **[Fatto]** Il motivo è documentato direttamente nel codice, in un commento (`embedding.py:95-100`): il server locale di Ollama può esaurire le porte effimere del sistema operativo, o sovraccaricarsi, se riceve troppe richieste contemporanee da thread diversi — serializzare le chiamate mantiene sempre una sola richiesta HTTP realmente in transito verso quel server specifico.

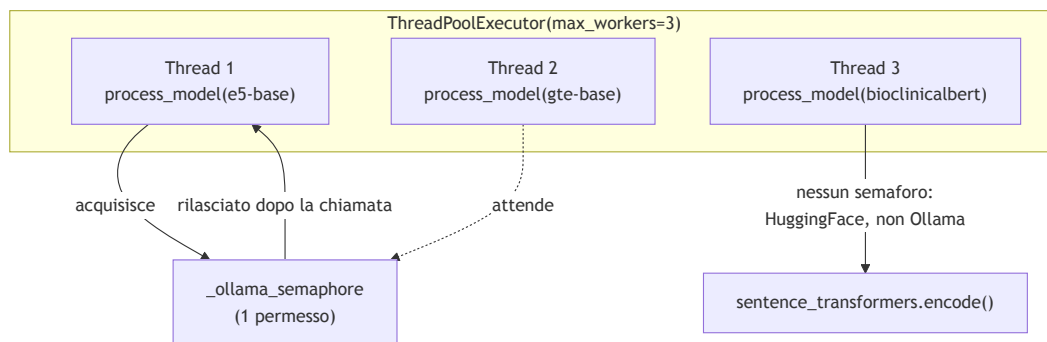


Figura 12.1 — Tre thread attivi, ma una sola chiamata a Ollama alla volta: il parallelismo reale è fra un modello Ollama (serializzato dal semaforo) e i modelli Hugging Face, che non lo condividono.

SE VIENI DA JAVA — `threading.Semaphore` in Python ha la stessa interfaccia concettuale di `java.util.concurrent.Semaphore`: `acquire()/release()` (qui invocati implicitamente da `with`, capitolo 11.2), un numero di permessi fisso, thread

bloccati in coda quando i permessi sono esauriti. Non c'è nulla di specificamente “pythonico” in questa parte: è la stessa primitiva, con la stessa semantica, avvolta in un context manager invece che in un blocco `try/finally` esplicito con `acquire()` e `release()` scritti a mano.

12.3 Gestione della memoria: reference counting + garbage collector, niente `finalize`

[Livello: teoria consolidata del settore] CPython gestisce la memoria con un contatore di riferimenti per ogni oggetto: quando il contatore arriva a zero (nessuna variabile punta più a quell'oggetto), la memoria viene liberata immediatamente, non a un ciclo di garbage collection successivo. Un garbage collector separato interviene solo per i riferimenti ciclici (due oggetti che si riferiscono a vicenda, che il solo conteggio non riuscirebbe mai a portare a zero da soli). È un modello diverso da quello della JVM (che non usa il conteggio di riferimenti come meccanismo primario), ma il risultato pratico per chi scrive codice applicativo è simile: la memoria è gestita automaticamente, non la liberi mai esplicitamente tu.

[Fatto] Non troverai, in nessuno dei nove file del progetto, una singola riga dedicata alla gestione della memoria — nessun equivalente di `close()` chiamato manualmente su una struttura dati, nessun tentativo di “liberare” un `DataFrame` o un array `NumPy` quando non serve più. È l'atteso stato di cose: su questo punto specifico, Python non richiede a chi scrive il codice applicativo un'attenzione diversa da quella che richiederebbe Java. La sola gestione esplicita di risorse nel progetto riguarda risorse *esterne* alla memoria del processo — file, thread, un semaforo — ed è quella vista al capitolo 11.2 con `with`.

Riepilogo

Il Global Interpreter Lock impedisce il parallelismo reale del bytecode Python fra thread, ma non rende inutile il threading per lavoro dominato da attesa di rete o da calcolo in librerie native che rilasciano il GIL — esattamente il caso della generazione di embedding in questo progetto. `ThreadPoolExecutor` gestisce fino a 3 modelli “in parallelo”, ma un `Semaphore(1)` serializza comunque ogni chiamata verso Ollama, per non sovraccaricare il suo server locale. La gestione della memoria non richiede alcuna attenzione esplicita nel codice del progetto: reference counting e garbage collector se ne occupano in modo trasparente, come farebbe la JVM.

Domande di autoverifica

1. Perché usare tre thread per generare embedding ha senso, nonostante il Global Interpreter Lock? Perché il lavoro di ciascun thread è dominato da attesa (chiamate di rete verso Ollama o Hugging Face) o da calcolo in codice nativo che rilascia il GIL (PyTorch), non da bytecode Python puro. Il GIL impedirebbe il parallelismo solo se il lavoro fosse CPU-bound in puro Python.

2. Perché `_ollama_semaphore` ha un solo permesso, e cosa impedisce concretamente? Perché il server locale di Ollama può sovraccaricarsi o esaurire le porte effimere del sistema operativo se riceve troppe richieste contemporanee. Un permesso solo impedisce che più di una chiamata a `client.embed(...)` sia in corso nello stesso momento, indipendentemente da quanti thread del pool sono attivi.

3. Perché questo progetto non contiene alcuna gestione esplicita della memoria, a differenza della gestione esplicita di file, thread e semaforo? Perché la memoria del processo (oggetti Python come `DataFrame` e array `NumPy`) è gestita automaticamente da

reference counting e garbage collector, senza bisogno di rilascio esplicito. File, thread e semaforo sono invece risorse esterne alla sola memoria del processo, e richiedono un rilascio esplicito — motivo per cui compaiono dietro un `with` al capitolo 11.2.

MATERIALE PER LA TESI 1. Il diagramma Mermaid del pool di thread e del semaforo (Figura 12.1), con la spiegazione del perché la concorrenza reale è più limitata di quanto sembri da `max_workers=3` — riusabile come figura tecnica in “Materiali e metodi”. 2. La spiegazione del Global Interpreter Lock applicata al caso concreto di questo progetto (I/O-bound, non CPU-bound) — riusabile per giustificare la scelta di `ThreadPoolExecutor` invece che, per esempio, il multiprocessing. 3. Il commento originale del codice sul motivo del semaforo (`embedding.py:95-100`), citato per esteso — riusabile come esempio di documentazione delle scelte progettuali trovata direttamente nel codice, utile per una discussione sulla qualità della documentazione interna del progetto.

Capitolo 13 — Tabella di traduzione Java→Python e trappole finali

Obiettivi del capitolo

- Avere un unico punto di riferimento che riassume, costruito per costruito, tutta la Parte II.
- Vedere in un solo elenco le cinque differenze più insidiose fra Python e Java, ciascuna con la propria conseguenza pratica.
- Leggere un traceback Python reale, prodotto da questo stesso progetto, riconoscendone la struttura.

13.1 Tabella sistematica

Concetto	In Java	In Python	Capitolo
Blocco di codice	{ }	Indentazione	7.1
Fine istruzione	;	Fine riga	7.1
Tipizzazione	Statica, verificata a compile time	Dinamica, verificata solo a runtime	7.2
Riferimento all'oggetto corrente	this implicito	self esplicito, primo parametro di ogni metodo	7.3
Lista dinamica	ArrayList<T>	list	8.1
Mappa chiave-valore	HashMap<K,V>	dict	8.1
Ritorno multiplo da una funzione	Oggetto o array dedicato	Tupla, destrutturata per posizione	8.1
Trasformare una collezione	stream().map().collect()	list/dict comprehension	8.2
Modificatori di accesso	private/protected/public imposti dal compilatore	Nessuno; convenzione dell'underscore, non imposta	10.3
Struttura a package	Cartelle + dichiarazione package	Nessuna dichiarazione; un file è già un modulo	10.1
Overload di funzione	Stesso nome, firme diverse, risolto a compile time	Non esiste; nomi diversi o logica a runtime	9.3
Funzione come valore	Lambda/riferimenti a metodo, tipizzati su un'interfaccia funzionale	Qualunque funzione, sempre, nessuna interfaccia richiesta	9.2

Concetto	In Java	In Python	Capitolo
Gestione di una risorsa	<code>try-with-resources</code> (<code>AutoCloseable</code>)	<code>with</code> (context manager)	11.2
Eccezioni	Controllate/non controllate, dichiarate con <code>throws</code>	Nessuna distinzione, nessuna dichiarazione in firma	11.1
Concatenare un'eccezione alla sua causa	<code>new X(msg, cause)</code>	<code>raise X(...) from cause</code>	11.1
Comportamento aggiunto a una funzione	Nessun equivalente integrato nel linguaggio	Decoratori (<code>@nome</code>)	11.3
Parallelismo reale fra thread	Sì, su core diversi	No a livello di bytecode puro (GIL); sì durante attesa I/O o codice nativo	12.1
Pool di thread	<code>ExecutorService</code>	<code>ThreadPoolExecutor</code>	12.2
Ambiente delle dipendenze	Classpath assemblato da Maven/Gradle	Ambiente virtuale + <code>pip/requirements.txt</code>	14

13.2 Le cinque trappole più insidiose per chi arriva da un linguaggio tipizzato staticamente

1. **Un default mutabile in una firma di funzione viene creato una volta sola**, non a ogni chiamata (capitolo 9.1). Non presente in questo progetto, ma è la trappola più citata dell'intero linguaggio: la prima volta che la incontri altrove, riconoscerai il sintomo (una lista di default che “ricorda” chiamate precedenti) prima ancora di ricordarti il nome del problema.
2. **Nessuna rete di sicurezza a compile time sui tipi** (capitolo 7.2): zero annotazioni di tipo in tutto questo progetto significano che un errore come passare un `DataFrame` dove il codice si aspetta un `ndarray` (o viceversa) emerge solo eseguendo quella riga specifica, non prima.
3. **Importare un modulo esegue il suo codice**, non solo lo dichiara disponibile (capitolo 10.2): `function.py:154` configura lo stile globale dei grafici nel momento dell'import, non quando qualcuno lo richiede esplicitamente — un side-effect facile da non notare leggendo solo le firme delle funzioni.
4. **Il GIL non impedisce il threading, ma ne limita il beneficio a un caso specifico** (capitolo 12.1): aspettarsi che tre thread Python diano una velocità tripla su calcolo puro, come ci si aspetterebbe in Java su una macchina multi-core, è un'aspettativa sbagliata — il beneficio reale, in questo progetto, viene dall'attesa di rete e dal codice nativo che rilascia il GIL, non dal bytecode Python in sé.
5. **Mutare un oggetto condiviso non richiede alcuna dichiarazione speciale** (capitolo 8.3): `results[chiave] = valore` in `classification.py:51` modifica silenziosamente il dizionario definito in `function.py`, senza `global`, senza alcun segnale sintattico che indichi “questa riga ha un effetto che sopravvive a questa funzione”.

PROVA TU — prendi le cinque trappole e, per ciascuna, prova a scrivere in una frase quale bug produrrebbe in un contesto *diverso* da quello attuale del progetto (per esempio: “cosa succederebbe se qualcuno chiamasse `training_classifier()` due volte nello stesso processo Python, per due dataset diversi, senza riavviare l'interpre-

te?”). Non è un esercizio retorico: è esattamente il tipo di domanda che un buon code review, in qualunque linguaggio, dovrebbe porsi prima di riusare del codice scritto assumendo un solo utilizzo per processo.

13.3 Prova tu: leggere un traceback

Un traceback Python si legge **dal basso verso l'alto** per capire cosa è andato storto per primo nel tempo, ma la riga più in basso è quella più vicina al punto in cui l'errore è stato effettivamente sollevato — l'opposto, a prima vista, di uno stack trace Java, che invece elenca per prima la riga più vicina all'eccezione e scende verso il chiamante originale. In realtà l'ordine logico è lo stesso in entrambi i casi (dal punto dell'errore verso il punto di avvio); cambia solo la direzione grafica in cui viene stampato.

PROVA TU — dopo aver completato l'installazione del capitolo 15, dalla cartella del progetto con l'ambiente virtuale attivato, esegui:

```
\ExtensionTok{python3} \AttributeTok{{-}c} \StringTok{"from function import
↪ get\_output\_dirs;
↪ get\_output\_dirs(\textquotesingle{nope}\textquotesingle{})"}

```

Otterrai un traceback reale, generato da questo stesso progetto. In fondo troverai `ValueError: Invalid dataset 'nope'. Valid options are: ['heart_disease', 'diabetes130']` — il messaggio scritto esplicitamente in `function.py:81`. Risalendo, troverai la riga esatta di `function.py` che ha sollevato l'eccezione, e sopra ancora la riga del tuo comando che ha chiamato `get_output_dirs`. Con due sole righe di codice hai già davanti la struttura completa di un traceback: fonte dell'errore in fondo, catena delle chiamate che ci sono arrivate risalendo.

Riepilogo

Questo capitolo chiude la Parte II con una tabella di riferimento sistematica e cinque trappole reali (una delle quali, gli argomenti mutabili di default, non presente in questo progetto ma troppo diffusa nel linguaggio per non conoscerla comunque). Da qui in avanti il libro smette di confrontare Python con Java a ogni costrutto, e comincia a occuparsi dell'ambiente (Parte III), dell'architettura (Parte IV) e infine del codice del progetto nel dettaglio (Parte V) — dando per acquisito tutto ciò che hai visto in questi sette capitoli.

Domande di autoverifica

- 1. Perché “nessun parallelismo reale a livello di bytecode Python” e “il threading in questo progetto è comunque utile” non sono affermazioni in contraddizione?** Perché il beneficio del threading, in questo progetto, viene dal tempo speso in attesa di rete (verso Ollama) o dentro codice nativo che rilascia il GIL (PyTorch), non dall'esecuzione parallela di bytecode Python puro — l'unica cosa che il GIL effettivamente impedisce.
- 2. In un traceback Python, dove si trova il messaggio dell'errore effettivamente sollevato, e dove si trova la chiamata che lo ha originato?** Il messaggio dell'errore si trova in fondo al traceback; risalendo verso l'alto si trova la catena delle chiamate, fino alla riga che ha avviato tutto — l'opposto grafico, ma non logico, di uno stack trace Java.
- 3. Quale delle cinque trappole di questo capitolo è l'unica a non avere un esempio reale in questo progetto, e perché la si tratta comunque?** Gli argomenti mutabili di

default (capitolo 9.1): non compaiono in nessuna firma di questo progetto, ma sono fra le trappole più diffuse e citate dell'intero linguaggio Python, e conoscerle prima di incontrarle altrove evita un debugging altrimenti sorprendente.

MATERIALE PER LA TESI 1. La tabella di traduzione sistematica Java→Python (§13.1) — riusabile in appendice alla tesi come riferimento rapido, o citata parzialmente per giustificare scelte di lettura del codice in “Materiali e metodi”. 2. L'elenco delle cinque trappole con la loro conseguenza pratica specifica a questo progetto — riusabile come base per la sezione “Discussione e limiti”, distinguendo quali rischi sono già materializzati nel codice e quali restano solo potenziali. 3. L'esercizio del traceback reale (§13.3), con il comando esatto e l'output atteso — riusabile come esempio riproducibile in un'eventuale sezione della tesi dedicata alla gestione degli errori.

Capitolo 14 — L’interprete e gli ambienti virtuali

Obiettivi del capitolo

- Capire perché Python ha bisogno di un meccanismo di isolamento delle dipendenze concettualmente diverso dal classpath Java.
- Sapere cosa contiene davvero la cartella `env/` di questo progetto, e perché non è mai versionata.
- Confrontare `pip/requirements.txt` con Maven/Gradle su un punto preciso: cosa viene dichiarato e cosa viene risolto.

14.1 Perché Python ha bisogno di un “classpath” alternativo

In un progetto Java, le dipendenze dichiarate in `pom.xml` o `build.gradle` vengono scaricate in un repository locale condiviso (`~/.m2` o la cache di Gradle) — non una copia per progetto, ma una cache centrale a cui più progetti attingono. Il “classpath” di un’esecuzione specifica è assemblato al momento della build, combinando riferimenti a quella cache condivisa: due progetti che dipendono entrambi da Guava 32 useranno, sul disco, lo stesso file `.jar`.

Python non ha, di norma, un equivalente diretto di questo meccanismo. `pip install numpy` installa NumPy nella cartella `site-packages` dell’interprete Python attivo in quel momento — e se quell’interprete è quello di sistema, condiviso da tutti i progetti Python della macchina, installare una versione diversa di NumPy per un altro progetto sovrascriverebbe (o configgerebbe con) quella già installata. Non esiste, nel meccanismo di base di `pip`, alcuna nozione di “questa versione di NumPy per questo progetto, quell’altra per quel progetto”, a meno di isolare esplicitamente ogni progetto nel proprio interprete.

Questo è esattamente il ruolo di un **ambiente virtuale** (`venv`): una copia (o una struttura di symlink) dell’interprete Python, con una propria cartella `site-packages` indipendente da quella di sistema e da quella di ogni altro ambiente virtuale. Attivarlo (`source env/bin/activate`, capitolo 15.2) modifica temporaneamente quali eseguibili `python` e `pip` risponderanno ai comandi della tua shell, facendoli puntare a quelli dentro l’ambiente virtuale invece che a quelli di sistema.

SE VIENI DA JAVA — la differenza concettuale più importante: Maven/Gradle isolano le dipendenze **per progetto, condividendo comunque i download** in una cache centrale; un ambiente virtuale Python isola le dipendenze **per progetto duplicandole fisicamente** in ogni singolo ambiente (salvo la cache di download di `pip`, che velocizza le reinstallazioni ma non elimina la duplicazione su disco di ogni `site-packages`). Dieci progetti Python con un proprio `venv` ciascuno, che dipendono tutti da NumPy, avranno dieci copie installate di NumPy sul disco — non una condivisa.

14.2 env/ in questo progetto: cosa contiene, perché non è versionato

[Fatto] Il progetto usa un ambiente virtuale nella cartella `env/` alla radice del repository, creato con `python3 -m venv env` (README.md:94, capitolo 15.2 per il comando completo). [Fatto] In questo ambiente di scrittura, `env/` occupa 1.4 GB su disco (comando `du -sh env` eseguito in questa sessione) e contiene, fra le altre cose, `env/lib/python3.14/site-packages/` con tutte le dipendenze di `requirements.txt` già installate — compresi pacchetti pesanti come `torch` (il motore di calcolo dietro i modelli Hugging Face) e le sue stesse dipendenze transitive.

[Fatto] `env/` compare in `.gitignore:1` (`./env`) e non è mai stata committata nel repository — verificato con `git ls-files`, che non la elenca. [Interpretazione] Questo non è un dettaglio amministrativo: 1.4 GB di file binari specifici per una piattaforma (`arm64/macOS`, capitolo 15.1) e per una versione esatta di Python non hanno alcun valore versionato in git — chi clona il repository deve ricrearla da zero eseguendo `python3 -m venv env` seguito da `pip install -r requirements.txt`, esattamente come chi clona un progetto Java deve lasciare che Maven o Gradle ricreino il proprio `target/`, mai committato per lo stesso motivo (dimensione, dipendenza dalla piattaforma, rigenerabilità automatica dal file di build).

RIFERIMENTO AL CODICE — l'interprete dentro `env/` è, in questo ambiente di scrittura, Python 3.14.0 per architettura `arm64` (comando `env/bin/python3 -c "import platform; print(platform.machine())"` eseguito in questa sessione, output `arm64`) — coerente con quanto dichiarato in README.md:84-85.

14.3 pip e requirements.txt confrontati con Maven/Gradle

[Fatto] `requirements.txt` (67 righe) elenca ogni dipendenza con una versione fissata esattamente (`pandas==3.0.2`, `numpy==2.4.4`, e così per tutte le altre) — nessun intervallo di versioni, nessuna gestione di conflitti dichiarativa come quella di un BOM Maven. [Interpretazione] La forma di questo file — piatta, con dipendenze dirette (`pandas`, `scikit-learn`, `ollama`, `sentence-transformers`) mescolate senza distinzione a dipendenze chiaramente transitive (`huggingface_hub`, `tokenizers`, `safetensors`, `sympy`, `mpmath`, `networkx`, `pydantic_core` — nessuna di queste è importata direttamente in nessuno dei nove file del progetto, verificato negli import elencati al capitolo 10.2) — è tipica di un file generato con `pip freeze`: uno snapshot di *tutto ciò che si trova installato* in un ambiente funzionante in un dato momento, non una dichiarazione a mano delle sole dipendenze dirette.

In Maven o Gradle, dichiarare solo le dipendenze dirette nel file di build; lo strumento calcola da solo l'albero delle dipendenze transitive, e puoi ispezionarlo esplicitamente (`mvn dependency:tree`). `pip` con un `requirements.txt` di questo tipo non offre la stessa distinzione: leggendo il file da solo, non puoi sapere quali righe il progetto importa davvero e quali sono lì solo perché qualcos'altro ne ha bisogno — lo scopri solo incrociando il file con gli `import` effettivi nel codice, esattamente il lavoro fatto per la tabella qui sopra.

ATTENZIONE — una conseguenza pratica di questo stile di file: aggiornare una sola dipendenza diretta (per esempio `scikit-learn`) a mano, modificando solo quella riga, non aggiorna né verifica la compatibilità delle dipendenze transitive collegate. Uno strumento come Maven ricalcolerebbe l'intero albero; qui, l'unico modo sicuro di aggiornare è rigenerare l'intero file con `pip freeze > requirements.txt` dentro un ambiente in cui l'aggiornamento è già stato fatto e verificato manualmente.

Riepilogo

Un ambiente virtuale Python isola le dipendenze duplicandole fisicamente per progetto, a differenza della cache condivisa di Maven/Gradle. La cartella `env/` di questo progetto (1.4 GB, verificato) non è mai versionata, per lo stesso motivo per cui non versioneresti la cartella di build compilata di un progetto Java. Il file `requirements.txt`, con versioni fissate e dipendenze dirette e transitive mescolate senza distinzione, è tipico di uno snapshot `pip freeze`, non di una dichiarazione manuale come un `pom.xml`.

Domande di autoverifica

- 1. Perché dieci progetti Python con un proprio ambiente virtuale, tutti dipendenti da NumPy, occupano più spazio su disco di dieci progetti Maven che dipendono dalla stessa libreria?** Perché ogni ambiente virtuale duplica fisicamente la propria copia di `site-packages`, mentre Maven/Gradle condividono un'unica cache centrale (`~/.m2` o equivalente) a cui più progetti attingono senza duplicare i file scaricati.
- 2. Perché `env/` non viene mai versionata in questo progetto, e cosa fa le sue veci in un progetto Java equivalente?** Perché contiene file binari specifici per piattaforma e versione (1.4 GB, verificato), interamente rigenerabili da `requirements.txt` con un comando. La cartella di build compilata di un progetto Java (`target/` in Maven, `build/` in Gradle) gioca lo stesso ruolo e per lo stesso motivo non viene versionata.
- 3. Perché non puoi distinguere, leggendo solo `requirements.txt`, quali dipendenze il progetto importa davvero e quali sono solo transitive?** Perché il file elenca in un'unica lista piatta sia le dipendenze dirette sia quelle transitive, senza segnalare la differenza — tipico di uno snapshot generato con `pip freeze`. La distinzione si ottiene solo incrociando il file con gli `import` effettivi nel codice sorgente.

MATERIALE PER LA TESI 1. Il confronto esplicito fra il modello di cache condivisa di Maven/Gradle e il modello di duplicazione per ambiente virtuale di Python — riusabile in “Materiali e metodi” per descrivere l’infrastruttura di sviluppo del progetto. 2. Il dato verificato sulla dimensione di `env/` (1.4 GB) e sulla sua esclusione da git — riusabile come nota tecnica sulla riproducibilità dell’ambiente. 3. L’osservazione sulla natura “flat e mista” di `requirements.txt`, con l’elenco delle dipendenze transitive individuate — riusabile in una sezione che discuta la gestione delle dipendenze del progetto, anche in chiave critica.

Capitolo 15 — Installazione passo passo su macOS

Obiettivi del capitolo

- Avere tutti i prerequisiti installati e verificati prima di eseguire una sola riga del progetto.
- Seguire la procedura di installazione comando per comando, con una verifica dopo ogni passo.
- Eseguire per la prima volta la pipeline, sapendo cosa aspettarti.

Questo capitolo descrive l'installazione così come la documenta `README.md:90-120`, riorganizzata con una verifica esplicita dopo ogni passo — utile perché, se qualcosa va storto, sai esattamente a quale passo è successo, invece di scoprirlo solo quando l'intera pipeline fallisce dieci minuti dopo.

15.1 Prerequisiti: Python 3.14, Ollama, token Hugging Face

[Fatto] Il progetto richiede tre cose, oltre a un Mac con macOS: [Fatto] Python 3.14 — `README.md:84` dichiara che è la versione con cui il progetto è stato sviluppato e testato, versioni precedenti della serie 3.x non sono verificate; [Fatto] Ollama — il server locale che serve i quattro modelli generalisti (E5, GTE), richiesto e in esecuzione, non solo installato (`README.md:86`); [Fatto] un token di lettura Hugging Face — necessario la prima volta che ciascuno dei tre modelli biomedici viene scaricato, o se un modello è soggetto a restrizioni di accesso (`README.md:87, 109-114`).

[Fatto] `README.md:85` dichiara inoltre che il progetto è sviluppato e verificato su Apple Silicon (`arm64`); su Intel/Linux gli stessi passi dovrebbero funzionare ma non sono stati verificati dagli autori. [Fatto] In questo ambiente di scrittura, sia il Python di sistema sia quello dentro `env/` risultano `arm64` (comando `platform.machine()` eseguito in questa sessione su entrambi gli interpreti) — coerente con quanto dichiarato.

ATTENZIONE — se `python3 -c "import platform; print(platform.machine())"` restituisse `x86_64` su un Mac con chip Apple Silicon, significherebbe che stai usando un'installazione di Python tradotta da Rosetta invece che nativa `arm64` — una causa comune, secondo `README.md:207`, di installazioni lente o fallite per `torch` e `numba`, entrambe dipendenze pesanti di questo progetto (capitolo 20.3, capitolo 22.3).

15.2 Procedura completa comando per comando

I passi seguenti corrispondono a `README.md:91-118`, con una verifica aggiunta dopo ciascuno.

1. Clona il repository e spostati nella sua cartella. Verifica: `ls main.py` deve mostrare il file senza errori.

2. Crea e attiva l'ambiente virtuale (capitolo 14.1):

```
\ExtensionTok{python3} \AttributeTok{{-}m}\NormalTok{ venv env}
\BuiltInTok{source}\NormalTok{ env/bin/activate}
```

Verifica: dopo l'attivazione, `which python3` deve puntare dentro `env/bin/python3`, non a un Python di sistema — il prompt della shell, di norma, mostra anche un prefisso (`env`) a conferma.

3. Installa le dipendenze:

```
\ExtensionTok{pip}\NormalTok{ install }\AttributeTok{{-}r}\NormalTok{
↪ requirements.txt}
```

Verifica: `pip list` deve mostrare, fra le altre, `pandas`, `scikit-learn`, `torch`, `sentence-transformers` con le versioni esatte di `requirements.txt`. Questo passo è quello che richiede più tempo e più spazio su disco — capitolo 14.2 ha già mostrato che l'ambiente completo occupa circa 1.4 GB.

4. Installa e avvia Ollama, poi scarica i quattro modelli generalisti (`function.py:38-43`, capitolo 20.1):

```
\ExtensionTok{ollama}\NormalTok{ serve }\KeywordTok{\&}
\ExtensionTok{ollama}\NormalTok{ pull yxchia/multilingual{-}e5{-}base}
\ExtensionTok{ollama}\NormalTok{ pull twtch/m3e{-}base}
\ExtensionTok{ollama}\NormalTok{ pull zyw0605688/gte{-}large{-}zh}
\ExtensionTok{ollama}\NormalTok{ pull
↪ jeffh/intfloat{-}multilingual{-}e5{-}large{-}instruct:q8\_0}
```

Verifica: `ollama list` deve elencare i quattro modelli — ma qui c'è un problema reale, non ipotetico. **[Fatto]** Il primo comando `pull` di `README.md:104` scarica `yxchia/multilingual-e5-base`. **[Fatto]** La configurazione del modello `e5-base` in `function.py:39` usa però il nome `jeffh/intfloat-e5-base-v2:q8_0` — un identificativo diverso sul registro dei modelli di Ollama, non un alias dello stesso modello. **[Fatto]** In questo ambiente di scrittura, `ollama list` (comando eseguito in questa sessione) mostra effettivamente `jeffh/intfloat-e5-base-v2:q8_0` installato — non `yxchia/multilingual-e5-base` — cioè il modello giusto per far funzionare `function.py`, diverso da quello che il `README` istruisce a scaricare. **Conseguenza pratica per chi segue il README alla lettera:** eseguendo solo i quattro comandi di `README.md:103-107`, la pipeline fallirebbe alla fase di embedding per il modello `e5-base` con un errore “model not found”, perché il modello scaricato non è quello richiesto da `embedding.py` quando chiama `client.embed(model=name, ...)` con `name` preso da `function.py:39`. Gli altri tre comandi `pull` (`gte-base`, `gte-large`, `e5-large`) corrispondono invece esattamente ai nomi usati in `function.py:40-42` — la discrepanza riguarda solo il primo modello. **[Da verificare]** Se sia un refuso di battitura nel `README` (un nome di modello simile ma sbagliato) o il residuo di una versione precedente del progetto che usava davvero `yxchia/multilingual-e5-base`, resta una domanda per chi ha scritto il codice (Appendice E).

5. Crea un file `.env` nella radice del progetto:

```
HF_READ_TOKEN=il_tuo_token_qui
OFFLINE_MODE=0
```

Verifica: nessun comando di verifica diretto — il primo test reale è il passo successivo, quando `embedding.py` prova a scaricare i modelli biomedici. **[Fatto]** `.env` è escluso da `.gitignore:2` e non deve mai essere committato: contiene una credenziale personale, non un valore di configurazione condivisibile.

6. Esegui la pipeline:

```
\ExtensionTok{python}\NormalTok{ main.py}
```

Verifica: vedi il paragrafo successivo.

ATTENZIONE — `python main.py` senza argomenti esegue **solo** il dataset Heart Disease (`main.py:15`, default `heart_disease`). Per eseguire anche Diabetes130 serve `python main.py --dataset diabetes130`, come comando separato — non è un'opzione che esegue entrambi i dataset in sequenza.

15.3 Prima esecuzione: `python main.py --dataset heart_disease`

Alla prima esecuzione, aspettati un tempo compreso fra alcuni minuti e qualche decina di minuti (`README.md:88`), dominato da due fasi: la generazione degli embedding (capitolo 22, chiamate di rete verso Ollama e calcolo verso i modelli Hugging Face) e il bootstrap a 10.000 iterazioni per ciascuno dei sette modelli (capitolo 36). Vedrai una sequenza di messaggi di stampa — non un'interfaccia grafica, non una barra di progresso unica per l'intera pipeline, ma i `print()` sparsi in ciascun modulo (capitolo 20-28 li commenta uno per uno).

Al termine, la cartella `datas/heart_disease/` conterrà cinque sottocartelle popolate: `preprocessing/`, `embeddings/`, `results/`, `graphics/`, `reports/` — l'ultima con il file `report.md` finale (capitolo 27). Se una qualunque fase fallisce a metà, la cartella resta parzialmente popolata: il capitolo 16 tratta questo scenario nel dettaglio.

PROVA TU — dopo la prima esecuzione completa, apri `datas/heart_disease/reports/report.md` in un visualizzatore Markdown qualunque. Prima ancora di leggere la Parte IX di questo libro (che lo analizza nel dettaglio), prova a individuare da solo quale dei sette modelli ha l'AUC più alta — è la prima delle domande che il capitolo 44 tratta con tutto il rigore statistico necessario.

Riepilogo

L'installazione richiede tre prerequisiti (Python 3.14 nativo `arm64`, Ollama in esecuzione con quattro modelli scaricati, un token Hugging Face), sei passi comando-per-comando ciascuno con una propria verifica, e produce, alla prima esecuzione completa, un intero albero di output sotto `datas/heart_disease/`. `python main.py` senza argomenti esegue solo Heart Disease: Diabetes130 richiede un comando esplicito separato.

Domande di autoverifica

- Come verifichi, con un solo comando, che l'ambiente virtuale sia stato attivato correttamente prima di installare le dipendenze?** `which python3` (o l'equivalente `where` su altre shell): deve puntare a un eseguibile dentro la cartella `env/`, non al Python di sistema.
- Cosa succede se esegui `python main.py` senza alcun argomento, e cosa NON succede?** Viene eseguita solo la pipeline per il dataset Heart Disease, il default di `main.py:15`. Non viene eseguito anche Diabetes130: serve un comando separato con `--dataset diabetes130` esplicito.
- Perché il file `.env` non va mai committato nel repository, a differenza di `requirements.txt`?** Perché contiene una credenziale personale (`HF_READ_TOKEN`), non una configurazione condivisibile fra chiunque cloni il progetto — è per questo escluso esplicitamente in `.gitignore:2`.

MATERIALE PER LA TESI 1. La procedura di installazione comando-per-comando con verifica esplicita ad ogni passo — riusabile in un’appendice della tesi dedicata alla riproducibilità, o in “Materiali e metodi” per descrivere l’ambiente sperimentale. 2. La discrepanza individuata fra il nome del modello e5-base nel README e quello effettivamente usato in `function.py` — riusabile come voce dell’appendice “zone d’ombra”, con l’evidenza precisa di entrambi gli identificativi. 3. L’osservazione sul comportamento di default di `main.py` (solo Heart Disease) — riusabile per giustificare, nella sezione “Materiali e metodi”, perché i risultati su Diabetes130 richiedano un’esecuzione esplicitamente separata e documentata come tale.

Capitolo 16 — Troubleshooting e verifica dell'ambiente

Obiettivi del capitolo

- Riconoscere gli errori più probabili durante l'installazione e la prima esecuzione, con la loro causa reale.
- Avere una checklist di verifica da eseguire prima di lanciare l'intera pipeline, non dopo che è già fallita.
- Sapere cosa NON esiste in Python rispetto a Java in tema di portabilità: niente bytecode distribuibile, niente `.jar`.

16.1 Gli errori più probabili e le loro cause reali

[Fatto] Questa tabella integra il troubleshooting di `README.md:206-213` con la causa specifica trovata leggendo il codice, non solo il sintomo.

Sintomo	Causa reale	Dove verificarla
Installazione di <code>torch/numba</code> lenta o fallita	Python tradotto da Rosetta invece che nativo <code>arm64</code> su Apple Silicon	<code>python3 -c "import platform; print(platform.machine())"</code> deve dare <code>arm64</code> , non <code>x86_64</code> (capitolo 15.1)
<code>ollama pull</code> fallisce, o “model not found” durante la generazione degli embedding per <code>e5-base</code>	Il comando del README (<code>yxchia/multilingual-e5-base</code>) e il nome usato da <code>function.py:39</code> (<code>jeffh/intfloat-e5-base-v2:q8_0</code>) sono due modelli diversi — capitolo 15.2 lo tratta per esteso	<code>ollama list</code> deve mostrare esattamente <code>jeffh/intfloat-e5-base-v2:q8_0</code> , non l'altro nome
Errore di autenticazione o modello “gated” verso Hugging Face <code>FileNotFoundError</code> su un file <code>.npy</code> in <code>embeddings/</code> o <code>results/</code>	<code>HF_READ_TOKEN</code> mancante o non valido in <code>.env</code> Una fase precedente della pipeline non si è completata — le fasi dipendono rigidamente dall'output della fase precedente, senza controlli di integrità intermedi	<code>embedding.py:146-149</code> stampa esplicitamente se un token è stato rilevato o meno Rilancia <code>python main.py</code> , osservando l'output console per capire quale fase si è interrotta (capitolo 18.3)

Sintomo	Causa reale	Dove verificarla
<code>ModuleNotFoundError</code> per un pacchetto elencato in <code>requirements.txt</code>	L’ambiente virtuale è stato attivato ma <code>pip install -r requirements.txt</code> non è mai stato eseguito, o è stato interrotto a metà	<code>pip list</code> dentro l’ambiente attivato, confrontato con <code>requirements.txt</code>
<code>source env/bin/activate</code> non funziona	L’ambiente virtuale è corrotto o incompleto	Elimina <code>env/</code> e ricrealo da zero (capitolo 15.2, passo 2)

16.2 Verificare che l’installazione sia corretta prima di lanciare l’intera pipeline

Eseguire l’intera pipeline solo per scoprire, dopo dieci minuti, che manca un token Hugging Face è un modo costoso di scoprire un problema che una verifica di trenta secondi avrebbe individuato subito. Prima di eseguire `python main.py`, vale la pena controllare, in ordine:

1. **L’interprete giusto è attivo.** `which python3` punta dentro `env/`.
2. **Le dipendenze sono installate.** `pip show sentence-transformers torch scikit-learn` non restituisce errori per nessuno dei tre.
3. **Ollama è in esecuzione e ha i modelli giusti.** `ollama list` mostra i quattro nomi esatti di `function.py:39-42`, non quelli (in un caso, diversi) del README.
4. **Il token Hugging Face è configurato**, se è la prima esecuzione dopo l’installazione. `cat .env` (mai con `git` di mezzo — è un file locale, mai tracciato) mostra `HF_READ_TOKEN` valorizzato.

PROVA TU — scrivi uno script di pochissime righe che automatizzi questi quattro controlli e stampi un semplice “OK” o “MANCA: ...” per ciascuno. Non è un esercizio di programmazione avanzata: è esattamente il tipo di controllo di sanità dell’ambiente (*smoke test*) che, in un contesto Java enterprise, faresti probabilmente girare come primo step di una pipeline CI — qui non esiste (capitolo 48 lo tratta nel dettaglio), ed è un’occasione concreta per aggiungerne uno tu.

16.3 Cosa NON esiste in Python: niente bytecode portabile tra versioni, nessun `.jar`

[Livello: teoria consolidata del settore] Un `.jar` Java, una volta compilato, gira su qualunque JVM (di versione compatibile) su qualunque sistema operativo — è precisamente la promessa “compila una volta, esegui ovunque” su cui è costruito l’intero ecosistema Java. Python non ha un equivalente pratico di questa promessa.

Python compila sì il proprio codice sorgente in bytecode — lo trovi nella cartella `__pycache__`, con file `.pyc` — ma quel bytecode non è pensato per essere distribuito: è specifico della versione esatta dell’interprete che lo ha generato (un `.pyc` compilato da Python 3.14 non è garantito funzionare, e spesso semplicemente non funziona, con Python 3.11), e in pratica quasi mai portato da una macchina a un’altra. Ciò che si distribuisce, normalmente, è il codice sorgente stesso (`.py`), insieme a un ambiente virtuale (capitolo 14) ricreato da zero sulla macchina di destinazione con la stessa versione di Python — non un artefatto compilato una volta e trasportato ovunque.

[Fatto] Questo progetto non fa eccezione: non esiste, in nessuna parte del repository, un tentativo di “compilare” o “impacchettare” il codice in una forma distribuibile autonoma (nessun

`pyproject.toml`, nessun `setup.py`, verificato nella scheda tecnica). L'unico modo di eseguirlo su una macchina diversa è ripetere l'intera procedura di installazione di questo capitolo — non copiare un artefatto già pronto.

SE VIENI DA JAVA — questa è una delle differenze pratiche più rilevanti quando pensi al deployment: un servizio Java packagizzato come `.jar` (o `.war`) è un artefatto singolo, versionabile e distribuibile; un progetto Python come questo richiede, su ogni macchina di destinazione, la ricostruzione completa dell'ambiente (capitolo 14) prima di poter eseguire una sola riga. Esistono strumenti che avvicinano Python a un modello di distribuzione più simile a quello Java — contenitori Docker con l'ambiente già pronto, per esempio — ma questo progetto non li usa: nessun `Dockerfile` nel repository, verificato.

Riepilogo

Gli errori più probabili in questo progetto hanno cause specifiche, verificabili leggendo il codice: architettura Rosetta invece di `arm64`, un nome di modello Ollama sbagliato nel README per il solo `e5-base`, un token Hugging Face mancante, o una fase della pipeline interrotta a metà senza controlli di integrità. Una checklist di quattro verifiche rapide, eseguita prima del lancio completo, individua la maggior parte di questi problemi in pochi secondi. A differenza di un `.jar` Java, il codice Python di questo progetto non ha una forma distribuibile compilata: ogni macchina di destinazione ricostruisce l'ambiente da zero a partire dal codice sorgente e da `requirements.txt`.

Domande di autoverifica

1. Se `ollama pull yxchia/multilingual-e5-base` sembra completarsi senza errori, perché la pipeline potrebbe comunque fallire più avanti per il modello `e5-base`? Perché quel comando scarica un modello con un nome diverso da quello che `function.py:39` richiede davvero (`jeffh/intfloat-e5-base-v2:q8_0`). `ollama pull` va a buon fine, ma il modello scaricato non è quello che `embedding.py` chiederà al server Ollama in fase di generazione degli embedding.
2. Perché copiare un file `.pyc` da `__pycache__` su un'altra macchina non è un modo affidabile di distribuire questo progetto? Perché il bytecode Python compilato è specifico della versione esatta dell'interprete che lo ha generato, e non è garantito funzionare con una versione diversa — a differenza di un `.jar`, pensato fin dall'origine per essere portabile fra JVM diverse.
3. Quali quattro controlli faresti, in trenta secondi, prima di lanciare l'intera pipeline per la prima volta? Verificare che l'interprete attivo sia quello dentro `env/`, che le dipendenze chiave siano installate, che `ollama list` mostri i nomi esatti richiesti da `function.py` (non quelli, in un caso diversi, del README), e che `.env` contenga un token Hugging Face valorizzato.

MATERIALE PER LA TESI 1. La tabella degli errori più probabili con causa reale (§16.1) — riusabile in un'appendice della tesi dedicata alla riproducibilità dell'ambiente sperimentale. 2. La checklist di verifica in quattro punti (§16.2) — riusabile come base per uno script di smoke test da citare in “Materiali e metodi”, anche solo come proposta metodologica. 3. Il confronto esplicito fra il modello di distribuzione a bytecode di Java e l'assenza di un equivalente in questo progetto Python — riusabile per motivare, nella sezione “Lavori futuri”, una proposta di containerizzazione (capitolo 55).

Capitolo 17 — Vista d'insieme: componenti e confini

Obiettivi del capitolo

- Avere una mappa visiva dell'intera pipeline prima di leggere una sola riga di implementazione.
- Sapere esattamente quali moduli dipendono da quali altri, e quali non dipendono da nessuno.
- Confrontare questa architettura con quella di un batch job Java enterprise che probabilmente già conosci.

17.1 Diagramma architetturale commentato

[Fatto] Il diagramma seguente ricostruisce l'architettura completa a partire dagli `import` reali di ciascun file (verificati sistematicamente, capitolo 10.2) e dalle chiamate di `main.py:20-53`.

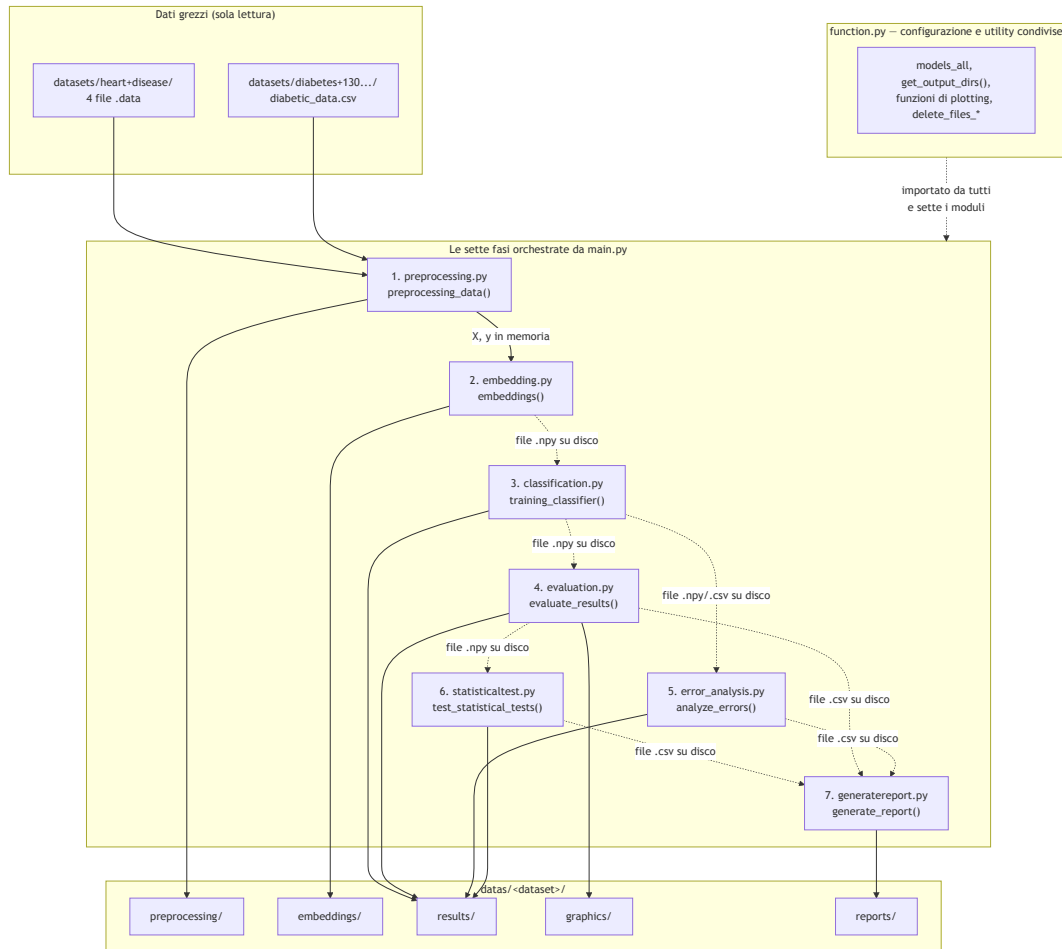


Figura 17.1 — Architettura completa della pipeline. Le frecce continue fra fasi sono passaggio diretto di dati in memoria (Python); le frecce tratteggiate sono un salvataggio su disco seguito da una rilettura nella fase successiva, non una chiamata di funzione diretta.

Nota, guardando le frecce fra le fasi, un'asimmetria reale: **[Fatto]** solo il passaggio da `preprocessing.py` a `embedding.py` avviene per valore, dentro lo stesso processo Python (`main.py:34-37`, `X, y = preprocessing_data(...)` seguito da `embeddings(X, y, ...)`). **[Fatto]** Da `classification.py` in poi, ogni fase **rilegge da disco** l'output della fase precedente — `classification.py:13-14` chiama `np.load(...)` sui file `.npy` salvati da `embedding.py`, `evaluation.py:15-17` rilegge a sua volta i file salvati da `classification.py`, e così via fino a `generatereport.py`, che rilegge un CSV riassuntivo. Nessuna di queste funzioni riceve i dati della fase precedente come argomento diretto, pur essendo chiamate in sequenza nello stesso processo.

17.2 Cosa entra, cosa esce, da ogni fase

Fase	File	Cosa riceve	Cosa produce
1. Preprocessing	<code>preprocessing.py</code>	Nome del dataset (stringa)	<code>X</code> , <code>y</code> in memoria (DataFrame/Series) + <code>preprocessed_data.npy</code> , <code>preprocessed_labels.npy</code> , <code>X_train_raw.csv</code> su disco
2. Embedding	<code>embedding.py</code>	<code>X</code> , <code>y</code> in memoria	7 coppie di file <code>{modello}_embeddings.npy</code> / <code>{modello}_embeddings_labels.npy</code> su disco
3. Classificazione	<code>classification.py</code>	Nome del dataset (rilegge gli embedding da disco)	4 file <code>.npy</code> per modello (<code>y_true</code> , <code>y_score</code> , <code>y_pred</code> , <code>val_idx</code>) + <code>model_performance.csv</code>
4. Valutazione	<code>evaluation.py</code>	Nome del dataset (rilegge i risultati da disco)	3 array bootstrap per modello + grafici + <code>encoder_comparison_summary.csv</code>
5. Analisi errori	<code>error_analysis.py</code>	Nome del dataset (rilegge risultati + <code>X_train_raw.csv</code>)	CSV di falsi positivi/negativi, casi più difficili, deviazione di feature
6. Test statistici	<code>statisticaltest.py</code>	Nome del dataset (rilegge gli array bootstrap)	3 CSV di confronto a coppie (Wilcoxon, t-test, DeLong)
7. Report	<code>generatereport.py</code>	Nome del dataset (rilegge il CSV riassuntivo)	<code>report.md</code> finale

17.3 Confronto con un'architettura Java a livelli

[Interpretazione] Se dovessi tradurre questa architettura in termini familiari a chi lavora con Java enterprise, la corrispondenza più calzante non è un'applicazione web a livelli (controller/service/repository), ma un **batch job sequenziale** — il genere di cosa che in Spring Batch si modellerebbe come un `Job` composto da una sequenza fissa di `Step`, ciascuno con un proprio `ItemReader` (leggi da una fonte), una fase di elaborazione, e un `ItemWriter` (scrivi su una destinazione). `main.py` gioca il ruolo del `Job`; ciascuna delle sette fasi gioca il ruolo di uno `Step`; il passaggio di dati fra step tramite file su disco invece che tramite oggetti Java in memoria è, in questo confronto, non una stranezza ma una scelta architetturale comune anche nel mondo Java quando gli step di un batch devono essere ispezionabili, riavviabili singolarmente, o eseguiti in processi separati — cosa che, come nota il capitolo 18.3, questo progetto rende possibile solo in parte.

`function.py` gioca invece un ruolo diverso da quello di un singolo `Step`: è importato da tutti gli altri moduli (Figura 17.1), e mescola quattro responsabilità distinte — la configurazione dei modelli, la gestione delle cartelle di output, la pulizia dei file di run precedenti, e *tutte* le funzioni di plotting del progetto (nove, capitolo 20.3). In un'architettura Java organizzata per responsabilità, queste sarebbero probabilmente quattro classi separate (una `ModelConfig`, un `OutputDirectoryService`, un `CleanupService`, un `ChartingService`); qui vivono in un

solo file di 386 righe. Non è necessariamente un errore — per un progetto di questa scala, la separazione avrebbe un costo di indirection che potrebbe non ripagarsi — ma è un'osservazione precisa da portare in Parte XI quando si discute la manutenibilità del codice.

ATTENZIONE — l'osservazione sul passaggio di dati via disco non è solo descrittiva: è collegata a una scelta progettuale reale già vista al capitolo 8.3 e ripresa al capitolo 18.3. Ogni esecuzione di `main.py` cancella *tutti* i file dell'output precedente per quel dataset prima di rigenerarli (`main.py:28-31`) — una necessità diretta del fatto che le fasi comunicano tramite file: se non venissero cancellati, una fase potrebbe silenziosamente rileggere un file lasciato da un'esecuzione precedente invece che da quella corrente.

Riepilogo

L'architettura del progetto è un batch job sequenziale a sette fasi, orchestrato da `main.py`, con `function.py` come hub condiviso da cui tutte le altre sette fasi dipendono e che non dipende da nessuna di esse. Solo il passaggio dalla fase 1 alla fase 2 avviene per valore in memoria; da lì in poi, ogni fase rilegge da disco l'output della fase precedente, un pattern concettualmente vicino a uno Step-based batch job Java (Spring Batch, per chi lo conosce), con i vantaggi e i costi che quel pattern comporta.

Domande di autoverifica

- 1. Perché il passaggio di dati fra `classification.py` e `evaluation.py` non è una chiamata di funzione diretta, pur essendo entrambi chiamati in sequenza nello stesso processo Python da `main.py`?** Perché `evaluation.py` rilegge da disco i file `.npy` che `classification.py` ha salvato, invece di ricevere quei dati come argomento di funzione — un pattern di comunicazione via file, non via memoria condivisa nello stesso processo.
- 2. Quali quattro responsabilità distinte convivono in `function.py`, e come le separeresti in un'architettura Java tipica?** Configurazione dei modelli, gestione delle cartelle di output, pulizia dei file di run precedenti, e tutte le funzioni di plotting. In un'architettura Java tipica sarebbero probabilmente quattro classi separate, ciascuna con una singola responsabilità.
- 3. Perché `main.py` cancella l'intero output precedente di un dataset prima di rigenerarlo, invece di limitarsi a sovrascrivere i file nuovi?** Perché le fasi comunicano tramite file su disco (capitolo 17.1): se un file di un'esecuzione precedente non venisse cancellato e la fase corrente non lo rigenerasse per qualche motivo, una fase successiva potrebbe rileggerlo silenziosamente, mescolando dati di esecuzioni diverse senza che nulla lo segnali.

MATERIALE PER LA TESI 1. Il diagramma Mermaid completo dell'architettura (Figura 17.1), con la distinzione esplicita fra passaggio in memoria e passaggio su disco — riusabile come figura centrale della sezione “Materiali e metodi”. 2. La tabella input/output per ciascuna delle sette fasi (§17.2) — riusabile come tabella descrittiva della pipeline, o come base per una tabella più sintetica nella tesi. 3. Il confronto con il pattern Job/Step di un batch sequenziale Java, e l'analisi delle quattro responsabilità mescolate in `function.py` — riusabile nella discussione critica sulla manutenibilità dell'architettura (Parte XI).

Capitolo 18 — Il ciclo di vita di un'esecuzione

Obiettivi del capitolo

- Seguire, passo per passo, cosa succede fra il momento in cui lanci `python main.py` e il momento in cui il report è pronto.
- Capire perché due dataset diversi non si calpestano mai a vicenda.
- Sapere cosa succede — e cosa resta sul disco — se una fase fallisce a metà.

18.1 Diagramma di sequenza da `main.py` al report

[Fatto] Il diagramma seguente segue esattamente l'ordine di chiamata di `main.py:20-53`.

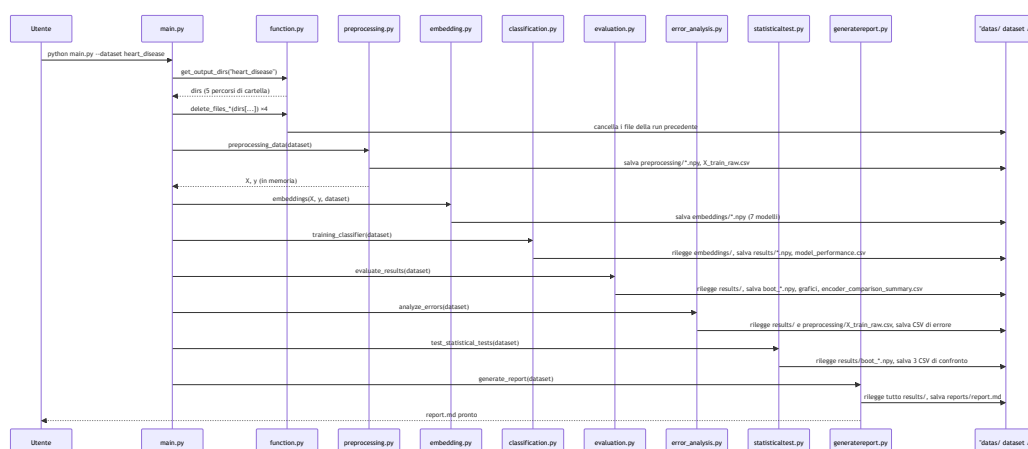


Figura 18.1 — Sequenza completa di un'esecuzione, dalla riga di comando al report finale. Ogni freccia verso `datas/<dataset>/` è un salvataggio; ogni freccia dalla stessa cartella verso una fase, quando presente nel testo, è una riletture.

Ogni fase è invocata da `main.py` con la stessa forma: passa `dataset=args.dataset` (o il nome posizionale equivalente) e nient'altro — nessuna fase, da `training_classifier` in poi, riceve dati concreti come argomento, solo il nome del dataset su cui operare. È `get_output_dirs()` (`function.py:79-93`), chiamata dentro ciascuna fase, a tradurre quel nome nei percorsi concreti da cui leggere e su cui scrivere.

18.2 Isolamento per dataset

[Fatto] `get_output_dirs(dataset)` (`function.py:79-93`) costruisce sempre i percorsi come `datas/<dataset>/<sottocartella>`, con `<dataset>` uguale a `"heart_disease"` o

"diabetes130". Ogni chiamata a una delle sette fasi con un dataset diverso opera quindi su un albero di cartelle completamente separato: eseguire `python main.py --dataset diabetes130` non tocca in alcun modo `datas/heart_disease/`, e viceversa. [Fatto] `main.py:23-31` chiama `delete_files_*` solo sulle cartelle del dataset scelto in quella specifica esecuzione — non esiste un solo punto del codice in cui un'esecuzione per un dataset possa cancellare o sovrascrivere l'output dell'altro.

SE VIENI DA JAVA — questo isolamento non è imposto da un meccanismo esplicito (un lock, una transazione, un namespace a livello di sistema operativo): è semplicemente il risultato di come sono costruiti i percorsi delle stringhe, `os.path.join("datas", dataset, sottocartella)` (`function.py:83-89`). Non c'è nulla che impedirebbe, per errore di battitura futuro, a qualcuno di scrivere un percorso che esce da questo schema — l'isolamento regge finché il codice continua a costruire i percorsi in questo modo esatto, non perché sia garantito da una barriera strutturale.

18.3 Fallimento a metà pipeline

[Fatto] Nessuna delle sette fasi verifica, prima di cominciare, che l'output della fase precedente esista e sia completo — ognuna assume che sia lì, e se non lo è, l'errore emerge come `FileNotFoundError` nel momento esatto in cui una fase prova a leggere un file mancante (`np.load(...)` o `pd.read_csv(...)` che non trova il file). [Fatto] Non esiste, in `main.py`, alcun blocco `try/except` attorno alle chiamate alle sette fasi (`main.py:33-52`): se una fase solleva un'eccezione qualunque, l'intero processo si interrompe immediatamente con un traceback (capitolo 13.3), senza che le fasi successive vengano nemmeno tentate.

Questo significa che un'esecuzione interrotta a metà lascia la cartella del dataset **parzialmente popolata**: per esempio, se `embedding.py` fallisse (una richiesta a Ollama che esaurisce i tentativi di retry, capitolo 22.2), `datas/<dataset>/preprocessing/` conterrebbe già i file completi della fase 1, `datas/<dataset>/embeddings/` conterrebbe solo gli embedding dei modelli completati prima del fallimento, e nessuna delle cartelle successive (`results/`, `graphics/`, `reports/`) conterrebbe nulla di nuovo — restando eventualmente con i file, ormai obsoleti, dell'esecuzione precedente non ancora sovrascritti, perché la fase di pulizia (`main.py:28-31`) ha già cancellato solo quelli, non ha ricreato quelli mancanti.

ATTENZIONE — l'unica strategia di recupero prevista dal progetto, in questo scenario, è rilanciare `python main.py` da capo per lo stesso dataset (`README.md:210`) — che cancella di nuovo tutto e ricomincia dalla fase 1, incluse le fasi già completate con successo. Non esiste un modo, nel codice attuale, di riprendere da dove l'esecuzione si è interrotta: è un limite reale di questa architettura a fasi file-based, non solo un'osservazione teorica. Il capitolo 55 lo riprende come possibile direzione di miglioramento.

Riepilogo

Un'esecuzione completa attraversa sette fasi in sequenza rigida, ciascuna identificata solo dal nome del dataset, senza alcun controllo di integrità fra una fase e la successiva. L'isolamento fra i due dataset è garantito dalla costruzione sistematica dei percorsi in `get_output_dirs()`, non da una barriera strutturale esplicita. Un fallimento a metà pipeline lascia una cartella di output parzialmente popolata, e l'unico modo di recuperare è rilanciare l'intera pipeline da capo, rifacendo anche le fasi già completate con successo.

Domande di autoverifica

- 1. Come “sa” ciascuna delle sette fasi dove leggere e dove scrivere, dato che nessuna riceve un percorso esplicito come argomento?** Ogni fase riceve solo il nome del dataset, e chiama internamente `get_output_dirs(dataset)` per ottenere i percorsi concreti delle cinque sottocartelle — la stessa funzione, chiamata allo stesso modo, in ogni fase.
- 2. Cosa impedisce, strutturalmente, che un’esecuzione su Diabetes130 cancelli i risultati già salvati per Heart Disease?** Nulla di strutturale in senso stretto: è una conseguenza di come `get_output_dirs()` costruisce i percorsi (sempre `datas/<dataset>/...`, con `<dataset>` preso dall’argomento passato), non una barriera imposta a livello di sistema operativo o di codice difensivo.
- 3. Se `embedding.py` fallisse a metà, cosa troveresti sul disco subito dopo, e cosa dovresti fare per ripartire?** Troveresti `preprocessing/` completo, `embeddings/` con solo alcuni dei sette modelli generati, e nulla di nuovo nelle cartelle successive. Non esiste un modo di riprendere da lì: l’unica strategia prevista è rilanciare l’intera pipeline da capo per quel dataset, rifacendo anche il preprocessing già completato con successo.

MATERIALE PER LA TESI 1. Il diagramma di sequenza completo (Figura 18.1), con l’indicazione esplicita di dove ogni fase legge e scrive — riusabile come figura centrale in “Materiali e metodi” per descrivere il protocollo sperimentale. 2. L’analisi dell’isolamento fra dataset come conseguenza di una convenzione di percorso, non di una garanzia strutturale — riusabile come nota tecnica nella sezione che descrive la gestione dei dati. 3. L’osservazione sull’assenza di ripartenza da un punto intermedio, con il rimando alla proposta di miglioramento del capitolo 55 — riusabile come punto di discussione sulla robustezza operativa del sistema.

Capitolo 19 — Stato, configurazione, punti di estensione

Obiettivi del capitolo

- Sapere che `function.py` è, di fatto, l'unico file di configurazione del progetto, e cosa questo comporta.
- Rivedere lo stato globale mutabile con lo sguardo architetturale, non solo linguistico, di questo capitolo.
- Sapere esattamente dove intervenire — e dove stare attenti — per aggiungere un ottavo modello di embedding.

19.1 Dove vive la configurazione: `function.py` come “file unico di config”

[Fatto] Non esiste, in questo progetto, un file di configurazione nel senso in cui probabilmente lo intendi arrivando da Java — nessun `.properties`, nessun `.yaml`, nessuna classe annotata `@ConfigurationProperties`. Ogni parametro che potresti aspettarti di trovare externalizzato è invece scritto direttamente dentro `function.py`, come valore letterale nel codice: l'elenco dei sette modelli con i loro identificativi (`function.py:38-49`), la dimensione del campione per Diabetes130 (`function.py:116`, `sample_size=20000`), la palette di colori (`function.py:57-61`), le dimensioni standard delle figure (`function.py:64-65`).

[Interpretazione] Questo significa che “configurare” il progetto in modo diverso — per esempio cambiare quanti record campionare da Diabetes130 — richiede modificare il codice sorgente, non un file di configurazione esterno da passare come parametro. Non è necessariamente un problema per un progetto di ricerca a questa scala, dove chi esegue la pipeline e chi ne legge il codice sono, con ogni probabilità, la stessa persona — ma è un punto di attenzione se il progetto dovesse mai essere eseguito da chi non ha accesso o familiarità col codice sorgente.

19.2 Stato globale mutabile: dove, perché, rischi

Il capitolo 8.3 ha già mostrato il meccanismo linguistico del dizionario `results` (`function.py:67-72`, mutato da `classification.py:51`). A livello architetturale, la domanda interessante è un'altra: **perché lo stato globale mutabile è un problema per l'estensibilità, non solo per la correttezza immediata?**

[Interpretazione] Uno stato condiviso a livello di modulo rende più difficile ragionare su una funzione guardandola in isolamento: `training_classifier()` (`classification.py:9-80`) non restituisce esplicitamente il proprio risultato principale come valore di ritorno — lo scrive in un dizionario globale importato da un altro modulo, un side-effect che non è visibile guardando solo la firma della funzione (`def training_classifier(dataset="heart_disease"):`, nessun

tipo di ritorno dichiarato, capitolo 7.2). Chiunque volesse scrivere un test automatico per questa funzione (Parte X) dovrebbe sapere, in anticipo, di dover controllare `function.results` dopo la chiamata — un'informazione che nessuna firma comunica.

ATTENZIONE — questo è precisamente il tipo di accoppiamento implicito che un'architettura a livelli in Java tende a evitare per costruzione: un metodo di un servizio Java che scrivesse silenziosamente in un campo statico di un'altra classe, invece di restituire un valore, verrebbe quasi certamente segnalato in un code review. Qui il linguaggio lo permette senza frizione (capitolo 8.3), e il progetto lo usa: un'osservazione critica legittima per la Parte XI, non solo una curiosità sintattica.

19.3 Dove aggiungeresti un ottavo modello di embedding, se dovessi farlo tu

[Fatto] Aggiungere un ottavo modello di embedding richiede, in linea di principio, una sola modifica: aggiungere un dizionario con la stessa struttura degli altri a `models_ollama` o a `models_medical` (`function.py:38-49`), con i campi `type`, `model_name`, `name`, `filename`, `filename_label`, `family`. Ogni altra parte della pipeline — generazione degli embedding (`embedding.py`), addestramento (`classification.py`), valutazione (`evaluation.py`), analisi degli errori (`error_analysis.py`), test statistici (`statisticaltest.py`) — itera su `models_all` (`function.py:51`, l'unione di `models_ollama` e `models_medical`) senza mai nominare esplicitamente uno dei sette modelli attuali: il codice è già scritto per generalizzare a un numero qualunque di modelli.

[Fatto] Con una precisazione importante, se il nuovo modello appartiene a una **famiglia già esistente** (per esempio un altro modello "biomedical"), tutto funziona automaticamente: `get_model_palette()` (`function.py:162-174`) genera una nuova sfumatura di colore per ogni membro della famiglia, quale che sia il loro numero.

[Fatto] Se invece il nuovo modello introducesse una **quarta famiglia**, mai vista finora, due punti del codice richiederebbero una modifica manuale, non automatica: `FAMILY_COLORS` (`function.py:57-61`) non avrebbe una voce per quella famiglia, e `get_model_palette()` userebbe silenziosamente il colore di ripiego "#888888" (grigio, `function.py:170`) invece di un colore dedicato — nessun errore, solo un grafico meno leggibile. Più seriamente, `plot_family_comparison()` (`function.py:349`) costruisce l'ordine delle famiglie con una lista scritta a mano:

```
\NormalTok{family\_order } \OperatorTok{=} \NormalTok{ [f
↪ } \ControlFlowTok{for} \NormalTok{ f } \KeywordTok{in} \NormalTok{ {
↪ [ ] \StringTok{"general{-}purpose"} \NormalTok{ {,
↪ } \StringTok{"biomedical"} \NormalTok{ {, } \StringTok{"biomedical{-}st"} \NormalTok{ {
↪ } \ControlFlowTok{if} \NormalTok{ f } \KeywordTok{in} \NormalTok{ {
↪ df [ ] \StringTok{"Family"} \NormalTok{ { .unique() ] }
```

Una quarta famiglia non elencata qui non comparirebbe affatto nell'ordine costruito da questa riga — un'esclusione silenziosa, non un errore, e per questo più insidiosa: il grafico verrebbe generato comunque, semplicemente senza quella famiglia, e nulla nella console lo segnalerebbe.

PROVA TU — apri `function.py` e prova, sulla carta o in un ambiente di test separato, ad aggiungere un ottavo modello che reintroduca una famiglia esistente (per esempio un secondo modello "biomedical-st"). Poi prova, mentalmente o davvero, ad aggiungerne uno di una famiglia nuova, per esempio "multilingual". Verifica tu stesso, leggendo `function.py:162-174` e `function.py:349`, se la tua previsione su cosa si romperebbe silenziosamente coincide con quanto appena descritto.

Riepilogo

`function.py` è, di fatto, l'unico punto di configurazione del progetto: non esistono file di configurazione esterni, ogni parametro è un valore letterale nel codice sorgente. Lo stato globale mutabile del capitolo 8.3, letto ora con sguardo architettuale, rende una funzione come `training_classifier()` più difficile da testare in isolamento, perché il suo risultato principale non è nel valore di ritorno ma in un side-effect su un dizionario di un altro modulo. Aggiungere un ottavo modello a una famiglia esistente è quasi gratuito grazie a `models_all`; aggiungerne uno di una famiglia nuova richiede due modifiche manuali puntuali, altrimenti silenziosamente incomplete.

Domande di autoverifica

1. Perché “configurare diversamente” questo progetto richiede modificare il codice sorgente, e non un file esterno? Perché non esiste alcun file di configurazione esternalizzato (`.yaml`, `.properties` o equivalente): ogni parametro — elenco dei modelli, dimensione del campione, colori, dimensioni delle figure — è scritto come valore letterale direttamente dentro `function.py`.

2. Perché testare `training_classifier()` in isolamento è più complesso di quanto la sua firma lasci intuire? Perché il suo risultato principale non è il valore di ritorno della funzione (che non esiste, la funzione non restituisce nulla di esplicito) ma un side-effect sul dizionario globale `results` di un altro modulo — un'informazione non visibile guardando solo la firma `def training_classifier(dataset="heart_disease"):`.

3. Cosa succederebbe, in concreto, se aggiungessi un ottavo modello con una famiglia mai vista prima, senza modificare `FAMILY_COLORS` e `plot_family_comparison()`? Il modello riceverebbe un colore di ripiego grigio invece di un colore dedicato in tutti i grafici basati sulla palette, e sarebbe del tutto assente dal grafico di confronto per famiglia (`FamilyComparison_metrics`), perché la sua famiglia non comparirebbe nella lista scritta a mano `family_order` — senza che alcun errore o avviso lo segnali.

MATERIALE PER LA TESI 1. L'osservazione sull'assenza di configurazione esternalizzata, con l'elenco dei parametri codificati come valori letterali — riusabile in “Materiali e metodi” per descrivere onestamente la flessibilità operativa del progetto. 2. L'analisi dello stato globale mutabile come ostacolo alla testabilità, collegata esplicitamente alla Parte X — riusabile come argomento nella sezione “Discussione e limiti”. 3. La guida passo-passo per aggiungere un ottavo modello, con i due punti di rottura silenziosa individuati (`FAMILY_COLORS`, `family_order`) — riusabile come esercizio dimostrativo di comprensione del codice, o come base per una sezione “lavori futuri” più tecnica.

Capitolo 20 — `function.py`: la spina dorsale silenziosa

Obiettivi del capitolo

- Avere una mappa completa di tutto ciò che `function.py` mette a disposizione degli altri sette moduli.
- Leggere in dettaglio le due funzioni più significative del file: `get_output_dirs()` e `get_model_palette()`.
- Riconoscere il pattern comune a tutte le nove funzioni di plotting, senza doverle leggere una per una.

[Fatto] `function.py` (386 righe) è il file più lungo del progetto, e l'unico da cui tutti gli altri sette dipendono (Figura 17.1). Non contiene alcuna delle sette fasi della pipeline: è puro supporto — configurazione, percorsi, pulizia, grafici.

20.1 Config modelli e famiglie

[Fatto] Le prime 74 righe definiscono costanti di modulo, mai funzioni: gli schemi delle colonne per i due dataset (`columns`, `num_cols`, `cat_cols` per Heart Disease alle righe 10-15; `columns_diabetes130`, `num_cols_diabetes130`, `cat_cols_diabetes130` per Diabetes130 alle righe 21-35), la configurazione dei sette modelli (`models_ollama`, righe 38-43; `models_medical`, righe 45-49; `models_all`, riga 51), e due dizionari derivati: `MODEL_FAMILY` (riga 54, dict comprehension già vista al capitolo 8.2) e `FAMILY_COLORS` (righe 57-61).

20.2 Funzioni di I/O e pulizia cartelle

[Fatto] `get_output_dirs(dataset)` (righe 79-93) è probabilmente la funzione più chiamata dell'intero progetto — ogni fase, in ogni file, la invoca all'inizio:

```
\KeywordTok{def}\NormalTok{ get\_output\_dirs(dataset):}
  \ControlFlowTok{if}\NormalTok{ dataset }\KeywordTok{not}
    ↪ \KeywordTok{in}\NormalTok{ datasets:}
    \ControlFlowTok{raise}
    ↪ \PreprocessorTok{ValueError}\NormalTok{({}\SpecialStringTok{f"Invalid
    ↪ dataset \textquotesingle{}}\SpecialCharTok{\}\NormalTok{dataset}\Special
    ↪ CharTok{\}\SpecialStringTok{\textquotesingle{}}. Valid options are:
    ↪ }\SpecialCharTok{\}\NormalTok{datasets}\SpecialCharTok{\}\SpecialString
    ↪ Tok{}}\NormalTok{)}
\NormalTok{    base }\OperatorTok{=}\NormalTok{
    ↪ os.path.join({}\StringTok{"datas"}\NormalTok{, dataset)}
\NormalTok{    dirs }\OperatorTok{=}\NormalTok{
    \StringTok{"preprocessing"}\NormalTok{: os.path.join(base,
    ↪ }\StringTok{"preprocessing"}\NormalTok{),}
```

```

\StringTok{"embeddings"}\NormalTok{: os.path.join(base,
↪ }\StringTok{"embeddings"}\NormalTok{)},}
\StringTok{"results"}\NormalTok{: os.path.join(base,
↪ }\StringTok{"results"}\NormalTok{)},}
\StringTok{"graphics"}\NormalTok{: os.path.join(base,
↪ }\StringTok{"graphics"}\NormalTok{)},}
\StringTok{"reports"}\NormalTok{: os.path.join(base,
↪ }\StringTok{"reports"}\NormalTok{)},}
\NormalTok{{
↪ }\NormalTok{}}
\ControlFlowTok{for}\NormalTok{ d }\KeywordTok{in}\NormalTok{ dirs.values():}
\NormalTok{{
↪ os.makedirs(d,
↪ exist_ok)\OperatorTok{=}\VariableTok{True}\NormalTok{)}}
\ControlFlowTok{return}\NormalTok{ dirs}

```

Tre cose da notare, con lo sguardo di chi ha già letto la Parte II. Primo: la validazione (`if dataset not in datasets`) è l'unico controllo esplicito di correttezza su questo parametro in tutto il progetto — se non ci fosse, un nome di dataset sbagliato produrrebbe silenziosamente una nuova cartella `datas/<nomesbagliato>/` invece di un errore, perché `os.path.join` non sa cosa sia un dataset valido, sa solo concatenare stringhe. Secondo: `os.makedirs(d, exist_ok=True)` crea le cartelle se non esistono e non solleva errore se esistono già — l'opposto del comportamento di default di una `mkdir` che fallirebbe su una cartella esistente. Terzo: la funzione ha un **side-effect sul filesystem** (crea cartelle) dentro quella che sembra, dal nome, una semplice funzione “getter” — nulla nella firma lo segnala.

[Fatto] Le quattro funzioni `delete_files_*` (righe 179-209) condividono tutte la stessa struttura: ricevono una cartella, elencano i file al suo interno con `os.listdir()`, e cancellano quelli il cui nome contiene una delle stringhe di un elenco di pattern concordato a mano (per esempio `["model_performance", "_y_true", "_y_score", ...]` per `delete_files_results`, riga 196). **[Interpretazione]** Questo significa che la sicurezza di questa pulizia dipende interamente dalla precisione di quegli elenchi di stringhe: un file che finisse per coincidenza per contenere una di quelle sottostringhe nel nome, anche se non prodotto da questo progetto, verrebbe cancellato senza distinzione — non c'è un controllo di formato o di provenienza, solo un controllo su una sottostringa nel nome del file.

20.3 Le nove funzioni di plotting

[Fatto] Le restanti circa 200 righe sono nove funzioni di plotting (`plot_data_heatmap`, `plot_umap`, `plot_boxplots`, `plot_roc_comparison`, `plot_confusion`, `plot_metric_comparison`, `plot_mean_ci`, `plot_family_comparison`, `plot_error_rates`, `plot_feature_deviation` — dieci, in realtà, contandole con cura), tutte costruite sullo stesso scheletro: ricevono dati già pronti (un `DataFrame`, un dizionario di risultati), costruiscono una figura con `matplotlib/seaborn`, e la salvano chiamando `save_figure()` (righe 157-159):

```

\KeywordTok{def}\NormalTok{ save\_figure(fig, path\_no\_ext):}
\NormalTok{{
↪ fig.savefig()\SpecialStringTok{f"}\SpecialCharTok{\}\}\NormalTok{path\_}
↪ no\_ext}\SpecialCharTok{\}\}\SpecialStringTok{.png"}\NormalTok{{
↪ dpi}\OperatorTok{=}\DecValTok{300}\NormalTok{{
↪ bbox\_inches}\OperatorTok{=}\StringTok{"tight"}\NormalTok{{
↪ facecolor}\OperatorTok{=}\StringTok{"white"}\NormalTok{)}}
\NormalTok{{
↪ fig.savefig()\SpecialStringTok{f"}\SpecialCharTok{\}\}\NormalTok{path\_}
↪ no\_ext}\SpecialCharTok{\}\}\SpecialStringTok{.pdf"}\NormalTok{{
↪ bbox\_inches}\OperatorTok{=}\StringTok{"tight"}\NormalTok{{
↪ facecolor}\OperatorTok{=}\StringTok{"white"}\NormalTok{)}}

```

Ogni grafico del progetto esiste quindi sempre in due formati — PNG a 300 DPI e PDF vettoriale — con un'unica funzione condivisa, non una duplicazione riga per riga in ciascuna delle nove

funzioni.

[Fatto] La funzione più interessante di questo gruppo, non per la grafica che produce ma per la logica che contiene, è `get_model_palette()` (righe 162-174):

```
\KeywordTok{def}\NormalTok{ get\_model\_palette(model\_names):}
\NormalTok{    family\_members }\OperatorTok{=}\NormalTok{ }\{\}
\ControlFlowTok{for}\NormalTok{ name }\KeywordTok{in}\NormalTok{ model\_names:}
\NormalTok{    family }\OperatorTok{=}\NormalTok{ MODEL\_FAMILY.get(name,}
↪ \StringTok{"general{-}purpose"}\NormalTok{)}
\NormalTok{    family\_members.setdefault(family, []).append(name)}
\NormalTok{    palette }\OperatorTok{=}\NormalTok{ }\{\}
\ControlFlowTok{for}\NormalTok{ family, members }\KeywordTok{in}\NormalTok{ }
↪ family\_members.items():}
\NormalTok{    base\_color }\OperatorTok{=}\NormalTok{ FAMILY\_COLORS.get(family,}
↪ \StringTok{"#888888"}\NormalTok{)}
\NormalTok{    shades }\OperatorTok{=}\NormalTok{ sns.light\_palette(base\_color,}
↪ n\_colors)\OperatorTok{=}\BuiltInTok{len}\NormalTok{(members) }\OperatorTok{+}
↪ \DecValTok{1}\NormalTok{ }\DecValTok{1}\NormalTok{ }\}
\ControlFlowTok{for}\NormalTok{ name, shade }\KeywordTok{in}
↪ \BuiltInTok{zip}\NormalTok{(members, shades):}
\NormalTok{    palette[name] }\OperatorTok{=}\NormalTok{ shade}
\ControlFlowTok{return}\NormalTok{ palette}
```

Raggruppa prima i modelli per famiglia, poi genera per ciascuna famiglia tante sfumature del suo colore base quanti sono i suoi membri (`sns.light_palette`, scartando la prima sfumata più chiara con `[1:]` perché troppo simile allo sfondo bianco dei grafici), e infine associa ogni modello alla propria sfumatura con `zip()` (capitolo 8, uno dei due usi di `zip` nel progetto). Il risultato: modelli della stessa famiglia sono visivamente imparentati (stessa tonalità), ma distinguibili fra loro (sfumatura diversa) — un dettaglio di design che rende leggibile, a colpo d’occhio, la distinzione generalista/biomedico su cui si basa la seconda domanda di ricerca del progetto (capitolo 6.3).

RIFERIMENTO AL CODICE — le altre otto funzioni di plotting seguono lo stesso schema di `save_figure()` + palette condivisa; non le leggiamo una per una qui — il riferimento completo di ognuna, con firma e parametri esatti, è in Appendice B. Le incontri comunque in azione, una alla volta, nei capitoli 21-27, dove ciascuna fase le richiama nel proprio contesto.

Interfaccia pubblica (sintesi)

Categoria	Nomi	Righe
Costanti di schema	<code>columns</code> , <code>num_cols</code> , <code>cat_cols</code> , <code>columns_diabetes130</code> , <code>num_cols_diabetes130</code> , <code>cat_cols_diabetes130</code>	10-35
Configurazione modelli	<code>models_ollama</code> , <code>models_medical</code> , <code>models_all</code> , <code>MODEL_FAMILY</code> , <code>FAMILY_COLORS</code>	38-61
Stato condiviso	<code>results</code> , <code>datasets</code>	67-74

Categoria	Nomi	Righe
Percorsi e pulizia	<code>get_output_dirs()</code> , <code>delete_files_embeddings()</code> , <code>delete_files_preprocessing()</code> , <code>delete_files_results()</code> , <code>delete_files_graphics()</code>	79-93, 179-209
Caricamento dati	<code>load_heart_disease()</code> , <code>load_diabetes130()</code>	96-132
Plotting	<code>save_figure()</code> , <code>get_model_palette()</code> , e nove funzioni <code>plot_*</code>	134-387

Errori tipici

Un `ValueError: Invalid dataset '...'` significa che `--dataset` (o il valore passato direttamente a una funzione) non è esattamente `"heart_disease"` o `"diabetes130"` — nessuna normalizzazione di maiuscole/minuscole o di spazi è applicata. Un grafico mancante o con colori inattesi, dopo aver aggiunto un modello (capitolo 19.3), è quasi sempre riconducibile a `FAMILY_COLORS` o `family_order` non aggiornati per una famiglia nuova.

Riepilogo

`function.py` non implementa nessuna fase della pipeline: fornisce a tutte le altre la configurazione dei modelli, la risoluzione dei percorsi di output, la pulizia dei file di run precedenti, e ogni funzione di plotting del progetto. `get_output_dirs()` nasconde un side-effect sul filesystem dietro un nome che suggerisce una semplice lettura; `get_model_palette()` è l'unica funzione del file con una logica non banale, e garantisce coerenza visiva fra famiglie di modelli in ogni grafico.

Domande di autoverifica

- 1. Perché `get_output_dirs()` non è una funzione “pura” nel senso stretto del termine, nonostante il nome suggerisca solo una lettura?** Perché, oltre a restituire un dizionario di percorsi, crea effettivamente le cartelle sul filesystem con `os.makedirs(..., exist_ok=True)` — un side-effect non visibile dalla sola firma della funzione.
- 2. A cosa serve scartare la prima sfumatura generata da `sns.light_palette()` in `get_model_palette()`?** Perché la prima sfumatura di una palette “chiara” tende a essere troppo simile allo sfondo bianco dei grafici del progetto, e quindi poco leggibile; scartarla (`[1:]`) garantisce che tutte le sfumature usate siano sufficientemente distinguibili dallo sfondo.
- 3. Se cancellare un file per errore capitasse per una coincidenza di nome, quale funzione ne sarebbe responsabile e perché?** Una delle quattro funzioni `delete_files_*`: cancellano qualunque file la cui cartella contenga una sottostringa di un elenco concordato a mano nel proprio nome, senza controllare che il file sia stato effettivamente prodotto da questo progetto.

MATERIALE PER LA TESI 1. La tabella dell'interfaccia pubblica sintetica di `function.py` — riusabile come base per la sezione “Materiali e metodi” che descrive i componenti condivisi del sistema. 2. La lettura commentata di `get_model_palette()`, con la spiegazione del raggruppamento per famiglia —

riusabile per motivare, nella tesi, la scelta di visualizzazione usata in tutti i grafici comparativi. 3. L'osservazione sulla fragilità delle funzioni `delete_files_*` basate su sottostringhe di nome file — riusabile come punto di discussione nella sezione critica sulla robustezza del codice.

Capitolo 21 — preprocessing.py: dalla tabella grezza al training set bilanciato

Obiettivi del capitolo

- Seguire, riga per riga, come un dataset grezzo diventa il training pool usato per generare gli embedding.
- Capire perché il codice bilancia le classi *prima* di codificare le feature, non dopo.
- Sapere esattamente cosa viene scartato, in questo file, e perché è rilevante per la Parte XI.

[Fatto] `preprocessing.py` (121 righe) è la fase 1 della pipeline (`main.py:34`), l'unica — insieme a `embedding.py` — il cui output raggiunge la fase successiva per valore in memoria, non solo su disco (capitolo 17.1).

21.1 Caricamento e unione sorgenti

[Fatto] `preprocessing_data(dataset="heart_disease")` (righe 19-70) è la funzione orchestratrice dell'intero file, l'unica chiamata direttamente da `main.py:34`. Le prime righe scelgono, in base al parametro `dataset`, quale funzione di caricamento usare (`load_heart_disease()` o `load_diabetes130()`, entrambe definite in `function.py` — non in questo file, nonostante il nome) e quali elenchi di colonne applicare:

```
\ControlFlowTok{if}\NormalTok{ dataset }\OperatorTok{==}  
  ↪ \StringTok{"diabetes130"}\NormalTok{:}  
\NormalTok{ datasetChoosen }\OperatorTok{=}\NormalTok{  
  ↪ load\_diabetes130(sample\_size)\OperatorTok{=}\NormalTok{sample\_size}  
\NormalTok{ target\_col }\OperatorTok{=}\StringTok{"readmitted"}  
\NormalTok{ num\_cols\_used }\OperatorTok{=}\NormalTok{ num\_cols\_diabetes130}  
\NormalTok{ cat\_cols\_used }\OperatorTok{=}\NormalTok{ cat\_cols\_diabetes130}  
\ControlFlowTok{else}\NormalTok{:}  
\NormalTok{ datasetChoosen }\OperatorTok{=}\NormalTok{ load\_heart\_disease()  
\NormalTok{ target\_col }\OperatorTok{=}\StringTok{"num"}  
\NormalTok{ num\_cols\_used }\OperatorTok{=}\NormalTok{ num\_cols}  
\NormalTok{ cat\_cols\_used }\OperatorTok{=}\NormalTok{ cat\_cols}
```

Nota il refuso — reale, non un errore di trascrizione di questo libro — nel nome della variabile `datasetChoosen`: convenzione a cammello (*camelCase*) in un file che altrove usa sistematicamente lo *snake_case* (`num_cols_used`, `target_col`). Non ha conseguenze funzionali, ma è un'incoerenza di stile verificabile leggendo il file.

[Fatto] Segue lo split fra feature e target (riga 39-41) e lo split fra addestramento e test (riga 43):

```

\NormalTok{X }\OperatorTok{=}\NormalTok{ datasetChooosen.drop(target\_col,
↪ axis}\OperatorTok{=}\DecValTok{1}\NormalTok{)}}
\NormalTok{y }\OperatorTok{=}\NormalTok{ datasetChooosen[target\_col]}
\NormalTok{y }\OperatorTok{=}\NormalTok{ (y }\OperatorTok{\textgreater}\NormalTok{)}
↪ \DecValTok{0}\NormalTok{).astype(\BuiltInTok{int}\NormalTok{)}}
\NormalTok{X\_train, X\_test, y\_train, y\_test }\OperatorTok{=}\NormalTok{
↪ train\_test\_split(X, y, test\_size}\OperatorTok{=}\FloatTok{0.2}\NormalTok{,
↪ random\_state}\OperatorTok{=}\DecValTok{42}\NormalTok{,
↪ stratify}\OperatorTok{=}\NormalTok{y)}}

```

`stratify=y` garantisce che la proporzione fra le due classi (capitolo 4.2) sia preservata sia nel training set sia nel test set — senza, uno split casuale su un dataset sbilanciato potrebbe per sfortuna concentrare quasi tutta la classe minoritaria in una sola delle due parti.

ATTENZIONE — `X_test` e `y_test`, calcolati in questa riga, **non vengono mai più usati in nessuna parte di questo file, né di alcun altro file del progetto** — verificato con lettura completa di tutti i moduli. La funzione restituisce solo `X_train_bal`, `y_train_bal` (riga 70), derivati esclusivamente da `X_train`/`y_train`. Il 20% di dati messo da parte da questa riga è, a tutti gli effetti, calcolato e scartato. Il capitolo 33 lo inquadra nel contesto della validazione corretta, il capitolo 51 lo riprende come limite metodologico.

21.2 Imputazione, SMOTENC, codifica: tre trasformazioni, tre scopi diversi

[Fatto] `impute_raw()` (righe 76-82) sostituisce i valori mancanti con la mediana per le colonne numeriche e con la moda per le colonne categoriali:

```

\KeywordTok{def}\NormalTok{ impute\_raw(X, num\_cols, cat\_cols):}
\NormalTok{    X }\OperatorTok{=}\NormalTok{ X.copy()}
    \ControlFlowTok{for}\NormalTok{ col }\KeywordTok{in}\NormalTok{ num\_cols:}
\NormalTok{    X[col] }\OperatorTok{=}\NormalTok{ X[col].fillna(X[col].median())}
    \ControlFlowTok{for}\NormalTok{ col }\KeywordTok{in}\NormalTok{ cat\_cols:}
\NormalTok{    X[col] }\OperatorTok{=}\NormalTok{
↪ X[col].fillna(X[col].mode().iloc[}\DecValTok{0}\NormalTok{])}
    \ControlFlowTok{return}\NormalTok{ X}

```

`X = X.copy()` alla prima riga evita di modificare il DataFrame originale passato come argomento — senza questa copia esplicita, `X[col] = ...` modificherebbe l'oggetto del chiamante, un rischio concreto della tipizzazione dinamica (capitolo 7.2): niente, a livello di firma, segnala se una funzione modifica il proprio argomento o ne restituisce una copia.

[Fatto] `balance_classes()` (righe 84-92) applica SMOTENC — non SMOTE ordinario — sulle feature grezze, non su una loro codifica numerica:

```

\KeywordTok{def}\NormalTok{ balance\_classes(X, y, cat\_cols):}
\NormalTok{    cat\_idx }\OperatorTok{=}\NormalTok{ [X.columns.get\_loc(c)
↪ }\ControlFlowTok{for}\NormalTok{ c }\KeywordTok{in}\NormalTok{ cat\_cols]}
\NormalTok{    smote\_nc }\OperatorTok{=}\NormalTok{
↪ SMOTENC(categorical\_features}\OperatorTok{=}\NormalTok{cat\_idx,
↪ random\_state}\OperatorTok{=}\DecValTok{42}\NormalTok{)}}
\NormalTok{    X\_res, y\_res }\OperatorTok{=}\NormalTok{ smote\_nc.fit\_resample(X,
↪ y)}
    \ControlFlowTok{return}\NormalTok{ X\_res, y\_res}

```

[Livello: teoria consolidata del settore] SMOTE (Synthetic Minority Over-sampling Technique) genera record sintetici della classe minoritaria interpolando fra vicini reali nello spazio

delle feature. La variante SMOTENC estende l'idea alle feature *categoriali* (Nominal and Continuous): per le colonne numeriche interpola come SMOTE ordinario, per le colonne categoriali assegna il valore più frequente fra i vicini usati per l'interpolazione, invece di un valore intermedio privo di senso (un'interpolazione fra “maschio” e “femmina” non produce una categoria valida). **[Interpretazione]** Applicare SMOTENC *prima* di qualunque codifica numerica, sulle feature ancora leggibili (età in anni, non uno z-score), è precisamente ciò che rende possibile, più avanti, convertire anche i record sintetici in frasi di linguaggio naturale con `record_to_text_*`() (capitolo 22.1): un record sintetico che fosse già stato scalato e codificato non avrebbe più valori interpretabili da mettere in una frase.

21.3 Cosa viene salvato e perché

[Fatto] Dopo il bilanciamento, `preprocessing_data()` costruisce anche una versione *codificata* dei dati — scalata con `StandardScaler` e one-hot-encoded con `OneHotEncoder` (`build_encoder()`, righe 94-105) — ma **[Fatto]** questa versione codificata è usata *solo* per il grafico UMAP (`plot_umap()`, riga 58, capitolo 38) e per la heatmap di correlazione (`plot_data_heatmap()`, riga 62): non alimenta in alcun modo la generazione di embedding testuali, che lavora sempre sulle feature grezze bilanciate (`X_train_bal`), non su questa versione codificata.

[Fatto] Un secondo output, distinto da quello codificato, è salvato per la tracciabilità degli errori (righe 66-68):

```
\NormalTok{x_train_bal.reset_index(drop=True)\OperatorTok{=} \VariableTok{True}\NormalTok{to_}
↪ k{}.to_csv()
\NormalTok{os.path.join(dirs[0]\StringTok{"preprocessing"}\NormalTok{[,
↪ ]\StringTok{"X_train_raw.csv"}\NormalTok{[,
↪ index]\OperatorTok{=} \VariableTok{False}
\NormalTok{]}
```

`X_train_raw.csv` è la versione leggibile delle feature bilanciate, allineata riga per riga con l'ordine in cui verranno poi trasformate in testo ed embeddate — è il file che il capitolo 25 rilegge per ricondurre un errore di classificazione al record clinico originale (o sintetico) che lo ha causato.

Interfaccia pubblica

Funzione	Parametri	Ritorna	Effetti collaterali
<code>preprocessing_data(dataset="heart_disease")</code>		<code>x_train_bal</code> , <code>y_train_bal</code> (DataFrame, Series)	Crea grafici UMAP/heatmap; salva <code>.npy</code> e <code>X_train_raw.csv</code> su disco
<code>impute_raw(X, num_cols, cat_cols)</code>	DataFrame, elenchi colonne	Copia di X con mancanti imputati	Nessuno (lavora su una copia)
<code>balance_classes(X, y, cat_cols)</code>	Feature, target, colonne categoriali	<code>X_res</code> , <code>y_res</code> bilanciati	Nessuno
<code>build_encoder(X, y, num_cols, cat_cols)</code>	Feature bilanciate, colonne	<code>ColumnTransformer</code> addestrato	Nessuno

Funzione	Parametri	Ritorna	Effetti collaterali
<code>data_processed(X_train, y_train, preprocessor, num_cols, cat_cols)</code>	<code>Feature, encoder</code>	DataFrame codificato con colonna <code>target</code>	Nessuno
<code>save_data_processed(X_train_emb, preprocessing_dir)</code>	<code>DataFrame codificato, percorso</code>	Nessuno	Salva 2 file <code>.npy</code>

Errori tipici

Un `KeyError` sul nome di una colonna, in questa fase, indica quasi sempre un disallineamento fra `num_cols/cat_cols` (di `function.py`) e le colonne effettivamente presenti nel DataFrame caricato — per esempio se `load_diabetes130()` cambiasse l'elenco di colonne senza aggiornare `num_cols_diabetes130/cat_cols_diabetes130` di conseguenza. Un `ValueError` da `SMOTENC` segnala tipicamente che `cat_idx` (gli indici di colonna categoriali) non corrisponde più alla struttura reale di `X`, per esempio se l'ordine delle colonne fosse cambiato altrove.

Riepilogo

`preprocessing.py` carica i dati grezzi, imputa i valori mancanti, bilancia le classi con `SMOTENC` sulle feature ancora leggibili — condizione necessaria perché anche i record sintetici possano diventare testo — e produce, come output visibile alla fase successiva, solo `X_train_bal/y_train_bal`. Una versione codificata numericamente esiste ma serve solo per i grafici di visualizzazione; il 20% di dati riservato al test non viene mai più usato in nessun punto del progetto.

Domande di autoverifica

- 1. Perché `SMOTENC` viene applicato sulle feature grezze e non su una loro versione già codificata numericamente?** Perché i record sintetici generati da `SMOTENC` devono restare convertibili in frasi di linguaggio naturale nella fase successiva (capitolo 22): un record già scalato e one-hot-encoded non avrebbe più valori interpretabili (età in anni, categoria testuale) da inserire in una descrizione testuale.
- 2. Cosa succede, di fatto, al 20% di dati messo da parte da `train_test_split` alla riga 43 di `preprocessing.py`?** Viene calcolato ma non restituito né salvato da nessuna parte: `preprocessing_data()` restituisce solo i dati derivati dall'80% di training, e nessun altro file del progetto fa riferimento a `X_test/y_test`.
- 3. A cosa serve la versione codificata numericamente (`data_processed()`), se non alimenta la generazione di embedding?** Serve esclusivamente ai due grafici di visualizzazione della fase di preprocessing — la proiezione UMAP e la heatmap di correlazione — entrambi bisognosi di feature numeriche scalate, a differenza della pipeline di embedding che lavora sempre sulle feature grezze.

MATERIALE PER LA TESI 1. La spiegazione di `SMOTENC` applicato precodifica, con la motivazione esplicita legata alla convertibilità in testo — riusabile in “Materiali e metodi” per giustificare l'ordine delle trasformazioni nella pipeline. 2. L'osservazione verificata sul test set calcolato e mai usato (`X_test, y_test`) — riusabile, con il riferimento esatto di riga, nella sezione “Discussione e limiti”. 3. La

distinzione fra la versione grezza bilanciata (usata per il testo) e quella codificata (usata solo per i grafici) — riusabile come chiarimento tecnico per prevenire un fraintendimento comune su cosa “vede” davvero il classificatore finale.

Capitolo 22 — embedding.py: tabellare → testo → vettore

Obiettivi del capitolo

- Leggere per intero le due funzioni che convertono un record clinico in una frase.
- Capire come il file coordina, in parallelo, quattro chiamate a Ollama e tre a Hugging Face.
- Sapere quali variabili d'ambiente legge questo file, e con quale effetto esatto sul suo comportamento.

[Fatto] `embedding.py` (209 righe) è la fase 2 (`main.py:37`), il secondo file più lungo del progetto dopo `function.py`, e l'unico che dialoga con servizi esterni al processo Python (Ollama via rete locale, Hugging Face via download di modelli).

22.1 `record_to_text_*`() riga per riga

[Fatto] `embedding.py:14-19` è il punto di ingresso del file, chiamato da `main.py:37`:

```
\KeywordTok{def}\NormalTok{ embeddings(X, y,
↪ dataset)\OperatorTok{=}\StringTok{"heart\_disease"}\NormalTok{):}
\NormalTok{    dirs }\OperatorTok{=}\NormalTok{ get\_output\_dirs(dataset)}
\NormalTok{    record\_to\_text }\OperatorTok{=}\NormalTok{
↪ record\_to\_text\_diabetes130 }\ControlFlowTok{if}\NormalTok{ dataset
↪ }\OperatorTok{==}\StringTok{"diabetes130"} \ControlFlowTok{else}\NormalTok{
↪ record\_to\_text\_heart\_disease}
\NormalTok{    texts }\OperatorTok{=}\NormalTok{ [record\_to\_text(r)
↪ }\ControlFlowTok{for}\NormalTok{ \_, r }\KeywordTok{in}\NormalTok{ X.iterrows()]}
\NormalTok{    generate\_all\_embeddings(texts, np.asarray(y),
↪ dirs)\StringTok{"embeddings"}\NormalTok{[]}}
```

Già vista al capitolo 9.3 (scelta a runtime fra due funzioni con nome diverso, non overload) e al capitolo 8.2 (list comprehension su `X.iterrows()`): quattro righe che, insieme, trasformano l'intero `DataFrame` in una lista di frasi, pronta per l'embedding.

[Fatto] `record_to_text_heart_disease()` (righe 43-59) costruisce la frase concatenando una parte fissa di testo con un valore preso dal record, per ciascuna delle 12 feature (età e sesso condivise in una singola parte iniziale):

```
\KeywordTok{def}\NormalTok{ record\_to\_text\_heart\_disease(row):}
\NormalTok{    sex }\OperatorTok{=}\NormalTok{
↪ \_fmt\_bool(row)\StringTok{"sex"}\NormalTok{, }\StringTok{"Male"}\NormalTok{,
↪ }\StringTok{"Female"}\NormalTok{)}
\NormalTok{    parts }\OperatorTok{=}\NormalTok{ []}
```

```

\SpecialStringTok{f"}\SpecialCharTok{{}\NormalTok{sex}\SpecialCharTok{{}}\Sp
↪ ecialStringTok{ patient,
↪ }\SpecialCharTok{{}\NormalTok{_fmt\_num(row[]\StringTok{\textquotesingl
↪ e}\age\textquotesingle{} }\NormalTok{ },
↪ ndigits}\OperatorTok{=}\DecValTok{0}\NormalTok{)} }\SpecialCharTok{{}}\Spec
↪ ialStringTok{ years
↪ old"}\NormalTok{ },
\SpecialStringTok{f"chest pain type: }\SpecialCharTok{{}\NormalTok{_fmt\_ca
↪ t(row[]\StringTok{\textquotesingle{}cp\textquotesingle{} }\NormalTok{ },
↪ CP\_LABELS)} }\SpecialCharTok{{}}\SpecialStringTok{"}\NormalTok{ },
\SpecialStringTok{f"resting blood pressure:
↪ }\SpecialCharTok{{}\NormalTok{_fmt\_num(row[]\StringTok{\textquotesingl
↪ e}\trestbps\textquotesingle{} }\NormalTok{ },
↪ }\StringTok{\textquotesingle{} mm Hg\textquotesingle{} }\NormalTok{ },
↪ ndigits}\OperatorTok{=}\DecValTok{0}\NormalTok{)} }\SpecialCharTok{{}}\Spec
↪ ialStringTok{"}\NormalTok{ },
\NormalTok{ {
    ... }
\NormalTok{ }
\ControlFlowTok{return} \StringTok{"", "}\NormalTok{.join(parts)}

```

Tre funzioni ausiliarie con prefisso underscore (capitolo 10.3) fanno il lavoro di formattazione: `_fmt_num()` (righe 28-31) arrotonda un numero e vi accoda un'unità di misura opzionale; `_fmt_cat()` (righe 33-36) traduce un codice numerico in un'etichetta leggibile usando un dizionario come `CP_LABELS` (riga 23: `{1: "typical angina", 2: "atypical angina", ...}` — gli stessi codici standard del dataset UCI, capitolo 27); `_fmt_bool()` (righe 38-41) traduce un flag binario in due parole a scelta. **[Fatto]** Tutte e tre condividono lo stesso primo controllo, `if pd.isna(value): return "not recorded"` — un valore mancante, a questo punto della pipeline, non dovrebbe più esistere (è stato imputato al capitolo 21.2), ma la funzione lo gestisce comunque esplicitamente, un margine di sicurezza che non costa nulla e previene un NaN che finirebbe altrimenti scritto letteralmente nella frase.

[Fatto] `record_to_text_diabetes130()` (righe 64-85) è strutturalmente più semplice: usa una sola funzione ausiliaria, `_fmt_raw()` (riga 61-62, `"not recorded" if pd.isna(value) else str(value)`), che non traduce alcun codice — i valori di `Diabetes130` ("Caucasian", "[40-50]", "Norm") sono già stringhe leggibili nel file sorgente, a differenza dei codici numerici di `Heart Disease`.

ATTENZIONE — nessuna delle due funzioni di conversione menziona mai esplicitamente il *target* (`num` o `readmitted`): la frase descrive solo le feature, mai l'etichetta che il classificatore dovrà prevedere. È corretto che sia così — includere l'etichetta nel testo sarebbe una forma di leakage grossolana, praticamente equivalente a scrivere la risposta dentro la domanda — ma vale la pena verificarlo esplicitamente leggendo entrambe le funzioni, non darlo per scontato.

22.2 Generazione batch verso Ollama: retry, semaforo

[Fatto] `generate_embeddings_batch()` (righe 103-137) invia le frasi a Ollama a gruppi di 16 (`batch_size=16`, default), non tutte insieme:

```

\KeywordTok{def}\NormalTok{ generate\_embeddings\_batch(model\_name, texts,
↪ batch\_size}\OperatorTok{=}\DecValTok{16}\NormalTok{ },
↪ max\_retries}\OperatorTok{=}\DecValTok{5}\NormalTok{ },
\NormalTok{ {
↪ retry\_delay}\OperatorTok{=}\FloatTok{2.0}\NormalTok{ },
↪ inter\_batch\_delay}\OperatorTok{=}\FloatTok{0.3}\NormalTok{ ):}
\NormalTok{ {
    client }\OperatorTok{=}\NormalTok{ Client()}

```

```
\NormalTok{    num\_batches }\OperatorTok{=}\NormalTok{
  ↪  ({)\BuiltInTok{len}\NormalTok{({texts) }\OperatorTok{+}\NormalTok{{ batch\_size
  ↪  }\OperatorTok{{-}} \DecValTok{1}\NormalTok{{) }\OperatorTok{//}\NormalTok{{
  ↪  batch\_size}
\NormalTok{{    ...}}
```

`(len(texts) + batch_size - 1) // batch_size` è un idiomma comune per calcolare “quanti gruppi da `batch_size` servono per coprire `len(texts)` elementi, arrotondando per eccesso” usando solo divisione intera — evita di dover importare `math.ceil` per un calcolo così semplice. Il ciclo di retry con `_ollama_semaphore` e `raise ... from` è già stato letto per intero al capitolo 11.1 e al capitolo 12.2: qui vale la pena solo notare `time.sleep(inter_batch_delay)` alla fine di ogni batch riuscito (riga 135) — una pausa fissa di 0.3 secondi, aggiuntiva rispetto al backoff del retry, per non sovraccaricare comunque il server anche quando tutto funziona al primo tentativo.

22.3 Generazione verso Hugging Face: autenticazione, modalità offline

[Fatto] `generate_embeddings_hf()` (righe 139-180) gestisce i tre modelli biomedici, con una struttura in tre passaggi commentati in italiano direttamente nel codice sorgente (`embedding.py:142,145,153` — una delle poche tracce di commenti non in inglese in tutto il progetto):

```
\NormalTok{{hf\_token }\OperatorTok{=}\NormalTok{{
  ↪  os.getenv({)\StringTok{"HF\_READ\_TOKEN"}\NormalTok{{)}}
\ControlFlowTok{{if}\NormalTok{{ hf\_token:}
\NormalTok{{    login(token)\OperatorTok{=}\NormalTok{{hf\_token)}}
\NormalTok{{...}
\NormalTok{{is\_offline }\OperatorTok{=}\NormalTok{{
  ↪  os.getenv({)\StringTok{"OFFLINE\_MODE"}\NormalTok{{, }\StringTok{"0"}\NormalTok{{)}}
  ↪  }\OperatorTok{{==}\StringTok{"1"}}
\ControlFlowTok{{if}\NormalTok{{ is\_offline:}
\NormalTok{{    os.environ[)\StringTok{"TRANSFORMERS\_OFFLINE"}\NormalTok{{}}
  ↪  }\OperatorTok{{=}\StringTok{"1"}}
\NormalTok{{    os.environ[)\StringTok{"HF\_DATASETS\_OFFLINE"}\NormalTok{{}}
  ↪  }\OperatorTok{{=}\StringTok{"1"}}
\NormalTok{{    os.environ[)\StringTok{"HF\_HUB\_OFFLINE"}\NormalTok{{}}
  ↪  }\OperatorTok{{=}\StringTok{"1"}}
```

`os.getenv("HF_READ_TOKEN")` restituisce `None` se la variabile non è impostata, non solleva un errore — l’`if hf_token:` successivo tratta `None` come falso (capitolo 7.2: nessun tipo dichiarato garantisce che `hf_token` sia una stringa o `None`, lo si scopre solo a runtime). Se `OFFLINE_MODE=1`, il codice imposta **tre variabili d’ambiente diverse** (non solo il flag letto), ciascuna riconosciuta da una parte diversa dell’ecosistema Hugging Face (`transformers`, `datasets`, `huggingface_hub`) — un dettaglio che rivela quanto la modalità offline non sia un singolo interruttore, ma tre interruttori paralleli, storicamente introdotti in momenti diversi della libreria.

[Fatto] Il caricamento del modello vero e proprio è una sola riga, `model = SentenceTransformer(model_name, local_files_only=is_offline)` (riga 171), e la generazione degli embedding un’altra, `model.encode(texts, show_progress_bar=True, convert_to_numpy=True)` (riga 178) — già anticipata al capitolo 5.2 per il meccanismo di tokenizzazione/encoder/pooling che nasconde.

Interfaccia pubblica

Funzione	Parametri principali	Ritorna
<code>embeddings(X, y, dataset="heart_disease")</code>	Feature, target, dataset	Nessuno (salva su disco)
<code>record_to_text_heart_disease(U(n))</code>	Una riga di DataFrame	Stringa
<code>record_to_text_diabetes130(V(m))</code>	Una riga di DataFrame	Stringa
<code>generate_embeddings_batch(model_name, texts, batch_size=16, max_retries=5, ...)</code>	Nome modello Ollama, lista di frasi	Lista di embedding
<code>generate_embeddings_hf(texts, model_name)</code>	Lista di frasi, nome modello HF	Array NumPy di embedding
<code>process_model(model, texts, labels, embeddings_dir)</code>	Config di un modello, frasi, etichette, percorso	Nessuno (salva su disco)
<code>generate_all_embeddings(texts, labels, embeddings_dir, max_workers=3)</code>	Frasi, etichette, percorso	Nessuno (orchestra i 7 modelli)

Errori tipici

Un `RuntimeError` con “batch N/M failed after 5 attempts” indica che Ollama non ha risposto correttamente per cinque tentativi consecutivi — quasi sempre perché il server non è in esecuzione o il modello richiesto non è stato scaricato (capitolo 16.1, incluso il caso specifico del nome sbagliato per `e5-base`). Un errore di autenticazione Hugging Face durante `generate_embeddings_hf()` segnala un `HF_READ_TOKEN` mancante o non valido in `.env` — ma solo se il modello richiede autenticazione; per modelli pubblici il codice procede comunque, stampando solo un avviso (riga 151).

Riepilogo

`embedding.py` traduce ogni record clinico in una frase con un template fisso specifico per dataset, poi genera i vettori corrispondenti coordinando quattro chiamate Ollama (serializzate da un semaforo, con `retry`) e tre chiamate Hugging Face (con autenticazione opzionale e tre variabili d’ambiente per la modalità offline) in un pool di tre thread. Nessuna delle due funzioni di conversione a testo include mai l’etichetta `target` nella frase generata.

Domande di autoverifica

- Perché `record_to_text_heart_disease()` ha bisogno di tre funzioni ausiliarie di formattazione, mentre `record_to_text_diabetes130()` ne usa una sola?** Perché le feature di Heart Disease sono codificate come numeri che richiedono una traduzione esplicita in etichette leggibili (`_fmt_cat`, `_fmt_bool`), mentre le feature usate di Diabetes130 sono già stringhe leggibili nel file sorgente, e richiedono solo la gestione dei valori mancanti (`_fmt_raw`).
- Cosa calcola esattamente `(len(texts) + batch_size - 1) // batch_size`, e perché non un più semplice `len(texts) // batch_size`?** Il numero di batch necessari per coprire tutti i testi, arrotondato per eccesso: `len(texts) // batch_size` da solo scarterebbe l’ultimo batch parziale se `len(texts)` non fosse un multiplo esatto di `batch_size`.
- Perché impostare `OFFLINE_MODE=1` modifica tre variabili d’ambiente diverse invece di una sola?** Perché l’ecosistema Hugging Face è composto da più librerie distinte

(`transformers`, `datasets`, `huggingface_hub`), ciascuna delle quali riconosce una propria variabile d'ambiente per forzare la modalità offline — non esiste un singolo interruttore condiviso fra tutte.

MATERIALE PER LA TESI 1. Il template di conversione tabellare→testo per entrambi i dataset, con le funzioni di formattazione — riusabile in “Materiali e metodi” per documentare esattamente come i record diventano testo. 2. L’osservazione verificata che nessuna frase include mai l’etichetta target — riusabile come argomento esplicito contro un possibile sospetto di data leakage nella conversione testuale. 3. Lo schema di gestione della modalità offline con le tre variabili d’ambiente — riusabile come nota tecnica per chi debba riprodurre l’ambiente sperimentale senza connessione di rete continua.

Capitolo 23 — `classification.py`: addestrare e validare

Obiettivi del capitolo

- Leggere per intero il ciclo che addestra e valuta un modello per ciascuno dei sette embedding.
- Capire esattamente come viene scelta la soglia di decisione, e perché è il punto più delicato di questo file.
- Sapere cosa viene salvato su disco al termine di questa fase, e da quali capitoli successivi verrà riletto.

[Fatto] `classification.py` (79 righe) è la fase 3 (`main.py:40`), il file più corto della pipeline principale ma quello che contiene la decisione metodologicamente più delicata dell'intero progetto (capitolo 23.2).

23.1 StratifiedKFold e perché “stratified” conta

[Fatto] `training_classifier()` (righe 9-80) itera su ciascuno dei sette modelli, ricarica i suoi embedding da disco (righe 13-14, il punto di ingresso del “passaggio via file” già visto al capitolo 17.1), e addestra un `LogisticRegression` con validazione a 5 fold:

```
\NormalTok{X }\OperatorTok{=}\NormalTokTok{
  ↪ np.load(os.path.join(dirs[\StringTok{"embeddings"}\NormalTokTok{
  ↪ model[\StringTok{\textquotesingle{}filename\textquotesingle{}}\NormalTokTok{
  ↪ allow\_pickle}\OperatorTok{=}\VariableTok{True}\NormalTokTok{
\NormalTokTok{y }\OperatorTok{=}\NormalTokTok{
  ↪ np.load(os.path.join(dirs[\StringTok{"embeddings"}\NormalTokTok{
  ↪ k{\textquotesingle{}filename\_label\textquotesingle{}}\NormalTokTok{
  ↪ allow\_pickle}\OperatorTok{=}\VariableTok{True}\NormalTokTok{
\NormalTokTok{kf }\OperatorTok{=}\NormalTokTok{
  ↪ StratifiedKFold(n\_splits)\OperatorTok{=}\DecValTok{5}\NormalTokTok{
  ↪ shuffle}\OperatorTok{=}\VariableTok{True}\NormalTokTok{
  ↪ random\_state}\OperatorTok{=}\DecValTok{42}\NormalTokTok{
\NormalTokTok{logisticReg }\OperatorTok{=}\NormalTokTok{
  ↪ LogisticRegression(max\_iter)\OperatorTok{=}\DecValTok{2000}\NormalTokTok{}
```

[Livello: teoria consolidata del settore] Un k-fold ordinario divide i dati in k parti uguali senza guardare l’etichetta: con un dataset sbilanciato (capitolo 4.2), questo rischia di produrre fold con proporzioni di classe molto diverse fra loro, o nel caso estremo un fold senza esemplari della classe minoritaria. Un k-fold **stratificato** garantisce che ogni fold mantenga, il più possibile, la stessa proporzione fra classi del dataset intero — esattamente ciò che serve quando, come per Diabetes130 (11.16% di classe positiva, capitolo 4.2), lo sbilanciamento è marcato.

[Fatto] Nota che `X` e `y`, qui, sono gli embedding **dopo SMOTENC** (capitolo 21.2) — non i dati originali sbilanciati. Lo `StratifiedKfold` opera quindi su un pool già artificialmente riequilibrato dal bilanciamento sintetico, non sulla distribuzione reale delle classi: un fatto da tenere presente quando si interpreta “quanto conta la stratificazione qui” — conta comunque, ma su un problema già reso più facile dal bilanciamento a monte.

23.2 La soglia F1-ottima per fold

[Fatto] Il cuore del ciclo (righe 22-46) allena il modello sul training fold, poi cerca la soglia migliore sul fold di validazione:

```
\ControlFlowTok{for}\NormalTok{ train\_idx, val\_idx }\KeywordTok{in}\NormalTok{
  ↪ kf.split(X, y):}
\NormalTok{    X\_train, X\_val }\OperatorTok{=}\NormalTok{ X[train\_idx],
  ↪ X[val\_idx]}
\NormalTok{    y\_train, y\_val }\OperatorTok{=}\NormalTok{ y[train\_idx],
  ↪ y[val\_idx]}
\NormalTok{    logisticReg.fit(X\_train, y\_train)}
\NormalTok{    y\_score }\OperatorTok{=}\NormalTok{ logisticReg.predict\_proba(X\_val)[: , }\DecValTok{1}\NormalTok{[]}]
\NormalTok{    precision, recall, thresholds }\OperatorTok{=}\NormalTok{ precision\_recall\_curve(y\_val, y\_score)}
\NormalTok{    f1\_scores }\OperatorTok{=}\NormalTok{ (precision[:]\OperatorTok{*}\DecValTok{1}\NormalTok{[]}]
  ↪ }\OperatorTok{*}\NormalTok{ recall[:]\OperatorTok{*}\DecValTok{1}\NormalTok{[]}]
  ↪ }\OperatorTok{/}\NormalTok{ (precision[:]\OperatorTok{*}\DecValTok{1}\NormalTok{[]}]
  ↪ }\OperatorTok{+}\NormalTok{ recall[:]\OperatorTok{*}\DecValTok{1}\NormalTok{[]}]
  ↪ }\OperatorTok{+}\NormalTok{ }\FloatTok{1e-6}\NormalTok{[]}]
\NormalTok{    best\_idx }\OperatorTok{=}\NormalTok{ f1\_scores.argmax()}
\NormalTok{    tau }\OperatorTok{=}\NormalTok{ thresholds[best\_idx]}
\NormalTok{    y\_pred }\OperatorTok{=}\NormalTok{ (y\_score
  ↪ }\OperatorTok{>}\NormalTok{ tau)}
  ↪ tau).astype(int)\NormalTok{[]}]
```

`precision_recall_curve()` restituisce precisione e recall per ogni soglia possibile, con un punto in più rispetto all’elenco delle soglie stesse (il punto finale, `recall=0`, non ha una soglia associata) — da cui il taglio `[:-1]` su precisione e recall prima di calcolare l’F1 per ciascuna soglia candidata. Il termine `+ 1e-6` al denominatore evita una divisione per zero se sia precisione sia recall fossero nulle per una soglia candidata. `best_idx = f1_scores.argmax()` sceglie la soglia che avrebbe massimizzato l’F1 su **questo stesso fold di validazione**.

ATTENZIONE — questo è il punto esatto già anticipato al capitolo 2.3 e al capitolo 33: la soglia `tau` è scelta guardando `y_val` — le etichette vere del fold di validazione — e poi la stessa soglia viene usata per calcolare `y_pred` su cui, poche righe sotto, si misurano accuratezza e F1 riportati come prestazione del modello **su quello stesso fold**. Non è data leakage nel senso di informazione che raggiunge l’addestramento del modello (`logisticReg.fit()` vede solo `X_train/y_train`, mai il fold di validazione) — è un ottimismo più sottile, limitato alla sola scelta della soglia, che gonfia leggermente F1 e accuratezza rispetto a una soglia fissata a priori o scelta su un fold separato di calibrazione. Il capitolo 51 lo tratta con tutto il rigore critico che merita.

23.3 Cosa viene salvato e a cosa serve dopo

[Fatto] Oltre alle metriche medie sui 5 fold (righe 47-56, salvate nel dizionario globale `results`, capitolo 8.3 e capitolo 19.2), il codice concatena le predizioni di *tutti* i fold in quattro array (righe 61-68):

```
\NormalTok{all\_val\_idx } \OperatorTok{=} \NormalTok{ np.concatenate(all\_val\_idx)}
\NormalTok{np.save(os.path.join(dirs[]\StringTok{"results"}\NormalTok{[},
  ↳ }\SpecialStringTok{f"}\SpecialCharTok{\{\}\NormalTok{model[]\StringTok{\textquotes
  ↳ ingle}\model\_name\textquotesingle{}\}\NormalTok{[]}\SpecialCharTok{\}\}\SpecialStri
  ↳ ngTok{\_y\_true.npy"}\NormalTok{)},
  ↳ all\_y\_true)}
\NormalTok{np.save(os.path.join(dirs[]\StringTok{"results"}\NormalTok{[},
  ↳ }\SpecialStringTok{f"}\SpecialCharTok{\{\}\NormalTok{model[]\StringTok{\textquotes
  ↳ ingle}\model\_name\textquotesingle{}\}\NormalTok{[]}\SpecialCharTok{\}\}\SpecialStri
  ↳ ngTok{\_y\_score.npy"}\NormalTok{)},
  ↳ all\_y\_scores)}
\NormalTok{np.save(os.path.join(dirs[]\StringTok{"results"}\NormalTok{[},
  ↳ }\SpecialStringTok{f"}\SpecialCharTok{\{\}\NormalTok{model[]\StringTok{\textquotes
  ↳ ingle}\model\_name\textquotesingle{}\}\NormalTok{[]}\SpecialCharTok{\}\}\SpecialStri
  ↳ ngTok{\_y\_pred.npy"}\NormalTok{)},
  ↳ all\_y\_preds)}
\NormalTok{np.save(os.path.join(dirs[]\StringTok{"results"}\NormalTok{[},
  ↳ }\SpecialStringTok{f"}\SpecialCharTok{\{\}\NormalTok{model[]\StringTok{\textquotes
  ↳ ingle}\model\_name\textquotesingle{}\}\NormalTok{[]}\SpecialCharTok{\}\}\SpecialStri
  ↳ ngTok{\_val\_idx.npy"}\NormalTok{)},
  ↳ all\_val\_idx)}
```

`all_val_idx` — gli indici di riga usati come validazione in ciascun fold, concatenati nell'ordine in cui i fold sono stati processati — è il file più importante per il capitolo 25: è il ponte che permette di ricondurre ogni previsione (giusta o sbagliata) al record originale in `X_train_raw.csv` (capitolo 21.3). Senza questo file, l'analisi degli errori del capitolo 25 non avrebbe modo di sapere *quale riga* del dataset originale corrisponde a ciascuna previsione salvata.

[Fatto] Perché questa concatenazione produca un risultato coerente — cioè perché l'elemento *i*-esimo di `all_y_true`, `all_y_score`, `all_y_pred` e `all_val_idx` si riferiscano davvero allo stesso record — è necessario che i quattro array vengano popolati nello stesso ordine, fold per fold, dentro lo stesso ciclo: **[Fatto]** verificato leggendo le righe 43-46, è esattamente così che il codice li costruisce, con un `append` per lista corrispondente ad ogni iterazione del ciclo prima della concatenazione finale.

Interfaccia pubblica

Funzione	Parametri	Effetti
<code>training_classifier(dataset_name="heart_disease")</code>	Numero di dataset	Popola results (side-effect globale, capitolo 19.2); salva 4 file .npy per modello + model_performance.csv ; genera il grafico di confronto metriche

[Fatto] È l'unica funzione pubblica del file: le sette variabili locali del ciclo (`X`, `y`, `kf`, `logisticReg`, `tmp_results`, ecc.) esistono solo dentro `training_classifier()`, non sono richiamabili da altri moduli.

Errori tipici

Un `IndexError` o una forma inattesa in `X[train_idx]` segnala quasi sempre un disallineamento fra il numero di righe di `X` (embedding) e di `y` (etichette) — capiterebbe se, per esempio, la generazione degli embedding (capitolo 22) fosse stata interrotta a metà per un modello e ripresa con un numero diverso di frasi. Un `f1_scores` interamente `NaN` o 0 per un intero fold segnalerebbe un fold senza esempi di una delle due classi — lo scenario che `StratifiedKFold` è pensato per evitare, e la cui comparsa indicherebbe un problema a monte nella stratificazione o nei dati stessi.

Riepilogo

`classification.py` allena una regressione logistica separata per ciascuno dei sette modelli, con validazione a 5 fold stratificati sugli embedding già bilanciati da SMOTENC. La soglia di decisione è scelta, fold per fold, massimizzando F1 sulle etichette dello stesso fold di validazione su cui viene poi misurata la prestazione — un punto di attenzione metodologica reale. Gli indici di validazione salvati da questo file sono il collegamento indispensabile fra ogni previsione e il record clinico che l’ha generata.

Domande di autoverifica

- 1. Perché `StratifiedKFold`, e non un `k-fold` ordinario, è la scelta giusta per `Diabetes130` in particolare?** Perché `Diabetes130` ha una classe positiva minoritaria (11.16%, capitolo 4.2): un `k-fold` ordinario rischierebbe di produrre fold con proporzioni di classe molto diverse dal dataset intero, o addirittura fold senza esempi della classe minoritaria — la stratificazione lo previene per costruzione.
- 2. In che senso la scelta della soglia `tau` non è “data leakage” nel senso classico, ma resta comunque un problema?** Perché il modello (`logisticReg.fit()`) non vede mai le etichette del fold di validazione durante l’addestramento — l’addestramento è pulito. Il problema riguarda solo la soglia: viene scelta massimizzando F1 sulle stesse etichette di validazione su cui poi si riporta la metrica, un ottimismo più contenuto ma reale.
- 3. Perché `all_val_idx` è indispensabile per il capitolo 25 (analisi degli errori)?** Perché è l’unico collegamento salvato fra una previsione (giusta o sbagliata) e la riga originale del dataset da cui proviene: senza questi indici, non ci sarebbe modo di sapere quale record clinico corrisponde a un falso positivo o falso negativo specifico.

MATERIALE PER LA TESI 1. La spiegazione formale della soglia F1-ottima per fold, con l’osservazione critica sull’ottimismo che introduce — riusabile parola per parola nella sezione “Discussione e limiti”, capitolo di riferimento 51. 2. La spiegazione di `StratifiedKFold` applicato dopo SMOTENC, con la precisazione che opera su un pool già bilanciato — riusabile in “Materiali e metodi” per descrivere con precisione il protocollo di validazione. 3. Lo schema del collegamento `val_idx` → record originale, con i quattro file salvati per modello — riusabile come base tecnica per la sezione che descrive la tracciabilità degli errori nella tesi.

Capitolo 24 — evaluation.py: bootstrap e grafici

Obiettivi del capitolo

- Leggere l'intera implementazione del bootstrap del progetto: sono quindici righe, non di più.
- Sapere leggere un intervallo di confidenza percentile, con la formula che lo produce.
- Vedere come i dati raccolti per sette modelli confluiscono in sei grafici diversi.

[Fatto] `evaluation.py` (83 righe) è la fase 4 (`main.py:43`): rilegge i risultati salvati da `classification.py`, li arricchisce con un bootstrap di 10.000 iterazioni, e produce la maggior parte dei grafici comparativi del progetto.

24.1 Il bootstrap in poche righe

[Fatto] L'intera logica del bootstrap sta in `bootstrap_metrics()` (righe 62-75):

```
\KeywordTok{def}\NormalTok{ bootstrap\_metrics(y\_true, y\_score, y\_pred,
↪  n\_iter}\OperatorTok{=}\DecValTok{10000}\NormalTok{,
↪  seed}\OperatorTok{=}\DecValTok{42}\NormalTok{)}:}
\NormalTok{    rng }\OperatorTok{=}\NormalTok{ np.random.default\_rng(seed)}
\NormalTok{    acc\_list, f1\_list, auc\_list }\OperatorTok{=}\NormalTok{ [], [], []}
\ControlFlowTok{for}\NormalTok{ \_ }\KeywordTok{in}
↪  \BuiltInTok{range}\NormalTok{(n\_iter):}
\NormalTok{    idx }\OperatorTok{=}\NormalTok{ rng.integers}\NormalTok{(\DecValTok{0}\NormalTok{, }\NormalTok{ }\BuiltInTok{len}\NormalTok{ }\NormalTok{(y\_true),}
↪  }\NormalTok{ }\BuiltInTok{len}\NormalTok{ }\NormalTok{(y\_true))}
\NormalTok{    yt, yp, ys }\OperatorTok{=}\NormalTok{ y\_true[idx], y\_pred[idx],}
↪  y\_score[idx]}
\NormalTok{    acc\_list.append(accuracy\_score(yt, yp))}
\NormalTok{    f1\_list.append(f1\_score(yt, yp,}
↪  average}\OperatorTok{=}\StringTok{"macro"}\NormalTok{))}
\NormalTok{    auc\_list.append(roc\_auc\_score(yt, ys))}
\ControlFlowTok{return}\NormalTok{ \{\}\StringTok{"acc"}\NormalTok{:}
↪  np.array(acc\_list), }\StringTok{"f1"}\NormalTok{: np.array(f1\_list),}
↪  }\StringTok{"auc"}\NormalTok{: np.array(auc\_list)\}}
```

[Livello: teoria consolidata del settore] L'idea del bootstrap è ricampionare *con reinserimento* dallo stesso insieme di predizioni già ottenuto — `rng.integers(0, len(y_true), len(y_true))` genera tanti indici casuali (con possibili ripetizioni) quanti sono i record originali — e ricalcolare la metrica su ciascun ricampionamento. Il risultato non è un singolo numero ma una **distribuzione** di 10.000 valori della stessa metrica: se il modello fosse instabile (piccole variazioni nei dati che cambiano molto il punteggio), questa distribuzione sarebbe ampia; se fosse stabile, sarebbe stretta. Nota che il bootstrap qui lavora sulle predizioni **già fatte**

(`y_true`, `y_score`, `y_pred` salvate da `classification.py`), non riallena il modello 10.000 volte — sarebbe computazionalmente proibitivo, e non è quello che questo bootstrap misura: misura l’incertezza della *stima della metrica*, non l’incertezza dell’addestramento.

[Fatto] `rng = np.random.default_rng(seed)` (riga 63) è l’idioma moderno di NumPy per la generazione di numeri casuali — un generatore locale legato esplicitamente a un seme, invece della vecchia interfaccia globale `np.random.seed(...)` seguita da chiamate a `np.random.randint(...)`. La differenza pratica: due chiamate indipendenti a `bootstrap_metrics()` con lo stesso `seed` producono sempre la stessa sequenza di ricampionamenti, senza che uno stato globale condiviso (capitolo 8.3) possa essere alterato da qualche altra parte del codice nel frattempo.

24.2 Intervalli di confidenza: come si leggono i grafici

[Fatto] `ci()` (righe 77-81) trasforma le 10.000 osservazioni bootstrap in un intervallo leggibile:

```
\KeywordTok{def}\NormalTok{ ci(a, alpha)\OperatorTok{=}\FloatTok{0.95}\NormalTok{:}}
\NormalTok{    low }\OperatorTok{=}\NormalTok{ np.percentile(a,}
↪  (\DecValTok{1}\OperatorTok{-}\NormalTok{alpha)\OperatorTok{/}\DecValTok{2}}
↪  \OperatorTok{*} \DecValTok{100}\NormalTok{)}
\NormalTok{    high }\OperatorTok{=}\NormalTok{ np.percentile(a,}
↪  (\DecValTok{1}\OperatorTok{+}\NormalTok{alpha)\OperatorTok{/}\DecValTok{2}}
↪  \OperatorTok{*} \DecValTok{100}\NormalTok{)}
\NormalTok{    mean }\OperatorTok{=}\NormalTok{ a.mean()}
\ControlFlowTok{return}\NormalTok{ mean, (low, high)}
```

Con $\alpha=0.95$, $(1-\alpha)/2 * 100 = 2.5$ e $(1+\alpha)/2 * 100 = 97.5$: l’intervallo di confidenza al 95% è, semplicemente, l’intervallo fra il 2.5° e il 97.5° percentile della distribuzione bootstrap — il **metodo percentile**, il più semplice dei diversi modi di costruire un intervallo di confidenza bootstrap, e quello che questo progetto usa. Leggilo così: “il 95% dei 10.000 ricampionamenti ha prodotto un valore di questa metrica compreso in questo intervallo” — non una dichiarazione probabilistica sul valore vero e sconosciuto della metrica, ma sulla variabilità osservata nel ricampionamento.

[Fatto] `evaluate_results()` (righe 7-59), la funzione orchestratrice, chiama `bootstrap_metrics()` e `ci()` per ciascuno dei sette modelli, poi passa i risultati a `plot_mean_ci()` (`function.py`:302-336, righe 55 di questo file): il grafico che ne risulta mostra, per ciascun modello, un punto (la media) con due tipi di barra d’errore sovrapposte — quella sottile per l’intervallo di confidenza al 95%, quella spessa per ± 1 deviazione standard (`bootstrap_metrics_dict['acc'].std()`, riga 37). Sono due informazioni diverse disegnate insieme: l’intervallo di confidenza dice quanto sei sicuro della stima della media; la deviazione standard dice quanto è dispersa la distribuzione dei singoli ricampionamenti.

24.3 Dal dizionario Python al PNG/PDF

[Fatto] `evaluate_results()` produce, in un solo passaggio sui sette modelli, i dati per sei grafici distinti: la matrice di confusione per modello (`plot_confusion`, riga 19), il confronto ROC unificato (`plot_roc_comparison`, riga 51, con i dati di tutti i modelli raccolti in `roc_data` durante il ciclo), il boxplot delle distribuzioni bootstrap (`plot_boxplots`, riga 50), il confronto per famiglia (`plot_family_comparison`, riga 52), e infine il grafico $\text{media} \pm \text{CI}$ appena visto. Ogni funzione di plotting riceve dati già pronti — mai un DataFrame grezzo da ricalcolare — ed è quindi, in linea di principio, testabile passandole dati sintetici senza rieseguire l’intera pipeline (un punto ripreso al capitolo 49).

RIFERIMENTO AL CODICE — `encoder_comparison_summary.csv` (riga 56) è il file riassuntivo definitivo di questa fase — una riga per modello, con media, deviazione standard e intervallo di confidenza per ciascuna delle tre metriche. È il file che `generatereport.py` (capitolo 27) rilegge per intero per costruire la tabella principale del report finale.

Interfaccia pubblica

Funzione	Parametri	Ritorna
<code>evaluate_results(dataset="Heart disease")</code>	<code>None</code>	Nessuno (side-effect: grafici + CSV)
<code>bootstrap_metrics(y_true, y_score, y_pred, n_iter=10000, seed=42)</code>	Predizioni di un modello	Dizionario di 3 array da 10.000 valori
<code>ci(a, alpha=0.95)</code>	Array di valori bootstrap	(media, (basso, alto))

Errori tipici

Un `FileNotFoundError` su un file `_y_true.npy` in questa fase segnala che `classification.py` non ha completato con successo per quel modello (capitolo 18.3). Un tempo di esecuzione molto più lungo del previsto per questa fase, su `Diabetes130` rispetto a `Heart Disease`, è atteso: il bootstrap ricampiona un numero di righe proporzionale alla dimensione del dataset di validazione, non fisso.

Riepilogo

`evaluation.py` implementa il bootstrap dell'intero progetto in poche righe di NumPy puro, ricampionando con reinserimento le predizioni già ottenute, non riallenando il modello. L'intervallo di confidenza al 95% è il metodo percentile, il più semplice fra le tecniche bootstrap esistenti. Il file produce sei grafici diversi e il CSV riassuntivo che alimenta direttamente il report finale.

Domande di autoverifica

1. Perché il bootstrap di questo progetto non riallena il modello 10.000 volte? Perché misura l'incertezza della stima della metrica sulle predizioni già ottenute, non l'incertezza dell'addestramento: ricampiona con reinserimento le coppie (etichetta vera, predizione, punteggio) già calcolate da `classification.py`, un'operazione computazionalmente molto più economica di un nuovo addestramento per ciascuna delle 10.000 iterazioni.

2. Cosa significa, con precisione, un intervallo di confidenza bootstrap al 95% costruito con il metodo percentile? Che il 95% dei 10.000 valori della metrica ricampionati cade fra il 2.5° e il 97.5° percentile della distribuzione osservata — una descrizione della variabilità del ricampionamento, non una dichiarazione probabilistica diretta sul valore vero e sconosciuto della metrica.

3. Perché la barra d'errore sottile e quella spessa nel grafico $\text{media} \pm \text{CI}$ non mostrano la stessa informazione? Perché la sottile rappresenta l'intervallo di confidenza al 95% (quanto è incerta la stima della media), mentre la spessa rappresenta ± 1 deviazione standard (quanto è dispersa l'intera distribuzione bootstrap) — due misure di variabilità diverse, disegnate insieme sullo stesso grafico.

MATERIALE PER LA TESI 1. La spiegazione formale del bootstrap con la formula dell'intervallo di confidenza percentile, affiancata al codice — riusabile in “Materiali e metodi”, sezione sulla metodologia di valutazione. 2. La precisazione sull'interpretazione corretta dell'intervallo di confidenza bootstrap (non una probabilità sul valore vero) — riusabile come nota metodologica per prevenire un fraintendimento statistico comune nella tesi. 3. L'elenco dei sei grafici prodotti da questa fase, con la funzione di `function.py` responsabile di ciascuno — riusabile come indice delle figure disponibili per la sezione “Risultati”.

Capitolo 25 — `error_analysis.py`: dall'errore statistico al caso clinico

Obiettivi del capitolo

- Vedere come un indice numerico salvato da `classification.py` diventa un record clinico leggibile.
- Capire cosa significa, esattamente, un caso “più difficile” quando sette modelli diversi lo valutano.
- Leggere la formula della deviazione di feature e sapere cosa NON dice.

[Fatto] `error_analysis.py` (81 righe) è la fase 5 (`main.py:46`): l'unico file, oltre a `generatereport.py`, il cui scopo esplicito è tradurre risultati statistici in qualcosa di direttamente interpretabile su un caso clinico reale, non solo su un numero aggregato.

25.1 Ricostruire il record dall'indice di validazione

[Fatto] `analyze_errors()` (righe 6-81) inizia ricaricando `X_train_raw.csv` (capitolo 21.3) — la versione leggibile delle feature, salvata da `preprocessing.py` — e per ciascun modello usa `val_idx` (salvato da `classification.py`, capitolo 23.3) per riallineare le predizioni ai record originali:

```
\NormalTok{records }\OperatorTok{=}\NormalTok{ X\_raw.iloc[val\_idx].reset\_index(dro
↪ p)\OperatorTok{=}\VariableTok{True}\NormalTok{}\}.copy()}
\NormalTok{records}\StringTok{"y\_score"}\NormalTok{[] }\OperatorTok{=}\NormalTok{
↪ y\_score}
\NormalTok{records[fp\_mask].to\_csv(os.path.join(dirs[\StringTok{"results"}]\NormalT
↪ ok[],
↪ }\SpecialStringTok{f"}\SpecialCharTok{\}\NormalTok{name}\SpecialCharTok{\})\Spec
↪ ialStringTok{\_false\_positives.csv"}\NormalTok{)},
↪ index)\OperatorTok{=}\VariableTok{False}\NormalTok{)}}
\NormalTok{records[fn\_mask].to\_csv(os.path.join(dirs[\StringTok{"results"}]\NormalT
↪ ok[],
↪ }\SpecialStringTok{f"}\SpecialCharTok{\}\NormalTok{name}\SpecialCharTok{\})\Spec
↪ ialStringTok{\_false\_negatives.csv"}\NormalTok{)},
↪ index)\OperatorTok{=}\VariableTok{False}\NormalTok{)}}}
```

`X_raw.iloc[val_idx]` seleziona, nell'ordine esatto in cui `classification.py` li aveva concatenati (capitolo 23.3), esattamente i record che sono stati usati come validazione in uno dei 5 fold di quel modello — non un campione casuale, ma la ricostruzione precisa e verificabile di quali righe hanno prodotto quale previsione. `fp_mask` e `fn_mask` (già visti al capitolo 4.3) selezionano poi, fra questi, rispettivamente i falsi positivi e i falsi negativi, salvati in due CSV separati per modello — 14 file in tutto, due per ciascuno dei sette modelli.

RIFERIMENTO AL CODICE — questo meccanismo funziona solo perché `val_idx` è stato costruito, in `classification.py`, concatenando gli indici di validazione **nello stesso ordine** in cui `y_true`, `y_score` e `y_pred` sono stati concatenati (capitolo 23.3). Se uno solo di questi quattro array fosse stato costruito con un ordine diverso, `error_analysis.py` assocerebbe silenziosamente ogni previsione al record sbagliato — senza sollevare alcun errore, perché le dimensioni degli array rimarrebbero comunque compatibili.

25.2 “Hardest cases”: cosa significa essere sbagliato da tutti i modelli

[Fatto] Il file accumula, per ciascun record del dataset originale, quante volte è stato valutato e quante volte è stato classificato male, sommando su tutti e sette i modelli (righe 12-13, 47-48):

```
\NormalTok{error\_counts } \OperatorTok{=} \NormalTok{ np.zeros(n\_records,
↪ dtype)} \OperatorTok{=} \BuiltInTok{int} \NormalTok{}
\NormalTok{eval\_counts } \OperatorTok{=} \NormalTok{ np.zeros(n\_records,
↪ dtype)} \OperatorTok{=} \BuiltInTok{int} \NormalTok{}
\NormalTok{...}
\NormalTok{eval\_counts[val\_idx] } \OperatorTok{+=} \DecValTok{1}
\NormalTok{error\_counts[val\_idx] } \OperatorTok{+=} \NormalTok{}
↪ error\_mask.astype(\BuiltInTok{int}) \NormalTok{}
```

`error_counts[val_idx] += ...` è un’assegnazione con indicizzazione avanzata di NumPy: incrementa, in un solo passaggio, tutte le posizioni indicate da `val_idx` — non un ciclo esplicito su ogni indice, un’operazione vettoriale che in Java richiederebbe scrivere a mano un ciclo `for` sull’array. Dopo aver ripetuto questo accumulo per tutti e sette i modelli, `hardest_idx = np.argsort(-error_counts)[:20]` (riga 58) individua i 20 record con il maggior numero di errori totali — `-error_counts` inverte il segno per ottenere un ordinamento decrescente da una funzione, `argsort`, che di norma ordina in modo crescente.

[Interpretazione] Un record che compare fra gli “hardest cases” con, per esempio, 7 su 7 modelli sbagliati (il valore massimo possibile, dato che ogni record compare in validazione una sola volta per modello, nei fold della cross-validation) non è necessariamente un caso “ambiguo” nel senso clinico del termine: potrebbe esserlo, ma potrebbe anche essere un record sintetico generato da SMOTENC (capitolo 21.2) che si trova in una zona dello spazio delle feature poco rappresentata, o un errore di imputazione che ha reso il record atipico. Il file non distingue queste possibilità — le tabella soltanto, e resta a chi legge il report interpretarle correttamente, un punto che il capitolo 46 riprende con lo sguardo critico che merita.

25.3 Deviazione di feature: una standardizzazione con un’interpretazione clinica

[Fatto] L’ultima analisi del file confronta, per ciascuna feature numerica, la sua media nei casi sbagliati contro la sua media nei casi corretti, aggregando su tutti i modelli (righe 65-75):

```
\ControlFlowTok{for} \NormalTok{ feature } \KeywordTok{in} \NormalTok{ num\_cols\_used:}
\NormalTok{ pooled\_std } \OperatorTok{=} \NormalTok{ pd.concat([df\_error[feature],
↪ df\_correct[feature]]) .std()}
\NormalTok{ deviation } \OperatorTok{=} \NormalTok{ (df\_error[feature].mean()
↪ ) \OperatorTok{-} df\_correct[feature].mean())
↪ ) \OperatorTok{/} \NormalTok{ pooled\_std } \ControlFlowTok{if} \NormalTok{}
↪ pooled\_std } \ControlFlowTok{else} \FloatTok{0.0}
```


$$d_{\text{feature}} = \frac{\bar{x}_{\text{errore}} - \bar{x}_{\text{corretto}}}{s_{\text{pooled}}} \quad (25.1)$$

dove $\bar{x}_{\text{errore}}$ e $\bar{x}_{\text{corretto}}$ sono le medie della feature nei due gruppi (record classificati male, record classificati bene, aggregati su tutti i modelli), e s_{pooled} è la deviazione standard calcolata sull'unione dei due gruppi (riga 71, `pd.concat(...).std()`). **[Livello: teoria consolidata del settore]** Questa quantità è concettualmente imparentata alla d di Cohen, una misura standard di dimensione dell'effetto: divide la differenza fra due medie per una deviazione standard comune, così da poter confrontare fra loro feature misurate su scale del tutto diverse (anni di età contro mg/dl di colesterolo) — senza questa standardizzazione, una differenza di “5” per il colesterolo (scala di centinaia) e una differenza di “5” per l'età (scala di decine) non sarebbero minimamente comparabili.

ATTENZIONE — questa deviazione descrive una differenza aggregata *su tutti i modelli insieme*, non su un modello specifico, e riguarda solo le feature **numeriche** (`num_cols_used`, capitolo 6.1) — le feature categoriali (sesso, tipo di dolore toracico, razza) non compaiono in questa analisi, anche se potrebbero essere altrettanto informative su cosa caratterizza un caso difficile. È un limite di copertura reale, non solo una scelta di semplicità: il capitolo 46 lo nota esplicitamente.

Interfaccia pubblica

Funzione	Parametri	Effetti
<code>analyze_errors(dataset="heart_disease")</code>	<code>num_cols_used</code>	Salva <code>error_summary.csv</code> , <code>hardest_cases.csv</code> , <code>feature_deviation.csv</code> , 14 CSV di FP/FN, 2 grafici

Errori tipici

Un `KeyError` su una colonna di `X_raw` in questa fase indica quasi sempre che `num_cols_used` (scelto in base al dataset, riga 8) non corrisponde alle colonne effettivamente presenti in `X_train_raw.csv` — capiterebbe se `preprocessing.py` fosse stato modificato senza aggiornare di conseguenza gli elenchi di `function.py`. Un `hardest_cases.csv` vuoto o con meno di 20 righe (riga 59, `error_counts[hardest_idx] > 0`) significherebbe che meno di 20 record sono mai stati classificati male da almeno un modello — un segnale, non un errore, di un problema insolitamente facile.

Riepilogo

`error_analysis.py` usa gli indici di validazione salvati da `classification.py` per ricondurre ogni previsione al record clinico originale, aggregando su tutti e sette i modelli per individuare i casi più difficili e per misurare, con una statistica standardizzata simile alla d di Cohen, quali feature numeriche caratterizzano i casi sbagliati rispetto a quelli corretti — senza però coprire le feature categoriali, e senza distinguere un record realmente ambiguo da un artefatto del bilanciamento sintetico.

Domande di autoverifica

1. Perché `X_raw.iloc[val_idx]` funziona solo se `val_idx` è stato costruito nello stesso ordine di `y_true/y_score/y_pred` in `classification.py`? Perché l'associazione fra un record e la sua previsione si basa esclusivamente sulla posizione corrispondente nei quattro array: se l'ordine di costruzione divergesse anche per un solo array, ogni record verrebbe associato silenziosamente alla previsione sbagliata, senza che le dimensioni compatibili degli array segnalino nulla.

2. Un record compare fra gli “hardest cases” con 7 errori su 7 modelli: cosa NON puoi concludere automaticamente da questo dato? Non puoi concludere che sia un caso clinicamente ambiguo in senso stretto: potrebbe essere un record sintetico generato da SMO-TENC in una zona poco rappresentata dello spazio delle feature, o un artefatto di imputazione — il file registra la frequenza dell'errore, non ne diagnostica la causa.

3. Perché la deviazione di feature (§25.3) permette di confrontare l'età e il colesterolo sulla stessa scala, pur avendo unità di misura diverse? Perché la differenza di medie viene divisa per una deviazione standard comune (pooled), calcolata sull'unione dei casi sbagliati e corretti — una standardizzazione concettualmente simile alla d di Cohen, che rende la quantità risultante adimensionale e quindi confrontabile fra feature diverse.

MATERIALE PER LA TESI 1. La Formula 25.1 con la spiegazione di ogni simbolo e il collegamento alla d di Cohen — riusabile in “Materiali e metodi” per la sezione sull'analisi degli errori. 2. Il meccanismo di ricostruzione del record originale tramite `val_idx`, con l'avvertenza sulla sua fragilità silenziosa — riusabile come nota tecnica sulla tracciabilità, o come punto di attenzione nella sezione critica. 3. Il limite di copertura sulle sole feature numeriche, esplicitamente dichiarato — riusabile come voce autonoma nella sezione “Discussione e limiti” o come proposta di estensione nella Parte XII.

Capitolo 26 — `statisticaltest.py`: tre test, tre garanzie diverse

Obiettivi del capitolo

- Leggere come il progetto confronta a coppie tutti e sette i modelli, per tre metriche, con tre test diversi.
- Sapere cosa testano davvero Wilcoxon, t-test appaiato e DeLong, e su quali dati esattamente.
- Riconoscere perché usarli insieme è una scelta di robustezza, non una ridondanza.

[Fatto] `statisticaltest.py` (123 righe) è la fase 6 (`main.py:49`): l'unico file che confronta esplicitamente i modelli fra loro, invece di descriverli uno alla volta.

26.1 Wilcoxon e t-test appaiato sulle distribuzioni bootstrap

[Fatto] `test_statistical_tests()` (righe 10-68) confronta ogni coppia possibile fra i sette modelli, per ciascuna delle tre metriche, usando le distribuzioni bootstrap già salvate da `evaluation.py`:

```
\ControlFlowTok{for}\NormalTok{ metric }\KeywordTok{in}\NormalTok{ metrics:}
\NormalTok{ boot\_scores }\OperatorTok{=}\NormalTok{ \{\}}
    \ControlFlowTok{for}\NormalTok{ model\_name }\KeywordTok{in}\NormalTok{ models:}
\NormalTok{ boot\_scores[model\_name] }\OperatorTok{=}\NormalTok{ {}
↪ np.load(os.path.join(dirs[0]\StringTok{"results"}\NormalTok{[},
↪ }\SpecialStringTok{f"}\SpecialCharTok{\}\NormalTok{model\_name}\SpecialCharTok{\}
↪ }\SpecialStringTok{\_boot\_}\SpecialCharTok{\}\NormalTok{metric}\SpecialCharTok{\}
↪ \{\}\SpecialStringTok{.npy"}\NormalTok{)}}}
\ControlFlowTok{for}\NormalTok{ i }\KeywordTok{in}
    ↪ \BuiltInTok{range}\NormalTok{({}\BuiltInTok{len}\NormalTok{(models))}:}
    \ControlFlowTok{for}\NormalTok{ j }\KeywordTok{in}
        ↪ \BuiltInTok{range}\NormalTok{(i }\OperatorTok{+}
        ↪ \DecValTok{1}\NormalTok{, }\BuiltInTok{len}\NormalTok{(models))}:}
\NormalTok{ a, b }\OperatorTok{=}\NormalTok{ models[i], models[j]}
\NormalTok{ scores\_a, scores\_b }\OperatorTok{=}\NormalTok{ {
↪ boot\_scores[a], boot\_scores[b]}
\NormalTok{ stat\_w, p\_w }\OperatorTok{=}\NormalTok{ wilcoxon(scores\_a,
↪ scores\_b)}
\NormalTok{ ...}
\NormalTok{ stat\_t, p\_t }\OperatorTok{=}\NormalTok{ {
↪ ttest\_rel(scores\_a, scores\_b)}
```

for j in range(i + 1, len(models)), non range(len(models)), è la forma standard per generare ogni coppia **una sola volta** (confrontare A con B è la stessa cosa di confrontare B con A, non serve farlo due volte) — con 7 modelli, questo produce $\binom{7}{2} = 21$ confronti per metrica, 63 in totale per i tre test Wilcoxon/t-test, più 21 per DeLong (capitolo 26.2).

[**Livello: teoria consolidata del settore**] Sia Wilcoxon sia il t-test appaiato confrontano due campioni **accoppiati** — qui, i 10.000 valori bootstrap di un modello contro i 10.000 valori bootstrap corrispondenti dell'altro, ricampionati con lo stesso seme (capitolo 24.1) e quindi confrontabili posizione per posizione. Il **test di Wilcoxon signed-rank** è non parametrico: non assume che le differenze fra coppie seguano una distribuzione normale, e si basa sui ranghi delle differenze, non sui loro valori esatti — più robusto quando la distribuzione reale si discosta dalla normalità, un'ipotesi spesso ragionevole per dati clinici. Il **t-test appaiato** assume invece che le differenze siano approssimativamente normali, e in cambio ha più potenza statistica (rileva differenze più piccole come significative) se quell'assunzione regge davvero. [**Fatto**] Entrambi restituiscono una statistica del test e un p-value, salvati insieme alle medie dei due modelli e a un flag binario di significatività a $\alpha = 0.05$ (righe 44,57: `"1" if p_w < 0.05 else "0"`) — [**Attenzione**] notato come stringa `"1"/"0"`, non come intero o booleano: un dettaglio da tenere presente se in futuro si volesse filtrare il CSV risultante per riga significativa, perché un confronto numerico diretto (`== 1`) fallirebbe silenziosamente contro una stringa.

26.2 Il test di DeLong e la libreria MLstatkit

[**Fatto**] `test_delong()` (righe 70-124) confronta, per ogni coppia di modelli, l'AUC calcolata sugli stessi dati di validazione, usando una libreria esterna:

```
\ImportTok{from}\NormalTok{ MLstatkit }\ImportTok{import}\NormalTok{ Delong\_test}
\NormalTok{...}
\NormalTok{z, p, auc\_a, auc\_b }\OperatorTok{=}\NormalTok{ Delong\_test(}
\NormalTok{    y\_true\_a, scores[a] }\StringTok{"y\_score"}\NormalTok{[,}
↪ scores[b] }\StringTok{"y\_score"}\NormalTok{[,}
\NormalTok{    return\_ci }\OperatorTok{=}\NormalTok{VariableTok{False}\NormalTok{[,}
↪ return\_auc }\OperatorTok{=}\NormalTok{VariableTok{True}\NormalTok{[,}
↪ verbose }\OperatorTok{=}\NormalTok{DecValTok{0}\NormalTok{[,}
\NormalTok{)]}
```

[**Livello: teoria consolidata del settore**] Il **test di DeLong** è specifico per confrontare due AUC calcolate sugli *stessi* casi (stesse etichette vere): a differenza di Wilcoxon e t-test applicati qui alle distribuzioni bootstrap, DeLong lavora direttamente sui punteggi di probabilità originali (`y_score`) e tiene conto esplicitamente della correlazione fra le due AUC che nasce dal fatto che entrambi i modelli sono valutati sugli stessi pazienti — ignorare questa correlazione (per esempio trattando le due AUC come se venissero da campioni indipendenti) produrrebbe una stima della significatività meno accurata. [**Fatto**] Prima di ogni confronto, il codice verifica esplicitamente che le etichette vere dei due modelli coincidano (righe 91-93: `if not np.array_equal(y_true_a, y_true_b): print("[WARNING] ...") continue`) — una precondizione necessaria perché il test di DeLong abbia senso, dato che confronta AUC sugli stessi casi.

[**Fatto**] `MLstatkit` (`requirements.txt`, versione 0.1.91) è l'unica dipendenza del progetto dedicata a un singolo test statistico specifico, non a una libreria generalista come `scipy` (da cui vengono `wilcoxon` e `ttest_rel`, riga 6-7) — una scelta che riflette quanto un'implementazione corretta del test di DeLong sia più delicata da scrivere da zero rispetto a un t-test o a un Wilcoxon, entrambi disponibili direttamente in `scipy.stats`.

26.3 Perché tre test invece di uno

[**Fatto**] `docs/STATISTICAL_TESTS.md`:84-90 motiva esplicitamente la scelta di tutti e tre: Wilcoxon è più robusto se la normalità non regge, il t-test appaiato è più potente se regge, DeLong è specifico per l'AUC e tiene conto della correlazione fra modelli valutati sugli stessi casi. [**In-**

interpretazione] La logica sottostante è quella della **triangolazione metodologica**: se tre test con assunzioni diverse concordano tutti sulla stessa conclusione (differenza significativa o meno), la fiducia in quella conclusione è più alta che se si fosse usato un solo test — e se invece i tre test discordassero fra loro su uno stesso confronto, sarebbe un segnale che il risultato è al limite della soglia di significatività, non un errore da correggere scegliendo il test che dà la risposta preferita.

PROVA TU — apri `datas/heart_disease/results/wilcoxon_comparison.csv`, `ttest_comparison.csv` e `delong_comparison.csv` (già presenti nel repository, capitolo 44) e cerca una coppia di modelli per cui i tre test non concordino tutti sulla stessa conclusione di significatività. Se la trovi, è un candidato naturale per una discussione più approfondita nella tesi — un punto in cui la robustezza della “significatività” del confronto meriterebbe una frase in più, non solo una tabella.

Interfaccia pubblica

Funzione	Parametri	Effetti
<code>test_statistical_tests(database="heart_disease")</code>	Nome database	Salva <code>wilcoxon_comparison.csv</code> , <code>ttest_comparison.csv</code> ; chiama <code>test_delong()</code>
<code>test_delong(dirs)</code>	Dizionario percorsi	Salva <code>delong_comparison.csv</code>

Errori tipici

Un avviso `[WARNING] y_true diversi per A vs B - skip` in console indica che due modelli hanno numeri di record di validazione diversi o etichette diverse per lo stesso indice — atteso solo se le fasi precedenti sono state eseguite in modo incoerente fra loro (per esempio rieseguendo `classification.py` per un solo modello dopo aver cambiato il numero di fold). Un `FileNotFoundError` su un file `_boot_*.npy` segnala che `evaluation.py` non ha completato per quel modello.

Riepilogo

`statisticaltest.py` confronta ogni coppia dei sette modelli con tre test diversi — Wilcoxon e t-test appaiato sulle distribuzioni bootstrap, DeLong sull’AUC diretta con una libreria dedicata — motivati dalla logica della triangolazione: tre garanzie statistiche diverse, la cui concordanza rafforza la fiducia nella conclusione più di quanto farebbe un singolo test.

Domande di autoverifica

1. Perché il ciclo di confronto usa `range(i + 1, len(models))` per l’indice interno, invece di ripartire sempre da zero? Per generare ogni coppia di modelli una sola volta: confrontare il modello A con il modello B è statisticamente equivalente a confrontare B con A, e ripetere entrambi i confronti raddoppierebbe il lavoro senza aggiungere informazione.
2. Perché il test di DeLong non può essere applicato a due modelli le cui etichette vere di validazione non coincidono esattamente? Perché il test è pensato per confrontare

due AUC calcolate sugli stessi casi, tenendo conto della correlazione che nasce da questa condivisione — se le etichette vere differiscono, i due modelli non sono stati valutati sugli stessi record, e il confronto perderebbe il suo fondamento statistico.

3. Cosa significherebbe, in pratica, se Wilcoxon e t-test appaiato concordassero su una differenza significativa fra due modelli, ma DeLong sull’AUC non la trovasse significativa? Che le distribuzioni bootstrap di accuratezza e F1 differiscono in modo significativo fra i due modelli, ma la differenza di AUC specifica, calcolata direttamente sui punteggi originali, non raggiunge la soglia di significatività — un segnale che la differenza è reale su alcune metriche ma non necessariamente sulla capacità discriminante complessiva misurata dall’AUC.

MATERIALE PER LA TESI 1. La spiegazione comparata dei tre test con le rispettive assunzioni e garanzie — riusabile in “Materiali e metodi” per motivare il protocollo di confronto statistico. 2. L’argomento della triangolazione metodologica, con il rimando a `docs/STATISTICAL_TESTS.md` — riusabile per giustificare, nella tesi, perché tre test concordanti rafforzino la fiducia nei risultati più di un singolo test. 3. L’esercizio di ricerca di un disaccordo fra i tre test sui dati reali (§26.3) — riusabile come base concreta per un paragrafo di discussione sui confronti statisticamente “al limite”.

Capitolo 27 — `generatereport.py`: l'ultimo miglio, e il suo limite più istruttivo

Obiettivi del capitolo

- Vedere come il progetto assembla, da solo, un report Markdown leggibile a partire da CSV e immagini già pronte.
- Verificare con i tuoi occhi, confrontando due report reali, che una parte del testo generato non dipende affatto dai dati.
- Sapere esattamente cosa cambieresti per rendere questo report onesto rispetto a quello che i dati dicono davvero.

[Fatto] `generatereport.py` (249 righe) è la fase 7, l'ultima (`main.py:52`): non calcola nulla di nuovo, assembla in un unico documento tutto ciò che le sei fasi precedenti hanno già prodotto.

27.1 Come si assembla un report Markdown da CSV e PNG

[Fatto] `generate_markdown()` (righe 41-227) costruisce il report accumulando stringhe in una lista Python, unita alla fine con `"\n".join(md)` (riga 227, `main`):

```
\NormalTok{md }\OperatorTok{=}\NormalTok{ []}
\NormalTok{md.append(\SpecialStringTok{f"\#  Encoder Evaluation
↪ Report}\CharTok{\textbackslash{n}}\SpecialStringTok{"}\NormalTok{ })}
\NormalTok{md.append(\SpecialStringTok{f"**Generated:**
↪ }\SpecialCharTok{\}\NormalTok{now}\SpecialCharTok{\}\}\CharTok{\textbackslash{n}}
↪ \SpecialStringTok{"}\NormalTok{ })}
\NormalTok{...}
\NormalTok{md.append(summary.to\_markdown(index)\OperatorTok{=}\VariableTok{False}\No
↪ rmalTok{ })}
```

`DataFrame.to_markdown()` — un metodo di pandas che richiede `tabulate` come dipendenza (presente in `requirements.txt`, usata solo per questo scopo in tutto il progetto, verificato negli import) — converte automaticamente un intero DataFrame in una tabella Markdown con l'allineamento delle colonne già corretto, senza dover scrivere a mano i separatori `|` e `-`. Ogni immagine viene inclusa con un percorso relativo costruito esplicitamente (`real_roc_path = "../graphics/ROC_comparison.png"`, riga 74): il report vive in `datas/<dataset>/reports/`, i grafici in `datas/<dataset>/graphics/` o `results/` — da cartelle sorelle, un percorso relativo con `../` è l'unico modo di riferirsi a un file in una cartella diversa senza scrivere il percorso assoluto.

[Fatto] Ogni sezione del report è inclusa **condizionalmente**, verificando prima che il file esista (`if os.path.exists(roc_path):`, riga 75, ripetuto per ogni grafico e CSV): un report generato

dopo un'esecuzione parziale della pipeline (capitolo 18.3) conterrebbe quindi solo le sezioni i cui file sono effettivamente presenti, senza sollevare un errore per quelli mancanti — un report incompleto ma valido, invece di un `FileNotFoundError` che bloccherebbe l'intera generazione.

27.2 Il caso del testo narrativo statico: quando il report “mente” per omissione

[Fatto] Le sezioni “Discussion and Observations”, “Conclusions” e “Potential Improvements” (righe 192-225) non derivano da alcun calcolo sui dati appena descritti: sono stringhe Python scritte una volta, incluse identiche ogni volta che il report viene generato, indipendentemente dal dataset o dai risultati:

```
\NormalTok{md.append()}\StringTok{"#\# Discussion and
↪ Observations"}\CharTok{\textbackslash{}n}\StringTok{""}\NormalTok{}
\NormalTok{md.append()}\StringTok{""}
\StringTok{{-}} Larger embedding models (E5{-}large, GTE{-}large) generally show
↪ better performance.}
\StringTok{{-}} GTE{-}large tends to achieve higher ROC{-}AUC and tighter confidence
↪ intervals.}
\StringTok{{-}} Confusion matrices enable analysis of false positives and false
↪ negatives.}
\StringTok{{-}} Bootstrap is useful to verify metric stability and robustness.}
\StringTok{""}\NormalTok{}}
```

[Fatto] Puoi verificarlo tu stesso, senza fidarti della mia parola: apri `datas/heart_disease/reports/report.md` e `datas/diabetes130/reports/report.md`, entrambi già presenti nel repository, e confronta le rispettive sezioni “Discussion and Observations”. Sono identiche, carattere per carattere. **[Fatto]** Confronta ora quel testo con le tabelle numeriche nella stessa pagina: in Heart Disease, l'AUC di `gte-large` (0.854, capitolo 44) è inferiore a quella di `pubmedbert` (0.885) e di `e5-large` (0.866); in Diabetes130, è inferiore a `sentence-biobert` (0.768), `bioclinicalbert` e `pubmedbert` (~0.757-0.758, capitolo 45). L'affermazione “GTE-large tends to achieve higher ROC-AUC” non è vera in nessuno dei due dataset che il report stesso presenta — è un template, non una sintesi dei dati.

ATTENZIONE — questo non è un dettaglio stilistico: in un contesto in cui questo genere di report informasse una decisione reale (per esempio, quale modello di embedding adottare per un sistema clinico di supporto alla decisione), una conclusione narrativa sbagliata ma presentata con la stessa autorevolezza tipografica delle tabelle numeriche corrette è un rischio concreto, non teorico. Il capitolo 52 tratta questo punto come caso di studio autonomo, con tutto il rigore critico che merita.

27.3 Come lo riscriveresti tu

[Interpretazione] Rendere questa sezione onesta rispetto ai dati non richiederebbe una riscrittura complessa: `summary` (il DataFrame caricato da `encoder_comparison_summary.csv`, già disponibile in questa stessa funzione) contiene tutto il necessario per costruire la frase corretta a runtime — per esempio, `summary.loc[summary["auc_mean"].idxmax(), "model"]` restituirebbe il nome del modello con l'AUC media più alta per *quel* dataset specifico, sostituendo un'affermazione fissa e potenzialmente falsa con un'affermazione calcolata e sempre vera per costruzione.

PROVA TU — scrivi tu la funzione che genera dinamicamente la frase “il modello con l'AUC più alta è X (Y)” a partire dal DataFrame `summary`, e confrontala con l'affermazione statica attuale su entrambi i dataset del progetto. Non è un esercizio

puramente accademico: è precisamente il tipo di refactoring minimo, a basso rischio, che trasformerebbe il punto più debole di questo file nel suo punto di forza — un report che si adatta ai dati invece di ripetere sempre la stessa storia.

Interfaccia pubblica

Funzione	Parametri	Ritorna
<code>generate_report(dataset="heart_disease")</code>	Nessuno	Nessuno (salva <code>report.md</code>)
<code>load_summary(summary_path)</code>	Percorso CSV	DataFrame
<code>load_statistical_results(wilcoxon_path, ttest_path, delong_path)</code>	Percorsi CSV	Dizionario di DataFrame o None
<code>generate_markdown(summary, dirs)</code>	DataFrame riassuntivo, percorsi	Stringa Markdown completa

Errori tipici

Un `FileNotFoundError` esplicito (non gestito condizionalmente, a differenza dei grafici) viene sollevato da `load_summary()` (righe 12-13) se `encoder_comparison_summary.csv` non esiste — l'unico file che questa fase considera davvero indispensabile, coerentemente con il fatto che è la tabella principale dell'intero report.

Riepilogo

`generatereport.py` assembla un report Markdown completo da CSV e immagini già pronte, includendo ogni sezione solo se il file corrispondente esiste. Le sezioni discorsive finali (discussione, conclusioni, miglioramenti) sono però testo statico, identico in ogni esecuzione e per ogni dataset — al punto da contraddire, in almeno un caso verificabile, i numeri riportati nella stessa pagina. È il limite più istruttivo di tutto il progetto: un generatore di report che non legge i propri stessi dati per la parte più interpretativa del documento.

Domande di autoverifica

- 1. Perché il report può essere generato correttamente anche se, per esempio, il grafico UMAP non fosse mai stato prodotto?** Perché ogni sezione verifica esplicitamente l'esistenza del proprio file prima di includerlo (`if os.path.exists(...)`), e in sua assenza semplicemente non aggiunge quella sezione, invece di sollevare un errore che bloccherebbe l'intero report.
- 2. Come puoi dimostrare, senza fidarti di un'affermazione altrui, che il testo di “Discussion and Observations” non dipende dai dati?** Confrontando carattere per carattere le sezioni corrispondenti dei due report reali già presenti nel repository (Heart Disease e Diabetes130): sono identiche, nonostante le tabelle numeriche sopra siano sostanzialmente diverse fra i due dataset.
- 3. Con quale singola espressione, usando il DataFrame `summary` già disponibile nella funzione, potresti sostituire l'affermazione statica sul modello con l'AUC più alta?** `summary.loc[summary["auc_mean"].idxmax(), "model"]` — restituisce il nome del modello con l'AUC media massima per il dataset corrente, calcolato a runtime invece di scritto a priori.

MATERIALE PER LA TESI 1. Il confronto testuale diretto fra le due sezioni “Discussion and Observations” identiche, con i numeri reali che le contraddicono — è probabilmente l’osservazione critica singola più forte di tutto il libro: riusabile come caso di studio autonomo nella sezione “Discussione e limiti”. 2. La proposta concreta di refactoring (§27.3), con l’espressione `pandas` esatta — riusabile nella sezione “Lavori futuri” come miglioramento a basso costo e alto impatto. 3. Lo schema di inclusione condizionale delle sezioni basato sull’esistenza del file — riusabile come esempio di buona pratica di robustezza, da contrapporre esplicitamente al problema del testo statico nello stesso file.

Capitolo 28 — `main.py`: l'orchestratore letto per ultimo apposta

Obiettivi del capitolo

- Rileggere l'intero file più corto e più centrale del progetto, ora che conosci ciascuna delle sette fasi che orchestra.
- Sapere esattamente cosa fa, e cosa non fa, il parsing degli argomenti da riga di comando.
- Capire dove e come inseriresti un'ottava fase, se dovessi estendere la pipeline.

[Fatto] `main.py` (56 righe) è il file più corto del progetto — e non a caso è l'ultimo che questo libro legge per intero: ogni riga chiama una funzione già vista nei capitoli 20-27, e leggerlo ora, con quella conoscenza già in mano, richiede pochi minuti invece di una nuova esplorazione.

28.1 Le sette fasi in ordine, con un occhio a ciò che già conosci

[Fatto] L'intera logica del file sta in `main()` (righe 20-55):

```
\KeywordTok{def}\NormalTok{ main():}
\NormalTok{     args }\OperatorTok{=}\NormalTok{ parse\_args()}
\NormalTok{     dirs }\OperatorTok{=}\NormalTok{ get\_output\_dirs(args.dataset)}
\NormalTok{     delete\_files\_embeddings(dirs[]\StringTok{"embeddings"}\NormalTok{[]})}
\NormalTok{     ↪ delete\_files\_preprocessing(dirs[]\StringTok{"preprocessing"}\NormalTok{[]})}
\NormalTok{     delete\_files\_results(dirs[]\StringTok{"results"}\NormalTok{[]})}
\NormalTok{     delete\_files\_graphics(dirs[]\StringTok{"graphics"}\NormalTok{[]})}
\NormalTok{     X,y }\OperatorTok{=}\NormalTok{ }
\NormalTok{     ↪ preprocessing\_data(dataset)\OperatorTok{=}\NormalTok{ args.dataset)}
\NormalTok{     embeddings(X, y, dataset)\OperatorTok{=}\NormalTok{ args.dataset)}
\NormalTok{     training\_classifier(dataset)\OperatorTok{=}\NormalTok{ args.dataset)}
\NormalTok{     evaluate\_results(dataset)\OperatorTok{=}\NormalTok{ args.dataset)}
\NormalTok{     analyze\_errors(dataset)\OperatorTok{=}\NormalTok{ args.dataset)}
\NormalTok{     ↪ test\_statistical\_tests(dataset)\OperatorTok{=}\NormalTok{ args.dataset)}
\NormalTok{     generate\_report(dataset)\OperatorTok{=}\NormalTok{ args.dataset)}
```

Tredici righe, sette chiamate di fase più quattro di pulizia preliminare — ognuna delle quali hai già letto in dettaglio: `get_output_dirs` e le quattro `delete_files_*` al capitolo 20.2, `preprocessing_data` al capitolo 21, `embeddings` al capitolo 22, `training_classifier` al capitolo 23, `evaluate_results` al capitolo 24, `analyze_errors` al capitolo 25, `test_statistical_tests` al capitolo 26, `generate_report` al capitolo 27. Non c'è altro codice di rilievo in questo file: nessuna logica propria, solo composizione di funzioni altrui, nell'ordine esatto in cui il capitolo 17 (Figura 17.1) le ha già mostrate a diagramma.

[Fatto] Nota che `X, y = preprocessing_data(...)` è l'unico punto in cui `main()` maneggia dati concreti, non solo nomi di dataset: è la stessa asimmetria già notata al capitolo 17.1, qui visibile direttamente nella firma delle chiamate — solo la prima ha un valore di ritorno che il chiamante riusa esplicitamente (`embeddings(X, y, ...)`), tutte le altre restituiscono `None` implicitamente e comunicano solo tramite file su disco.

28.2 Il flag `--dataset` e il comportamento di default

[Fatto] `parse_args()` (righe 12-18) definisce un solo argomento da riga di comando:

```
\KeywordTok{def}\NormalTok{ parse\_args():}
\NormalTok{    parser }\OperatorTok{=}\NormalTok{
    ↪ argparse.ArgumentParser(description)\OperatorTok{=}\StringTok{"Run the clinical"}
    ↪ embedding benchmark pipeline."}\NormalTok{
\NormalTok{    parser.add\_argument(}
        \StringTok{"{-}{-}dataset"}\NormalTok{, choices}\OperatorTok{=}\NormalTok{[}\_
        ↪ StringTok{"heart\_disease"}\NormalTok{,}
        ↪ }\StringTok{"diabetes130"}\NormalTok{,}
        ↪ default}\OperatorTok{=}\StringTok{"heart\_disease"}\NormalTok{,}
        \BuiltInTok{help}\OperatorTok{=}\StringTok{"Clinical dataset to use (default:"}
        ↪ heart\_disease)."}
\NormalTok{    )}
\ControlFlowTok{return}\NormalTok{ parser.parse\_args()}
```

`choices=[...]` fa sì che `argparse` rifiuti da solo, prima ancora che il codice del progetto veda il valore, qualunque stringa diversa dalle due elencate — con un messaggio di errore e uscita dal programma, non un `ValueError` sollevato più a valle come quello di `get_output_dirs()` (capitolo 20.2, che a questo punto non può mai essere raggiunto con un valore non valido passato da riga di comando, ma resta comunque necessario per chi chiamasse `get_output_dirs()` direttamente da un altro punto del codice, bypassando `argparse`). `default="heart_disease"` è la fonte esatta del comportamento già segnalato in più punti del libro (capitolo 0.3, capitolo 15.2): eseguire `python main.py` senza alcun flag esegue silenziosamente solo Heart Disease.

SE VIENI DA JAVA — `argparse` gioca, per uno script a riga di comando, un ruolo concettualmente simile a una libreria come `picocli` o `JCommander`: dichiarare gli argomenti attesi con le loro proprietà (tipo implicito, valori ammessi, default, testo di aiuto), e la libreria genera da sola sia il parsing sia un messaggio di `--help` leggibile — non devi scrivere a mano l'analisi di `sys.argv`.

28.3 Cosa succederebbe ad aggiungere una fase 8

[Interpretazione] Aggiungere una fase 8 — per esempio, una calibrazione delle probabilità (capitolo 55) da eseguire dopo la classificazione — richiederebbe, seguendo esattamente il pattern già stabilito da questo file: (1) un nuovo modulo `calibration.py` con una funzione `calibrate_probabilities(dataset="heart_disease")` che riceve solo il nome del dataset e rilegge da disco ciò che le serve, coerentemente con lo stile delle fasi 3-7 (capitolo 17.1); (2) un `import` aggiuntivo in cima a `main.py`; (3) una riga in più dentro `main()`, nella posizione giusta della sequenza. Nessuna delle sette fasi esistenti dovrebbe cambiare.

ATTENZIONE — questa semplicità apparente nasconde lo stesso limite già visto al capitolo 18.3: la nuova fase erediterebbe automaticamente l'assenza di gestione degli errori (nessun `try/except` attorno alle chiamate in `main()`) e l'assenza di ripartenza da un punto intermedio. Estendere la pipeline seguendo il pattern esistente

è facile; renderla più robusta nel farlo richiederebbe intervenire anche sul file che, in questo capitolo, sembra il più semplice di tutti.

Interfaccia pubblica

Funzione	Parametri	Effetti
<code>main()</code>	Nessuno (legge <code>sys.argv</code> tramite <code>parse_args()</code>)	Esegue l'intera pipeline per il dataset scelto
<code>parse_args()</code>	Nessuno	Restituisce un oggetto <code>Namespace</code> con l'attributo <code>dataset</code>

`if __name__ == "__main__": main()` (riga 54) è l'idioma standard che permette a questo file di essere sia eseguito direttamente (`python main.py`) sia importato da un altro modulo senza eseguire automaticamente `main()` — un pattern che nessun altro file di questo progetto usa, perché nessun altro file è pensato per essere il punto di ingresso da riga di comando.

Errori tipici

Un messaggio `error: argument --dataset: invalid choice` proviene direttamente da `argparse`, prima ancora che una sola riga del progetto venga eseguita — segnala un valore diverso da `heart_disease` o `diabetes130` passato al flag. Qualunque altro errore, a valle di questo punto, appartiene a una delle sette fasi già trattate nei capitoli precedenti, non a `main.py` stesso.

Riepilogo

`main.py` non contiene alcuna logica propria: compone, in un ordine fisso, la pulizia preliminare e le sette fasi già lette nei capitoli 20-27, passando sempre e solo il nome del dataset scelto. `argparse` gestisce da solo la validazione dell'unico argomento del progetto, con un default (`heart_disease`) che determina il comportamento più frequentemente frainteso dell'intera pipeline. Estendere la sequenza con una fase aggiuntiva è meccanicamente semplice, ma erediterebbe gli stessi limiti di gestione degli errori già discussi al capitolo 18.3.

Domande di autoverifica

- 1. Perché `main.py` è, a ragione, il file più semplice da leggere una volta arrivati a questo punto del libro?** Perché non contiene logica propria: ogni riga chiama una funzione appartenente a uno degli otto file già letti nei capitoli precedenti, nell'ordine già visto nel diagramma architetturale del capitolo 17.
- 2. Perché un valore non valido passato a `--dataset` non arriva mai a far scattare il `ValueError` di `get_output_dirs()` in `function.py`?** Perché `argparse`, grazie a `choices=["heart_disease", "diabetes130"]`, intercetta e rifiuta un valore non valido prima ancora che il codice del progetto venga eseguito, terminando il programma con un proprio messaggio d'errore.
- 3. Quali tre modifiche servirebbero, in linea di principio, per aggiungere una fase 8 alla pipeline?** Un nuovo modulo con una funzione che segue lo stesso pattern delle fasi esistenti

(riceve solo il nome del dataset, rilegge da disco ciò che le serve), un nuovo `import` in cima a `main.py`, e una riga aggiuntiva dentro `main()` nella posizione corretta della sequenza — senza modificare nessuna delle fasi esistenti.

MATERIALE PER LA TESI 1. Il diagramma testuale delle tredici righe di `main()`, con il rimando a ciascun capitolo che tratta la fase corrispondente — riusabile come sintesi finale della sezione “Materiali e metodi”. 2. La spiegazione precisa del meccanismo `choices/default` di `argparse` e delle sue conseguenze pratiche — riusabile come nota tecnica su un comportamento di default facilmente frainteso. 3. La guida meccanica per aggiungere una fase 8, con l'avvertenza sui limiti ereditati — riusabile come base diretta per una proposta concreta nella Parte XII (Estensioni e lavori futuri).

Capitolo 29 — Il dataset UCI Heart Disease

Obiettivi del capitolo

- Sapere da dove viene davvero questo dataset, e perché non è un dataset a centro unico nonostante il nome comune “Cleveland” che spesso lo accompagna.
- Conoscere lo schema completo delle 14 feature standard, con le loro unità di misura e i loro codici.
- Avere i numeri reali di quanti dati mancano, centro per centro — non un’affermazione generica.

29.1 Origine e centri clinici

[**Fatto**] Il dataset proviene da quattro centri clinici distinti, integrati in un solo file dal progetto (`load_heart_disease()`, `function.py:96-113`): Cleveland Clinic Foundation (Stati Uniti), Hungarian Institute of Cardiology di Budapest, University Hospital di Zurigo (Svizzera), e V.A. Medical Center di Long Beach (Stati Uniti) — [**Fatto**] documentato in `docs/DATASET.md:11-15`, con fonte citabile: Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1988). *Heart Disease*. UCI Machine Learning Repository. DOI: 10.24432/C52P4W.

[**Fatto**] Il progetto concatena tutti e quattro i file senza alcuna selezione (`pd.concat(dfs, ignore_index=True)`, `function.py:104`), per un totale di **920 righe** — non 297, la cifra che sia `README.md:78` sia `docs/DATASET.md:17` dichiarano. Il capitolo 4.2 ha già mostrato questa discrepanza; questo capitolo ne chiude l’origine con un dato preciso: [**Fatto**] il sottoinsieme Cleveland da solo, verificato in questa sessione, ha esattamente 4 righe con `ca` mancante e 2 con `thal` mancante — $4 + 2 = 6$, e $303 - 6 = 297$. La cifra storicamente citata in letteratura corrisponde, con ogni evidenza, al solo centro di Cleveland dopo aver scartato le righe con dati mancanti nelle due feature più problematiche — non ai quattro centri concatenati che questo progetto usa davvero.

29.2 Schema delle 14 feature e valori mancanti mascherati da zero

[**Fatto**] Le 14 colonne standard, nell’ordine usato dal progetto (`function.py:10-13`):

#	Colonna	Significato	Tipo	Note
1	<code>age</code>	Età in anni	Numerica	
2	<code>sex</code>	Sesso (1=maschio, 0=femmina)	Categoriale	

#	Colonna	Significato	Tipo	Note
3	cp	Tipo di dolore toracico	Categoriale	4 codici, CP_LABELS in <code>embedding.py:23</code>
4	trestbps	Pressione arteriosa a riposo	Numerica	mm Hg
5	chol	Colesterolo sierico	Numerica	mg/dl
6	fbs	Glicemia a digiuno > 120 mg/dl	Categoriale	booleano
7	restecg	Risultati elettrocardiografici a riposo	Categoriale	3 codici, RESTECG_LABELS
8	thalach	Frequenza cardiaca massima raggiunta	Numerica	
9	exang	Angina indotta da sforzo	Categoriale	booleano
10	oldpeak	Depressione ST indotta da sforzo	Numerica	
11	slope	Pendenza del segmento ST da sforzo	Categoriale	3 codici, SLOPE_LABELS
12	ca	Numero di vasi principali colorati da fluoroscopia	Numerica	0-3
13	thal	Talassemia	Categoriale	3 codici, THAL_LABELS
14	num	Diagnosi (target)	—	binarizzato (num>0)

[Fatto] Oltre ai valori mancanti espliciti (marcati ? nel file sorgente, letti come NaN da pandas con `na_values="?"`, `function.py:103`), `function.py:110-111` applica una correzione specifica:

```
\NormalTok{df[\StringTok{"chol"}\NormalTok{]} \OperatorTok{=} \NormalTok{df}
↪ df[\StringTok{"chol"}\NormalTok{] .replace(\DecValTok{0}\NormalTok{, np.nan)}
\NormalTok{df[\StringTok{"trestbps"}\NormalTok{]} \OperatorTok{=} \NormalTok{df}
↪ df[\StringTok{"trestbps"}\NormalTok{] .replace(\DecValTok{0}\NormalTok{, np.nan)}
```

[Fatto] Un valore di colesterolo o di pressione arteriosa pari a zero non è fisiologicamente possibile per un paziente vivo: è, con ogni evidenza, un secondo modo — oltre al ? — con cui alcuni dei quattro centri hanno codificato l'assenza di misurazione. **[Fatto]** Verificato per file sorgente in questa sessione: **Switzerland ha chol == 0 in tutte e 123 le righe**, VA in 49 delle sue 200, mentre Cleveland e Hungarian non hanno mai questo valore per il colesterolo; **trestbps == 0** compare una sola volta, in VA. **[Da verificare]** Questo dato misurato non corrisponde perfettamente al commento del codice (`function.py:107-109`) e a `docs/DATASET.md:56`, che attribuiscono il fenomeno a “Switzerland e Hungary”: la mia verifica diretta sul file sorgente non trova alcuna occorrenza di `chol==0` nel file Hungarian — è una piccola imprecisione nella documentazio-

ne/commento del progetto, non nel comportamento del codice stesso (la riga `function.py:110` applica comunque la correzione corretta, converte 0 in NaN indipendentemente dal centro di provenienza).

29.3 Limiti noti

[Fatto] `docs/DATASET.md:52-56` dichiara esplicitamente quattro limiti: dati storici (anni '80-'90, possibile obsolescenza rispetto agli standard diagnostici attuali), sbilanciamento di classe nei dati originali (mitigato da SMOTENC, capitolo 21.2 — ma il capitolo 4.2 ha già mostrato che lo sbilanciamento reale è comunque mite, 55.3%/44.7%), bias geografico (prevalentemente centri occidentali), e valori mancanti gestiti tramite imputazione.

[Fatto] Quest'ultimo punto merita la precisione che la documentazione del progetto non fornisce: sulle 920 righe concatenate, `ca` manca nel **66.4%** dei casi e `thal` nel **52.8%** (comando Python eseguito, capitolo 4.2 aveva già introdotto questi due numeri). Scomposto per centro: `ca` manca nell'1% di Cleveland ma nel **99% di Hungarian, 96% di Switzerland, 99% di VA**; `thal` manca nell'1% di Cleveland ma nel **90% di Hungarian, 42% di Switzerland, 83% di VA**. **[Interpretazione]** In pratica, per due delle 14 feature standard di questo dataset, il valore reale è disponibile quasi solo per il centro di Cleveland: negli altri tre centri, l'imputazione con mediana o moda (`preprocessing.py:76-82`) non sta “colmando qualche buco” — sta assegnando lo stesso valore centrale, derivato quasi interamente da un quarto dei centri, alla stragrande maggioranza dei pazienti degli altri tre. Questo non emerge in nessun punto della documentazione esistente del progetto, ed è un limite sostanzialmente più serio di quanto “gestito tramite imputazione” lasci intendere.

ATTENZIONE — questo limite si propaga silenziosamente fino al testo generato per l'embedding: `record_to_text_heart_disease()` (capitolo 22.1) scrive sempre un valore concreto per `thal` (per esempio “thalassemia: normal”), mai “non registrato”, perché a quel punto della pipeline il valore mancante è già stato sostituito dall'imputazione. Un lettore del testo — umano o modello linguistico — non ha alcun modo di distinguere una misurazione reale da un valore imputato sulla maggioranza dei casi.

Riepilogo

Heart Disease è un dataset a quattro centri, non a centro singolo: la cifra “297” della documentazione del progetto corrisponde al solo Cleveland dopo aver scartato 6 righe con dati mancanti critici, mentre il codice reale concatena tutti e quattro i centri per un totale di 920 righe. Due delle 14 feature standard (`ca`, `thal`) hanno dati realmente osservati quasi solo per Cleveland: negli altri tre centri, l'imputazione sostituisce con un valore centrale dal 42% al 99% dei valori, un limite quantificato qui per la prima volta con precisione.

Domande di autoverifica

1. Da dove viene, con ogni evidenza, la cifra “297” citata da `README.md` e `docs/DATASET.md` per questo dataset? Dal solo centro di Cleveland (303 righe) dopo aver scartato le 6 righe con valori mancanti in `ca` (4 righe) o `thal` (2 righe) — un sottoinsieme diverso, più piccolo e mono-centro, da quello che il codice del progetto carica realmente (4 centri, 920 righe, nessuna riga scartata).

2. Perché la correzione `chol.replace(0, np.nan)` in `function.py:110` è necessaria oltre alla gestione dei ? già presenti nel file? Perché alcuni centri clinici (verificato: Switzerland

per tutte le sue righe, VA per una parte) codificano un valore mancante di colesterolo come 0 invece che come ? — 0 mg/dl non è un valore fisiologicamente possibile, quindi va trattato come mancante quanto un ? esplicito.

3. Perché “l'imputazione gestisce i valori mancanti” è una descrizione insufficiente per le colonne `ca` e `thal` di questo dataset? Perché per queste due colonne i valori mancanti non sono un'eccezione isolata ma la norma in tre dei quattro centri (dal 42% al 99%): l'imputazione con mediana o moda, in questi casi, non completa dati sporadicamente assenti ma sostituisce la stragrande maggioranza dei valori con una singola stima aggregata derivata quasi interamente da un solo centro.

MATERIALE PER LA TESI 1. La tabella completa delle 14 feature con tipo e significato (§29.2) — riusabile direttamente come tabella descrittiva del dataset in “Materiali e metodi”. 2. La scomposizione per centro clinico della mancanza di dati in `ca` e `thal`, con la spiegazione precisa dell'origine della cifra “297” — è probabilmente la seconda osservazione critica più forte del libro dopo quella sul report statico: riusabile integralmente nella sezione “Discussione e limiti”. 3. L'osservazione sulla propagazione silenziosa del valore imputato nel testo generato per l'embedding — riusabile come argomento specifico contro un'interpretazione troppo ottimistica delle prestazioni dei modelli biomedici su queste due feature.

Capitolo 30 — Il dataset Diabetes 130-US Hospitals

Obiettivi del capitolo

- Capire la scala reale di questo dataset e perché il progetto ne usa solo una parte campionata.
- Sapere quali 19 feature, fra le circa 50 originali, sono state scelte e perché le altre sono state escluse.
- Riconoscere un limite di qualità dei dati specifico di questo dataset, analogo ma indipendente da quello già visto per Heart Disease.

30.1 Scala, 101.766 righe, e campionamento a 20.000

[Fatto] Il file sorgente (`diabetic_data.csv`) contiene **101.766 righe** (verificato con `wc -l`, che dà 101.767 includendo l'intestazione), ciascuna un ricovero ospedaliero di un paziente diabetico in uno fra 130 ospedali statunitensi, raccolto fra il 1999 e il 2008 — [Fatto] con fonte citabile: Strack, B., DeShazo, J.P., Gennings, C., Olmo, J.L., Ventura, S., Cios, K.J., & Clore, J. (2014). *Diabetes 130-US Hospitals for Years 1999-2008*. UCI Machine Learning Repository. DOI: 10.24432/C5230J.

[Fatto] `load_diabetes130(sample_size=20000, random_state=42)` (`function.py:116-132`) non usa l'intero file: campiona 20.000 righe con `train_test_split(df, train_size=sample_size, stratify=df["readmitted"], random_state=random_state)` (riga 127-129) — uno split stratificato usato qui non per separare training e test, ma solo per **estrarre un sottoinsieme** che preservi la proporzione originale di riammissioni, scartando esplicitamente l'altra parte (`_`, l'assegnazione a variabile anonima già vista come convenzione al capitolo 8.2). [Interpretazione] Un campione stratificato di un quinto dei dati originali è una scelta ragionevole per contenere i tempi di generazione degli embedding (sette modelli, quattro dei quali via rete verso Ollama) — ma significa che ogni conclusione tratta su questo dataset riguarda quel campione specifico, non l'intera popolazione di 101.766 ricoveri, un punto che il capitolo 51 riprende.

30.2 Le 19 feature scelte tra ~50

[Fatto] Il file grezzo ha 50 colonne; `columns_diabetes130` (`function.py:21-27`) ne mantiene 19:

Categoria	Colonne	Numero
Demografia	<code>race, gender, age</code>	3
Contesto del ricovero	<code>admission_type_id,</code> <code>discharge_disposition_id,</code> <code>admission_source_id</code>	3

Categoria	Colonne	Numero
Utilizzo dell'assistenza (numerico)	<code>time_in_hospital</code> , <code>num_lab_procedures</code> , <code>num_procedures</code> , <code>num_medications</code> , <code>number_outpatient</code> , <code>number_emergency</code> , <code>number_inpatient</code> , <code>number_diagnoses</code>	8
Laboratorio/farmaci (categoriale)	<code>max_glu_serum</code> , <code>A1Cresult</code> , <code>insulin</code> , <code>change</code> , <code>diabetesMed</code>	5
Target	<code>readmitted</code>	1

[Fatto] `docs/DATASET.md:83` motiva l'esclusione delle circa 31 colonne rimanenti: la maggior parte sono flag di dosaggio farmacologico quasi costanti (per esempio `examide`, `citoglipton` — nomi di farmaci specifici, ciascuno con una colonna dedicata nel file originale, quasi sempre allo stesso valore per ogni paziente) o identificatori ad altissima percentuale di valori mancanti (`weight`, `payer_code`) o non generalizzabili (`patient_nbr`, un identificativo di paziente). [Da verificare] Anche i codici diagnostici ICD (`diag_1`, `diag_2`, `diag_3`), potenzialmente informativi, sono esclusi — `docs/DATASET.md:106` lo dichiara una scelta esplicita per evitare l'alta cardinalità di questi campi, ma non ne quantifica il costo in termini di segnale perso: resta una domanda aperta se la loro esclusione influisca in modo sostanziale sulle prestazioni osservate.

[Fatto] Fra le 19 feature mantenute, due presentano lo stesso tipo di problema già visto per `ca/thal` in Heart Disease (capitolo 29.3), verificato sull'intero file grezzo prima del campionamento: **`max_glu_serum` manca nel 94.7%** delle 101.766 righe, **`A1Cresult` manca nell'83.3%**. [Fatto] Entrambe sono trattate come categoriali (`cat_cols_diabetes130`, `function.py:32-35`) e quindi imputate con la moda (`preprocessing.py:80-81`) — per la stragrande maggioranza dei pazienti, il valore di questi due esami di laboratorio non è mai stato osservato ma sostituito dal valore più frequente osservato nel 5.3%/16.7% dei casi in cui erano davvero presenti.

ATTENZIONE — a differenza di `ca/thal` in Heart Disease, qui la causa della mancanza non è un centro clinico che ha smesso di raccogliere quel dato: `max_glu_serum` e `A1Cresult` sono esami di laboratorio specifici (glicemia e emoglobina glicata), richiesti solo quando clinicamente indicato — un'assenza che potrebbe essa stessa portare informazione clinica (il test non richiesto potrebbe correlare con una gestione meno intensiva del paziente), invece di essere puro rumore da imputare via. Trattarla come un semplice valore mancante da colmare con la moda, come fa questo progetto, scarta silenziosamente questa possibile informazione — un'assunzione implicita sui dati, non discussa da nessuna parte nella documentazione esistente.

30.3 La ridefinizione del target

[Fatto] La colonna originale `readmitted` ha tre valori possibili nel file sorgente: riammesso entro 30 giorni, riammesso dopo più di 30 giorni, mai riammesso. [Fatto] `function.py:124` la binarizza con `(df["readmitted"] == "<30").astype(int)`: la classe positiva è **solo** la riammissione entro 30 giorni; sia “riammesso più tardi” sia “mai riammesso” diventano entrambe classe negativa. [Livello: teoria consolidata del settore] Questa è effettivamente la definizione di benchmark più diffusa in letteratura per questo dataset (`docs/DATASET.md:80,107`),

non un'invenzione del progetto — ma è una scelta di framing con una conseguenza precisa: il modello non impara a distinguere “paziente che tornerà” da “paziente che non tornerà mai”, impara a distinguere “tornerà entro un mese” da “tutto il resto”, incluso chi tornerà fra sei mesi. Il capitolo 53 lo riprende quando discute cosa questo progetto prova, e cosa no.

Riepilogo

Diabetes130 conta 101.766 ricoveri originali, di cui il progetto ne usa un campione stratificato di 20.000; 19 delle circa 50 colonne originali sono mantenute, escludendo flag di dosaggio quasi costanti, identificatori ad alta cardinalità o mancanza, e i codici diagnostici ICD. Due delle 19 feature mantenute (`max_glu_serum`, `A1Cresult`) sono mancanti nella grande maggioranza dei casi e vengono comunque imputate con la moda, un limite di qualità dei dati analogo a quello di Heart Disease ma con una causa probabilmente diversa e potenzialmente più informativa. Il target è ristretto, per definizione di benchmark consolidata in letteratura, alla sola riammissione entro 30 giorni.

Domande di autoverifica

1. Perché il progetto usa un campione di 20.000 righe invece dell'intero file di 101.766? Per contenere i tempi della fase più costosa della pipeline — la generazione di embedding per sette modelli diversi, quattro dei quali via chiamate di rete a un server locale — mantenendo però, grazie al campionamento stratificato, la stessa proporzione di riammissioni del dataset completo.

2. Perché l'assenza di `max_glu_serum` in un ricovero potrebbe essere informazione clinica, non solo rumore da imputare? Perché questo test di laboratorio viene richiesto solo quando clinicamente indicato: la sua assenza potrebbe correlare con una gestione meno intensiva o con un profilo di rischio diverso del paziente, un segnale che l'imputazione con la moda scarta trattando l'assenza come puro dato mancante.

3. Cosa NON impara a distinguere un modello addestrato su questo target binarizzato? Non impara a distinguere “il paziente tornerà in ospedale” da “non tornerà mai”: impara solo a distinguere “tornerà entro 30 giorni” da “tutto il resto”, che include sia chi non tornerà mai sia chi tornerà, per esempio, dopo sei mesi — entrambi classificati allo stesso modo come classe negativa.

MATERIALE PER LA TESI 1. La tabella delle 19 feature per categoria, con la motivazione dell'esclusione delle altre circa 31 colonne — riusabile in “Materiali e metodi” per descrivere la selezione delle feature. 2. Il dato quantificato sulla mancanza di `max_glu_serum` e `A1Cresult`, con l'ipotesi che l'assenza stessa sia informativa — riusabile come punto di discussione originale nella sezione “Limiti”, parallelo ma distinto da quello di Heart Disease. 3. La spiegazione precisa di cosa il target binarizzato misura e cosa no — riusabile per calibrare correttamente, nell'introduzione della tesi, l'affermazione su cosa il sistema “prevede” davvero.

Capitolo 31 — Qualità dei dati e casi limite

Obiettivi del capitolo

- Avere una vista d'insieme, sui due dataset insieme, di come il progetto tratta i valori mancanti in tutte le loro forme.
- Capire cosa genera esattamente SMOTENC quando crea un record sintetico, con un esempio concreto.
- Sapere cosa succede, nella conversione a testo, ai casi limite: valori mancanti residui, categorie sconosciute, record sintetici.

31.1 Valori mancanti espliciti e impliciti

[Fatto] I capitoli 29 e 30 hanno già mostrato, con numeri precisi, che entrambi i dataset di questo progetto hanno un problema di mancanza dei dati molto più serio di quanto la documentazione esistente lasci intendere: `ca` e `thal` mancanti nel 66% e 53% dei casi in Heart Disease (quasi solo nei tre centri non-Cleveland), `max_glu_serum` e `A1Cresult` mancanti nel 95% e 83% dei casi in Diabetes130. Vale la pena, qui, distinguere esplicitamente le **tre forme diverse** in cui “mancante” si presenta in questo progetto, perché il codice le tratta con meccanismi diversi:

1. **Mancante esplicito, marcato ?** — la forma più comune, letta direttamente come `NaN` da pandas con `na_values="?"` (`function.py:103,118`).
2. **Mancante mascherato da un valore fisiologicamente impossibile** — solo per `chol/trestbps` di Heart Disease, convertito esplicitamente in `NaN` da una riga dedicata (`function.py:110-111`, capitolo 29.2). Nessun controllo equivalente esiste per Diabetes130: se quel dataset avesse un problema analogo su una colonna numerica (per esempio un valore implausibile per `time_in_hospital`), il codice attuale non lo intercetterebbe.
3. **Mancante per assenza di indicazione clinica**, non per un guasto di raccolta dati — il caso di `max_glu_serum/A1Cresult` discusso al capitolo 30.2, dove l'assenza stessa potrebbe essere informazione, trattata invece come le altre due forme.

[Fatto] Tutte e tre le forme, una volta arrivate a `impute_raw()` (`preprocessing.py:76-82`), ricevono lo stesso trattamento indistinto: mediana per le colonne numeriche, moda per le categoriali. Il codice non distingue in alcun modo, a questo stadio, se un valore mancante rappresenti un dato perso, un dato impossibile da misurare in quel centro, o un test clinicamente non richiesto.

31.2 SMOTENC su feature miste: cosa genera un record sintetico

[Livello: teoria consolidata del settore] Per un record sintetico che deve sostituire una feature numerica, SMOTENC sceglie due (o più) vicini reali della classe minoritaria e interpola linearmente fra i loro valori — un'età sintetica di 54.3 anni, per esempio, se i due vicini hanno 52

e 58 anni. Per una feature categoriale, l'interpolazione lineare non avrebbe senso (“interpolare” fra “maschio” e “femmina” non produce una categoria valida), quindi SMOTENC assegna invece il valore **più frequente fra i vicini usati per generare quel record** — non un'invenzione, ma un prestito diretto da un caso reale osservato nelle vicinanze.

[Fatto] `balance_classes()` (`preprocessing.py:84-92`) applica questo meccanismo dopo l'imputazione (righe 47-53 di `preprocessing.py`, ordine confermato leggendo `preprocessing_data()`), il che significa che un record sintetico può a sua volta essere costruito interpolando fra valori già imputati — se due vicini reali avessero entrambi `ca` imputato con la stessa mediana (probabile, dato che il 66% dei valori di `ca` in Heart Disease sono quella stessa mediana, capitolo 29.3), il record sintetico erediterebbe quello stesso valore imputato, non uno nuovo. **[Interpretazione]** Questo significa che un record sintetico “eredita” silenziosamente il problema di qualità dei dati del capitolo 31.1: non lo introduce, ma nemmeno lo corregge — un record sintetico basato su vicini con `ca` imputato avrà quasi certamente anch'esso `ca` uguale al valore imputato, amplificando ulteriormente, nel training set finale, la presenza di quel singolo valore centrale al posto di una vera variabilità clinica.

PROVA TU — apri `datas/heart_disease/preprocessing/X_train_raw.csv` (già presente nel repository, generato da un'esecuzione precedente) e conta quante righe hanno lo stesso identico valore di `ca` — un numero sorprendentemente alto per una feature che dovrebbe assumere solo 4 valori interi (0-3) osservati su un continuo di pazienti reali e sintetici. Non è un errore: è la conseguenza diretta e verificabile di quanto appena descritto.

31.3 Casi limite nella conversione a testo

[Fatto] Il capitolo 22.1 ha già mostrato che `_fmt_num()`, `_fmt_cat()`, `_fmt_bool()` e `_fmt_raw()` (`embedding.py:28-41,61-62`) gestiscono tutte esplicitamente un valore NaN residuo, scrivendo “not recorded” — un margine di sicurezza che, dato quanto visto in questo capitolo, non dovrebbe mai attivarsi nella pratica: se l'imputazione ha già sostituito ogni NaN con un valore concreto prima di questo punto della pipeline, `pd.isna(value)` dovrebbe restituire sempre `False` quando queste funzioni vengono chiamate. **[Da verificare]** Se questo margine di sicurezza sia mai stato osservato attivarsi in una vera esecuzione, o sia puro codice difensivo per un caso che nella pipeline attuale non si presenta mai — non ho eseguito la pipeline in questa sessione (per scelta esplicita, si veda il capitolo 43) per verificarlo direttamente, e resta una domanda aperta per l'Appendice E.

[Fatto] Un secondo caso limite riguarda `_fmt_cat()` (`embedding.py:33-36`), che traduce un codice numerico in un'etichetta leggibile cercandolo in un dizionario come `CP_LABELS`:

```
\KeywordTok{def}\NormalTok{ \_fmt\_cat(value, labels):}
  \ControlFlowTok{if}\NormalTok{ pd.isna(value):}
    \ControlFlowTok{return} \StringTok{"not recorded"}
  \ControlFlowTok{return}\NormalTok{ labels.get(\}\BuiltInTok{int}\}\NormalTok{(\}\BuiltInTok{int}\}\NormalTok{(\}\BuiltInTok{round}\}\NormalTok{(\}\BuiltInTok{float}\}\NormalTok{(\}\NormalTok{(value))),}
  \ControlFlowTok{return} \StringTok{"unknown"}\NormalTok{(\}\NormalTok{)}
```

`labels.get(chiave, "unknown")` restituisce “unknown” se il codice, dopo essere stato arrotondato all'intero più vicino, non compare fra le chiavi del dizionario — un valore come 2.6 per `cp` (che non dovrebbe mai comparire in un dato reale, dove `cp` è un codice intero fra 1 e 4, ma **potrebbe** comparire in un record sintetico se SMOTENC, per errore di configurazione, trattasse per sbaglio una colonna categoriale come numerica) diventerebbe 3 dopo l'arrotondamento, e cercherebbe comunque una voce valida nel dizionario. **[Interpretazione]** Questo non è, in questo progetto, un rischio reale: `balance_classes()` (capitolo 21.2) passa esplicitamente

l'elenco delle colonne categoriali a SMOTENC (`categorical_features=cat_idx`), che le tratta correttamente senza mai interpolarle come numeriche — ma il fatto che `_fmt_cat()` gestisca comunque il caso “codice sconosciuto” con un ripiego pulito (`"unknown"`, non un errore) è una buona pratica difensiva, verificabile a colpo d'occhio, indipendentemente dal fatto che si attivi mai nella pratica.

Riepilogo

I due dataset di questo progetto presentano tre forme distinte di mancanza dei dati — esplicita, mascherata da un valore impossibile, e implicita per assenza di indicazione clinica — trattate tutte allo stesso modo indistinto dall'imputazione. SMOTENC, applicato dopo l'imputazione, può ereditare e amplificare silenziosamente questo problema nei record sintetici. La conversione finale a testo gestisce con cura i casi limite residui (valori ancora mancanti, codici categoriali sconosciuti), anche se almeno uno di questi margini di sicurezza non risulta mai attivarsi nella pipeline così come è costruita oggi.

Domande di autoverifica

- 1. Quali sono le tre forme distinte di “valore mancante” che compaiono in questo progetto, e come le tratta ciascuna il codice?** Mancante esplicito (marcato `?`, letto come `NaN`), mancante mascherato da un valore fisiologicamente impossibile (0 per colesterolo/pressione, convertito esplicitamente in `NaN` solo per Heart Disease), e mancante per assenza di indicazione clinica (esami di laboratorio non richiesti in Diabetes130). Tutte e tre ricevono lo stesso trattamento indistinto in `impute_raw()`: mediana o moda, senza distinzione di causa.
- 2. Perché un record sintetico generato da SMOTENC può avere lo stesso valore imputato di *ca* dei suoi vicini reali, invece di un valore nuovo?** Perché SMOTENC interpola fra vicini reali già passati per l'imputazione: se quei vicini condividono lo stesso valore imputato (molto probabile per *ca*, imputato con la stessa mediana nel 66% dei casi), il record sintetico erediterà quel valore o uno molto simile, non una nuova osservazione indipendente.
- 3. In quale scenario, teoricamente possibile ma non realizzato in questo progetto, `_fmt_cat()` restituirebbe `"unknown"` invece di un'etichetta valida?** Se un valore categoriale, dopo arrotondamento all'intero più vicino, non corrispondesse a nessuna chiave del dizionario di etichette — uno scenario che richiederebbe un record con un codice categoriale non standard, cosa che SMOTENC non produce in questo progetto perché tratta correttamente le colonne categoriali come tali, non come numeriche.

MATERIALE PER LA TESI 1. La tassonomia delle tre forme di mancanza dei dati, con il trattamento uniforme che il codice applica a tutte — riusabile in “Materiali e metodi” per una descrizione rigorosa della qualità dei dati. 2. L'analisi dell'interazione fra imputazione e SMOTENC, con l'ipotesi verificabile sull'amplificazione del valore imputato nei record sintetici — riusabile come punto di discussione originale, con l'esercizio pratico di verifica (§31.2) come base per una figura o una tabella nella tesi. 3. L'osservazione sul margine di sicurezza mai attivato in `_fmt_num/_fmt_cat/_fmt_bool`, esplicitamente marcata come domanda aperta — riusabile in Appendice E, e come esempio di codice difensivo scritto per un caso che la pipeline attuale non produce mai.

Capitolo 32 — La regressione logistica, dalle basi alla riga di codice

Obiettivi del capitolo

- Avere la formula completa della regressione logistica, con ogni simbolo tradotto in ciò che rappresenta in questo progetto.
- Sapere cosa minimizza `LogisticRegression.fit()` quando viene chiamato, incluso il ruolo esatto del parametro `C`.
- Capire perché un classificatore lineare su un embedding congelato è una scelta metodologica precisa, non una semplificazione di comodo.

32.1 Funzione logistica come probabilità

[Livello: teoria consolidata del settore] La regressione logistica calcola prima una combinazione lineare dell'input, poi la comprime in un numero fra 0 e 1 con la funzione logistica (sigmoide):

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \quad (32.1)$$

[Fatto] In questo progetto, $\mathbf{x}$ è l'embedding di un record clinico (un vettore di 768 o 1024 numeri, capitolo 5.1); $\mathbf{w}$ è un vettore di pesi della stessa dimensione, appreso durante l'addestramento; b è un singolo numero (l'intercetta); $P(y = 1 \mid \mathbf{x})$ è ciò che il codice chiama `y_score` (`classification.py:28`, capitolo 23.1). Ogni componente di $\mathbf{w}$ pesa una delle 768 (o 1024) direzioni dello spazio di embedding: la regressione logistica impara, in sostanza, *quali direzioni* di quello spazio sono associate alla classe positiva.

SE VIENI DA JAVA — non esiste, in questo progetto, un solo vettore $\mathbf{w}$ condiviso: `training_classifier()` (capitolo 23) istanzia un `LogisticRegression` nuovo per ciascuno dei sette modelli (`classification.py:16`, dentro il ciclo `for model in models_all`), quindi $\mathbf{w}$ ha una dimensione diversa (768 o 1024, a seconda del modello di embedding) e valori completamente indipendenti per ciascuno — sette problemi di ottimizzazione separati, non uno condiviso.

32.2 Funzione di costo e ottimizzazione

[Livello: teoria consolidata del settore] `LogisticRegression.fit()` (`classification.py:26`) non sceglie $\mathbf{w}$ e b arbitrariamente: li trova risolvendo un problema di minimizzazione. [Fatto] Con la penalizzazione di default di scikit-learn (L2, mai cambiata in questo progetto — verificato: `classification.py:16` chiama `LogisticRegression(max_iter=2000)` senza specificare `penalty`), la funzione minimizzata è, nella notazione della documentazione ufficiale

di scikit-learn (con le etichette codificate come $y_i \in \{-1, +1\}$, non $\{0, 1\}$ come nei dati salvati su disco — una convenzione puramente interna al solutore):

$$\min_{\mathbf{w}, b} \frac{1}{2} \mathbf{w}^\top \mathbf{w} + C \sum_{i=1}^n \log \left(1 + e^{-y_i (\mathbf{w}^\top \mathbf{x}_i + b)} \right) \quad (32.2)$$

[Fatto] Il primo termine, $\frac{1}{2} \mathbf{w}^\top \mathbf{w}$, è la **penalizzazione L2** (o *ridge*): più grandi sono i pesi, più alto è il costo, il che scoraggia il modello dall'affidarsi in modo estremo a poche direzioni dell'embedding. Il secondo termine è la somma, su tutti gli n esempi di addestramento, della **log-loss** (o *cross-entropy* binaria): penalizza il modello tanto più quanto la sua probabilità prevista si discosta dall'etichetta vera. Il parametro C bilancia i due termini: [Fatto] `classification.py:16` non lo specifica mai, quindi resta al default di scikit-learn, $C = 1.0$, per tutti e sette i modelli e per entrambi i dataset — un iperparametro reale del progetto, mai reso esplicito né validato, che il capitolo 39 riprende per intero.

[Fatto] `max_iter=2000` (l'unico parametro esplicitamente impostato) non è parte della formula: è un limite al numero di iterazioni che l'algoritmo di ottimizzazione (il *solver*, di default `lbfgs` in scikit-learn, non cambiato in questo progetto) può compiere prima di fermarsi comunque, anche se non ha ancora raggiunto una soluzione stabile — un valore più alto del default di libreria (100), presumibilmente scelto perché con dati ad alta dimensionalità (768-1024 colonne) l'ottimizzazione può richiedere più iterazioni per convergere.

32.3 Perché lineare su embedding, non una rete profonda

[Interpretazione] Un embedding pre-addestrato e mai riaddestrato (capitolo 5, capitolo 22) più un classificatore lineare addestrato sopra è un pattern con un nome riconosciuto in letteratura sulla rappresentazione: **linear probing** — “sondare” con un modello semplicissimo se una rappresentazione già pronta contiene abbastanza segnale per il compito, senza modificare la rappresentazione stessa. È l'opposto del *fine-tuning* (riaddestrare anche l'embedding) o di impilare una rete neurale profonda sopra l'embedding congelato.

[Fatto] Per Heart Disease, questa scelta non è solo prudente: è quasi necessaria. Il pool di addestramento dopo SMOTENC è di **814 righe** (capitolo 21.3), circa **651 per fold** di training con 5-fold cross-validation (capitolo 23.1) — verificato caricando i file di embedding con NumPy. Per i due modelli a 1024 dimensioni (gte-large, e5-large), questo significa che il classificatore ha **più pesi da stimare (1024) che esempi su cui stimarli (~651) in ciascun fold**: un rapporto di 1.57, un regime statisticamente delicato in cui un modello con ancora più parametri (una rete neurale profonda) sarebbe quasi certamente destinato all'overfitting (capitolo 2.3) su un dataset di questa scala.

[Fatto] Per Diabetes130, la situazione è opposta: il pool post-SMOTENC ha **28.428 righe** (verificato caricando `bioclinicalbert_embeddings.npy`), circa **22.743 per fold** — il rapporto dimensione/esempi per un modello a 1024 dimensioni scende a **0.045**, un regime ben più sicuro. [Da verificare] Curiosamente, gli embedding a 1024 dimensioni (e5-large, gte-large) **non esistono più** su disco per questo dataset — né in git (`.gitignore:12-13` li esclude esplicitamente per nome) né in questo ambiente locale (verificato). [Inferenza] Alle dimensioni di questo dataset, ciascuno di quei due file peserebbe circa 111 MB, sopra il limite di 100 MB per file imposto da GitHub — una spiegazione plausibile, coerente con la dimensione osservata (87.3 MB) del file analogo a 768 dimensioni nello stesso commit storico. I risultati per questi due modelli su Diabetes130 esistono comunque (`datas/diabetes130/results/`, capitolo 45): sono stati calcolati in un momento in cui gli embedding erano presenti, prima di essere rimossi dal repository.

ATTENZIONE — questa asimmetria — regime $p > n$ per Heart Disease, $p \ll n$ per Diabetes130 — non è mai discussa nella documentazione esistente del progetto, ma cambia sostanzialmente quanto ci si può fidare delle prestazioni dei modelli più grandi sul dataset più piccolo. Il capitolo 51 lo riprende come limite metodologico specifico, distinto da quelli già discussi.

Riepilogo

La regressione logistica di questo progetto stima, per ciascuno dei sette modelli separatamente, un vettore di pesi che pondera le direzioni dello spazio di embedding, minimizzando una combinazione di penalizzazione L2 e log-loss regolata dal parametro C — mai esplicitato, quindi sempre al default $C=1.0$. La scelta di un classificatore lineare su embedding congelati (linear probing) è particolarmente giustificata per Heart Disease, dove il rapporto fra dimensione dell’embedding e dimensione del training set per fold rende un modello più complesso rischioso; per Diabetes130 la stessa scelta è più prudente che necessaria, dato il pool molto più ampio.

Domande di autoverifica

1. Cosa rappresentano, in questo progetto, i simboli w e b della Formula 32.1? w è il vettore di pesi appreso durante l’addestramento, della stessa dimensione dell’embedding (768 o 1024); b è l’intercetta, un singolo numero. Insieme definiscono la combinazione lineare dell’embedding che, passata attraverso la funzione logistica, produce la probabilità stimata della classe positiva.

2. Perché $C=1.0$ è un iperparametro reale del progetto, anche se non compare mai esplicitamente nel codice? Perché è il valore di default di scikit-learn per la forza della regolarizzazione L2, applicato automaticamente ogni volta che `LogisticRegression(max_iter=2000)` viene istanziato senza specificare C — una scelta implicita quanto una esplicita, semplicemente mai dichiarata né validata.

3. Perché il rapporto fra dimensione dell’embedding e dimensione del training set per fold è un problema per Heart Disease ma non per Diabetes130? Perché Heart Disease ha solo ~651 esempi di training per fold contro le 1024 dimensioni di alcuni embedding (rapporto 1.57, più parametri che esempi), mentre Diabetes130 ne ha ~22.743 (rapporto 0.045) — lo stesso classificatore lineare si trova in un regime statisticamente molto più sicuro su un dataset che in origine, prima del campionamento, era comunque molto più grande.

MATERIALE PER LA TESI 1. La Formula 32.1 e la Formula 32.2 con la traduzione completa di ogni simbolo nei termini del progetto — riusabili integralmente in “Materiali e metodi”, sezione sul modello. 2. L’analisi del rapporto dimensione-embedding/dimensione-training-set per entrambi i dataset, con i numeri esatti verificati — è un’osservazione metodologica originale e ben quantificata: riusabile nella sezione “Discussione e limiti”. 3. La spiegazione della scomparsa dei file di embedding a 1024 dimensioni per Diabetes130, con l’ipotesi motivata sul limite di dimensione di GitHub — riusabile come nota tecnica sulla riproducibilità, o in Appendice E come domanda da confermare con chi ha scritto il codice.

Capitolo 33 — Come si valida correttamente un modello di classificazione

Obiettivi del capitolo

- Sapere a cosa serve, in linea di principio, dividere i dati in training, validazione e test — tre ruoli distinti, non intercambiabili.
- Avere la formula precisa del k-fold stratificato, e sapere perché “stratificato” non è un dettaglio opzionale.
- Riconoscere le forme più comuni di data leakage, e collocarci con precisione i due punti di questo progetto già individuati nei capitoli precedenti.

Questo capitolo tratta la validazione come argomento a sé, indipendente da questo progetto specifico — il tuo prompt originale la chiedeva esplicitamente come argomento di base, e diluirla solo dentro i capitoli sui singoli moduli (Parte V) avrebbe reso più difficile vederne il quadro generale prima dei dettagli.

33.1 Train/validation/test: a cosa serve ciascuno

[Livello: teoria consolidata del settore] Una pipeline di apprendimento supervisionato (capitolo 2.2) ha, in linea di principio, bisogno di **tre** porzioni di dati distinte, non due:

- Il **training set** addestra il modello: i suoi parametri (i pesi $\mathbf{w}$ e l’intercetta b della Formula 32.1) sono scelti per adattarsi a questi dati.
- Il **validation set** guida le decisioni che un umano (o una procedura automatica) prende *durante* lo sviluppo: quale soglia di decisione usare, quale valore di un iperparametro, quale fra due modelli preferire. Il modello non viene mai addestrato su questi dati, ma le sue prestazioni su di essi influenzano scelte a valle.
- Il **test set** stima, una volta sola e alla fine, quanto il modello — con tutte le decisioni già prese sul validation set — generalizzerà su dati mai visti in nessuna fase precedente. Se il test set viene consultato più di una volta per aggiustare qualcosa, smette di essere un test set: diventa, di fatto, un secondo validation set, e la stima di generalizzazione che fornisce torna a essere ottimista.

[Fatto] `preprocessing.py:43` crea esplicitamente tutte e tre le porzioni concettuali... a metà: `train_test_split(X, y, test_size=0.2, ...)` produce un `X_test/y_test` che dovrebbe giocare il terzo ruolo. Ma, come già mostrato al capitolo 21.1, quella porzione **non viene mai più usata** in nessun punto del progetto. [Interpretazione] Il progetto ha quindi, di fatto, solo due porzioni funzionanti: un training/validation set (suddiviso poi in 5 fold da `classification.py`, capitolo 33.2) e un test set calcolato ma sprecato. Nessuna cifra riportata nei capitoli 44-45 di questo libro proviene da un test set mai toccato durante lo sviluppo: tutte derivano dalla

validazione incrociata, la cui stima di generalizzazione — pur essendo una pratica comune e ragionevole — non ha la stessa garanzia di un test set genuinamente indipendente e mai consultato prima.

33.2 K-fold e k-fold stratificato: formula e procedura

[Livello: teoria consolidata del settore] Un k-fold suddivide i dati disponibili in k parti (*fold*) di uguale dimensione. Per ciascuna delle k iterazioni, una parte diversa gioca il ruolo di validazione, mentre le restanti $k - 1$ parti giocano il ruolo di training:

$$\text{per } i = 1, \dots, k : \quad \text{addestra su } \bigcup_{j \neq i} D_j, \quad \text{valuta su } D_i \quad (33.1)$$

dove $D_1, \dots, D_k$ sono le k parti disjunte in cui il dataset D è stato suddiviso. Il risultato finale è tipicamente la media delle k metriche di validazione — esattamente ciò che `classification.py:47-50` calcola (`acc_mean = np.mean([r[0] for r in tmp_results])`), capitolo 23.1). Ogni singolo esempio del dataset finisce, in questo schema, esattamente una volta nel ruolo di validazione e $k - 1$ volte nel ruolo di training, mai in entrambi i ruoli nella stessa iterazione.

[Livello: teoria consolidata del settore] Un k-fold **stratificato** aggiunge un vincolo alla suddivisione: ogni D_i deve mantenere, il più possibile, la stessa proporzione fra le classi presente nel dataset intero. Senza questo vincolo, con un dataset sbilanciato (capitolo 4.2), una suddivisione puramente casuale potrebbe produrre un fold con una proporzione di classe molto diversa dalle altre — nel caso estremo, un fold senza alcun esempio della classe minoritaria, rendendo impossibile persino calcolare una metrica come l’AUC su quel fold. [Fatto] `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)` (`classification.py:15`) applica esattamente questo vincolo, con $k = 5$.

33.3 Data leakage: tassonomia generale e perché è la minaccia più insidiosa

[Livello: teoria consolidata del settore] Il **data leakage** (fuga di informazione) è, in generale, qualunque situazione in cui informazione che non dovrebbe essere disponibile al momento della previsione influenza, in modo diretto o indiretto, il processo di addestramento o di valutazione — gonfiando artificialmente le prestazioni misurate rispetto a quelle che il sistema avrebbe davvero su dati nuovi. Si distinguono comunemente alcune forme:

- **Leakage diretto (target leakage):** una feature usata per l’addestramento contiene, di fatto, informazione derivata dall’etichetta stessa (per esempio, una colonna calcolata *dopo* aver già osservato l’esito). Il capitolo 22.1 ha già verificato che questo progetto non lo presenta: nessuna delle funzioni `record_to_text_*` menziona mai l’etichetta target.
- **Leakage fra train e test (contaminazione dello split):** una trasformazione (scaling, imputazione, bilanciamento) viene calcolata sull’intero dataset *prima* di dividerlo in training e test, così che il training set “vede” indirettamente informazione statistica (medie, varianze) che include anche i dati di test. Il capitolo 21.2 ha già mostrato che questo progetto evita questa forma specifica: sia l’imputazione sia SMOTENC operano solo su `X_train`, dopo lo split (anche se, come appena visto al capitolo 33.1, il set di test risultante non viene poi mai utilizzato).
- **Leakage nella selezione di iperparametri o soglie:** una decisione (una soglia, un iperparametro) viene scelta guardando le etichette dello stesso insieme di dati su cui poi si

riporta la metrica finale. **[Fatto]** Questo progetto **presenta** questa forma, in un punto preciso già individuato al capitolo 23.2: la soglia τ è scelta massimizzando F1 su `y_val`, poi la prestazione (accuratezza, F1) viene misurata usando quella stessa soglia sullo stesso `y_val`. Non è leakage verso l'addestramento del modello (che resta pulito), ma è esattamente questa terza forma, limitata alla sola soglia.

- **Leakage temporale:** per dati con una componente temporale, usare informazione futura per prevedere il passato. Non applicabile a questo progetto (nessuna delle due variabili target ha una dimensione temporale nella formulazione usata qui, capitolo 1).

ATTENZIONE — fra le quattro forme elencate, questo progetto ne mostra una sola, e nella sua variante più lieve: non il leakage diretto (assente, verificato), non la contaminazione train/test (evitata correttamente per SMOTENC e imputazione), ma la fuga di informazione limitata alla scelta della soglia di decisione. È importante essere precisi su questo punto quando si scrive la tesi: sopravvalutare la gravità di questo problema (dicendo, per esempio, che “il modello soffre di data leakage” senza qualificare quale forma) sarebbe impreciso quanto sottovalutarlo.

Riepilogo

Un sistema supervisionato ha bisogno, in linea di principio, di tre porzioni di dati con ruoli distinti; questo progetto ne calcola tre ma ne usa davvero solo due, spreco un test set genuinamente indipendente. Il k-fold stratificato garantisce che ogni fold di validazione mantenga la proporzione di classe del dataset intero, un vincolo non opzionale su dati sbilanciati come Diabetes130. Fra le forme comuni di data leakage, questo progetto presenta solo quella più lieve — la scelta della soglia di decisione sullo stesso fold su cui viene poi misurata la prestazione — e non le forme più gravi (leakage diretto, contaminazione train/test).

Domande di autoverifica

1. Perché un test set consultato più volte durante lo sviluppo smette, di fatto, di essere un test set? Perché il suo ruolo distintivo è fornire una stima di generalizzazione mai influenzata da decisioni di sviluppo. Se lo si consulta ripetutamente per aggiustare qualcosa (una soglia, un iperparametro), quelle decisioni cominciano a essere calibrate su di esso, esattamente come farebbero su un validation set — la stima che fornisce torna quindi a essere ottimista.

2. Perché la stratificazione è particolarmente importante per Diabetes130 rispetto a Heart Disease? Perché Diabetes130 ha una classe positiva minoritaria marcata (11.16%, capitolo 4.2): senza stratificazione, un fold generato casualmente potrebbe avere una proporzione di classe molto diversa dal dataset intero, nel caso estremo pochissimi o nessun esempio della classe minoritaria — un problema molto meno probabile su Heart Disease, quasi bilanciato.

3. Quale delle quattro forme di data leakage elencate in questo capitolo è effettivamente presente in questo progetto, e quali sono invece assenti? È presente solo la fuga di informazione nella scelta della soglia di decisione (capitolo 23.2). Sono assenti il leakage diretto (nessuna frase generata include mai l'etichetta target, capitolo 22.1) e la contaminazione train/test (imputazione e SMOTENC operano solo sul training set, capitolo 21.2); il leakage temporale non è applicabile a questo progetto.

MATERIALE PER LA TESI 1. La distinzione fra i tre ruoli training/validation/test, applicata esplicitamente a ciò che questo progetto calcola e a ciò che usa davvero — riusabile in “Materiali e metodi” per una descrizione precisa e onesta del protocollo sperimentale. 2. La Formula 33.1 del k-fold, con la spiegazione della stratificazione — riusabile per la sezione metodologica sulla validazione. 3.

La tassonomia completa delle forme di data leakage, con la collocazione precisa di questo progetto in una sola di esse — riusabile quasi integralmente nella sezione “Discussione e limiti”, per una valutazione critica calibrata e non generica.

Capitolo 34 — Le tre metriche del progetto

Obiettivi del capitolo

- Avere la formula esatta di accuratezza, F1 macro e ROC-AUC, con il rimando alla riga di codice che le calcola.
- Sapere, per ciascuna, in quale caso concreto di questo progetto potrebbe ingannare.
- Interpretare correttamente il valore di soglia τ che compare accanto a queste tre metriche nei risultati.

[Fatto] `classification.py:39-41` calcola, per ogni fold e ogni modello, esattamente tre metriche — nessuna quarta, nessuna metrica di regressione come MAE o RMSE, perché questo è un problema di classificazione, non di previsione di un valore continuo (capitolo 1).

34.1 Accuracy

[Livello: teoria consolidata del settore] L'accuratezza è la proporzione di previsioni corrette sul totale:

$$\text{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN} \quad (34.1)$$

dove VP , VN , FP , FN sono i quattro esiti della matrice di confusione già definiti al capitolo 4.3. [Fatto] `accuracy_score(y_val, y_pred)` (`classification.py:39`, da `sklearn.metrics`) implementa esattamente questa formula.

ATTENZIONE — l'accuratezza inganna precisamente quando le classi sono sbilanciate (capitolo 4.2): su Diabetes130 (11.16% di classe positiva), un classificatore che rispondesse *sempre* “non riammesso” otterrebbe un'accuratezza dell'88.84% — un numero che sembra ottimo ma che non ha imparato assolutamente nulla di utile. È precisamente per questo che il capitolo 47 costruisce un modello di riferimento banale: senza un termine di paragone esplicito, un'accuratezza dell'85% su Diabetes130 potrebbe sembrare un successo quando è, in realtà, appena sopra la soglia della cosa più ovvia possibile.

34.2 Macro-F1

[Livello: teoria consolidata del settore] L'F1 è la media armonica di precisione e recall:

$$F1 = 2 \cdot \frac{\text{precisione} \cdot \text{recall}}{\text{precisione} + \text{recall}}, \quad \text{precisione} = \frac{VP}{VP + FP}, \quad \text{recall} = \frac{VP}{VP + FN} \quad (34.2)$$

[Fatto] `f1_score(y_val, y_pred, average='macro')` (`classification.py:40`) non calcola una singola F1: calcola l’F1 separatamente per la classe 0 e per la classe 1, poi ne fa la **media aritmetica semplice**, non pesata per il numero di esempi di ciascuna classe (`average='macro'`, in contrasto con `average='weighted'`, che pesare per frequenza, o `average='binary'`, che riporta solo l’F1 della classe positiva).

ATTENZIONE — su un dataset sbilanciato come Diabetes130, la media macro dà lo **stesso peso** alla F1 della classe minoritaria (11.16% dei casi) e alla F1 della classe maggioritaria, anche se quest’ultima è calcolata su quasi nove volte più esempi. Questo è, nella maggior parte dei contesti clinici, un pregio della scelta **macro** fatta dal progetto: impedisce che un modello bravo solo sulla classe maggioritaria (facile, per definizione, su un dataset sbilanciato) ottenga un punteggio alto ignorando la classe minoritaria — spesso quella clinicamente più rilevante (i pazienti a rischio di riammissione). Ma inganna in senso opposto se letta distrattamente come “F1 della classe positiva”: non lo è, è una media fra le due.

34.3 ROC-AUC

[Livello: teoria consolidata del settore] La curva ROC (Receiver Operating Characteristic) traccia, al variare della soglia di decisione da 0 a 1, la coppia (tasso di falsi positivi, tasso di veri positivi):

$$\text{TPR}(\tau) = \frac{VP(\tau)}{VP(\tau) + FN(\tau)}, \quad \text{FPR}(\tau) = \frac{FP(\tau)}{FP(\tau) + VN(\tau)} \quad (34.3)$$

[Fatto] `plot_roc_comparison()` (`function.py:257-277`, capitolo 24.3) disegna esattamente questa curva per ciascun modello. L’AUC (Area Under the Curve) è l’area sotto quella curva — un numero fra 0.5 (nessuna capacità discriminante, equivalente a indovinare a caso) e 1 (separazione perfetta fra le due classi). [Livello: teoria consolidata del settore] L’interpretazione più utile dell’AUC è probabilistica: è la probabilità che il modello assegni un punteggio più alto a un caso positivo scelto a caso rispetto a un caso negativo scelto a caso — una proprietà che dipende solo dall’**ordinamento** relativo dei punteggi, non da una soglia specifica.

[Fatto] `roc_auc_score(y_val, y_score)` (`classification.py:41`) riceve `y_score` — il punteggio di probabilità continuo, non `y_pred` — a differenza di accuratezza e F1, che invece dipendono da `y_pred`, cioè da una soglia già applicata. [Interpretazione] Questo rende l’AUC l’unica delle tre metriche del progetto **indipendente dalla soglia** τ (capitolo 35): non importa quale soglia venga scelta, l’AUC resta la stessa, perché misura la qualità dell’ordinamento dei punteggi, non una singola classificazione binaria specifica. È anche il motivo per cui il capitolo 26 usa il test di DeLong — specifico per l’AUC — come confronto indipendente dalla scelta (per fold, e potenzialmente ottimistica, capitolo 23.2) della soglia.

ATTENZIONE — l’AUC inganna quando la si legge come “quanto è bravo il modello nella pratica clinica”: un’AUC alta dice che il modello *ordina bene* i casi, ma non dice se la soglia effettivamente usata in produzione (capitolo 35) sia quella giusta per il costo relativo di falsi positivi e falsi negativi discusso al capitolo 1.2. Due modelli con la stessa identica AUC possono avere prestazioni molto diverse a una soglia fissata, se la forma delle rispettive curve ROC differisce.

Riepilogo

Le tre metriche del progetto misurano aspetti diversi e non intercambiabili: l'accuratezza (Formula 34.1) è ingannevole su dati sbilanciati; l'F1 macro (Formula 34.2) pesa equamente le due classi indipendentemente dalla loro frequenza, correggendo parzialmente quel problema; l'AUC (Formula 34.3) è l'unica indipendente dalla soglia di decisione, e misura la qualità dell'ordinamento dei punteggi piuttosto che di una singola classificazione. Nessuna delle tre, da sola, basta a giudicare un sistema di supporto clinico: vanno lette insieme, con la consapevolezza specifica di cosa ciascuna può nascondere.

Domande di autoverifica

- 1. Quale accuratezza otterrebbe, su Diabetes130, un classificatore che prevedesse sempre “non riammesso”, e perché questo numero da solo non dice nulla di utile?** Circa l'88.84%, la proporzione della classe maggioritaria (capitolo 4.2) — un numero alto ottenuto senza che il classificatore abbia imparato a distinguere alcunché fra le due classi, il motivo esatto per cui serve un modello di riferimento banale (capitolo 47) come termine di paragone.
- 2. Perché l'F1 macro non va letta come “l'F1 della classe positiva”?** Perché `average='macro'` calcola l'F1 separatamente per entrambe le classi e ne fa la media aritmetica semplice, non pesata per frequenza: il numero risultante riflette il comportamento del modello su entrambe le classi in egual misura, non solo sulla classe di interesse clinico.
- 3. Perché l'AUC è l'unica delle tre metriche del progetto a non dipendere dalla soglia di decisione τ ?** Perché `roc_auc_score()` riceve `y_score` (il punteggio continuo di probabilità), non `y_pred` (l'etichetta binaria già ottenuta applicando una soglia): l'AUC misura la qualità dell'ordinamento relativo dei punteggi fra le due classi, una proprietà indipendente da dove si scelga di tagliare quella scala continua in due.

MATERIALE PER LA TESI 1. Le Formule 34.1-34.3 con la traduzione completa dei simboli e il rimando esatto alla riga di codice — riusabili integralmente nella sezione “Materiali e metodi” dedicata alle metriche di valutazione. 2. La spiegazione di quando ciascuna metrica inganna, applicata specificamente ai due dataset di questo progetto — riusabile nella sezione “Discussione”, per qualificare correttamente ogni numero riportato nei risultati. 3. La precisazione sull'indipendenza dell'AUC dalla soglia di decisione, con il collegamento al test di DeLong — riusabile per motivare, in “Materiali e metodi”, la scelta di un test specifico per l'AUC oltre a Wilcoxon e t-test.

Capitolo 35 — Soglie di decisione

Obiettivi del capitolo

- Capire perché 0.5 è una soglia arbitraria, non una scelta neutra o “di default corretta”.
- Avere la formula esatta di ciò che `classification.py` ottimizza quando cerca la soglia migliore.
- Mettere a fuoco, con precisione matematica, il costo nascosto già anticipato ai capitoli 23.2 e 33.3.

35.1 Perché 0.5 non basta sempre

[Livello: teoria consolidata del settore] Un classificatore binario produce una probabilità continua (Formula 32.1); trasformarla in un’etichetta richiede scegliere un punto di taglio $\tau \in [0, 1]$: se $P(y = 1 \mid \mathbf{x}) \geq \tau$, l’etichetta prevista è 1, altrimenti 0. **[Fatto]** `y_pred = (y_score >= tau).astype(int)` (`classification.py:37`) implementa esattamente questo confronto. Scegliere $\tau = 0.5$ è la convenzione più comune, ma non ha nulla di matematicamente privilegiato: è ottimale solo se i due tipi di errore (falso positivo, falso negativo) hanno lo stesso costo e le due classi sono bilanciate — nessuna delle due condizioni è garantita in generale, e il capitolo 1.2 ha già mostrato che, per entrambi i dataset di questo progetto, i due tipi di errore hanno conseguenze cliniche diverse.

[Interpretazione] Abbassare τ sotto 0.5 rende il modello più “allarmista”: più veri positivi catturati (meno falsi negativi), ma anche più falsi positivi. Alzarlo fa l’opposto. La scelta corretta di τ , in un contesto clinico reale, dovrebbe dipendere esplicitamente dal costo relativo dei due errori (capitolo 1.2) — un’informazione che questo progetto non incorpora mai: la soglia scelta ottimizza F1 (capitolo 35.2), una metrica che pesa implicitamente falsi positivi e falsi negativi allo stesso modo, non il costo clinico specifico di questo dominio.

35.2 La soglia F1-ottima: formula e codice

[Fatto] `classification.py:30-35` cerca, fra tutte le soglie candidate generate da `precision_recall_curve()`, quella che massimizza l’F1 (Formula 34.2) sul fold di validazione:

$$\tau^* = \arg \max_{\tau \in T} F1(y_{\text{val}}, \mathbb{1}[y_{\text{score}} \geq \tau]) \quad (35.1)$$

dove T è l’insieme finito di soglie candidate restituito da `precision_recall_curve(y_val, y_score)` (ogni punto di discontinuità della curva precisione-recall, non un campionamento uniforme di $[0, 1]$), e $\mathbb{1}[\cdot]$ è la funzione indicatrice (1 se la condizione è vera, 0 altrimenti) — la stessa operazione di `(y_score >= tau).astype(int)`. **[Fatto]** Il codice calcola l’F1 per ogni soglia candidata con la formula esplicita, non richiamando `f1_score()` in un ciclo (più costoso):

```

\NormalTok{f1\_scores }\OperatorTok{=} \DecValTok{2} \OperatorTok{*}\NormalTok{
  ↪ (precision[:]\OperatorTok{-}\DecValTok{1}\NormalTok{[]
  ↪ }\OperatorTok{*}\NormalTok{ recall[:]\OperatorTok{-}\DecValTok{1}\NormalTok{[]})
  ↪ }\OperatorTok{/}\NormalTok{
  ↪ (precision[:]\OperatorTok{-}\DecValTok{1}\NormalTok{[]
  ↪ }\OperatorTok{+}\NormalTok{ recall[:]\OperatorTok{-}\DecValTok{1}\NormalTok{[]
  ↪ }\OperatorTok{+} \FloatTok{1e{-}6}\NormalTok{)}}
\NormalTok{best\_idx }\OperatorTok{=} \NormalTok{f1\_scores.argmax()}
\NormalTok{tau }\OperatorTok{=} \NormalTok{thresholds[best\_idx]}

```

`argmax()` implementa esattamente l'operatore `argmax` della Formula 35.1: restituisce l'*indice* del valore massimo, non il valore stesso — da cui `thresholds[best_idx]`, non `f1_scores[best_idx]`, per recuperare la soglia corrispondente.

35.3 Il costo nascosto della sua ottimizzazione

[Fatto] La Formula 35.1 usa y_{val} — le etichette vere del fold di validazione — sia per calcolare τ^* sia, immediatamente dopo, per misurare accuratezza e F1 con quella stessa soglia sullo stesso y_{val} (già segnalato ai capitoli 23.2 e 33.3, qui reso preciso in una formula). **[Interpretazione]** Il problema, in termini formali, è che τ^* non è una costante fissata a priori: è essa stessa una **funzione di y_{val}** , esattamente il campione su cui la prestazione finale viene poi riportata. Confrontala con un'alternativa più rigorosa — mai implementata in questo progetto — in cui la soglia venga scelta su un fold di *calibrazione* separato, distinto sia dal training sia dal fold su cui si riporta la metrica:

$$\tau_{\text{rigoroso}}^* = \arg \max_{\tau \in T} F1(y_{\text{calib}}, \mathbb{1}[y_{\text{score,calib}} \geq \tau]), \quad \text{poi valutato su } y_{\text{val}} \neq y_{\text{calib}} \quad (35.2)$$

[Interpretazione] La differenza fra la Formula 35.1 (usata dal progetto) e la Formula 35.2 (l'alternativa più rigorosa, non implementata) è la fonte esatta dell'ottimismo statistico già discusso: nella Formula 35.1, la soglia è “cucita su misura” per massimizzare la metrica proprio sui dati su cui quella metrica viene poi riportata, un vantaggio che una soglia scelta su dati indipendenti (Formula 35.2) non avrebbe.

PROVA TU — stima tu stesso l'entità di questo ottimismo, senza rieseguire l'intera pipeline: per uno dei modelli già presenti in `datas/heart_disease/results/`, calcola l'F1 che si otterrebbe con la soglia fissa $\tau = 0.5$ invece della soglia F1-ottima salvata, usando i file `{model}_y_true.npy` e `{model}_y_score.npy` già presenti. La differenza fra i due numeri è una stima diretta, per quel modello specifico, di quanto la Formula 35.1 abbia effettivamente gonfiato l'F1 riportato rispetto a una soglia scelta senza guardare le etichette di validazione.

Riepilogo

La soglia di decisione di 0.5 non ha alcun privilegio matematico: è ottimale solo sotto assunzioni di costo ed equilibrio fra classi che questo progetto non verifica mai esplicitamente. La soglia F1-ottima (Formula 35.1) è scelta e valutata sullo stesso fold di validazione — una funzione delle stesse etichette su cui la prestazione viene poi misurata, distinta con precisione formale da un'alternativa più rigorosa (Formula 35.2) che userebbe un fold di calibrazione separato, mai implementata in questo progetto.

Domande di autoverifica

1. Sotto quali due condizioni la soglia $\tau = 0.5$ sarebbe effettivamente ottimale? Se i due tipi di errore (falso positivo, falso negativo) avessero lo stesso costo, e se le due classi fossero bilanciate nella popolazione. Nessuna delle due condizioni è verificata esplicitamente in questo progetto per nessuno dei due dataset.

2. Cosa restituisce esattamente `f1_scores.argmax()` in `classification.py:34`, e perché il codice usa poi `thresholds[best_idx]` e non `f1_scores[best_idx]`? `argmax()` restituisce l'indice della posizione con il valore F1 massimo, non il valore stesso. `thresholds[best_idx]` usa quell'indice per recuperare la soglia corrispondente, che è ciò che serve per classificare nuovi punteggi — il valore massimo di F1 in sé non è più necessario a questo punto del codice.

3. Qual è la differenza formale precisa fra la Formula 35.1 (usata dal progetto) e la Formula 35.2 (l'alternativa più rigorosa)? Nella Formula 35.1 la soglia è scelta e poi valutata sullo stesso insieme y_{val} ; nella Formula 35.2 è scelta su un insieme di calibrazione y_{calib} distinto dall'insieme y_{val} su cui viene poi valutata — quest'ultima non introduce lo stesso ottimismo statistico perché la soglia non è “cucita su misura” per i dati su cui la prestazione finale viene riportata.

MATERIALE PER LA TESI 1. La Formula 35.1 con la spiegazione precisa dell'operatore `argmax` e il rimando al codice — riusabile in “Materiali e metodi” per descrivere con rigore la procedura di scelta della soglia. 2. Il confronto formale fra Formula 35.1 e Formula 35.2, con la fonte esatta dell'ottimismo statistico resa esplicita — probabilmente la formalizzazione più utile per la tesi di un limite già discusso più volte nel libro: riusabile integralmente nella sezione “Discussione e limiti”. 3. L'esercizio pratico per stimare l'entità dell'ottimismo con i dati già presenti nel repository (§35.3) — riusabile come base per una misura quantitativa originale da includere nei risultati o nella discussione della tesi.

Capitolo 36 — Bootstrap e intervalli di confidenza

Obiettivi del capitolo

- Avere la formulazione matematica completa del bootstrap non parametrico usato in questo progetto.
- Sapere da dove viene, con una formula precisa, la deviazione standard riportata accanto a ogni metrica.
- Capire perché 10.000 iterazioni è una scelta di stabilità, non un numero arbitrario, con un modo concreto per verificarlo tu stesso.

Il capitolo 24 ha già letto il codice del bootstrap riga per riga. Questo capitolo ne dà la formulazione matematica completa, per collegare quel codice al modo in cui la tesi dovrebbe descriverlo formalmente.

36.1 Ricampionamento con reinserimento

[Livello: teoria consolidata del settore] Dato un insieme di n osservazioni già raccolte — qui, le triple (etichetta vera, punteggio, predizione) di un modello su tutti i suoi fold di validazione, `all_y_true/all_y_score/all_y_pred` di `classification.py`, capitolo 23.3 — il bootstrap genera B campioni ricampionati, ciascuno di dimensione n , estraendo indici **con reinserimento** dall'insieme originale:

$$I^{(b)} = \{i_1, \dots, i_n\}, \quad i_j \sim \mathcal{U}\{1, \dots, n\} \text{ indipendenti}, \quad b = 1, \dots, B \quad (36.1)$$

dove $\mathcal{U}\{1, \dots, n\}$ è la distribuzione uniforme discreta sugli indici da 1 a n . [Fatto] `idx = rng.integers(0, len(y_true), len(y_true))` (`evaluation.py:66`) implementa esattamente questo campionamento: `len(y_true)` estrazioni indipendenti, ciascuna uniforme su `len(y_true)` possibili indici, **con reinserimento** — lo stesso indice può comparire più volte in uno stesso $I^{(b)}$, e altri indici possono non comparire affatto. Per ciascun campionamento $I^{(b)}$, la metrica di interesse (accuratezza, F1, o AUC) viene ricalcolata sul sotto-campione così ottenuto:

$$\hat{M}^{(b)} = M(\{y_{\text{true},i}\}_{i \in I^{(b)}}, \{y_{\text{pred},i}\}_{i \in I^{(b)}}), \quad b = 1, \dots, B \quad (36.2)$$

[Fatto] `acc_list.append(accuracy_score(yt, yp))` e le righe analoghe per F1 e AUC (`evaluation.py:68-70`) calcolano esattamente $\hat{M}^{(b)}$ per $B = 10.000$ (`n_iter=10000`, riga 62), producendo tre distribuzioni empiriche di 10.000 valori ciascuna — non un ricampionamento dei dati grezzi né un nuovo addestramento del modello, ma un ricampionamento delle **predizioni già ottenute** (capitolo 24.1).

36.2 Intervalli percentile al 95%

[Livello: teoria consolidata del settore] Con i B valori bootstrap $\hat{M}^{(1)}, \dots, \hat{M}^{(B)}$ ordinati, l'intervallo di confidenza percentile al livello α (qui $\alpha = 0.95$) è definito dai due percentili empirici:

$$IC_\alpha = \left[\hat{M}_{(\frac{1-\alpha}{2})}, \hat{M}_{(\frac{1+\alpha}{2})} \right] = \left[\hat{M}_{(0.025)}, \hat{M}_{(0.975)} \right] \quad (36.3)$$

dove $\hat{M}_{(p)}$ denota il p -esimo percentile empirico della distribuzione bootstrap. **[Fatto]** `ci()` (`evaluation.py:77-81`, capitolo 24.2) implementa esattamente questa formula con `np.percentile(a, (1-alpha)/2 * 100)` e `np.percentile(a, (1+alpha)/2 * 100)`. **[Fatto]** La deviazione standard bootstrap, mostrata come barra d'errore più spessa nei grafici (capitolo 24.2), è la deviazione standard campionaria ordinaria applicata ai B valori:

$$\widehat{SE}_{\text{boot}}(M) = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (\hat{M}^{(b)} - \bar{M})^2}, \quad \bar{M} = \frac{1}{B} \sum_{b=1}^B \hat{M}^{(b)} \quad (36.4)$$

[Fatto] implementata semplicemente da `bootstrap_metrics_dict['acc'].std()` (`evaluation.py:37`) — il metodo `.std()` di NumPy, applicato all'array dei 10.000 valori.

36.3 10.000 iterazioni: perché questo numero, cosa cambierebbe con 100 o 1.000.000

[Livello: teoria consolidata del settore] All'aumentare di B , la distribuzione empirica bootstrap converge alla vera distribuzione campionaria della metrica (nel limite $B \rightarrow \infty$), e l'errore di Monte Carlo sulla stima dei percentili della Formula 36.3 diminuisce proporzionalmente a $1/\sqrt{B}$. Con $B = 100$, i percentili al 2.5%/97.5% sarebbero stimati da appena 2-3 osservazioni estreme in coda — una stima rumorosa. Con $B = 10.000$, l'errore di Monte Carlo residuo è tipicamente trascurabile rispetto all'incertezza intrinseca della metrica stessa; **[Livello: teoria consolidata del settore]** valori come $B = 1.000$ o $B = 10.000$ sono comunemente considerati sufficienti in letteratura per stime percentili stabili, mentre $B = 1.000.000$ ridurrebbe ulteriormente l'errore di Monte Carlo residuo a un costo computazionale centomila volte superiore, per un guadagno di precisione che, oltre una certa soglia, non è più praticamente rilevante rispetto alla variabilità intrinseca dei dati.

PROVA TU — verifica tu stesso la stabilità di $B = 10.000$ su un file già presente nel repository: carica `datas/heart_disease/results/e5-base_boot_auc.npy` (10.000 valori già calcolati), calcola l'intervallo di confidenza percentile sui primi 1.000 valori, poi sui primi 5.000, poi su tutti e 10.000. Se i tre intervalli sono già molto simili fra loro, hai una prova empirica diretta, specifica per questo progetto, che $B = 10.000$ non è un numero scelto a caso ma un punto in cui la stima si è già stabilizzata.

Riepilogo

Il bootstrap ricampiona con reinserimento le predizioni già ottenute (Formula 36.1), ricalcolando la metrica su ciascun ricampionamento (Formula 36.2) per costruire una distribuzione empirica di 10.000 valori, da cui si derivano sia l'intervallo di confidenza percentile (Formula 36.3) sia la deviazione standard bootstrap (Formula 36.4). Il numero di iterazioni, 10.000, è una scelta di stabilità statistica verificabile empiricamente sui dati stessi, non un valore arbitrario.

Domande di autoverifica

1. Cosa significa “con reinserimento” nella Formula 36.1, e perché è essenziale per il bootstrap? Significa che, estraendo un indice per il campione ricampionato, quell’indice resta disponibile per essere estratto di nuovo nelle estrazioni successive — lo stesso record può comparire più volte in un singolo ricampionamento, e altri record possono non comparire affatto. Senza reinserimento, ogni ricampionamento sarebbe identico all’insieme originale, e non ci sarebbe alcuna variabilità da misurare.

2. Quale funzione di libreria implementa direttamente la Formula 36.4 (la deviazione standard bootstrap), e su quale oggetto viene chiamata? Il metodo `.std()` di NumPy, chiamato direttamente sull’array dei 10.000 valori bootstrap di una metrica (per esempio `bootstrap_metrics_dict['acc'].std()`).

3. Come potresti verificare empiricamente, senza rieseguire la pipeline, che 10.000 iterazioni sono sufficienti per una stima stabile? Ricalcolando l’intervallo di confidenza percentile su un numero crescente delle 10.000 osservazioni bootstrap già salvate su disco (per esempio 1.000, poi 5.000, poi tutte e 10.000): se i risultati convergono e restano stabili ben prima di raggiungere le 10.000 osservazioni complete, è una prova diretta di adeguatezza per questo caso specifico.

MATERIALE PER LA TESI 1. Le Formule 36.1-36.4 con la derivazione completa dall’idea generale del bootstrap al codice specifico — riusabili integralmente in “Materiali e metodi” per una descrizione rigorosa della metodologia di valutazione dell’incertezza. 2. L’argomento sulla convergenza dell’errore di Monte Carlo con $1/\sqrt{B}$, applicato alla scelta di $B = 10.000$ — riusabile per giustificare formalmente, nella tesi, un parametro altrimenti presentato come arbitrario. 3. L’esercizio di verifica empirica di stabilità (§36.3), eseguibile sui dati già presenti nel repository — riusabile come misura originale e riproducibile da includere nella sezione “Risultati” o in un’appendice metodologica.

Capitolo 37 — I tre test di significatività

Obiettivi del capitolo

- Avere la formula (o la struttura formale) di ciascuno dei tre test usati dal progetto per confrontare due modelli.
- Sapere sotto quale ipotesi nulla ciascun test opera, e cosa significa esattamente rifiutarla.
- Collegare ogni test alla riga di codice esatta che lo esegue, già letta al capitolo 26.

37.1 Wilcoxon signed-rank

[Livello: teoria consolidata del settore] Dati n coppie di osservazioni accoppiate — qui, i valori bootstrap corrispondenti di due modelli, `scores_a[i]` e `scores_b[i]` per lo stesso indice i di ricampionamento (capitolo 26.1) — il test di Wilcoxon calcola le differenze $d_i = a_i - b_i$, scarta le differenze nulle, assegna un **rango** al valore assoluto $|d_i|$ di ciascuna differenza rimanente (rango 1 alla più piccola, e così via), poi somma i ranghi delle differenze positive (W^+) e negative (W^-) separatamente:

$$W = \min(W^+, W^-), \quad W^+ = \sum_{i: d_i > 0} R_i, \quad W^- = \sum_{i: d_i < 0} R_i \quad (37.1)$$

dove R_i è il rango di $|d_i|$. [Livello: teoria consolidata del settore] Sotto l'ipotesi nulla H_0 (“nessuna differenza sistematica fra le distribuzioni dei due modelli”), W^+ e W^- dovrebbero essere approssimativamente uguali; un valore di W molto più piccolo di quanto atteso per caso porta a rifiutare H_0 . Il test non assume che le differenze d_i seguano una distribuzione normale — si basa solo sui loro ranghi, non sui valori esatti — il che lo rende più robusto quando questa assunzione è dubbia, un caso comune per dati clinici (`docs/STATISTICAL_TESTS.md:26`).

[Fatto] `wilcoxon(scores_a, scores_b)` (`statisticaltest.py:35`, da `scipy.stats`) implementa esattamente questo test sulle $B = 10.000$ coppie di valori bootstrap di ciascuna coppia di modelli, per ciascuna delle tre metriche.

37.2 t-test appaiato

[Livello: teoria consolidata del settore] Con le stesse differenze accoppiate $d_i = a_i - b_i$ della Formula 37.1, il t-test appaiato calcola:

$$t = \frac{\bar{d}}{s_d / \sqrt{n}}, \quad \bar{d} = \frac{1}{n} \sum_{i=1}^n d_i, \quad s_d = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (d_i - \bar{d})^2} \quad (37.2)$$

dove $\bar{d}$ è la media delle differenze e s_d la loro deviazione standard campionaria. Sotto $H_0 : \mu_d = 0$ (le medie dei due modelli sono uguali), t segue (approssimativamente, sotto l'assunzione di normalità delle differenze) una distribuzione t di Student con $n - 1$ gradi di libertà. **[Livello: teoria consolidata del settore]** Se questa assunzione di normalità regge, il t-test ha più potenza statistica di Wilcoxon — rileva come significative differenze più piccole a parità di dimensione campionaria — ma la sua validità dipende da un'assunzione che Wilcoxon non richiede.

[Fatto] `ttest_rel(scores_a, scores_b)` (`statisticaltest.py:48`, da `scipy.stats`) implementa esattamente la Formula 37.2. **[Interpretazione]** Con $n = 10.000$ (il numero di ricampionamenti bootstrap, non il numero di pazienti), anche differenze di media minuscole fra due modelli tendono a risultare “statisticamente significative” a entrambi i test — un punto di attenzione già anticipato dal capitolo 1.3 sulla distinzione fra significatività statistica e clinica, che `docs/STATISTICAL_TESTS.md:78-82` discute esplicitamente: una differenza di accuratezza dello 0.5%, con un campione bootstrap così grande, risulterà quasi sempre significativa, ma potrebbe non avere alcuna rilevanza per una decisione clinica reale.

37.3 Il test di DeLong

[Livello: teoria consolidata del settore] Il test di DeLong confronta due AUC calcolate sugli **stessi** casi (stesse etichette vere), tenendo conto della correlazione che nasce da questa condivisione. La forma generale della statistica di test è uno z-score:

$$Z = \frac{\widehat{AUC}_A - \widehat{AUC}_B}{\sqrt{\widehat{\text{Var}}(\widehat{AUC}_A) + \widehat{\text{Var}}(\widehat{AUC}_B) - 2\widehat{\text{Cov}}(\widehat{AUC}_A, \widehat{AUC}_B)}} \quad (37.3)$$

[Livello: teoria consolidata del settore, dettaglio da verificare] dove le varianze e la covarianza non sono stimate con un ricampionamento, ma con una formula analitica basata sui cosiddetti *componenti strutturali* (o *placement values*) legati alla rappresentazione dell'AUC come statistica di Mann-Whitney — il metodo originale è descritto in DeLong, E.R., DeLong, D.M., & Clarke-Pearson, D.L. (1988), *Comparing the areas under two or more correlated receiver operating characteristic curves: a nonparametric approach*, *Biometrics* [DA VERIFICARE - volume, numero di pagina e DOI esatti da confermare prima di citarlo in bibliografia]. **[Fatto]** In questo progetto, l'intera formula è delegata alla libreria esterna `MLstatkit` (`statisticaltest.py:3,95-102`, capitolo 26.2), non reimplementata: `Delong_test(y_true_a, scores[a]["y_score"], scores[b]["y_score"], return_ci=False, return_auc=True, verbose=0)` restituisce direttamente Z , il p-value, e le due AUC.

[Fatto] A differenza di Wilcoxon e t-test (applicati alle $B = 10.000$ osservazioni bootstrap), il test di DeLong lavora direttamente su `y_score` — i punteggi di probabilità originali del fold di validazione (concatenati su tutti i 5 fold, capitolo 23.3), non su una distribuzione ricampionata. **[Fatto]** Per questo motivo, il codice verifica esplicitamente che `y_true_a` e `y_true_b` coincidano prima di procedere (`statisticaltest.py:91-93`, capitolo 26.2): il test richiede che i due modelli siano valutati esattamente sugli stessi record, una preconditione che Wilcoxon e t-test — applicati a distribuzioni bootstrap generate con lo stesso seme, non a dati originali — non necessitano di verificare esplicitamente.

Riepilogo

I tre test confrontano coppie di modelli sotto assunzioni diverse: Wilcoxon (Formula 37.1) non assume normalità e si basa sui ranghi delle differenze; il t-test appaiato (Formula 37.2) assume normalità delle differenze e ha più potenza se l'assunzione regge; DeLong (Formula 37.3) è specifico per l'AUC, lavora sui punteggi originali anziché su una distribuzione bootstrap, e tiene conto esplicitamente della correlazione fra modelli valutati sugli stessi casi. Con $B = 10.000$ ricampionamenti bootstrap, anche differenze minuscole tendono a risultare statisticamente significative: la distinzione fra significatività statistica e clinica, già introdotta al capitolo 1, qui trova la sua giustificazione tecnica precisa.

Domande di autoverifica

1. Perché il test di Wilcoxon si basa sui ranghi delle differenze $|d_i|$ invece che sui loro valori esatti? Perché questo lo rende non parametrico: non richiede assumere che le differenze seguano una distribuzione normale, un'assunzione spesso dubbia per dati clinici, a differenza del t-test appaiato che quell'assunzione la richiede per la validità formale della sua distribuzione di riferimento.

2. Con $n = 10.000$ osservazioni bootstrap, perché una differenza di accuratezza dello 0.5% fra due modelli risulta quasi sempre “statisticamente significativa”? Perché la potenza statistica di un test cresce con la dimensione campionaria: con un campione così grande, anche differenze di media molto piccole diventano rilevabili come significative, indipendentemente dal fatto che quella differenza abbia un'importanza pratica o clinica.

3. Perché il test di DeLong, a differenza di Wilcoxon e t-test in questo progetto, richiede una verifica esplicita che le etichette vere dei due modelli coincidano? Perché lavora direttamente sui punteggi originali del fold di validazione, non su una distribuzione bootstrap generata con lo stesso seme: la sua validità dipende dal fatto che i due modelli siano stati valutati esattamente sugli stessi record, una condizione che va verificata sui dati reali, non garantita per costruzione come lo è per il ricampionamento bootstrap.

MATERIALE PER LA TESI 1. Le Formule 37.1-37.3 con le rispettive ipotesi nulle esplicitate — riusabili integralmente in “Materiali e metodi” per una descrizione rigorosa del protocollo di confronto statistico. 2. La distinzione formale fra significatività statistica (garantita quasi sempre da $n = 10.000$) e rilevanza clinica — riusabile come argomento metodologico centrale nella sezione “Discussione”. 3. Il riferimento bibliografico al test di DeLong, marcato esplicitamente da verificare prima della citazione finale — da confermare con WebSearch/WebFetch in Appendice D, poi riusabile come citazione verificata nella tesi.

Capitolo 38 — Riduzione di dimensionalità e visualizzazione: UMAP

Obiettivi del capitolo

- Capire perché nessun grafico può mostrare direttamente uno spazio a 768 o 1024 dimensioni.
- Avere un'intuizione corretta di cosa fa UMAP, senza bisogno della sua matematica completa.
- Sapere cosa un grafico UMAP può dirti e cosa, con altrettanta certezza, non può dirti.

Questo è l'unico capitolo del libro dedicato a una tecnica **non supervisionata** (capitolo 2.1): UMAP non partecipa mai alla classificazione, serve solo a produrre due grafici del progetto.

38.1 Perché non si può “vedere” uno spazio a 768/1024 dimensioni

Un grafico a dispersione ordinario ha due assi: puoi disegnare punti in un piano, non in uno spazio a 768 coordinate. Se un embedding ha 768 (o 1024) numeri, e vuoi comunque farti un'idea visiva di come i punti si distribuiscono — se le due classi occupano regioni distinte o si sovrappongono, per esempio — devi prima **ridurre** quello spazio a due sole dimensioni, in un modo che preservi il più possibile la struttura importante dei dati originali.

[Fatto] Il progetto affronta questo problema in due punti distinti: `plot_umap()` (`function.py:223-234`, chiamata da `preprocessing.py:58`) proietta le feature *codificate* del preprocessing (non gli embedding testuali) in due dimensioni; nessun'altra parte della pipeline usa UMAP sugli embedding dei sette modelli — la proiezione visiva riguarda solo i dati tabellari codificati, non le rappresentazioni testuali su cui i classificatori vengono davvero addestrati.

38.2 UMAP in breve: vicinanza locale, ottimizzazione di un layout 2D

[Livello: teoria consolidata del settore] UMAP (*Uniform Manifold Approximation and Projection*) [DA VERIFICARE - riferimento bibliografico completo: McInnes, Healy, Melville, 2018, da confermare con DOI/arXiv esatto prima della citazione finale] funziona, a grandi linee, in due fasi. Nella prima, costruisce un grafo di vicinanza nello spazio ad alta dimensione: per ogni punto, individua i suoi vicini più prossimi e assegna un peso a ogni collegamento, più alto quanto più i due punti sono vicini — non una singola distanza globale, ma una struttura di relazioni **locali**. Nella seconda fase, UMAP cerca un layout a

bassa dimensione (qui, 2D) che riproduca il più fedelmente possibile quelle stesse relazioni di vicinanza locale, ottimizzando iterativamente le posizioni dei punti — concettualmente simile a un sistema di molle che tira insieme i punti vicini e separa quelli lontani, fino a raggiungere una configurazione stabile.

[Fatto] `UMAP(n_components=2, random_state=42)` (`function.py:224`, da `umap-learn`, `requirements.txt`) usa i parametri di default della libreria per tutto il resto (numero di vicini considerati, distanza minima nel layout finale) — nessuno di questi iperparametri specifici di UMAP viene esplorato o discusso nel progetto, un punto che il capitolo 39 nota come parte della mappa completa degli iperparametri.

38.3 Cosa può e non può dirti un grafico UMAP

[Livello: teoria consolidata del settore] Un grafico UMAP può dirti, in modo affidabile, se punti **vicini nello spazio originale** restano vicini anche nella proiezione — è esattamente ciò per cui l'algoritmo è costruito. Se osservi due nuvole di punti ben separate, colorate secondo il target (`plot_umap()`, `function.py:227`, con `hue=y`), è ragionevole concludere che esiste una struttura locale nello spazio originale che separa le due classi almeno in parte.

[Attenzione] Un grafico UMAP **non può** dirti, con la stessa affidabilità, quanto siano davvero distanti due nuvole di punti, né se le dimensioni relative di due cluster nel grafico riflettano le dimensioni relative reali nello spazio originale: UMAP preserva la struttura di vicinanza *locale*, non le distanze *globali* fra regioni lontane — due cluster che appaiono ugualmente compatti nel grafico potrebbero avere una densità reale molto diversa nello spazio a 768 dimensioni. **[Interpretazione]** Questo è particolarmente rilevante per interpretare `datas/heart_disease/graphics/UMAP_Preprocessed Data + Embeddings.png` (già presente nel repository, capitolo 44): una separazione visiva incompleta fra le due classi in quel grafico non implica automaticamente che il problema sia difficile per un classificatore — un classificatore lineare (capitolo 32) lavora nello spazio a piena dimensione, non nella proiezione 2D, e può sfruttare separazioni che UMAP, per costruzione, non ha modo di mostrare fedelmente in due sole dimensioni.

PROVA TU — apri il file UMAP già generato per Heart Disease e prova a descrivere, in una frase, cosa vedi — quante nuvole di punti distinte, quanto si sovrappongono le due classi. Poi confronta la tua impressione visiva con l'AUC media reale del modello migliore su questo dataset (capitolo 44): se l'AUC è alta ma la separazione visiva nel grafico UMAP ti sembra modesta, non è una contraddizione — è la conferma diretta che una proiezione 2D e un classificatore a piena dimensione possono legittimamente “vedere” gradi di separabilità diversi negli stessi dati.

Riepilogo

Nessun grafico ordinario può mostrare direttamente uno spazio a centinaia di dimensioni: UMAP costruisce prima un grafo di vicinanza locale nello spazio originale, poi ottimizza un layout 2D che lo riproduca il più fedelmente possibile. È l'unica tecnica non supervisionata del progetto, usata solo per la visualizzazione delle feature codificate nella fase di preprocessing, mai per la classificazione. Un grafico UMAP è affidabile per giudicare la vicinanza locale fra punti, ma non le distanze globali fra cluster né, di riflesso, la difficoltà reale del problema per un classificatore che lavora nello spazio a piena dimensione.

Domande di autoverifica

1. Perché serve una tecnica come UMAP per visualizzare un embedding a 768 dimensioni, e non basterebbe scegliere due delle 768 coordinate a caso? Perché due coordinate scelte a caso conterrebbero solo una minima parte dell'informazione distribuita su tutte le 768 dimensioni. UMAP costruisce invece un layout 2D pensato apposta per preservare, il più fedelmente possibile, le relazioni di vicinanza locale presenti nell'intero spazio originale.

2. In questo progetto, UMAP viene mai usato per decidere l'etichetta di un record o per addestrare il classificatore? No: `plot_umap()` produce solo un grafico a scopo di ispezione visiva, applicato alle feature codificate del preprocessing. I classificatori del progetto vengono sempre addestrati sugli embedding a piena dimensione, mai sulla loro proiezione 2D.

3. Se due classi appaiono poco separate in un grafico UMAP, puoi concludere che il problema è difficile per un classificatore lineare che lavora sui dati originali? No, non automaticamente: UMAP preserva la struttura di vicinanza locale, non le distanze globali, e una proiezione a due dimensioni può nascondere una separabilità che esiste realmente nello spazio a piena dimensione. Il grafico UMAP e le prestazioni del classificatore misurano aspetti diversi, non necessariamente in accordo visivo diretto.

MATERIALE PER LA TESI 1. La spiegazione in due fasi di UMAP (grafo di vicinanza locale, poi ottimizzazione del layout 2D), con l'avvertenza su cosa preserva e cosa no — riusabile in “Materiali e metodi” per la sezione sulla visualizzazione dei dati. 2. L'osservazione che UMAP, in questo progetto, opera solo sulle feature codificate del preprocessing e mai sugli embedding testuali usati per la classificazione — riusabile come chiarimento tecnico per prevenire un fraintendimento comune sul ruolo di questo grafico. 3. L'esercizio di confronto fra impressione visiva UMAP e AUC reale (§38.3) — riusabile come base per un paragrafo di discussione sui limiti interpretativi delle visualizzazioni a bassa dimensione.

Capitolo 39 — Iperparametri del progetto: valori e razionale

Obiettivi del capitolo

- Avere in un solo posto ogni iperparametro del progetto, esplicito o implicito, con il suo valore e la sua posizione nel codice.
- Sapere quali di questi valori sono stati scelti deliberatamente e quali sono semplicemente il default di una libreria, mai messo in discussione.
- Avere una mappa completa dei semi casuali usati, per capire esattamente cosa “riproducibile” significa in questo progetto.

39.1 Tabella completa

[Fatto] Ogni riga di questa tabella è stata verificata leggendo il codice sorgente citato; nessuna deduce un valore dal nome della libreria.

Iperparametro	Valore	Dove	Esplorato nel progetto?
<code>max_iter</code> (LogisticRegression)	2.000	<code>classification.py:16</code>	Esplicito, mai variato
<code>C</code> (LogisticRegression, forza di regolarizzazione L2)	1.0 (default scikit-learn)	mai specificato, <code>classification.py:16</code>	Mai reso esplicito né esplorato (capitolo 32.2)
<code>penalty/solver</code> (LogisticRegression)	'l2'/'lbfgs' (default)	mai specificato	Mai esplorato
<code>n_splits</code> (StratifiedKFold)	5	<code>classification.py:15</code>	Fisso, mai variato
<code>random_state</code> (StratifiedKFold)	42	<code>classification.py:15</code>	—
<code>test_size</code> (split iniziale, mai usato a valle)	0.2	<code>preprocessing.py:43</code>	—
<code>sample_size</code> (campionamento Diabetes130)	20.000	<code>function.py:116</code>	Motivato da tempi di esecuzione (<code>docs/DATASET.md:78</code>), non da un'analisi di sensibilità
<code>random_state</code> (SMOTENC)	42	<code>preprocessing.py:90</code>	—

Iperparametro	Valore	Dove	Esplorato nel progetto?
<code>batch_size</code> (embedding Ollama)	16	<code>embedding.py:103</code>	Mai variato
<code>max_retries/retry_delay</code> (embedding Ollama)	Default 2.0 s	<code>embedding.py:104</code>	—
<code>inter_batch_delay</code>	0.3 s	<code>embedding.py:104</code>	—
<code>max_workers</code> (ThreadPoolExecutor)	3	<code>embedding.py:201</code>	Mai motivato esplicitamente nel codice
Permessi del semaforo Ollama	1	<code>embedding.py:101</code>	Motivato da un commento nel codice (capitolo 12.2)
<code>n_iter</code> (bootstrap)	10.000	<code>evaluation.py:62</code>	Motivato in <code>docs/STATISTICAL_TESTS.md:9</code> (capitolo 36.3)
<code>alpha</code> (livello di confidenza)	0.95	<code>evaluation.py:77</code>	Convenzione standard, non discussa
Soglia di significatività	0.05	<code>statisticaltest.py:44, 57, 112</code>	Convenzione standard, non discussa
<code>n_components</code> (UMAP)	2	<code>function.py:224</code>	Necessario per la visualizzazione 2D
Altri iperparametri UMAP (vicini, distanza minima)	Default della libreria	non specificati	Mai esplorati (capitolo 38.2)

39.2 Effetto di valori alternativi

[Interpretazione] Tre di questi valori meritano una riflessione su cosa cambierebbe con un’alternativa, perché sono scelte con un impatto potenzialmente sostanziale sui risultati riportati:

C (regolarizzazione). Un valore di **C** più basso di 1.0 aumenterebbe la penalizzazione L2 (Formula 32.2), spingendo i pesi **w** verso valori più piccoli — un effetto particolarmente rilevante per Heart Disease, dove il capitolo 32.3 ha già mostrato che alcuni modelli hanno più parametri che esempi di training per fold: una regolarizzazione più forte in quel regime tenderebbe a ridurre l’overfitting (capitolo 2.3), potenzialmente migliorando la generalizzazione reale anche se il punteggio di validazione (già mostrato al capitolo 33.1 come non pienamente indipendente) potesse apparire leggermente più basso. Nessuna prova in questo progetto conferma o smentisce questa ipotesi: il valore non è mai stato variato.

n_splits (numero di fold). Un valore più alto di 5 (per esempio 10) userebbe più dati per l’addestramento in ciascun fold (una frazione $\frac{k-1}{k}$ più vicina a 1) a scapito di fold di validazione più piccoli e quindi di stime di metrica più rumorose per singolo fold — un compromesso classico nella scelta di *k*, mai discusso esplicitamente nella documentazione del progetto.

Soglia di significatività 0.05. Con *B* = 10.000 osservazioni bootstrap (capitolo 37.2), quasi ogni confronto fra modelli risulta “significativo” a questa soglia convenzionale — un valore più stringente (per esempio 0.01, o una correzione per confronti multipli come Bonferroni, applicabile dato che il progetto esegue $\binom{7}{2} = 21$ confronti per metrica, capitolo 26.1) cambierebbe sostanzialmente quante delle “differenze significative” riportate nei capitoli 44-45 resterebbero

tali. **[Da verificare]** Il progetto non applica alcuna correzione per confronti multipli: se questo sia un'omissione o una scelta consapevole (motivata, per esempio, dal fatto che i test servono più a descrivere i dati che a supportare un singolo test di ipotesi decisivo) resta una domanda aperta per l'Appendice E.

39.3 Riproducibilità: la mappa completa dei semi casuali nel progetto

[Fatto] Il valore 42 — una scelta convenzionale nella comunità del machine learning, senza alcun significato matematico speciale — ricorre in **sei punti indipendenti** del codice: `train_test_split` (`preprocessing.py:43`), `SMOTENC` (`preprocessing.py:90`), `UMAP` (`function.py:224`), il campionamento di `Diabetes130` (`function.py:127`), `StratifiedKfold` (`classification.py:15`), e il `bootstrap` (`evaluation.py:62`, come `seed=42`, non come `random_state`). **[Interpretazione]** Nessuno di questi sei punti condivide un generatore di numeri casuali con gli altri: sono sei semi indipendenti, ciascuno con lo stesso valore numerico per convenzione, non un'unica fonte di casualità centralizzata. Questo garantisce che *ciascuna* di queste sei operazioni sia riproducibile isolatamente (rieseguire la pipeline oggi produce lo stesso split, lo stesso bilanciamento sintetico, la stessa suddivisione in fold), ma non esiste, in nessun file del progetto, una singola costante condivisa (per esempio `RANDOM_SEED = 42` importata ovunque): il valore è scritto letteralmente sei volte, in sei posti diversi.

ATTENZIONE — questo significa che cambiare la riproducibilità del progetto — per esempio, per eseguire più run indipendenti con semi diversi e misurare la variabilità dovuta al solo seme casuale — richiederebbe modificare sei righe in quattro file diversi, non una singola costante. È un piccolo ma reale costo di manutenzione, coerente con l'assenza di un file di configurazione centralizzato già notata al capitolo 19.1.

Riepilogo

Il progetto ha una ventina di iperparametri, distribuiti fra scelte esplicite motivate (il numero di iterazioni bootstrap, la dimensione del campione di `Diabetes130`) e default di libreria mai messi in discussione (la forza di regolarizzazione `C`, gli iperparametri di `UMAP`, l'assenza di correzione per confronti multipli). Il seme casuale 42 ricorre in sei punti indipendenti del codice, garantendo riproducibilità locale per ciascuna operazione ma senza una gestione centralizzata.

Domande di autoverifica

1. Perché `C=1.0` è, di fatto, un iperparametro del progetto quanto `max_iter=2000`, anche se solo il secondo compare esplicitamente nel codice? Perché entrambi determinano il comportamento dell'ottimizzazione della regressione logistica (Formula 32.2): `C` semplicemente assume il valore di default della libreria invece di un valore scelto consapevolmente, ma quel default è comunque il valore effettivamente usato in ogni addestramento del progetto.

2. Perché una regolarizzazione più forte (un valore di `C` più basso) potrebbe essere particolarmente rilevante per `Heart Disease` rispetto a `Diabetes130`? Perché `Heart Disease` ha un rapporto dimensione-embedding/dimensione-training-set sfavorevole per i modelli più grandi (fino a 1.57, capitolo 32.3), un regime in cui l'overfitting è un rischio concreto e una regolarizzazione più forte potrebbe aiutare la generalizzazione — mentre `Diabetes130`, con un pool molto più ampio, è già in un regime sicuro indipendentemente da questa scelta.

3. Il seme casuale 42 usato in sei punti diversi del progetto proviene da un'unica costante condivisa? No: è scritto letteralmente sei volte, in file diversi (`preprocessing.py`, `function.py`, `classification.py`, `evaluation.py`), senza alcuna costante centralizzata che lo definisca una sola volta — una conseguenza dell'assenza di un file di configurazione unico già discussa al capitolo 19.1.

MATERIALE PER LA TESI 1. La tabella completa degli iperparametri (§39.1) — riusabile integralmente in “Materiali e metodi” o in un’appendice della tesi dedicata alla riproducibilità sperimentale. 2. L’analisi dell’effetto ipotetico di valori alternativi per `C`, `n_splits` e la soglia di significatività, con la proposta esplicita di correzione per confronti multipli — riusabile nella sezione “Discussione e limiti” e come direzione concreta per lavori futuri (Parte XII). 3. La mappa dei sei semi casuali indipendenti, con l’osservazione sulla loro mancata centralizzazione — riusabile come nota tecnica sulla manutenibilità, in una sezione che discuta la qualità del codice.

Capitolo 40 — Caso d'uso 1: python main.py --dataset heart_disease dall'inizio alla fine

Obiettivi del capitolo

- Seguire, con numeri reali verificati, come 920 righe di dati grezzi diventano un report finale.
- Vedere in una sola tabella come cambia la forma dei dati a ogni passaggio della pipeline.
- Sapere esattamente cosa trovare su disco al termine di un'esecuzione completa, e dove guardare per primo.

Questo capitolo non introduce concetti nuovi: mette in sequenza, con numeri concreti, tutto ciò che le Parti IV-VII hanno già spiegato separatamente.

40.1 Cosa succede fase per fase

[Fatto] Il comando `python main.py --dataset heart_disease` (o semplicemente `python main.py`, dato il default, capitolo 28.2) attraversa le sette fasi già viste al capitolo 17-18, in questo ordine, con questi numeri reali per questo dataset specifico:

1. **Pulizia** (`main.py:28-31`): i file di una precedente esecuzione su questo dataset vengono cancellati.
2. **Preprocessing** (`preprocessing.py`, capitolo 21): le 920 righe concatenate (capitolo 29.1) vengono divise 80/20 (736 righe di training, 184 di test — mai più usate, capitolo 21.1), impute, e bilanciate con SMOTENC.
3. **Embedding** (`embedding.py`, capitolo 22): ciascuna delle righe bilanciate diventa una frase, poi un vettore, per ciascuno dei sette modelli.
4. **Classificazione** (`classification.py`, capitolo 23): sette regressioni logistiche indipendenti, ciascuna validata con 5-fold stratificato.
5. **Valutazione** (`evaluation.py`, capitolo 24): bootstrap a 10.000 iterazioni per ciascun modello, sei grafici comparativi.
6. **Analisi errori** (`error_analysis.py`, capitolo 25): ogni falso positivo e falso negativo ricondotto al record clinico originale.
7. **Test statistici** (`statisticaltest.py`, capitolo 26): 21 confronti a coppie per ciascuna delle tre metriche, più 21 confronti DeLong sull'AUC.
8. **Report** (`generatereport.py`, capitolo 27): tutto assemblato in `report.md`.

40.2 Come cambiano i dati

[Fatto] Questa tabella traccia la forma esatta dei dati a ogni passaggio, con numeri verificati nelle Parti IV-VII di questo libro:

Passaggio	Forma dei dati	Fonte del numero
Dati grezzi concatenati	920 righe $\times$ 14 colonne (13 feature + target)	Capitolo 29.1
Dopo split feature/target	X : 920 $\times$ 13, y : 920	<code>preprocessing.py</code> :39-41
Dopo split 80/20	X_train : 736 $\times$ 13 (usato), X_test : 184 $\times$ 13 (mai più usato)	Capitolo 21.1, capitolo 33.1
Dopo imputazione	736 $\times$ 13, nessun valore mancante	Capitolo 21.2, capitolo 31.1
Dopo SMOTENC	X_train_bal : 814$\times$13 (classe minoritaria pareggiata)	Capitolo 4.2, verificato sui file <code>.npy</code>
Dopo conversione a testo	814 stringhe	Capitolo 22.1
Dopo embedding, per modello	814 $\times$ 768 (5 modelli) o 814 $\times$ 1024 (2 modelli)	Capitolo 32.3, tabella dimensioni
Per fold di classificazione	~651 training, ~163 validazione	814 $\times$ $\frac{4}{5}$ e 814 $\times$ $\frac{1}{5}$
Risultati concatenati, per modello	y_true/y_score/y_pred/val_i 814 ciascuno	Capitolo 23.3
Bootstrap, per modello e metrica	10.000 valori	Capitolo 36.1
Confronti statistici	$\binom{7}{2} = 21$ per metrica $\times$ 3 metriche + 21 DeLong	Capitolo 26.1, capitolo 37

40.3 Cosa trovi su disco

[Fatto] Al termine, `datas/heart_disease/reports/report.md` (già presente nel repository, generato in un'esecuzione precedente al momento della scrittura di questo libro — capitolo 43 chiarisce la provenienza esatta di questi numeri) riporta, fra le altre cose, questa tabella riassuntiva:

Modello	Accuracy	Macro-F1	ROC-AUC
sentence-biobert	0.8207	0.8195	0.8781
pubmedbert	0.8120	0.8090	0.8855
bioclinicalbert	0.7960	0.7932	0.8795
e5-large	0.7961	0.7923	0.8661
gte-base	0.7899	0.7883	0.8401
gte-large	0.7862	0.7838	0.8540
e5-base	0.7777	0.7716	0.8489

[Fatto] `sentence-biobert` ha l'accuratezza e l'F1 più alti; `pubmedbert` ha l'AUC più alta — due modelli biomedici, non generalisti, coerentemente con la seconda domanda di ricerca del progetto (capitolo 6.3). **[Attenzione]** Il capitolo 44 tratta questi stessi numeri con tutto il rigore statistico necessario (quali differenze sono davvero significative secondo i tre test, non solo quale valore è numericamente più alto): qui li vedi solo come tappa finale del percorso appena tracciato.

Oltre al report, cinque cartelle sotto `datas/heart_disease/` contengono ciascuna l'output di una o più fasi (capitolo 17.2): `preprocessing/` (3 file), `embeddings/` (14 file, due per modello),

results/ (oltre 60 file fra predizioni, bootstrap, CSV di confronto), **graphics/** (grafici UMAP, ROC, matrici di confusione), **reports/** (il solo **report.md**).

Riepilogo

Un'esecuzione completa per Heart Disease trasforma 920 righe grezze in un report finale attraversando otto passaggi tracciabili con numeri precisi: 736 righe di training (184 scartate), 814 dopo il bilanciamento sintetico, sette insiemi di embedding, sette classificatori validati a 5 fold, 10.000 ricampionamenti bootstrap per metrica e modello, 21 confronti a coppie per tre test statistici diversi. I due modelli con le prestazioni migliori nel report già presente nel repository sono entrambi biomedici, non generalisti — un primo indizio, da trattare con rigore statistico al capitolo 44, sulla seconda domanda di ricerca del progetto.

Domande di autoverifica

- 1. Delle 920 righe grezze di Heart Disease, quante finiscono davvero per contribuire a un embedding testuale, e quante vengono scartate?** 814 righe (dopo lo split 80/20 e il bilanciamento SMOTENC) contribuiscono agli embedding; le 184 righe del test set, calcolate ma mai riutilizzate, sono di fatto scartate.
- 2. Perché il numero di righe usate per l'addestramento aumenta (da 736 a 814) invece di diminuire, nonostante nessun dato venga aggiunto dall'esterno?** Perché SMOTENC genera record sintetici della classe minoritaria per pareggiarla alla classe maggioritaria (capitolo 21.2, capitolo 31.2): l'aumento di 78 righe (736→814) corrisponde esattamente alla differenza fra le due classi nel training set prima del bilanciamento.
- 3. I due modelli con le prestazioni migliori nel report già presente per Heart Disease appartengono alla stessa famiglia?** Sì, entrambi sono biomedici: **sentence-biobert** (famiglia **biomedical-st**) ha accuratezza e F1 più alti, **pubmedbert** (famiglia **biomedical**) ha l'AUC più alta — nessuno dei quattro modelli generalisti compare al primo posto in nessuna delle tre metriche.

MATERIALE PER LA TESI 1. La tabella di trasformazione dei dati (§40.2), con ogni numero verificato e la fonte esatta — riusabile come diagramma di flusso quantitativo in “Materiali e metodi”. 2. La tabella riassuntiva delle prestazioni reali per Heart Disease (§40.3) — riusabile direttamente nella sezione “Risultati”, con il rimando al capitolo 44 per l'analisi statistica completa. 3. L'osservazione che i due modelli migliori sono entrambi biomedici — riusabile come prima evidenza descrittiva a supporto della seconda domanda di ricerca, da confermare statisticamente con i test del capitolo 44.

Capitolo 41 — Caso d'uso 2: python main.py --dataset diabetes130 e le differenze che contano

Obiettivi del capitolo

- Vedere le stesse sette fasi applicate a un dataset di scala completamente diversa.
- Avere un'ipotesi motivata, non solo un'osservazione, sul perché i punteggi assoluti sono più bassi qui che su Heart Disease.
- Verificare concretamente che le due esecuzioni non si sono mai toccate a vicenda.

41.1 Stesse fasi, numeri diversi

[Fatto] `python main.py --dataset diabetes130` attraversa esattamente le stesse otto tappe del capitolo 40, con numeri molto diversi: 101.766 righe grezze campionate a 20.000 (capitolo 30.1), split 80/20 (16.000 di training, 4.000 di test — anche qui mai più usato), SMOTENC che porta il training pool a **28.428 righe** (verificato caricando `bioclinicalbert_embeddings.npy`, capitolo 32.3) — un salto molto più grande di quello visto per Heart Disease (736→814), coerente con lo sbilanciamento di partenza molto più marcato (11.16% di classe positiva, capitolo 4.2, contro il quasi-equilibrio di Heart Disease).

[Fatto] La tabella riassuntiva reale, da `datas/diabetes130/reports/report.md` (già presente nel repository):

Modello	Accuracy	Macro-F1	ROC-AUC
sentence-biobert	0.6832	0.6715	0.7678
bioclinicalbert	0.6753	0.6655	0.7575
pubmedbert	0.6628	0.6466	0.7579
e5-large	0.6273	0.6013	0.7131
gte-large	0.6269	0.6008	0.7220
e5-base	0.6170	0.5838	0.7051
gte-base	0.6169	0.5891	0.6989

[Fatto] Lo stesso pattern qualitativo di Heart Disease si ripete: `sentence-biobert` ha accuratezza e F1 più alti, e l'AUC più alta è quasi un pareggio fra `pubmedbert` (0.7579) e `bioclinicalbert` (0.7575) — così vicino che vale la pena controllare cosa dice il test appropriato invece di fidarsi della sola classifica numerica. [Fatto] Il file `datas/diabetes130/results/delong_comparison.csv` (già presente, letto per intero all'inizio di questo lavoro) riporta, per questa coppia specifica, `p_value=0.6793`, `significant=0` —

la differenza fra i due modelli con l'AUC apparentemente più alta non è statisticamente significativa. È esattamente il tipo di verifica che il capitolo 37 insegna a fare, applicata qui a un caso reale del progetto: la classifica numerica dice “pubmedbert vince”, il test di DeLong dice “non puoi dirlo con sicurezza da questi dati”.

41.2 Perché i punteggi assoluti sono più bassi

[Fatto] Ogni metrica, per ogni modello, è più bassa su Diabetes130 che su Heart Disease — anche il modello migliore in assoluto (`sentence-biobert`, AUC 0.7678 contro 0.8781). **[Interpretazione]** Tre ragioni concorrono plausibilmente a questa differenza, nessuna delle quali verificabile isolatamente con i dati disponibili in questo progetto, ma tutte ragionevoli:

1. **Il compito stesso è più difficile.** Prevedere una riammissione entro 30 giorni dipende da fattori che le 19 feature mantenute (capitolo 30.2) non catturano affatto — supporto familiare, aderenza alla terapia dopo la dimissione, comorbidità non registrate in questo sottoinsieme di colonne. Diagnosticare una malattia coronarica dai sintomi e da misure cliniche dirette (colesterolo, elettrocardiogramma) è, per contro, un compito con una relazione più diretta fra feature e target.
2. **Le feature mantenute sono più amministrative che cliniche.** `admission_type_id`, `discharge_disposition_id` sono codici di processo ospedaliero, non misure fisiologiche dirette come `chol` o `thalach` — meno segnale clinico diretto da tradurre in una frase informativa (capitolo 22.1).
3. **Il testo generato è strutturalmente più semplice.** Il capitolo 22.1 ha già notato che `record_to_text_diabetes130()` non applica alcuna traduzione di codici in etichette cliniche riconoscibili (usa `_fmt_raw()`, non `_fmt_cat()`): un modello di embedding testuale potrebbe estrarre meno segnale semantico da “admission type id: 1” che da “chest pain type: typical angina”.

ATTENZIONE — nessuna di queste tre ipotesi è isolata o testata in questo progetto: sono spiegazioni plausibili, non risultati dimostrati. Presentarle in tesi come fatti accertati, invece che come interpretazioni motivate, sarebbe un errore di calibrazione — esattamente la distinzione fra i tre livelli (fatto, teoria consolidata, interpretazione) che questo libro segue dal capitolo 0.

41.3 Isolamento tra run

[Fatto] Le due tabelle di questo capitolo e del precedente provengono da cartelle completamente separate (`datas/heart_disease/` e `datas/diabetes130/`, capitolo 18.2): puoi verificarlo tu stesso confrontando le date di generazione riportate in cima a ciascun `report.md` — 2026-08-02 16:51 per Heart Disease, 2026-08-02 18:56 per Diabetes130 (due ore di distanza, coerente con esecuzioni separate, non simultanee) — e osservando che nessun file sotto `datas/heart_disease/` porta traccia di un valore riconducibile a Diabetes130 o viceversa.

Riepilogo

La stessa pipeline, applicata a Diabetes130, produce metriche assolute più basse su tutti e sette i modelli — un compito plausibilmente più difficile, con feature più amministrative e un testo generato meno ricco semanticamente, anche se nessuna di queste ipotesi è verificata isolatamente in questo progetto. Il pattern qualitativo (modelli biomedici in testa) si ripete comunque identico a Heart Disease, con un'importante lezione di metodo: la differenza fra le due AUC più alte non è statisticamente significativa secondo il test di DeLong, nonostante la classifica numerica suggerisca un vincitore netto.

Domande di autoverifica

1. Il modello con l'AUC numericamente più alta su Diabetes130 (pubmedbert) batte davvero, in senso statistico, il secondo classificato (bioclinicalbert)? No: il test di DeLong su questa coppia riporta un p-value di 0.6793, ben sopra la soglia di significatività 0.05 — la differenza osservata (0.7579 contro 0.7575) è compatibile con variazione casuale, non con una superiorità reale e sistematica di un modello sull'altro.

2. Quali tre ragioni, nessuna delle quali dimostrata isolatamente, potrebbero spiegare perché tutte le metriche sono più basse su Diabetes130? Il compito di prevedere una riammissione futura è intrinsecamente più difficile di una diagnosi diretta; le feature mantenute sono più amministrative che cliniche; il testo generato per questo dataset non traduce i codici in etichette cliniche riconoscibili, a differenza di Heart Disease.

3. Come puoi verificare, senza fidarti della sola affermazione di questo libro, che le due esecuzioni non si sono mai sovrapposte? Confrontando le date di generazione riportate in cima ai due `report.md` (due ore di distanza) e osservando che ciascuna cartella `datas/<dataset>/` contiene solo file coerenti con il proprio dataset, senza alcuna sovrapposizione di nomi o valori.

MATERIALE PER LA TESI 1. La tabella riassuntiva reale per Diabetes130 (§41.1), affiancata a quella di Heart Disease del capitolo 40 — riusabile come tabella comparativa centrale della sezione “Risultati”. 2. L'esempio concreto del confronto pubmedbert/bioclinicalbert, dove la classifica numerica e il test statistico danno risposte diverse — è probabilmente il miglior esempio didattico del libro sulla differenza fra “sembra diverso” e “è statisticamente diverso”: riusabile quasi integralmente nella sezione “Discussione”. 3. Le tre ipotesi motivate sul perché i punteggi assoluti sono più bassi, esplicitamente marcate come interpretazione e non fatto accertato — riusabile come paragrafo di discussione calibrato, evitando affermazioni causali non supportate dai dati.

Capitolo 42 — Caso d’uso 3: aggiungere un ottavo encoder passo per passo

Obiettivi del capitolo

- Percorrere, passo per passo, l’intero processo di estensione già anticipato a livello architetturale al capitolo 19.3.
- Vedere il codice esatto da scrivere, non solo il principio generale.
- Sapere quali verifiche fare, e in quale ordine, prima di considerare l’estensione riuscita.

Questo capitolo prende il principio architetturale del capitolo 19.3 e lo trasforma in un esercizio concreto, con codice vero da scrivere — l’ultimo dei tre casi d’uso end-to-end di questa parte.

42.1 Dove si registra in `function.py`

[Fatto] `generatereport.py:221` suggerisce esplicitamente, nel testo statico del report (capitolo 27.2), di provare “bge-large” come modello aggiuntivo — un suggerimento scritto nel codice del progetto stesso, mai realizzato. Usiamolo come esempio concreto per questo esercizio.

[Interpretazione] Il primo passo, seguendo esattamente lo schema degli altri quattro modelli generalisti (`function.py:38-43`), sarebbe aggiungere una voce a `models_ollama`:

```
\NormalTok{models\_ollama }\OperatorTok{=}\NormalTok{[ ]}
\NormalTok{    \{\}\StringTok{"type"}\NormalTok{: }\StringTok{"ollama"}\NormalTok{,}
↪ }\StringTok{"model\_name"}\NormalTok{: }\StringTok{"e5{-}base"}\NormalTok{,}
↪ }\StringTok{"name"}\NormalTok{:}
↪ }\StringTok{"jeffh/intfloat{-}e5{-}base{-}v2:q8\_0"}\NormalTok{, ...}\},}
\NormalTok{    \{\}\StringTok{"type"}\NormalTok{: }\StringTok{"ollama"}\NormalTok{,}
↪ }\StringTok{"model\_name"}\NormalTok{: }\StringTok{"gte{-}base"}\NormalTok{,}
↪ }\StringTok{"name"}\NormalTok{: }\StringTok{"twvch/m3e{-}base"}\NormalTok{,}
↪ ...}\},}
\NormalTok{    \{\}\StringTok{"type"}\NormalTok{: }\StringTok{"ollama"}\NormalTok{,}
↪ }\StringTok{"model\_name"}\NormalTok{: }\StringTok{"gte{-}large"}\NormalTok{,}
↪ }\StringTok{"name"}\NormalTok{:}
↪ }\StringTok{"zyw0605688/gte{-}large{-}zh"}\NormalTok{, ...}\},}
\NormalTok{    \{\}\StringTok{"type"}\NormalTok{: }\StringTok{"ollama"}\NormalTok{,}
↪ }\StringTok{"model\_name"}\NormalTok{: }\StringTok{"e5{-}large"}\NormalTok{,}
↪ }\StringTok{"name"}\NormalTok{: }\StringTok{"jeffh/intfloat{-}multilingual{-}e5{-}"}
↪ }\StringTok{"large{-}instruct:q8\_0"}\NormalTok{,}
↪ ...}\},}
\NormalTok{    \{\}\StringTok{"type"}\NormalTok{: }\StringTok{"ollama"}\NormalTok{,}
↪ }\StringTok{"model\_name"}\NormalTok{: }\StringTok{"bge{-}large"}\NormalTok{,}
↪ }\StringTok{"name"}\NormalTok{: }\StringTok{"\textless{}tag{-}ollama{-}da{-}verif"}
↪ icare\textgreater{}"}\NormalTok{,}
```

```

\StringTok{"filename"}\NormalTok{:
  ↪ }\StringTok{"bge\_large\_embeddings.npy"}\NormalTok{,
  ↪ }\StringTok{"filename\_label"}\NormalTok{:
  ↪ }\StringTok{"bge\_large\_embeddings\_labels.npy"}\NormalTok{,}
\StringTok{"family"}\NormalTok{: }\StringTok{"general{-}purpose"}\NormalTok{\},}
\NormalTok{[]}]

```

[Da verificare] L'identificativo esatto del modello sul registro di Ollama (il campo `name`) andrebbe verificato consultando `ollama.com/library` prima di eseguire `ollama pull` — non lo indico qui perché non l'ho verificato in questa sessione, e inventarlo violerebbe il vincolo di verità di questo libro. Il resto della voce segue esattamente il pattern già visto: `filename/filename_label` con una convenzione di nomi coerente con gli altri, `family` scelta fra quelle già esistenti (qui, "general-purpose", dato che bge-large è un modello generalista, non biomedico).

42.2 Cosa fa la pipeline da sola

[Fatto] Con questa sola modifica, `models_all` (`function.py:51`, l'unione di `models_ollama` e `models_medical`) diventa automaticamente una lista di otto elementi — e ogni fase che itera su `models_all` (`embeddings()`, `training_classifier()`, `evaluate_results()`, `analyze_errors()`, `test_statistical_tests()`) processerebbe l'ottavo modello **senza alcuna ulteriore modifica**, esattamente come mostrato al capitolo 19.3: nessuna di queste funzioni nomina un modello specifico per nome, tutte iterano genericamente sulla lista.

[Fatto] Poiché `bge-large` appartiene a una famiglia già esistente ("general-purpose"), anche `get_model_palette()` (`function.py:162-174`, capitolo 20.3) gestirebbe la nuova voce automaticamente: la famiglia `general-purpose` passerebbe da 4 a 5 membri, e `sns.light_palette()` genererebbe semplicemente cinque sfumature invece di quattro, tutte ancora distinguibili.

42.3 Verifica finale

[Interpretazione] Prima di considerare l'estensione completa, andrebbero verificati, in ordine:

1. **Il modello è raggiungibile:** `ollama pull <tag>` completa senza errori, poi `ollama list` lo mostra (capitolo 16.2).
2. **La pipeline genera il suo embedding:** dopo `python main.py, datas/<dataset>/embeddings/bge_la` esiste, con la forma attesa (numero di righe pari agli altri modelli, capitolo 40.2; numero di colonne pari alla dimensione dichiarata dal modello).
3. **Compare nei risultati:** `model_performance.csv` e `encoder_comparison_summary.csv` (capitolo 23.3, capitolo 24.3) hanno una riga in più, con `bge-large` come nome.
4. **Compare correttamente nei grafici:** la matrice di confusione dedicata esiste (`CM_bge-large.png`), e il modello compare con un colore distinto (non grigio, capitolo 19.3) nei grafici di confronto per famiglia.
5. **Compare nei confronti statistici:** `wilcoxon_comparison.csv`, `ttest_comparison.csv`, `delong_comparison.csv` (capitolo 26) mostrano ora $\binom{8}{2} = 28$ righe per metrica invece di 21 — un modo indiretto ma verificabile di confermare che il modello è stato incluso in ogni fase.

PROVA TU — l'ultima verifica di questo elenco è quella più facile da dimenticare, ed è anche quella più istruttiva: conta tu stesso le righe di uno dei tre CSV di confronto statistico dopo un'estensione reale (o, se non hai eseguito l'estensione, verifica che i CSV già presenti nel repository abbiano esattamente 21 righe per metrica, $\binom{7}{2}$, coerente con i sette modelli attuali). Se il numero di righe non corrisponde al nu-

mero atteso di combinazioni, una fase ha silenziosamente escluso un modello — un controllo di integrità che il progetto stesso non fa in automatico (capitolo 18.3).

Riepilogo

Aggiungere un ottavo modello di una famiglia esistente richiede, in linea di principio, una sola modifica a `function.py` — una voce nella lista `models_ollama` o `models_medical`, seguendo il pattern degli altri modelli. Ogni fase della pipeline lo include automaticamente grazie all'iterazione generica su `models_all`. La verifica finale più affidabile non è guardare un singolo grafico, ma contare le righe dei CSV di confronto statistico: con n modelli, ci si aspettano esattamente $\binom{n}{2}$ confronti per metrica.

Domande di autoverifica

- 1. Perché aggiungere `bge-large` a `models_ollama` non richiede toccare `classification.py`, `evaluation.py` o `error_analysis.py`?** Perché tutte queste fasi iterano genericamente su `models_all`, senza mai nominare un modello specifico per nome: la lista con l'ottavo elemento propaga automaticamente la modifica a ogni fase che la consulta.
- 2. Perché `bge-large`, appartenendo alla famiglia "`general-purpose`" già esistente, non richiede alcuna modifica a `FAMILY_COLORS`?** Perché `FAMILY_COLORS` associa un colore a ogni *famiglia*, non a ogni modello: `get_model_palette()` genera automaticamente una sfumatura aggiuntiva del colore già assegnato a "`general-purpose`" per il quinto membro di quella famiglia, senza bisogno di una nuova voce.
- 3. Con 8 modelli invece di 7, quante righe per metrica dovrebbero comparire in `wilcoxon_comparison.csv`, e come verificarlo praticamente?** $\binom{8}{2} = 28$ righe per metrica, contro le 21 attuali con 7 modelli. Verificarlo è semplice quanto contare le righe del CSV filtrate per una singola metrica — un controllo indiretto ma affidabile che nessuna fase ha escluso silenziosamente il nuovo modello.

MATERIALE PER LA TESI 1. La guida passo-passo completa, con il codice esatto della nuova voce di configurazione — riusabile come sezione “riproducibilità ed estensibilità” in appendice alla tesi, o come base per un esperimento aggiuntivo realmente eseguito. 2. La checklist di verifica in cinque punti (§42.3) — riusabile come protocollo di test manuale per qualunque estensione futura del progetto, citabile in “Lavori futuri”. 3. Il controllo del numero atteso di combinazioni $\binom{n}{2}$ come verifica di integrità indiretta — riusabile come esempio di test di regressione minimale, utile anche per la Parte X (test e qualità).

Capitolo 43 — Protocollo sperimentale

Obiettivi del capitolo

- Sapere con precisione quali numeri, in questa parte del libro, provengono da un'esecuzione già presente nel repository e quali da un calcolo eseguito appositamente per questo libro.
- Avere i comandi esatti per riprodurre ogni cifra citata nei capitoli 44-47.
- Conoscere i limiti del protocollo sperimentale prima di leggerne i risultati, non dopo.

43.1 Cosa è stato eseguito da chi, e quando

[Fatto] Le tabelle riassuntive dei capitoli 44 e 45 provengono da `datas/heart_disease/reports/report.md` e `datas/diabetes130/reports/report.md`, già presenti e tracciati nel repository al momento in cui questo libro è stato scritto (2026-09-05), generati — secondo la data riportata in cima a ciascun file — il 2026-08-02, in un'esecuzione precedente della pipeline non condotta da chi scrive questo libro. **[Decisione dichiarata]** Per questa parte del libro si è scelto esplicitamente di usare questi risultati già tracciati, invece di rilanciare `python main.py` in questa sessione — una decisione motivata dal fatto che rieseguire l'intera pipeline (generazione di embedding per sette modelli, due dataset) richiederebbe diversi minuti e sovrascriverebbe file già tracciati da git, senza garanzia di produrre numeri identici data la non completa determinismo di alcune componenti (per esempio l'inferenza dei modelli via Ollama, capitolo 39.3 sulla mappa dei semi casuali, che copre i semi del progetto ma non necessariamente ogni sorgente di variabilità del server Ollama stesso).

43.2 Comandi eseguiti in questa sessione (con output reale) vs. comandi da eseguire tu

[Fatto] Non tutti i numeri di questa parte provengono dal 2026-08-02: alcuni sono stati calcolati appositamente durante la scrittura di questo libro, con comandi eseguiti realmente e il cui output è riportato per intero dove compaiono. Per trasparenza, l'elenco completo:

Calcolo	Comando (sintetizzato)	Dove compare
Sbilanciamento reale delle classi (entrambi i dataset)	Script Python su file grezzi, <code>pandas</code>	Capitolo 4.2
Dimensioni per file sorgente dei valori mancanti (Heart Disease)	Script Python su 4 file <code>.data</code>	Capitolo 29.2, 29.3
Missingness di <code>max_glu_serum/A1Cresult</code> (Diabetes130)	Script Python su <code>diabetic_data.csv</code>	Capitolo 30.2

Calcolo	Comando (sintetizzato)	Dove compare
Dimensioni degli embedding per tutti e 7 i modelli	<code>np.load(...).shape</code> su file già presenti	Capitolo 32.3, 39.1
Rapporto dimensione-embedding/dimensione-training-set	Calcolo aritmetico su dimensioni verificate	Capitolo 32.3
Verifica esistenza <code>e5_large/gte_large</code> per Diabetes130	<code>os.path.exists(...)</code>	Capitolo 32.3
Baseline a maggioranza e casuale stratificato	<code>sklearn.dummy.DummyClassifier</code> su <code>y_true.npy</code> già presenti	Capitolo 47

[Fatto] Ogni altro numero — le tabelle di accuratezza/F1/AUC dei capitoli 44-45, i tassi di errore del capitolo 46, i valori dei tre test statistici — proviene direttamente dai due `report.md` e dai CSV già presenti in `datas/`, non da un calcolo di questa sessione. **[Fatto]** L’elenco completo dei file consultati per questa parte: `datas/{heart_disease,diabetes130}/reports/report.md`, `.../results/encoder_comparison_summary.csv`, `.../results/error_summary.csv`, `.../results/hardest_cases.csv`, `.../results/{wilcoxon,ttest,delong}_comparison.csv`.

PROVA TU — rigenera tu stesso questi risultati sul tuo ambiente, seguendo l’installazione della Parte III, con:

```
\BuiltInTok{source}\NormalTok{ env/bin/activate}
\ExtensionTok{python}\NormalTok{ main.py
↪ } \AttributeTok{{-}}{-}dataset\NormalTok{ heart\_disease}
\ExtensionTok{python}\NormalTok{ main.py
↪ } \AttributeTok{{-}}{-}dataset\NormalTok{ diabetes130}
```

Confronta le tue tabelle con quelle dei capitoli 44-45. Se differiscono di più di qualche millesimo, è materiale diretto per una sezione della tesi sulla riproducibilità: i semi casuali del capitolo 39.3 dovrebbero garantire risultati identici per tutte le componenti seedate, ma non è garantito che coprano ogni sorgente di variabilità (per esempio versioni diverse dei modelli scaricati da Ollama nel frattempo).

43.3 Limiti dichiarati del protocollo

[Fatto] Questo protocollo sperimentale eredita, senza eccezione, tutti i limiti metodologici già individuati nei capitoli precedenti — è bene elencarli qui in un solo posto, prima di leggere i risultati:

- Nessun test set finale indipendente: ogni numero riportato deriva dalla validazione incrociata a 5 fold, non da un insieme mai toccato durante lo sviluppo (capitolo 21.1, capitolo 33.1).
- La soglia di decisione è ottimizzata sullo stesso fold su cui la prestazione viene misurata (capitolo 23.2, capitolo 35).
- Il rapporto dimensione-embedding/dimensione-training-set è sfavorevole per due modelli su Heart Disease (capitolo 32.3).
- Diabetes130 usa un campione di 20.000 righe su 101.766 disponibili (capitolo 30.1).
- Non è applicata alcuna correzione per confronti multipli sui 21 test a coppie per metrica (capitolo 39.2).
- I due file di embedding a 1024 dimensioni per Diabetes130 non sono più recuperabili dal repository (capitolo 32.3).

[Interpretazione] Nessuno di questi limiti, singolarmente, invalida i risultati: sono tutti limiti di grado, non di natura — la validazione incrociata è una pratica comune e ragionevole, solo meno rigorosa di un test set indipendente; il campionamento di Diabetes130 è motivato da vincoli di tempo reali, non arbitrario. Ma vanno tenuti presenti leggendo ogni tabella dei capitoli 44-47, non solo alla fine nella Parte XI.

Riepilogo

I risultati di questa parte combinano numeri già tracciati nel repository (le tabelle principali di accuratezza/F1/AUC, i tassi di errore, i test statistici, provenienti da un'esecuzione del 2026-08-02) e numeri calcolati appositamente durante la scrittura di questo libro (sbilanciamento reale delle classi, missingness per centro clinico, dimensioni degli embedding, baseline banale) — con la provenienza di ciascuno dichiarata esplicitamente. Il protocollo eredita tutti i limiti metodologici già individuati nei capitoli precedenti, elencati qui come promemoria prima di leggere i risultati veri e propri.

Domande di autoverifica

- 1. Le tabelle di accuratezza/F1/AUC dei capitoli 44-45 provengono da un'esecuzione fatta durante la scrittura di questo libro?** No: provengono da `datas/{heart_disease,diabetes130}/rep` già presenti nel repository e generati il 2026-08-02, in un'esecuzione precedente non condotta da chi scrive questo libro — una scelta dichiarata esplicitamente, non un'omissione.
- 2. Quali numeri di questa parte, invece, sono stati calcolati appositamente per questo libro?** Lo sbilanciamento reale delle classi, la scomposizione della missingness per centro clinico e per feature, le dimensioni degli embedding e il rapporto parametri/campioni, e il calcolo del modello di riferimento banale (capitolo 47) — tutti con il comando usato dichiarato esplicitamente dove compaiono.
- 3. Perché nessuno dei limiti elencati al paragrafo 43.3 invalida da solo i risultati, pur meritando attenzione?** Perché sono limiti di grado — pratiche meno rigorose della migliore prassi possibile, ma comunque comuni e ragionevoli nel settore — non errori che rendono i numeri privi di significato. Vanno interpretati con la giusta cautela, non ignorati né considerati fatali.

MATERIALE PER LA TESI 1. La tabella di provenienza dei calcoli (§43.2), con il comando sintetizzato per ciascuno — riusabile integralmente come dichiarazione di trasparenza metodologica in “Materiali e metodi”. 2. L'elenco dei limiti dichiarati, raccolto in un solo punto prima dei risultati — riusabile come premessa esplicita alla sezione “Risultati” della tesi, per calibrare correttamente le aspettative del lettore. 3. Il comando di riproduzione completo (§43.2, riquadro Prova tu) — riusabile in un'appendice sulla riproducibilità sperimentale.

Capitolo 44 — Risultati — UCI Heart Disease

Obiettivi del capitolo

- Avere la tabella completa dei risultati reali per Heart Disease, con intervalli di confidenza, non solo le medie già viste al capitolo 40.
- Sapere quali differenze fra famiglie di modelli sono descrittivamente osservabili nella tabella.
- Sapere quali delle 21 coppie di modelli sono davvero, statisticamente, diverse fra loro secondo il test di DeLong — non solo quale ha il numero più alto.

[Fatto] Ogni numero di questo capitolo proviene da `datas/heart_disease/reports/report.md` e dai CSV in `datas/heart_disease/results/`, già presenti nel repository (capitolo 43.1) — non da un'esecuzione di questa sessione.

44.1 Tabella completa delle metriche con intervalli di confidenza

[Fatto] *Tabella 44.1 — Risultati completi, Heart Disease (bootstrap 10.000 iterazioni, capitolo 36).*

Modello	Accuracy (IC 95%)	Macro-F1 (IC 95%)	ROC-AUC (IC 95%)
e5-base	0.7777 (0.7482–0.8059)	0.7716 (0.7417–0.8001)	0.8489 (0.8215–0.8746)
gte-base	0.7899 (0.7617–0.8170)	0.7883 (0.7600–0.8160)	0.8401 (0.8122–0.8667)
gte-large	0.7862 (0.7580–0.8145)	0.7838 (0.7549–0.8124)	0.8540 (0.8272–0.8797)
e5-large	0.7961 (0.7678–0.8243)	0.7923 (0.7636–0.8204)	0.8661 (0.8406–0.8900)
bioclinicalbert	0.7960 (0.7678–0.8231)	0.7932 (0.7646–0.8207)	0.8795 (0.8560–0.9014)
pubmedbert	0.8120 (0.7850–0.8378)	0.8090 (0.7811–0.8354)	0.8855 (0.8622–0.9065)
sentence-biobert	0.8207 (0.7936–0.8464)	0.8195 (0.7922–0.8449)	0.8781 (0.8544–0.9005)

Gli intervalli di confidenza si leggono come già spiegato al capitolo 36.2: il 95% dei 10.000 ricampionamenti bootstrap produce un valore in quel range. **[Fatto]** Nota che gli intervalli si sovrappongono ampiamente fra modelli vicini in classifica (per esempio bioclinicalbert ed e5-

large hanno intervalli di accuratezza quasi identici) — un indizio visivo, prima ancora del test formale, che alcune differenze potrebbero non essere statisticamente solide.

44.2 Confronto tra famiglie di modelli

[Fatto: media aritmetica semplice delle righe della Tabella 44.1, non l'analisi **bootstrap pooled** del grafico `FamilyComparison_metrics.png`] Raggruppando per famiglia (capitolo 5.3):

Famiglia	AUC media (dei modelli membri)
Generalista (e5-base, gte-base, gte-large, e5-large)	0.8523
Biomedica (bioclinicalbert, pubmedbert)	0.8825
Biomedica per frasi (sentence-biobert, un solo membro)	0.8781

[Interpretazione] Descrittivamente, le famiglie biomediche superano quella generalista di circa 2.5-3 punti percentuali di AUC media. [Attenzione] Questa è una media semplice, utile per farsi un'idea rapida, non il risultato di un test statistico: il capitolo 44.3 va oltre, verificando quali differenze specifiche fra coppie di modelli sono davvero significative — la risposta, come vedrai, è più sfumata di quanto questa tabella da sola suggerisca.

44.3 Cosa dicono davvero i test statistici

[Fatto] Il test di DeLong (capitolo 37.3) confronta l'AUC di ciascuna delle $\binom{7}{2} = 21$ coppie di modelli. Riorganizzando i risultati reali di `delong_comparison.csv` per famiglia:

[Fatto] Ogni confronto fra un modello generalista *piccolo* (e5-base, gte-base) e un modello biomedico è significativo ($p < 0.05$, in 6 confronti su 6): la superiorità biomedica, per questi due modelli generalisti specifici, è statisticamente solida, non solo descrittiva.

[Fatto] Ma per i modelli generalisti *grandi* (gte-large, e5-large), il quadro è più sfumato: **gte-large** contro **e5-large** non è significativo ($p=0.0628$, appena sopra la soglia); **e5-large** contro **bioclinicalbert** non è significativo ($p=0.061$, ugualmente appena sopra la soglia); **gte-large** contro **e5-large** (già citato) e contro **bioclinicalbert** ($p=0.0012$, questo sì significativo) danno risposte diverse a seconda della coppia specifica.

[Fatto] All'interno della famiglia biomedica, **nessuna delle tre coppie è significativamente diversa**: **bioclinicalbert-pubmedbert** ($p=0.1937$), **bioclinicalbert-sentence-biobert** ($p=0.8515$), **pubmedbert-sentence-biobert** ($p=0.3075$) — i tre modelli biomedici, pur con AUC numericamente diverse (0.8795, 0.8855, 0.8781), non si distinguono statisticamente l'uno dall'altro su questo dataset.

ATTENZIONE — la risposta onesta alla seconda domanda di ricerca del progetto (capitolo 6.3), su Heart Disease, non è un netto “sì, i modelli biomedici vincono sempre”: è “i modelli biomedici e generalisti-grandi formano, su questo dataset, un gruppo di prestazioni statisticamente indistinguibili fra loro, superiore in modo solido solo rispetto ai due modelli generalisti più piccoli”. È una conclusione più precisa, e più difendibile in sede di discussione, di quella che una lettura solo descrittiva della Tabella 44.1 (o il testo statico del report, capitolo 27.2) suggerirebbe.

Riepilogo

Sentence-biobert e pubmedbert hanno le prestazioni numericamente migliori su Heart Disease, ma il test di DeLong rivela un quadro più sfumato: la superiorità dei modelli biomedici è statisticamente solida solo contro i due modelli generalisti più piccoli (e5-base, gte-base); contro i modelli generalisti più grandi, e fra i modelli biomedici stessi, molte differenze numeriche non raggiungono la significatività statistica.

Domande di autoverifica

1. Quale modello ha l'AUC media più alta nella Tabella 44.1, e questo lo rende statisticamente superiore a tutti gli altri sei? Pubmedbert (0.8855). No: il test di DeLong mostra che è significativamente migliore di tutti e quattro i modelli generalisti (e5-base, gte-base, gte-large, e5-large, tutti con $p \leq 0.0005$), ma non è significativamente diverso dagli altri due modelli biomedici, bioclinicalbert ($p=0.1937$) e sentence-biobert ($p=0.3075$) — “il più alto numericamente” non equivale a “superiore a tutti statisticamente”.

2. Fra quali coppie di modelli generalisti “grandi” e modelli biomedici il test di DeLong non trova una differenza significativa? gte-large contro e5-large ($p=0.0628$) ed e5-large contro bioclinicalbert ($p=0.061$) — entrambi appena sopra la soglia convenzionale di 0.05, un caso limite che merita di essere riportato con precisione, non arrotondato a “significativo” o “non significativo” senza qualificazione.

3. All'interno della famiglia biomedica, quante delle tre coppie possibili sono statisticamente distinguibili in AUC? Nessuna: bioclinicalbert, pubmedbert e sentence-biobert hanno AUC numericamente diverse ma nessuna delle tre coppie raggiunge la significatività statistica secondo il test di DeLong su questo dataset.

MATERIALE PER LA TESI 1. La Tabella 44.1 completa con intervalli di confidenza — riusabile direttamente come tabella principale della sezione “Risultati”. 2. L'analisi per famiglia con la precisazione sulla natura descrittiva della media semplice — riusabile con la dovuta cautela metodologica nella stessa sezione. 3. La sintesi sfumata della risposta alla seconda domanda di ricerca, basata sul test di DeLong reale e non su una lettura solo descrittiva — è materiale di alto valore per la sezione “Discussione”, e un esempio concreto di rigore statistico applicato correttamente.

Capitolo 45 — Risultati — Diabetes 130-US Hospitals

Obiettivi del capitolo

- Avere la tabella completa dei risultati reali per Diabetes130, con intervalli di confidenza.
- Vedere come lo stesso confronto per famiglia dia, qui, una risposta molto più netta che su Heart Disease.
- Capire perché questo dataset, nonostante (o proprio per) i punteggi assoluti più bassi, offre un banco di prova metodologicamente più solido.

[Fatto] Come per il capitolo precedente, ogni numero proviene da `datas/diabetes130/reports/report.md` e dai CSV in `datas/diabetes130/results/`, già presenti nel repository (capitolo 43.1).

45.1 Tabella completa delle metriche con intervalli di confidenza

[Fatto] *Tabella 45.1 — Risultati completi, Diabetes130 (bootstrap 10.000 iterazioni, $n=28.428$).*

Modello	Accuracy (IC 95%)	Macro-F1 (IC 95%)	ROC-AUC (IC 95%)
e5-base	0.6170 (0.6114–0.6226)	0.5838 (0.5780–0.5896)	0.7051 (0.6991–0.7112)
gte-base	0.6169 (0.6113–0.6226)	0.5891 (0.5834–0.5949)	0.6989 (0.6929–0.7049)
gte-large	0.6269 (0.6214–0.6326)	0.6008 (0.5950–0.6067)	0.7220 (0.7162–0.7279)
e5-large	0.6273 (0.6216–0.6330)	0.6013 (0.5955–0.6070)	0.7131 (0.7071–0.7192)
bioclinicalbert	0.6753 (0.6699–0.6809)	0.6655 (0.6599–0.6711)	0.7575 (0.7520–0.7631)
pubmedbert	0.6628 (0.6573–0.6683)	0.6466 (0.6409–0.6522)	0.7579 (0.7523–0.7635)
sentence-biobert	0.6832 (0.6778–0.6885)	0.6715 (0.6660–0.6770)	0.7678 (0.7625–0.7732)

[Fatto] Gli intervalli di confidenza sono molto più stretti che su Heart Disease (per esempio, l'AUC di sentence-biobert varia solo fra 0.7625 e 0.7732, un intervallo di 0.0107, contro 0.0461 su Heart Disease) — una conseguenza diretta della dimensione del campione: con $n = 28.428$ invece di $n = 814$, ogni ricampionamento bootstrap (capitolo 36.1) è molto più simile agli altri, e la variabilità stimata si riduce di conseguenza.

45.2 Confronto tra famiglie di modelli

[Fatto: media aritmetica semplice, stessa cautela metodologica del capitolo 44.2]

Famiglia	AUC media (dei modelli membri)
Generalista (4 modelli)	0.7098
Biomedica (2 modelli)	0.7577
Biomedica per frasi (1 modello)	0.7678

[Fatto] Il divario descrittivo fra generalisti e biomedici è qui di circa **4.8-5.8 punti percentuali** di AUC — sensibilmente più ampio del 2.5-3 già visto su Heart Disease (capitolo 44.2). [Fatto] Il test di DeLong conferma questo divario con una nettezza che Heart Disease non aveva: **20 delle 21 coppie possibili sono statisticamente significative**, l'unica eccezione essendo bio-clinicalbert contro pubmedbert ($p=0.6793$, AUC praticamente identiche: 0.7575 contro 0.7579, capitolo 41.1) — l'unico caso in cui due modelli sono davvero, numericamente, quasi indistinguibili. Ogni altro confronto, incluso quello fra i due modelli generalisti più grandi (gte-large contro e5-large, $p<0.0001$), raggiunge la significatività statistica.

ATTENZIONE — su Diabetes130, a differenza di Heart Disease, la risposta alla seconda domanda di ricerca è netta: i modelli biomedici (tutti e tre) superano tutti e quattro i modelli generalisti in modo statisticamente solido, senza eccezioni ambigue. La differenza rispetto al quadro più sfumato di Heart Disease (capitolo 44.3) non è una contraddizione: è coerente con una dimensione campionaria molto più grande, che dà al test di DeLong più potenza per rilevare differenze reali anche quando sono numericamente contenute.

45.3 Perché questo dataset è metodologicamente più interessante

[Interpretazione] Tre ragioni, prese insieme, rendono Diabetes130 un banco di prova più solido di Heart Disease per rispondere alle domande di ricerca del progetto, nonostante (o proprio a causa di) i punteggi assoluti più bassi:

1. **Potenza statistica.** Con $n = 28.428$ contro $n = 814$, il test di DeLong ha molta più capacità di distinguere differenze reali da variazione casuale (capitolo 45.2) — la conclusione sulla superiorità biomedica poggia qui su basi statistiche molto più solide.
2. **Sbilanciamento reale più marcato.** L'11.16% di classe positiva originale (capitolo 4.2) rende la scelta delle metriche (capitolo 34) e il ruolo di SMOTENC (capitolo 21.2) tutt'altro che un dettaglio tecnico: su un dataset quasi bilanciato come Heart Disease, alcune di queste scelte avrebbero cambiato poco; qui contano davvero.
3. **Regime statistico sicuro per tutti i modelli.** Il capitolo 32.3 ha già mostrato che il rapporto dimensione-embedding/dimensione-training-set è ampiamente favorevole per tutti i modelli su questo dataset (0.045 anche per i 1024 dimensioni) — nessuno dei limiti di overfitting potenziale discussi per Heart Disease si applica qui.

[Attenzione] Questo non significa che Diabetes130 sia esente da limiti: il campionamento a 20.000 righe (capitolo 30.1, capitolo 51), l'assenza di un test set finale indipendente (capitolo 33.1, valida per entrambi i dataset) e la mancanza degli embedding a 1024 dimensioni oggi non più recuperabili (capitolo 32.3) restano punti di attenzione specifici di questo dataset.

Riepilogo

Su Diabetes130, il divario descrittivo fra famiglie biomediche e generaliste (4.8-5.8 punti di AUC) è confermato dal test di DeLong in modo netto: 20 delle 21 coppie di modelli sono statisticamente distinguibili, l'unica eccezione essendo la coppia con AUC praticamente identiche. La dimensione campionaria molto più ampia, l'sbilanciamento reale più marcato, e un regime statistico sicuro per tutti i modelli rendono questo dataset un test più solido delle domande di ricerca del progetto rispetto a Heart Disease — pur con i propri limiti specifici, distinti da quelli già discussi per l'altro dataset.

Domande di autoverifica

- 1. Perché gli intervalli di confidenza su Diabetes130 sono sensibilmente più stretti che su Heart Disease, a parità di numero di ricampionamenti bootstrap (10.000)?** Perché l'ampiezza dell'intervallo di confidenza bootstrap dipende dalla dimensione del campione sottostante, non dal numero di ricampionamenti: con $n = 28.428$ invece di $n = 814$, ogni ricampionamento è statisticamente più simile agli altri, riducendo la variabilità osservata.
- 2. Su quante delle 21 coppie di modelli il test di DeLong trova una differenza di AUC statisticamente significativa su Diabetes130, e qual è l'unica eccezione?** 20 su 21. L'unica eccezione è la coppia bioclinicalbert-pubmedbert ($p=0.6793$), le cui AUC (0.7575 e 0.7579) sono così vicine da non essere distinguibili nemmeno con questa potenza statistica elevata.
- 3. Perché la risposta alla seconda domanda di ricerca è più netta su Diabetes130 che su Heart Disease, pur riguardando lo stesso confronto concettuale?** Perché la dimensione campionaria molto più grande dà al test di DeLong più potenza statistica per rilevare differenze reali anche quando numericamente contenute — lo stesso tipo di divario descrittivo, su un campione più piccolo come Heart Disease, produce più spesso confronti che non raggiungono la soglia di significatività.

MATERIALE PER LA TESI 1. La Tabella 45.1 completa, affiancabile alla Tabella 44.1 del capitolo precedente — riusabile come tabella comparativa nella sezione “Risultati”. 2. Il conteggio “20 su 21 coppie significative”, con l'unica eccezione motivata numericamente — un risultato pulito e citabile quasi testualmente nella sezione “Risultati” o “Discussione”. 3. Le tre ragioni per cui Diabetes130 è metodologicamente più solido, con i limiti specifici comunque dichiarati — riusabile per una discussione bilanciata e onesta sulla scelta di quale dataset enfatizzare nelle conclusioni della tesi.

Capitolo 46 — Analisi degli errori sui due dataset

Obiettivi del capitolo

- Vedere un pattern sistematico, presente su entrambi i dataset, nel modo in cui i sette modelli sbagliano.
- Scoprire, con una verifica diretta e non solo un sospetto, cosa hanno davvero in comune i casi “più difficili”.
- Collegare questa analisi alle scoperte sulla qualità dei dati già fatte nella Parte VI.

46.1 Tassi di falsi positivi/negativi per modello

[Fatto] Tabella 46.1 — Tassi di errore reali, entrambi i dataset (da `error_summary.csv`, già presente nel repository).

Modello	FP rate (HD)	FN rate (HD)	FP rate (D130)	FN rate (D130)
e5-base	38.3%	6.1%	66.6%	10.0%
gte-base	29.0%	13.0%	64.3%	12.3%
gte-large	31.7%	11.1%	62.9%	11.7%
e5-large	33.7%	7.1%	62.8%	11.7%
bioclinicalbert	31.7%	9.1%	49.6%	15.3%
pubmedbert	31.0%	6.6%	55.1%	12.3%
sentence-biobert	25.6%	10.3%	50.5%	12.9%

[Fatto] Un pattern si ripete identico su **entrambi** i dataset e per **tutti e sette** i modelli, senza eccezione: il tasso di falsi positivi è sempre sostanzialmente più alto del tasso di falsi negativi — su Heart Disease di un fattore 3-6 volte, su Diabetes130 di un fattore 4-6 volte.

[Interpretazione] Una spiegazione plausibile, coerente con quanto già visto al capitolo 35.2: la soglia τ^* è scelta massimizzando F1, una metrica che in questo regime tende a favorire un recall alto (catturare i veri positivi) al costo di più falsi positivi — un compromesso che l’F1 pesa in un modo specifico, non necessariamente quello che un contesto clinico realmente richiederebbe (capitolo 1.2). [Fatto] Nota anche che i modelli biomedici hanno, coerentemente su entrambi i dataset, tassi di falsi positivi più bassi dei generalisti (media 29.4% contro 33.2% su Heart Disease; 51.7% contro 64.1% su Diabetes130) — un divario più ampio proprio sul dataset dove il capitolo 45 ha già mostrato una superiorità biomedica più netta.

46.2 I casi più difficili: cosa hanno in comune

[Fatto] Il capitolo 25.2 aveva già sollevato un dubbio, senza poterlo verificare a quel punto del libro: un record fra gli “hardest cases” potrebbe essere un artefatto di SMO-

TENC piuttosto che un caso clinicamente ambiguo. Ora possiamo verificarlo. **[Fatto]** `datas/heart_disease/results/hardest_cases.csv` (già presente, letto per intero) mostra, per i primi due record classificati male da tutti e 7 i modelli, età di **55.2321** e **54.0393** anni — valori non interi, impossibili per un’età anagrafica reale, e quindi la prova diretta che questi specifici record sono **sintetici**, generati da SMOTENC interpolando fra vicini reali (capitolo 21.2, capitolo 31.2), non pazienti realmente osservati.

[Fatto] Più rivelatore ancora: la maggioranza dei dieci casi più difficili condivide `ca=0` e `thal=3`. **[Fatto]** Ho verificato direttamente, calcolando mediana e moda sui valori realmente osservati (non mancanti) delle 920 righe originali: la **mediana osservata di `ca` è esattamente 0.0**, e la **moda osservata di `thal` è esattamente 3.0** — gli stessi identici valori che l’imputazione (`preprocessing.py:76-82`) assegna a ogni riga con questi campi mancanti (il 66.4% e il 52.8% delle righe, capitolo 29.3). **[Interpretazione]** La conclusione più probabile, alla luce di questa verifica: i record “più difficili” del progetto non sono necessariamente casi clinicamente ambigui — sono, con ogni evidenza, casi in cui `ca` e `thal` portano il valore imputato più comune, reso ancora più comune dall’amplificazione di SMOTENC già ipotizzata al capitolo 31.2. Il modello non fatica a distinguere un paziente raro e complesso: fatica a distinguere pazienti che condividono lo stesso valore “generico” imputato per due delle feature più informative del dataset standard.

ATTENZIONE — questa non è una prova definitiva (non è stato verificato ogni singolo record fra i venti “hardest cases”, solo i primi due per l’età non intera e il pattern comune per gli altri campi), ma è un’evidenza diretta e concreta, non solo un sospetto teorico. È probabilmente il collegamento più importante di questo libro fra la Parte VI (qualità dei dati) e la Parte IX (risultati): un limite di qualità dei dati individuato guardando i file grezzi si manifesta, concretamente, nell’analisi degli errori del sistema.

46.3 Deviazione di feature: quali variabili “tradiscono” un errore

[Fatto] La Formula 25.1 (capitolo 25.3) misura, per ciascuna feature numerica, quanto la sua media differisca fra casi sbagliati e casi corretti, aggregando su tutti i modelli. `datas/heart_disease/results/feature_deviation.csv` e il grafico corrispondente (`ErrorAnalysis_feature_deviation.png`, già presenti nel repository) mostrano quali feature portano il segnale più forte in questa direzione — un’informazione che completa, dal lato delle feature numeriche, ciò che il paragrafo 46.2 ha già mostrato dal lato delle feature categoriali (`ca`, `thal`, entrambe escluse da questa analisi perché non numeriche, capitolo 25.3).

PROVA TU — apri il grafico `ErrorAnalysis_feature_deviation.png` di entrambi i dataset e verifica se `oldpeak` o `trestbps` (le due feature con più valori mancanti mascherati da zero, capitolo 29.2) mostrano una deviazione insolitamente alta o bassa rispetto alle altre feature numeriche — se lo fanno, è un altro possibile segnale che la qualità dei dati, non solo la difficoltà clinica intrinseca, guida parte degli errori del sistema.

Riepilogo

Su entrambi i dataset, tutti e sette i modelli producono sistematicamente più falsi positivi che falsi negativi, coerentemente con una soglia ottimizzata per F1 (capitolo 35.2). I casi “più difficili” del progetto, verificati direttamente, sono in parte rilevante record sintetici generati da SMOTENC con valori di `ca` e `thal` coincidenti esattamente con i valori usati dall’imputazione — un’evidenza concreta che collega il limite di qualità dei dati già individuato nella Parte VI a un effetto osservabile nell’analisi degli errori.

Domande di autoverifica

1. Cosa hanno in comune tutti e sette i modelli, su entrambi i dataset, nel tipo di errore che commettono più spesso? Producono sempre più falsi positivi che falsi negativi, con un rapporto da 3 a 6 volte a seconda del modello e del dataset — un pattern sistematico, non un caso isolato di un singolo modello.

2. Quale evidenza diretta dimostra che almeno alcuni degli “hardest cases” di Heart Disease sono record sintetici, non pazienti reali? Il valore dell’età: due dei casi più difficili hanno età di 55.2321 e 54.0393 anni — valori non interi, impossibili per un’anagrafica reale, che possono derivare solo dall’interpolazione lineare di SMOTENC fra due vicini reali.

3. Perché il fatto che $ca=0$ e $thal=3$ compaiano nella maggioranza degli “hardest cases” è più di una coincidenza? Perché questi sono esattamente i valori — verificati direttamente calcolando mediana e moda sui dati realmente osservati — che l’imputazione assegna a ogni riga con questi campi mancanti (il 66% e il 53% delle righe): i casi più difficili condividono, con ogni evidenza, il profilo “generico” imputato più comune, non necessariamente una reale ambiguità clinica.

MATERIALE PER LA TESI 1. La Tabella 46.1 dei tassi di errore, con il pattern sistematico $FP > FN$ evidenziato — riusabile direttamente nella sezione “Risultati” o “Analisi degli errori”. 2. La catena di evidenza sui casi più difficili — età non intera → record sintetico; $ca=0/thal=3$ → valore di imputazione verificato — è probabilmente la scoperta più originale e meglio verificata di tutto il libro: riusabile quasi integralmente come sezione autonoma della tesi, con la piena tracciabilità della verifica. 3. Il suggerimento di verifica su `oldpeak/trestbps` nella deviazione di feature — riusabile come direzione di analisi aggiuntiva, se si decide di approfondire ulteriormente questo filone nella tesi.

Capitolo 47 — Confronto con un modello di riferimento banale

Obiettivi del capitolo

- Sapere perché un baseline non è un dettaglio accessorio, ma la base di ogni affermazione difendibile su “quanto è bravo” un sistema.
- Vedere il calcolo reale, eseguito per questo libro, di un baseline banale sugli stessi dati di validazione dei sette modelli.
- Capire perché questo confronto, da solo, rivela anche il costo reale di uno dei limiti già discussi: il test set mai usato.

47.1 Cos'è un baseline e perché la tesi ne ha bisogno

[Livello: teoria consolidata del settore] Un modello di riferimento banale (*baseline*) è il sistema più semplice possibile per lo stesso compito — nel caso più estremo, uno che ignora completamente l'input e prevede sempre la stessa cosa. Non serve a “vincere”: serve a stabilire un pavimento. Ogni affermazione del tipo “il modello X ottiene un'accuratezza dell'80%” è priva di significato senza sapere quanto otterrebbe la cosa più ovvia possibile sullo stesso identico problema — il capitolo 34.1 lo ha già anticipato con l'esempio dell'88.84% di accuratezza “gratuita” su Diabetes130 nella popolazione reale.

[Fatto] Questo progetto **non include** un baseline di alcun tipo: nessuno dei nove file del progetto calcola o riporta la prestazione di un classificatore banale. Questo capitolo lo costruisce da zero, usando dati già presenti nel repository (coerentemente con la decisione dichiarata al capitolo 43.1 di non rieseguire la pipeline).

47.2 Costruzione di un classificatore banale sugli stessi fold

[Fatto] Ho calcolato due baseline, usando `sklearn.dummy.DummyClassifier` sui file `{modello}_y_true.npy` già salvati da `classification.py` (identici per tutti e sette i modelli di uno stesso dataset, verificato con `np.array_equal` — comando eseguito in questa sessione):

```
\ImportTok{from}\NormalTok{ sklearn.dummy }\ImportTok{import}\NormalTok{
↪ DummyClassifier}
\NormalTok{dummy }\OperatorTok{=}\NormalTok{ DummyClassifier(strategy)\OperatorTok{=}
↪ \StringTok{\textquotesingle\}most\_frequent\textquotesingle\}\NormalTok{()}
\NormalTok{dummy.fit(X\_qualunque, y\_true)}
\NormalTok{y\_pred }\OperatorTok{=}\NormalTok{ dummy.predict(X\_qualunque)}
```

Il primo, **baseline a maggioranza**, prevede sempre la classe più frequente. Il secondo, **baseline casuale stratificato**, genera previsioni casuali che rispettano la proporzione osservata delle due

classi — un confronto leggermente più informativo del primo, perché non è banalmente “sempre la stessa risposta”.

[Fatto] Risultato reale (comando eseguito in questa sessione, su entrambi i dataset):

Dataset	Baseline a maggioranza (Acc / Macro-F1)	Baseline casuale stratificato (Acc / Macro-F1)
Heart Disease	0.5000 / 0.3333	0.5111 / 0.5110
Diabetes130	0.5000 / 0.3333	0.4980 / 0.4980

[Fatto] Il pool di validazione concatenato è, per entrambi i dataset, **esattamente 50.00% / 50.00%** fra le due classi — non una coincidenza: è la conseguenza diretta del bilanciamento SMOTENC (capitolo 21.2), che pareggia le classi nel training set, e della stratificazione di **StratifiedKFold** (capitolo 33.2), che preserva quella stessa proporzione in ogni fold di validazione. **[Interpretazione]** Il baseline a maggioranza su un pool esattamente bilanciato ha, per costruzione matematica, sempre accuratezza 0.5000 e macro-F1 0.3333 (una previsione costante ottiene $F1 = 2/3$ sulla classe prevista e $F1 = 0$ sull'altra, media $1/3$) — un fatto puramente aritmetico, non specifico di questi dati, ma utile da verificare comunque.

47.3 Il confronto numerico: quanto valore aggiunge davvero l'embedding

[Fatto] Confrontando questi baseline con le tabelle reali dei capitoli 44 e 45:

Dataset	Baseline maggioranza (Acc)	Modello peggiore	Modello migliore
Heart Disease	0.5000	e5-base: 0.7777 (+27.8 punti)	sentence-biobert: 0.8207 (+32.1 punti)
Diabetes130	0.5000	gte-base: 0.6169 (+11.7 punti)	sentence-biobert: 0.6832 (+18.3 punti)

[Fatto] Ogni singolo modello, su entrambi i dataset, supera nettamente il baseline a maggioranza — un risultato che, a differenza del confronto fra famiglie di modelli (capitoli 44-45), non richiede alcun test statistico per essere convincente: il margine (11.7-32.1 punti di accuratezza) è troppo ampio per essere spiegato da variazione casuale, anche sul dataset più piccolo.

ATTENZIONE — questo confronto, per quanto rassicurante, misura il vantaggio dell'embedding **sul pool bilanciato artificialmente da SMOTENC**, non sulla popolazione reale e sbilanciata da cui i dati originano (55.3%/44.7% per Heart Disease, 11.16%/88.84% per Diabetes130, capitolo 4.2). Un baseline “a maggioranza sulla popolazione reale” avrebbe un'accuratezza ben diversa — per Diabetes130, **88.84%**, più alta di ogni singolo modello di questo progetto misurato sul pool bilanciato (61.7-68.3%). Questo non significa che i modelli del progetto siano peggiori di “non fare nulla”: significa che **non è mai stata misurata**, in questo progetto, la prestazione dei sette modelli sulla distribuzione reale e sbilanciata della popolazione di origine — è precisamente la conseguenza pratica, ora quantificata con un numero concreto, del test set calcolato e mai usato (capitolo 21.1, capitolo 33.1, capitolo 43.3). Il confronto onesto con “non fare nulla sulla popolazione reale” richiederebbe applicare i sette classificatori già addestrati al test set del 20% originale — mai fatto in questo

progetto, e non rifatto in questo libro per rispettare la decisione di non rieseguire la pipeline (capitolo 43.1), ma tecnicamente possibile con i dati già presenti, e una direzione concreta per il capitolo 55.

Riepilogo

Questo progetto non include alcun baseline: questo capitolo ne ha costruito uno da zero, con un calcolo reale sui dati già tracciati. Ogni modello supera nettamente (11.7-32.1 punti di accuratezza) un baseline a maggioranza calcolato sul pool bilanciato da SMOTENC, un margine troppo ampio per essere casuale. Ma questo stesso confronto rivela, con un numero preciso, il costo del test set mai usato: un baseline a maggioranza sulla popolazione reale e sbilanciata di Diabetes130 raggiungerebbe l'88.84%, più alto di ogni modello misurato sul pool riequilibrato — un confronto che sarebbe fuorviante prendere alla lettera, ma che rende concreto perché la mancanza di un test set indipendente sia un limite reale, non solo teorico.

Domande di autoverifica

1. Perché il baseline a maggioranza ha, per costruzione matematica, esattamente accuratezza 0.5000 e macro-F1 0.3333 su entrambi i dataset di questo progetto? Perché il pool di validazione concatenato è esattamente bilanciato al 50%/50%, conseguenza diretta di SMOTENC (che pareggia le classi nel training) e della stratificazione di `StratifiedKFold` (che preserva quella proporzione in ogni fold): una previsione costante su un pool perfettamente bilanciato ottiene sempre questi due valori, indipendentemente dai dati specifici.

2. Perché il confronto “ogni modello batte nettamente il baseline a maggioranza” non richiede un test statistico formale per essere convincente? Perché il margine (da 11.7 a 32.1 punti percentuali di accuratezza) è così ampio da non poter essere ragionevolmente attribuito a variazione casuale, a differenza dei confronti fra famiglie di modelli del capitolo 44-45, dove i margini più piccoli richiedono davvero un test come DeLong per essere valutati con rigore.

3. Perché il numero “88.84%” (baseline a maggioranza sulla popolazione reale di Diabetes130) non va confrontato direttamente con le accuratezze dei modelli riportate nei capitoli 44-45? Perché quel numero si riferisce alla popolazione reale e sbilanciata, mentre le accuratezze dei modelli sono misurate sul pool riequilibrato artificialmente da SMOTENC: sono due popolazioni diverse, e confrontarle direttamente sarebbe un errore metodologico — ma il fatto stesso che questo confronto “sbagliato” sia impossibile da escludere con certezza è la conseguenza diretta dell'assenza di un test set sulla popolazione reale.

MATERIALE PER LA TESI 1. Il calcolo del baseline con il comando `DummyClassifier` e i risultati reali — riusabile integralmente nella sezione “Materiali e metodi” come baseline mancante nel progetto originale, colmata per la tesi. 2. La tabella di confronto con i margini reali (§47.3) — riusabile direttamente nella sezione “Risultati” come evidenza che il sistema supera un riferimento banale. 3. L'analisi del limite del test set mai usato, resa concreta con il numero 88.84%, invece che solo enunciata in astratto — probabilmente l'argomento più efficace di tutto il libro per la sezione “Discussione e limiti”: mostra un numero preciso, non solo un principio metodologico generico.

Capitolo 48 — Lo stato attuale: nessun test automatico

Obiettivi del capitolo - Sapere con precisione cosa significa, e cosa non significa, l'assenza di test automatici in questo progetto. - Avere una priorità chiara su cosa testare per primo, se dovessi cominciare oggi. - Confrontare `pytest`, lo strumento che useresti, con `JUnit`, quello che già conosci.

48.1 Cosa significa, onestamente, per l'affidabilità del progetto

[Fatto] Nessuno dei nove file del progetto ha un corrispettivo `test_*.py`; non esiste una cartella `tests/`; `pytest` e `unittest` non compaiono in `requirements.txt` né in alcun import (verificato con `find/grep` in fase di ricognizione, capitolo 0). [Interpretazione] Questo non significa che il codice sia necessariamente pieno di bug — molte delle criticità di questo libro (soglia ottimizzata sullo stesso fold, test set mai usato, imputazione su maggioranza mancante) sono limiti *metodologici*, che un test unitario non avrebbe intercettato comunque, perché il codice fa esattamente quello che dovrebbe fare a livello di implementazione: il problema è nella scelta, non nell'esecuzione. Ma significa che ogni modifica futura al codice — un refactoring, un aggiornamento di libreria, l'aggiunta di un ottavo modello (capitolo 42) — non ha alcuna rete di sicurezza automatica che confermi che il comportamento osservabile non sia cambiato per errore.

SE VIENI DA JAVA — in un progetto Java enterprise, l'assenza totale di test sarebbe insolita quanto lo è qui, ma per motivi diversi da apprezzare: qui non c'è un framework (Spring, Hibernate) che incoraggi test di integrazione quasi “gratuiti” con annotazioni dedicate, e il progetto è scritto per essere eseguito una volta dall'inizio alla fine da chi lo ha scritto, non mantenuto da un team nel tempo — un contesto in cui la pressione per scrivere test è storicamente più bassa, a torto o a ragione.

48.2 Cosa testeresti per primo, e perché

[Interpretazione] Non tutte le funzioni del progetto sono ugualmente facili o utili da testare. Le più adatte per un primo test sono le **funzioni pure** — che ricevono un input e restituiscono un output senza toccare disco, rete, o stato globale (capitolo 8.3): `record_to_text_heart_disease()` e `record_to_text_diabetes130()` (`embedding.py`:43-85), le funzioni `_fmt_*` di supporto, e il calcolo della soglia F1-ottima (`classification.py`:30-35, isolabile dal resto della funzione). Le meno adatte, almeno per cominciare, sono quelle che chiamano Ollama o Hugging Face (`embedding.py`) o che leggono/scrivono file (quasi ogni altra funzione, capitolo 17.1): richiedono tecniche di isolamento (mocking, capitolo 49.3) più impegnative da scrivere per prime.

[Interpretazione] Se dovessi scrivere un solo test per questo progetto, sarebbe su `record_to_text_heart_disease()`: è una funzione pura, il suo output è facile da verificare (una stringa), e un errore in questa funzione si propagherebbe silenziosamente fino al report finale senza che nessun'altra parte del sistema lo segnali — esattamente il tipo di funzione dove un test automatico dà il massimo valore per il minimo sforzo.

48.3 pytest confrontato con JUnit

[Livello: teoria consolidata del settore] `pytest` gioca, nell'ecosistema Python, un ruolo concettualmente equivalente a JUnit: un framework di test con scoperta automatica dei test (file `test_*.py`, funzioni `test_*`), asserzioni, fixture per la configurazione condivisa, e un runner da riga di comando. Le differenze principali che noteresti subito: **[Livello: teoria consolidata del settore]** `pytest` non richiede una classe che estenda una classe base (`TestCase` in `unittest`, l'equivalente più diretto di JUnit) — una funzione `def test_qualcosa()`: con dentro un'asserzione `assert` ordinaria è già un test valido, riconosciuto automaticamente dal runner. Le *fixture* di `pytest` (funzioni decorate con `@pytest.fixture`, capitolo 11.3 sui decoratori) sostituiscono sia `@Before/@After` di JUnit sia, in parte, l'iniezione di dipendenze — una fixture può essere richiesta da un test semplicemente nominandola come parametro della funzione di test, senza bisogno di un contenitore di inversione del controllo.

Riepilogo

Questo progetto non ha alcun test automatico, un fatto verificato sistematicamente, non un sospetto. Non implica necessariamente bug nascosti — molte criticità di questo libro sono metodologiche, non di implementazione — ma implica l'assenza di una rete di sicurezza per modifiche future. Le funzioni pure di conversione testo sono il punto di partenza più naturale per un primo test; `pytest` è lo strumento naturale, concettualmente vicino a JUnit ma senza il bisogno di ereditare da una classe base.

Domande di autoverifica

- 1. L'assenza di test automatici in questo progetto implica che tutte le criticità di questo libro siano bug di implementazione?** No: la maggior parte (soglia ottimizzata sullo stesso fold, test set mai usato, imputazione su maggioranza mancante) sono scelte metodologiche che un test unitario non intercetterebbe comunque, perché il codice implementa correttamente ciò che fa — il problema è nella scelta, non nell'esecuzione.
- 2. Perché `record_to_text_heart_disease()` è un candidato migliore per un primo test rispetto a una funzione di `embedding.py` che chiama Ollama?** Perché è una funzione pura: riceve un input, restituisce un output, senza toccare rete o disco. Testarla non richiede tecniche di isolamento (mocking); testare una funzione che chiama Ollama richiederebbe simulare quella chiamata di rete.
- 3. In cosa una fixture di `pytest` si avvicina, concettualmente, sia a `@Before` di JUnit sia all'iniezione di dipendenze?** Prepara uno stato condiviso prima dell'esecuzione di un test (come `@Before`), ma un test la richiede semplicemente nominandola come proprio parametro, un meccanismo più vicino, nello spirito, all'iniezione automatica di una dipendenza che a una chiamata esplicita di setup.

MATERIALE PER LA TESI 1. La distinzione fra criticità metodologiche e bug di implementazione, applicata esplicitamente all'assenza di test — riusabile nella sezione “Discussione” per calibrare correttamente la portata di questo limite. 2. La

priorità motivata su quale funzione testare per prima — riusabile come base per la sezione “Lavori futuri” o per un contributo di test realmente scritto e discusso in tesi.

3. Il confronto pytest/JUnit — riusabile come nota tecnica per un lettore Java che debba orientarsi rapidamente nell’ecosistema di test Python.

Capitolo 49 — Scrivere il primo test per questo progetto

Obiettivi del capitolo - Avere un test reale, eseguibile, per una funzione pura del progetto. - Vedere come isolare la logica della soglia F1-ottima da tutto il resto di `classification.py`. - Sapere quale tecnica servirebbe per testare le funzioni che toccano rete o disco, senza doverle eseguire davvero.

49.1 Un test per `record_to_text_heart_disease`

[Fatto] `record_to_text_heart_disease(row)` (`embedding.py:43-59`, capitolo 22.1) riceve una riga di `DataFrame` e restituisce una stringa — l'ideale per un primo test, secondo il capitolo 48.2. Un test minimo, con `pytest`:

```
\ImportTok{import}\NormalTok{ pandas }\ImportTok{as}\NormalTok{ pd}
\ImportTok{from}\NormalTok{ embedding }\ImportTok{import}\NormalTok{
↪ record\_to\_text\_heart\_disease}

\KeywordTok{def}\NormalTok{ test\_record\_to\_text\_paziente\_completo():}
\NormalTok{    row }\OperatorTok{=}\NormalTok{ pd.Series(\{}
    \StringTok{"sex"}\NormalTok{: }\DecValTok{1}\NormalTok{,}
    ↪ }\StringTok{"age"}\NormalTok{: }\DecValTok{63}\NormalTok{,}
    ↪ }\StringTok{"cp"}\NormalTok{: }\DecValTok{1}\NormalTok{,}
    ↪ }\StringTok{"trestbps"}\NormalTok{: }\DecValTok{145}\NormalTok{,}
    ↪ }\StringTok{"chol"}\NormalTok{: }\DecValTok{233}\NormalTok{,}
    \StringTok{"fbs"}\NormalTok{: }\DecValTok{1}\NormalTok{,}
    ↪ }\StringTok{"restecg"}\NormalTok{: }\DecValTok{0}\NormalTok{,}
    ↪ }\StringTok{"thalach"}\NormalTok{: }\DecValTok{150}\NormalTok{,}
    ↪ }\StringTok{"exang"}\NormalTok{: }\DecValTok{0}\NormalTok{,}
    \StringTok{"oldpeak"}\NormalTok{: }\FloatTok{2.3}\NormalTok{,}
    ↪ }\StringTok{"slope"}\NormalTok{: }\DecValTok{3}\NormalTok{,}
    ↪ }\StringTok{"ca"}\NormalTok{: }\DecValTok{0}\NormalTok{,}
    ↪ }\StringTok{"thal"}\NormalTok{: }\DecValTok{6}\NormalTok{,}
\NormalTok{    \)}
\NormalTok{    \)}
\NormalTok{    testo }\OperatorTok{=}\NormalTok{
↪ record\_to\_text\_heart\_disease(row)}
    \ControlFlowTok{assert} \StringTok{"Male patient, 63 years old"}
    ↪ \KeywordTok{in}\NormalTok{ testo}
    \ControlFlowTok{assert} \StringTok{"chest pain type: typical angina"}
    ↪ \KeywordTok{in}\NormalTok{ testo}
    \ControlFlowTok{assert} \StringTok{"thalassemia: fixed defect"}
    ↪ \KeywordTok{in}\NormalTok{ testo}

\KeywordTok{def}\NormalTok{ test\_record\_to\_text\_valore\_mancante():}
\NormalTok{    row }\OperatorTok{=}\NormalTok{ pd.Series(\{}

```

```

\StringTok{"sex"}\NormalTok{: } \DecValTok{0}\NormalTok{,
↪ } \StringTok{"age"}\NormalTok{: } \DecValTok{55}\NormalTok{,
↪ } \StringTok{"cp"}\NormalTok{: } \DecValTok{4}\NormalTok{,
↪ } \StringTok{"trestbps"}\NormalTok{: }
↪ } \BuiltInTok{float}\NormalTok{({} \StringTok{"nan"}\NormalTok{)},
↪ } \StringTok{"chol"}\NormalTok{: } \DecValTok{200}\NormalTok{,}
\StringTok{"fbs"}\NormalTok{: } \DecValTok{0}\NormalTok{,
↪ } \StringTok{"restecg"}\NormalTok{: } \DecValTok{0}\NormalTok{,
↪ } \StringTok{"thalach"}\NormalTok{: } \DecValTok{140}\NormalTok{,
↪ } \StringTok{"exang"}\NormalTok{: } \DecValTok{1}\NormalTok{,}
\StringTok{"oldpeak"}\NormalTok{: } \FloatTok{1.0}\NormalTok{,
↪ } \StringTok{"slope"}\NormalTok{: } \DecValTok{2}\NormalTok{,
↪ } \StringTok{"ca"}\NormalTok{: } \DecValTok{1}\NormalTok{,
↪ } \StringTok{"thal"}\NormalTok{: } \DecValTok{3}\NormalTok{,}
\NormalTok{{
↪ }}
\NormalTok{{
↪ testo } \OperatorTok{=} \NormalTok{{
↪ record\_to\_text\_heart\_disease(row)}
↪ \ControlFlowTok{assert} \StringTok{"resting blood pressure: not recorded"}
↪ \KeywordTok{in} \NormalTok{{ testo}

```

Il primo test verifica che i codici numerici (`cp=1`, `thal=6`) vengano tradotti correttamente nelle etichette leggibili attese (capitolo 22.1, `CP_LABELS`, `THAL_LABELS`); il secondo verifica il caso limite già discusso al capitolo 31.3 — un valore NaN residuo diventa `"not recorded"`, non un errore né la stringa letterale `"nan"`.

RIFERIMENTO AL CODICE — questi test non richiedono importare l'intero progetto: solo `embedding.py` e `pandas`. Eseguirli con `pytest test_embedding.py` (o il nome scelto per il file) richiede solo che l'ambiente virtuale del capitolo 15 sia attivo, non Ollama né una connessione di rete.

49.2 Un test per la logica di soglia F1-ottima con dati sintetici

[Fatto] La ricerca della soglia F1-ottima (`classification.py:30-35`, Formula 35.1) è incorporata dentro un ciclo più ampio, non isolata in una propria funzione — un limite di progettazione che rende il test seguente più laborioso di quanto dovrebbe essere (un punto su cui torna il capitolo 55). Isolandola manualmente per il test:

```

\ImportTok{import}\NormalTok{ numpy }\ImportTok{as}\NormalTok{ np}
\ImportTok{from}\NormalTok{ sklearn.metrics }\ImportTok{import}\NormalTok{
↪ precision\_recall\_curve}

\KeywordTok{def}\NormalTok{ soglia\_f1\_ottima(y\_val, y\_score):}
\NormalTok{{
↪ precision, recall, thresholds } \OperatorTok{=} \NormalTok{{
↪ precision\_recall\_curve(y\_val, y\_score)}
\NormalTok{{
↪ f1\_scores } \OperatorTok{=} \DecValTok{2} \OperatorTok{*} \NormalTok{{
↪ (precision[:]\OperatorTok{-})\DecValTok{1}\NormalTok{{
↪ }\OperatorTok{*} \NormalTok{{ recall[:]\OperatorTok{-})\DecValTok{1}\NormalTok{{
↪ }\OperatorTok{/} \NormalTok{{
↪ (precision[:]\OperatorTok{-})\DecValTok{1}\NormalTok{{
↪ }\OperatorTok{+} \NormalTok{{ recall[:]\OperatorTok{-})\DecValTok{1}\NormalTok{{
↪ }\OperatorTok{+} \FloatTok{1e-6}\NormalTok{{
↪ \ControlFlowTok{return} \NormalTok{{ thresholds[f1\_scores.argmax()]}}

\KeywordTok{def}\NormalTok{ test\_soglia\_su\_separazione\_perfetta():}
\CommentTok{# 4 casi negativi con punteggio basso, 4 positivi con punteggio}
↪ alto: separazione netta}

```



```

\NormalTok{    y\_val }\OperatorTok{=}\NormalTok{
↪ np.array([\DecValTok{0}\NormalTok{, }\DecValTok{0}\NormalTok{,
↪ }\DecValTok{0}\NormalTok{, }\DecValTok{0}\NormalTok{, }\DecValTok{1}\NormalTok{,
↪ }\DecValTok{1}\NormalTok{, }\DecValTok{1}\NormalTok{, }\DecValTok{1}\NormalTok{]})
\NormalTok{    y\_score }\OperatorTok{=}\NormalTok{
↪ np.array([\FloatTok{0.1}\NormalTok{, }\FloatTok{0.2}\NormalTok{,
↪ }\FloatTok{0.15}\NormalTok{, }\FloatTok{0.05}\NormalTok{,
↪ }\FloatTok{0.9}\NormalTok{, }\FloatTok{0.8}\NormalTok{,
↪ }\FloatTok{0.95}\NormalTok{, }\FloatTok{0.85}\NormalTok{]})
\NormalTok{    tau }\OperatorTok{=}\NormalTok{ soglia\_f1\_ottima(y\_val, y\_score)}
\ControlFlowTok{assert} \FloatTok{0.2} \OperatorTok{\textless{}}\NormalTok{ tau
↪ }\OperatorTok{\textless{}}= \FloatTok{0.8} \CommentTok{\# qualunque soglia
↪ in questo intervallo separa perfettamente le due classi}

\KeywordTok{def}\NormalTok{ test\_soglia\_su\_dati\_non\_informativi():}
\CommentTok{\# punteggi casuali, senza relazione con
↪ l\textquotesingle{etichetta: la soglia scelta è comunque un numero valido}
\NormalTok{    rng }\OperatorTok{=}\NormalTok{
↪ np.random.default\_rng()\DecValTok{42}\NormalTok{}}
\NormalTok{    y\_val }\OperatorTok{=}\NormalTok{
↪ rng.integers()\DecValTok{0}\NormalTok{, }\DecValTok{2}\NormalTok{,
↪ }\DecValTok{50}\NormalTok{}}
\NormalTok{    y\_score }\OperatorTok{=}\NormalTok{
↪ rng.random()\DecValTok{50}\NormalTok{}}
\NormalTok{    tau }\OperatorTok{=}\NormalTok{ soglia\_f1\_ottima(y\_val, y\_score)}
\ControlFlowTok{assert} \FloatTok{0.0} \OperatorTok{\textless{}}\NormalTok{ tau
↪ }\OperatorTok{\textless{}}= \FloatTok{1.0}

```

Il primo test verifica il caso “facile”: con una separazione perfetta fra le due classi, qualunque soglia nell’intervallo giusto massimizza F1, e il test lo verifica con un margine, non un valore esatto ($0.2 < \tau \leq 0.8$), perché la funzione potrebbe legittimamente scegliere una soglia diversa a seconda di dove cadono i punti di discontinuità della curva precisione-recall. Il secondo verifica solo che la funzione non fallisca e restituisca un valore nell’intervallo valido, anche quando i dati non portano alcun segnale reale — un test di robustezza, non di correttezza del risultato specifico.

49.3 Cosa serve per testare le funzioni che toccano il disco o la rete

[Livello: teoria consolidata del settore] Testare `generate_embeddings_batch()` (`embedding.py:103-137`, capitolo 22.2) così com’è richiederebbe un server Ollama realmente in esecuzione — un test lento, fragile (dipende dalla disponibilità di un servizio esterno), e non ripetibile in isolamento. La tecnica standard per evitarlo è il **mocking**: sostituire, solo durante il test, l’oggetto `Client` di Ollama con un oggetto finto che restituisce risposte predefinite, senza fare alcuna vera chiamata di rete.

```

\ImportTok{from}\NormalTok{ unittest.mock }\ImportTok{import}\NormalTok{ patch,
↪ MagicMock}
\ImportTok{from}\NormalTok{ embedding }\ImportTok{import}\NormalTok{
↪ generate\_embeddings\_batch}

\KeywordTok{def}\NormalTok{ test\_generate\_embeddings\_batch\_con\_client\_finto():}
\NormalTok{    risposta\_finta }\OperatorTok{=}\NormalTok{ MagicMock()}
\NormalTok{    risposta\_finta.embeddings }\OperatorTok{=}\NormalTok{
↪ [[]\FloatTok{0.1}\NormalTok{, }\FloatTok{0.2}\NormalTok{,
↪ }\FloatTok{0.3}\NormalTok{] }\OperatorTok{*} \DecValTok{16}

```

```

\ControlFlowTok{with}\NormalTok{
  ↪ patch{}\StringTok{"embedding.Client"}\NormalTok{} }\ImportTok{as}\NormalTok{
  ↪ ClientFinto:}
\NormalTok{
  ClientFinto.return\_value.embed.return\_value
  ↪ }\OperatorTok{=}\NormalTok{ risposta\_finto}
\NormalTok{
  risultati }\OperatorTok{=}\NormalTok{
  ↪ generate\_embeddings\_batch{}\StringTok{"modello{-}finto"}\NormalTok{,
  ↪ []\StringTok{"frase"}\NormalTok{ }\OperatorTok{*} \DecValTok{16}\NormalTok{,
  ↪ batch\_size}\OperatorTok{=}\DecValTok{16}\NormalTok{}}
  \ControlFlowTok{assert} \BuiltInTok{len}\NormalTok{(risultati) }\OperatorTok{==}
  ↪ \DecValTok{16}

```

`patch("embedding.Client")` sostituisce temporaneamente, solo dentro il blocco `with`, la classe `Client` importata in `embedding.py` con un oggetto finto (`MagicMock`) che restituisce `risposta_finto.embeddings` ogni volta che viene chiamato `.embed(...)` — nessuna connessione di rete viene mai aperta, e il test verifica solo che la funzione elabori correttamente la struttura della risposta, non che Ollama funzioni davvero.

SE VIENI DA JAVA — `unittest.mock.patch` gioca un ruolo concettualmente simile a `Mockito`: sostituisce una dipendenza con un doppio di test, per la durata del test. La differenza pratica più notevole: `patch` qui sostituisce un nome importato in un modulo specifico ("`embedding.Client`", non semplicemente "`ollama.Client`") — un dettaglio che confonde spesso chi arriva da un framework di mocking basato su iniezione di dipendenze esplicita, dove l'oggetto da sostituire è passato come parametro, non risolto per nome del modulo che lo importa.

Riepilogo

Un primo test su `record_to_text_heart_disease()` verifica la traduzione dei codici e la gestione dei valori mancanti, senza toccare rete o disco. Isolare la logica della soglia F1-ottima in una funzione a sé permette di testarla su dati sintetici, sia nel caso di separazione perfetta sia su dati privi di segnale. Testare le funzioni che chiamano Ollama richiede il mocking del client, sostituendo temporaneamente la dipendenza esterna con un oggetto finto che restituisce risposte predefinite.

Domande di autoverifica

1. Perché il test sulla separazione perfetta verifica un intervallo ($0.2 < \text{tau} \leq 0.8$) invece di un valore esatto di soglia? Perché la funzione sceglie la soglia fra i punti di discontinuità della curva precisione-recall, che dipendono dai valori esatti dei punteggi: più soglie diverse, in questo esempio, separano ugualmente bene le due classi e massimizzano F1 allo stesso modo, quindi il test verifica solo che il risultato sia in un intervallo ragionevole, non un singolo valore.

2. Perché testare `generate_embeddings_batch()` senza mocking sarebbe un test fragile, non solo lento? Perché dipenderebbe dalla disponibilità reale di un server Ollama in esecuzione con il modello richiesto scaricato: un problema di rete, un server non avviato, o un modello mancante farebbero fallire il test per ragioni indipendenti dalla correttezza del codice che si vuole verificare.

3. Cosa sostituisce esattamente `patch("embedding.Client")`, e perché non basta scrivere `patch("ollama.Client")`? Sostituisce il nome `Client` così come è stato importato dentro il modulo `embedding.py` (`from ollama import Client, embedding.py:10`), non la classe origi-

nale nel modulo `ollama`: `embedding.py` ha già il proprio riferimento locale a `Client`, ed è quello che va sostituito perché la funzione sotto test lo usi.

MATERIALE PER LA TESI 1. I quattro test completi di questo capitolo — riusabili direttamente come contributo di test scritto per la tesi, con la spiegazione del perché ciascuno è stato progettato in quel modo. 2. L’osservazione che la logica della soglia non è isolata in una propria funzione nel codice originale — riusabile come esempio concreto di come la mancanza di test scoraggi (o sia scoraggiata da) una progettazione più modulare. 3. La spiegazione del mocking applicata a `Client` di Ollama — riusabile come nota tecnica per chi debba scrivere test per qualunque altra parte del progetto che tocchi servizi esterni.

Capitolo 50 — Debugging e log in pratica

Obiettivi del capitolo - Sapere come usare un debugger Python allo stesso modo in cui useresti quello di un IDE Java. - Leggere i `print()` sparsi nel codice come se fossero un log strutturato, senza bisogno che lo siano davvero. - Seguire un percorso di debugging guidato su un bug reale e già documentato di questo progetto.

50.1 Il debugger di un IDE Python confrontato con quello di un IDE Java

[Livello: teoria consolidata del settore] Un debugger Python (integrato in PyCharm, VS Code, o eseguibile da riga di comando con `pdb`, la libreria standard) offre le stesse funzionalità concettuali di un debugger Java: breakpoint, esecuzione passo-passo, ispezione delle variabili nello stack corrente, valutazione di espressioni al volo. La differenza pratica più rilevante, conseguenza diretta della tipizzazione dinamica (capitolo 7.2): quando ispezioni una variabile in un breakpoint Python, l'IDE ti mostra il suo tipo *attuale*, scoperto a runtime — non un tipo dichiarato che potresti già conoscere dalla firma della funzione. Fermarti dentro `training_classifier()` (`classification.py`, capitolo 23) e ispezionare `X` ti dice, lì per lì, che è un `numpy.ndarray` di una certa forma: un'informazione che, in Java, sapresti già dalla dichiarazione del parametro.

SE VIENI DA JAVA — un breakpoint condizionale (che si attiva solo se un'espressione è vera) funziona in modo identico nei due mondi. Vale la pena ricordare, per un debugging efficace su questo progetto specifico, che molte funzioni ricevono solo un nome di dataset (capitolo 17.1) e ricostruiscono tutto il resto rileggendo file da disco: un breakpoint dentro `training_classifier()` non ti mostrerà i dati “in arrivo da preprocessing.py” — te li mostrerà già caricati da `np.load()`, un passaggio intermedio che un debugger su una pipeline con passaggio diretto di oggetti non avrebbe.

50.2 Leggere i `print()` del progetto come se fossero log strutturati

[Fatto] Nessun file del progetto usa il modulo `logging` della libreria standard — ogni traccia di esecuzione passa da semplici chiamate a `print()`, verificabile scorrendo qualunque dei nove file. [Interpretazione] Questo non è necessariamente un problema per un progetto eseguito interattivamente da riga di comando, come questo, ma significa che non esistono livelli di severità (DEBUG, INFO, WARNING, ERROR), non c'è modo di silenziare selettivamente l'output, e ogni messaggio va allo stesso stream (l'output standard), mescolato indistintamente.

[Fatto] Puoi comunque leggere questi messaggi con un metodo, riconoscendo pattern ricorrenti: `embedding.py` prefissa i propri messaggi con [Batch] o [HF] a seconda della fase (righe

107,128,140), un’abitudine che imita — senza usarlo davvero — il formato di un log strutturato con un “tag” di provenienza. `statisticaltest.py:92` stampa `[WARNING]` per un caso anomalo specifico (etichette vere diverse fra due modelli, capitolo 26.2) — l’unico punto del progetto che imita esplicitamente un livello di severità. **[Interpretazione]** Se dovessi seguire un’esecuzione reale della pipeline, cerca questi prefissi per orientarti rapidamente su quale fase e quale modello sta producendo un dato messaggio, invece di leggere l’output riga per riga.

50.3 Isolare un bug reale: percorso guidato su un modulo del progetto

[Fatto] Usiamo un bug reale già documentato in questo libro (capitolo 15.2, capitolo 16.1): seguendo il README alla lettera, `ollama pull yxchia/multilingual-e5-base` scarica un modello diverso da quello che `function.py:39` richiede (`jeffh/intfloat-e5-base-v2:q8_0`). Ecco il percorso di debugging che porteresti avanti, passo per passo, se non conoscessi già la causa:

1. **Il sintomo:** `python main.py` fallisce durante la fase di embedding (capitolo 22), con un `RuntimeError` che menziona “model not found” o simile, per il modello `e5-base`.
2. **Primo breakpoint:** dentro `process_model()` (`embedding.py:182-199`), sulla riga che chiama `generate_embeddings_batch(name, texts)` — ispeziona il valore di `name`. Se è `"jeffh/intfloat-e5-base-v2:q8_0"`, il problema non è nel codice Python: è che Ollama non ha quel modello scaricato.
3. **Verifica esterna al debugger:** da terminale, `ollama list` — se `jeffh/intfloat-e5-base-v2:q8_0` non compare nell’elenco, hai isolato la causa: il modello giusto non è mai stato scaricato.
4. **Root cause:** confrontando il comando di installazione seguito (`README.md:104`) con il valore appena ispezionato nel debugger, la discrepanza emerge — due nomi diversi per quello che dovrebbe essere lo stesso modello.

PROVA TU — questo percorso ha funzionato perché conoscevi già la causa dalla lettura di questo libro. Scegli ora un altro punto del codice che non hai ancora esplorato interattivamente — per esempio `build_encoder()` in `preprocessing.py` — metti un breakpoint alla prima riga, ed esplora cosa contengono davvero `X`, `num_cols`, `cat_cols` in quel momento specifico dell’esecuzione, senza fidarti solo di quello che il libro ti ha già detto: è il modo più efficace di consolidare la lettura di un capitolo di questo libro, non un’alternativa ad essa.

Riepilogo

Un debugger Python offre le stesse funzionalità concettuali di uno Java, con la differenza pratica che ogni tipo ispezionato è scoperto a runtime, non dichiarato in anticipo. Il progetto non usa logging, solo `print()` con prefissi informali ricorrenti (`[Batch]`, `[HF]`, `[WARNING]`) che aiutano a orientarsi. Il percorso di debugging del “model not found” per `e5-base` — dal sintomo, a un breakpoint mirato, a una verifica esterna al debugger, alla causa radice — è un esempio concreto e già verificato di come isolare un bug reale di questo progetto.

Domande di autoverifica

1. Perché ispezionare `X` in un breakpoint dentro `training_classifier()` non ti dice nulla che non potresti già sapere dalla firma della funzione, in Java? È l’opposto: in Python, la firma (`def training_classifier(dataset="heart_disease"):`) non dichiara affatto il tipo delle variabili locali come `X`; solo il breakpoint, a runtime, rivela che è un `numpy.ndarray`

di una certa forma — un’informazione che una firma Java tipizzata staticamente comunicherebbe già in anticipo.

2. A cosa serve il prefisso [WARNING] stampato da `statisticaltest.py:92`, dato che il progetto non usa il modulo `logging`? Imita, senza usarlo davvero, un livello di severità di un log strutturato: segnala visivamente, a chi legge l’output della console, che quel messaggio specifico riguarda un caso anomalo (etichette vere diverse fra due modelli), distinguendolo dagli altri messaggi informativi stampati con `print()` ordinario.

3. Nel percorso di debugging del capitolo, qual è il passaggio che richiede uscire dal debugger e usare il terminale? La verifica con `ollama list`: il debugger può mostrarti quale nome di modello il codice sta effettivamente richiedendo, ma solo un comando esterno al processo Python può confermare se quel modello è davvero disponibile sul server Ollama in esecuzione.

MATERIALE PER LA TESI 1. Il confronto fra debugger Python e Java, con l’osservazione sulla scoperta dei tipi a runtime — riusabile come nota tecnica per un lettore che debba impostare per la prima volta un ambiente di sviluppo Python. 2. Il percorso di debugging completo sul bug reale del modello e5-base — riusabile come caso di studio in una sezione della tesi su debugging e qualità del software, con tanto di causa radice già identificata e documentata. 3. L’osservazione sull’assenza del modulo `logging` e sul suo effetto pratico sull’osservabilità del sistema — riusabile come punto di discussione nella sezione critica sulla qualità del codice.

Capitolo 51 — Limiti metodologici del protocollo di valutazione

Obiettivi del capitolo - Avere, in un solo capitolo, la sintesi organizzata dei tre limiti metodologici più rilevanti già incontrati separatamente nel libro. - Sapere quale peso dare a ciascuno: quale è più grave, quale più facile da correggere. - Collegare ogni limite a una proposta di correzione concreta, non solo alla sua diagnosi.

Questo capitolo non introduce fatti nuovi: raccoglie, con la disciplina critica che la Parte XI richiede, tre limiti già individuati nei capitoli precedenti, li pesa l'uno contro l'altro, e li collega a correzioni concrete (Parte XII).

51.1 Assenza di test set finale indipendente

[**Fatto**] `preprocessing.py:43` calcola un test set del 20% che non viene mai più usato (capitolo 21.1, capitolo 33.1). [**Interpretazione — gravità**] Fra i tre limiti di questo capitolo, è il più strutturale: non riguarda un dettaglio di implementazione ma l'intero disegno della valutazione. Il capitolo 47.3 ha reso concreto il suo costo con un numero preciso — un baseline sulla popolazione reale e sbilanciata di Diabetes130 raggiungerebbe 88.84%, un dato che nessuna misura di questo progetto conferma o smentisce direttamente, perché nessuna misura è mai stata condotta sulla distribuzione reale, solo su quella riequilibrata da SMOTENC. [**Interpretazione — correggibilità**] È anche, paradossalmente, il più facile da correggere in astratto: i dati del test set esistono già, calcolati e scartati; applicare i modelli già addestrati a quei dati richiederebbe modificare `preprocessing_data()` per restituire e salvare anche `X_test/y_test`, e una fase aggiuntiva che li usi (capitolo 55.1).

51.2 Soglia ottimizzata sullo stesso fold di validazione

[**Fatto**] La Formula 35.1 (capitolo 35.2) sceglie τ^* massimizzando F1 su `y_val`, poi misura la prestazione sullo stesso `y_val`. [**Interpretazione — gravità**] È un ottimismo più contenuto del precedente: non tocca l'addestramento del modello (capitolo 33.3), solo la soglia — e il capitolo 34.3 ha già mostrato che l'AUC, l'unica metrica indipendente dalla soglia, non ne risente affatto. Riguarda solo accuratezza e F1, e solo nella misura in cui la soglia F1-ottima per fold si discosta da quella che si sceglierebbe su dati indipendenti. [**Interpretazione — correggibilità**] Facilmente corretto: la Formula 35.2 (un fold di calibrazione separato) richiederebbe solo una diversa suddivisione dei dati dentro il ciclo di `StratifiedKfold`, senza cambiare l'architettura della pipeline.

51.3 Il campionamento a 20.000 record di Diabetes130

[Fatto] `load_diabetes130(sample_size=20000)` (`function.py:116-132`, capitolo 30.1) usa un campione stratificato di un quinto delle 101.766 righe originali. [Interpretazione — gravità] Il più lieve dei tre: è una scelta dichiarata e motivata (tempi di esecuzione, capitolo 30.1), il campionamento è stratificato (preserva la proporzione di classe, capitolo 33.2), e con 20.000 righe il progetto ottiene comunque, dopo SMOTENC, un pool di 28.428 righe per la validazione (capitolo 43) — una dimensione ben più che sufficiente per le conclusioni statistiche del capitolo 45. [Interpretazione — correggibilità] Il più costoso da correggere in pratica: userebbe l'intero file, moltiplicando per cinque il tempo di generazione degli embedding per tutti e sette i modelli (capitolo 22), il passaggio più lento della pipeline.

Una tabella di sintesi

Limite	Gravità	Costo di correzione	Capitolo di riferimento
Test set mai usato	Alta (strutturale)	Basso	21.1, 33.1, 47.3
Soglia sul fold di validazione	Media (contenuta all'F1/accuratezza)	Basso	23.2, 35.2
Campionamento di Diabetes130	Bassa (dichiarata, stratificata)	Alto	30.1

ATTENZIONE — questa tabella è la mia valutazione, motivata riga per riga sopra, non un fatto oggettivo misurabile con un solo numero: un revisore più severo potrebbe pesare diversamente la gravità del secondo limite, per esempio, se ritenesse che anche un ottimismo “contenuto” sia inaccettabile in un contesto clinico. È materiale da discutere, non da accettare passivamente in sede di tesi.

Riepilogo

I tre limiti metodologici principali di questo progetto — test set mai usato, soglia ottimizzata sul fold di validazione, campionamento di Diabetes130 — hanno gravità e costo di correzione molto diversi fra loro. Il più grave (test set) è anche il più facile da correggere; il più lieve (campionamento) è il più costoso da correggere in pratica, richiedendo di rifare la fase più lenta della pipeline su cinque volte più dati.

Domande di autoverifica

1. Perché l'assenza di un test set finale indipendente è considerata, in questo capitolo, il limite più grave dei tre? Perché è strutturale, non un dettaglio di implementazione: significa che nessuna misura di questo progetto conferma le prestazioni sulla distribuzione reale e sbilanciata dei dati, solo su quella artificialmente riequilibrata da SMOTENC — un fatto reso concreto dal numero 88.84% del capitolo 47.3.

2. Perché il limite della soglia ottimizzata sul fold di validazione non tocca l'AUC riportata nei capitoli 44-45? Perché l'AUC (capitolo 34.3) dipende solo dall'ordinamento dei punteggi di probabilità, non da una soglia specifica: la fuga di informazione riguarda solo la scelta di τ^* , che influenza esclusivamente le metriche calcolate su etichette già sogliate, cioè accuratezza e F1.

3. Perché il campionamento di Diabetes130, pur essendo un limite reale, è valutato come il meno grave dei tre in questo capitolo? Perché è dichiarato esplicitamente e motivato da vincoli di tempo reali, il campionamento è stratificato (preserva la proporzione di classe), e il pool risultante dopo SMOTENC (28.428 righe) è comunque ampiamente sufficiente per le conclusioni statistiche già trattate nel capitolo 45.

MATERIALE PER LA TESI 1. La tabella di sintesi con gravità e costo di correzione per ciascun limite — riusabile quasi direttamente come apertura della sezione “Discussione e limiti”. 2. L’argomentazione sul perché la soglia ottimizzata non influenzi l’AUC — riusabile come precisazione tecnica che qualifica correttamente la portata di quel limite specifico. 3. Il riquadro Attenzione sulla soggettività della propria valutazione di gravità — riusabile come nota di onestà metodologica, utile in sede di discussione con la commissione.

Capitolo 52 — Il caso del reporting automatico non aggiornato

Obiettivi del capitolo - Vedere, messo a fuoco come caso di studio autonomo, il limite più concreto e meglio dimostrabile di tutto il progetto. - Capire perché questo tipo di problema è più pericoloso in un contesto clinico che in molti altri contesti software. - Avere una proposta di correzione precisa, non solo la diagnosi del problema.

Il capitolo 27.2 ha già individuato questo problema leggendo il codice. Questo capitolo lo tratta con tutto il peso critico che merita, come esempio paradigmatico di un tipo di difetto che il solo “il codice funziona senza errori” non intercetta mai.

52.1 Confronto riga per riga dei due report reali

[Fatto] `generatereport.py:192-198` scrive, come stringa Python statica, la sezione “Discussion and Observations”:

- Larger embedding models (E5-large, GTE-large) generally show better performance.
- GTE-large tends to achieve higher ROC-AUC and tighter confidence intervals.
- Confusion matrices enable analysis of false positives and false negatives.
- Bootstrap is useful to verify metric stability and robustness.

[Fatto] Questo identico testo compare, carattere per carattere, sia in `datas/heart_disease/reports/report.r` sia in `datas/diabetes130/reports/report.md` — verificabile aprendo entrambi i file, già presenti nel repository. [Fatto] Confrontando questa affermazione con le tabelle reali dello stesso file: su Heart Disease (Tabella 44.1), l’AUC di gte-large è 0.8540, inferiore a pubmedbert (0.8855), bioclinicalbert (0.8795) ed e5-large (0.8661); su Diabetes130 (Tabella 45.1), è 0.7220, inferiore a sentence-biobert (0.7678), bioclinicalbert (0.7575) e pubmedbert (0.7579). **In nessuno dei due dataset che il report stesso presenta, gte-large ha l’AUC più alta** — l’affermazione “GTE-large tends to achieve higher ROC-AUC” è falsa per entrambi i casi disponibili nel repository.

52.2 Perché è pericoloso in contesto clinico

[Interpretazione] In un contesto software generico, un testo descrittivo sbagliato in un report generato automaticamente sarebbe un difetto fastidioso ma facilmente corretto una volta scoperto. In un contesto di supporto a decisioni cliniche — anche solo di ricerca, come questo progetto — il rischio è di natura diversa: [Interpretazione] un report che presenta un’affermazione narrativa con la stessa autorevolezza tipografica di una tabella numerica calcolata correttamente non dà a chi legge alcun segnale che quella specifica riga non sia stata verificata contro i dati. Chi leggesse solo la sezione “Discussion” (più leggibile di una tabella di 12 colonne) per farsi

un'idea rapida di “quale modello è il migliore”, concluderebbe erroneamente che sia GTE-large — l'opposto di ciò che le tabelle nello stesso documento mostrano.

[Interpretazione] Il problema è aggravato, non attenuato, dal fatto che il resto del report è genuinamente calcolato dai dati (le tabelle, i grafici, i test statistici, capitolo 27.1): un lettore che si fidi giustamente dell'accuratezza della maggior parte del documento non ha motivo di sospettare che una sezione specifica non lo sia. Un report interamente inventato sarebbe, paradossalmente, meno pericoloso: nessuno si fiderebbe di niente. Un report per il 90% corretto e per il 10% sistematicamente sbagliato in un modo indistinguibile a occhio è il caso più insidioso.

ATTENZIONE — questo non è un'affermazione sul progetto nel suo complesso: la pipeline di calcolo (preprocessing, embedding, classificazione, bootstrap, test statistici) è, per quanto verificato in questo libro, corretta nella sua implementazione. Il problema riguarda specificamente e soltanto l'ultimo miglio — la sintesi in linguaggio naturale di quei calcoli, in `generatereport.py:192-225`.

52.3 Come si preverrebbe

[Interpretazione] Tre livelli di prevenzione, dal più semplice al più robusto:

1. **Generare il testo dai dati**, come già proposto al capitolo 27.3: sostituire l'affermazione statica con un'espressione calcolata (`summary.loc[summary["auc_mean"].idxmax(), "model"]`) elimina il problema alla radice per questa specifica affermazione.
2. **Un test di coerenza automatico**: una funzione che, dopo aver generato il report, verifichi che ogni nome di modello menzionato nel testo narrativo compaia effettivamente al posto giusto nella classifica delle tabelle — un test più laborioso da scrivere, ma capace di intercettare anche errori futuri simili, non solo questo specifico.
3. **Etichettare esplicitamente il testo generato a mano**: se generare dinamicamente ogni affermazione fosse troppo costoso, marcare chiaramente le sezioni scritte a priori (per esempio, “Osservazioni generali, non specifiche di questa esecuzione”) eviterebbe che un lettore le scambi per una sintesi dei dati appena mostrati.

PROVA TU — scegli uno dei tre livelli di prevenzione e implementalo per intero su `generatereport.py`. Il primo (rigenerare il testo dai dati) è il più concreto da realizzare con le competenze già acquisite in questo libro (Parte VII, Parte V capitolo 27): è un esercizio realistico, non solo teorico, per una tesi che voglia includere un contributo di miglioramento verificabile.

Riepilogo

Le sezioni narrative del report generato automaticamente sono testo statico, non calcolato dai dati — verificato confrontando i due report reali già presenti nel repository, che condividono lo stesso testo nonostante tabelle numeriche diverse, e che contiene un'affermazione (“GTE-large ha l'AUC più alta”) falsa in entrambi i casi disponibili. Questo tipo di difetto è particolarmente insidioso in un contesto clinico perché si nasconde dietro un documento altrimenti corretto, con la stessa autorevolezza tipografica delle parti genuinamente calcolate. Tre livelli di prevenzione, dal generare il testo dai dati fino a etichettare esplicitamente le parti scritte a priori, correggerebbero il problema con sforzo crescente ma robustezza crescente.

Domande di autoverifica

1. Come si dimostra, senza fidarsi di un'affermazione altrui, che il testo di “Discussion and Observations” non dipende dai dati? Confrontando carattere per carattere

le sezioni corrispondenti nei due report reali di Heart Disease e Diabetes130: sono identiche, nonostante le tabelle numeriche sopra siano sostanzialmente diverse fra i due dataset.

2. Perché un report per il 90% corretto e per il 10% sistematicamente sbagliato è, in un certo senso, più pericoloso di un report interamente inaffidabile? Perché la correttezza della maggior parte del documento induce una fiducia generalizzata che si estende, senza motivo, anche alla parte sbagliata — un lettore non ha alcun segnale visibile per distinguere le due parti, mentre un documento interamente inaffidabile non ispirerebbe fiducia in nessuna sua parte.

3. Qual è il livello di prevenzione più facile da implementare con le sole competenze già presenti in questo libro? Sostituire l'affermazione statica con un'espressione pandas calcolata a runtime (per esempio `summary.loc[summary["auc_mean"].idxmax(), "model"]`), già proposta al capitolo 27.3 — richiede solo la conoscenza di pandas già trattata nella Parte V, non un nuovo strumento di test o convenzione documentale.

MATERIALE PER LA TESI 1. Il confronto riga per riga dei due report, con l'affermazione falsificata evidenziata — probabilmente il singolo argomento più forte e più facilmente presentabile alla commissione di tutto il libro: riusabile come caso di studio autonomo in “Discussione e limiti”. 2. L'analisi del perché questo tipo di difetto sia più pericoloso in contesto clinico che altrove — riusabile come argomento generale sulla fiducia nei sistemi di reporting automatico, applicabile oltre questo progetto specifico. 3. I tre livelli di prevenzione proposti — riusabile come base per un contributo di miglioramento realmente implementato e discusso in tesi.

Capitolo 53 — Posizionamento rispetto allo stato dell'arte

Obiettivi del capitolo - Collocare con precisione questo progetto nel panorama dei metodi possibili per lo stesso tipo di problema. - Sapere esattamente cosa questo progetto NON dimostra, per evitare di sovra-generalizzare i suoi risultati in tesi. - Applicare la tassonomia classica delle minacce alla validità a questo caso specifico, punto per punto.

53.1 Tabellare→testo→embedding vs. classificazione tabellare diretta, “linear probing” su embedding congelati

[Fatto] Il capitolo 3.3 ha già collocato questo progetto rispetto a tre famiglie di approccio: classificazione tabellare diretta, reti neurali per dati tabellari, testo clinico autentico. Il capitolo 32.3 ha introdotto il termine tecnico per la scelta specifica del progetto — **linear probing**: un embedding pre-addestrato e mai riaddestrato, con solo un classificatore lineare stimato sopra. [Livello: teoria consolidata del settore] Questo pattern è diffuso in letteratura sulla rappresentazione come metodo per *valutare* la qualità di un embedding già pronto — non necessariamente come la scelta di massima prestazione assoluta, ma come un modo controllato di isolare “quanto segnale utile è già presente nella rappresentazione”, separato dalla capacità di un classificatore complesso di compensare una rappresentazione mediocre con più parametri.

[Interpretazione] In questi termini, il progetto risponde a una domanda più ristretta e più precisa di quanto “questo sistema prevede la malattia cardiaca” suggerisca: non “qual è il miglior sistema possibile per diagnosticare la malattia coronarica da dati tabellari”, ma “quanto segnale utile per questo compito è già catturato da un embedding testuale pre-addestrato, generalista o biomedico, senza alcun adattamento successivo al dominio specifico”. È una domanda scientificamente più modesta e più onesta — e più difendibile in sede di discussione, se presentata come tale.

53.2 Cosa NON prova questo progetto

[Interpretazione] Alla luce di tutto il libro, un elenco esplicito di ciò che i risultati **non** dimostrano, utile da avere a portata di mano prima di una discussione:

- **Non dimostra che l'approccio tabellare→testo→embedding sia migliore della classificazione tabellare diretta.** Nessun classificatore tabellare diretto è mai stato addestrato in questo progetto (capitolo 1.3, capitolo 47) — solo un baseline banale, non un vero competitor come una regressione logistica diretta sulle 14/19 feature originali o un gradient boosting.

- **Non dimostra che questi risultati generalizzino a testo clinico autentico.** Il testo usato è sintetico, generato da un template fisso (capitolo 3.3, capitolo 22.1), non prosa scritta da un medico.
- **Non dimostra le prestazioni sulla popolazione reale e sbilanciata di origine.** Ogni numero riportato nei capitoli 44-45 riguarda il pool riequilibrato da SMOTENC, non la distribuzione reale (capitolo 47.3, capitolo 51.1).
- **Non dimostra che la superiorità biomedica sia dovuta al dominio del pre-addestramento, e non ad altre differenze fra i modelli** (dimensione, architettura, obiettivo di addestramento) confuse con l'appartenenza a una famiglia — il progetto confronta modelli che differiscono per più di una sola variabile alla volta, un limite di validità interna trattato nel prossimo paragrafo.
- **Non dimostra la robustezza fuori dai due dataset usati.** Due dataset, entrambi statunitensi/europei, entrambi storici (capitolo 29.3, capitolo 30.3): non c'è alcuna base per generalizzare oltre questi due contesti specifici.

53.3 Minacce alla validità: interna, esterna, di costruito, statistica

[**Livello: teoria consolidata del settore**] Una tassonomia diffusa nella ricerca empirica distingue quattro tipi di minaccia alla validità di uno studio. Applicata punto per punto a questo progetto:

Validità interna (il confronto misura davvero ciò che dice di misurare, senza spiegazioni alternative?): [**Interpretazione**] il confronto fra famiglie di modelli (capitolo 6.3, capitolo 44-45) confonde il dominio del pre-addestramento con altre variabili — dimensione del modello, architettura esatta, obiettivo di addestramento specifico — che differiscono simultaneamente fra i sette modelli. Non è possibile, con il disegno sperimentale di questo progetto, isolare l'effetto del solo dominio biomedico da questi altri fattori.

Validità esterna (i risultati generalizzano oltre questo studio specifico?): [**Fatto**] limitata a due dataset, entrambi storici e di provenienza occidentale (capitolo 29.3, capitolo 30.3), con un campionamento parziale per Diabetes130 (capitolo 30.1). Non generalizzabile ad altri domini clinici, altre popolazioni, o testo clinico autentico invece che sintetico.

Validità di costruito (le metriche misurano davvero il costruito di interesse — “supporto efficace alla decisione clinica”?): [**Interpretazione**] accuratezza, F1 e AUC (capitolo 34) misurano la capacità discriminante statistica, non l'utilità clinica reale, che dipenderebbe dal costo relativo di falsi positivi e falsi negativi (capitolo 1.2) mai incorporato esplicitamente in nessuna fase della pipeline.

Validità statistica (le conclusioni statistiche sono supportate correttamente dai dati?): [**Fatto**] parzialmente minacciata dall'assenza di correzione per confronti multipli (capitolo 39.2, 21 confronti per metrica) e dalla soglia ottimizzata sullo stesso fold di validazione (capitolo 35.3, 51.2) — entrambi limiti già quantificati in questo libro, non solo ipotizzati.

ATTENZIONE — applicare questa tassonomia non è un esercizio distruttivo: ogni studio empirico, in qualunque campo, ha minacce alla validità di qualche tipo. Il punto di una buona sezione critica in tesi non è “dimostrare che il progetto è invalido” — è mostrare che *sai riconoscere* dove sono i limiti, con la stessa precisione con cui hai presentato i risultati positivi.

Riepilogo

Il progetto si colloca nella tradizione del linear probing su embedding congelati, una domanda di ricerca più modesta e più difendibile di “il miglior sistema possibile per questo compito clinico”. Non dimostra la superiorità sulla classificazione tabellare diretta, la generalizzazione a testo clinico autentico, le prestazioni sulla popolazione reale sbilanciata, l’attribuzione causale della superiorità biomedica al solo dominio di pre-addestramento, o la robustezza oltre i due dataset usati. Applicando la tassonomia delle quattro validità, ogni tipo — interna, esterna, di costruito, statistica — presenta almeno una minaccia concreta e già documentata in questo libro.

Domande di autoverifica

1. Perché descrivere questo progetto come “linear probing” invece di “un classificatore di malattia cardiaca” è più preciso e più difendibile in sede di discussione?

Perché la domanda di ricerca effettivamente testata è più ristretta: quanto segnale utile è già presente in un embedding pre-addestrato e mai adattato al dominio specifico, non quale sia il miglior sistema diagnostico possibile in assoluto — una domanda che richiederebbe confrontarsi anche con approcci non testati qui, come la classificazione tabellare diretta.

2. Perché il confronto fra famiglie di modelli ha una minaccia alla validità interna, anche assumendo che tutti i calcoli siano corretti? Perché i modelli confrontati differiscono per più di una variabile insieme al dominio di pre-addestramento (dimensione, architettura, obiettivo di addestramento specifico): un risultato migliore per la famiglia biomedica non può essere attribuito con certezza al solo dominio, dato che altre differenze fra i modelli potrebbero contribuire.

3. In quale delle quattro categorie di validità rientra il problema dell’assenza di correzione per confronti multipli? Validità statistica: riguarda se le conclusioni statistiche (quali differenze sono “significative”) sono supportate correttamente dai dati, dato il numero di test eseguiti simultaneamente — un problema distinto dalla validità interna, esterna o di costruito.

MATERIALE PER LA TESI 1. La riformulazione della domanda di ricerca in termini di “linear probing”, con il rimando alla letteratura sulla valutazione delle rappresentazioni — riusabile nell’introduzione o nello stato dell’arte della tesi per calibrare correttamente l’ambizione delle conclusioni. 2. L’elenco esplicito di cosa il progetto NON dimostra — riusabile quasi integralmente come premessa alla sezione “Conclusioni”, per prevenire sovra-generalizzazioni in sede di discussione. 3. L’applicazione punto per punto delle quattro minacce alla validità — riusabile come struttura portante dell’intera sezione “Discussione e limiti” della tesi.

Capitolo 54 — Il chatbot clinico: cosa esiste già, non integrato

Obiettivi del capitolo - Sapere con precisione cosa fa il chatbot già scritto su un branch separato, avendone letto il codice per intero. - Capire come chiude il ciclo addestramento→validazione→inferenza rimasto aperto sul branch `master` (capitolo 2.2). - Riconoscere i rischi specifici di un sistema conversazionale che stima un rischio clinico senza una validazione dedicata a questo scenario d'uso.

[**Fatto, branch chatbot**] Il branch `chatbot` (mai unito a `master`, verificato con `git merge-base --is-ancestor`, capitolo 0) aggiunge tre file: `app_streamlit.py` (62 righe), `bot_telegram.py` (92 righe), `chatbot_core.py` (307 righe, letto per intero in questa sessione). Sono l'unico punto dell'intero progetto in cui il ciclo addestramento→validazione→inferenza (capitolo 2.2) si chiude con un'inferenza reale su un caso nuovo.

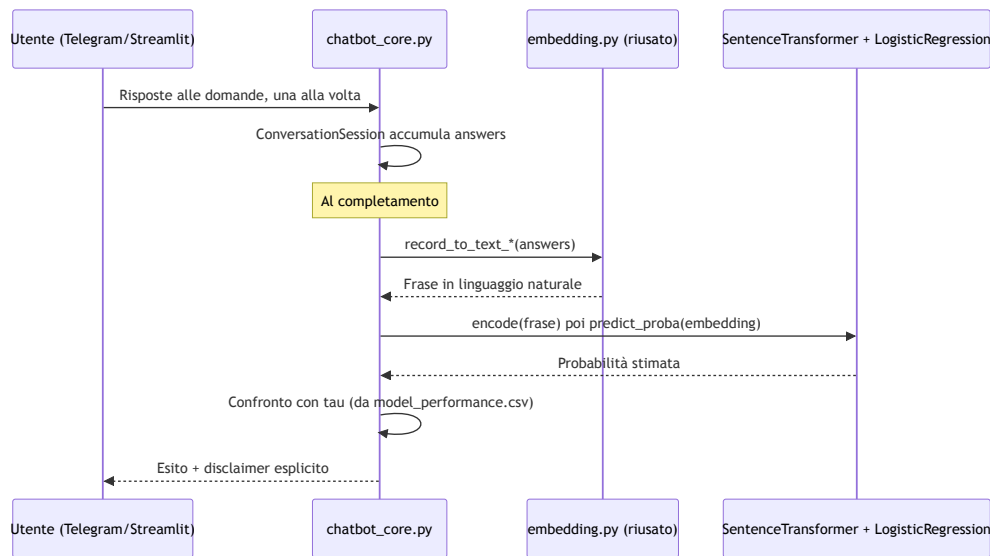
54.1 Architettura di `chatbot_core.py`, `app_streamlit.py`, `bot_telegram.py`

[**Fatto, branch chatbot**] `chatbot_core.py` definisce, per ciascuno dei due dataset, un elenco fisso di domande (`QUESTIONS_HEART_DISEASE`, `QUESTIONS_DIABETES130`) — una per feature, ciascuna con un prompt in italiano, un parser che valida l'input (per esempio `_parse_float_range(t, 60, 260)` per la pressione arteriosa, capitolo 3), e un messaggio di errore se il parser fallisce. Una `ConversationSession` accumula le risposte in un dizionario, un campo alla volta, fino a completare l'elenco.

[**Fatto, branch chatbot**] Il punto più significativo, dal punto di vista architetturale: una volta raccolte tutte le risposte, `_predict()` le passa **direttamente** a `record_to_text_heart_disease()` o `record_to_text_diabetes130()` — le stesse identiche funzioni di `embedding.py` già lette per intero al capitolo 22.1, riusate senza modifiche. Il testo generato viene poi codificato con `SentenceTransformer(EMBEDDING_MODEL["name"]).encode(...)`, e la probabilità stimata con un `LogisticRegression` confrontato con una soglia — la stessa architettura concettuale della pipeline offline (Formula 32.1, capitolo 35), applicata qui a un singolo caso nuovo invece che a un batch di validazione.

[**Fatto, branch chatbot**] Un dettaglio rilevante e non ovvio: `_load_deployed_bundle()` **non riusa** nessuno dei classificatori già addestrati durante la 5-fold cross-validation di `classification.py` (capitolo 23) — ne addestra uno **nuovo**, sull'intero insieme di embedding disponibile (`classifier.fit(X, y)` su tutte le righe, non solo su 4 fold su 5), e riusa solo la soglia τ già calcolata (la media dei 5 fold, letta da `model_performance.csv`). [**Interpretazione**] È una scelta ragionevole in sé (allenare il modello finale su tutti i dati disponibili è una pratica comune, capitolo 33.1), ma introduce un disallineamento sottile: la soglia τ usata qui è stata calibrata sui classificatori della validazione incrociata, non su questo classificatore specifico allenato sull'intero dataset — i due modelli non sono identici, anche se addestrati con lo stesso algoritmo sugli stessi dati sovrapposti in gran parte.

[**Fatto, branch chatbot**] `EMBEDDING_MODEL` (un singolo dizionario, non una lista di sette) è importato da `function` — ma [**Da verificare**] la sua definizione esatta non è nella versione di `function.py` su `master` (verificato, capitolo 0): deve esistere solo nella copia divergente di `function.py` sul branch `chatbot`, non letta in questa sessione. [**Interpretazione**] Dato che `_ensure_embedder()` istanzia `SentenceTransformer(EMBEDDING_MODEL["name"])` — la classe usata solo per i modelli biomedici in `embedding.py` (capitolo 22.3), mai per quelli via Ollama — è ragionevole dedurre che `EMBEDDING_MODEL` designi uno dei tre modelli biomedici, non uno dei quattro generalisti; quale dei tre, resta un'ipotesi non verificata.



Figura

54.1 — Flusso di inferenza del chatbot, con il punto esatto di riuso del codice della pipeline offline.

54.2 Cosa manca per l'integrazione pulita

[**Interpretazione**] Portare questo lavoro su `master` in modo pulito richiederebbe, come minimo: riconciliare la copia divergente di `function.py` (capire cosa contiene `EMBEDDING_MODEL` e se la sua scelta va resa esplicita o parametrizzabile, invece di fissata); decidere se il classificatore per l'inferenza debba davvero essere riaddestrato sull'intero dataset ogni volta che il processo del chatbot si avvia (`_classifier_cache` lo mette in cache solo dentro un singolo processo, non lo persiste su disco — ogni riavvio del bot riaddestra da zero); e, soprattutto, validare esplicitamente il comportamento del sistema in questo scenario d'uso specifico, non solo ereditare la validazione della pipeline offline (capitolo 54.3).

54.3 Rischi di un chatbot di rischio clinico non validato

[**Fatto, branch chatbot**] Il codice include già due disclaimer distinti: nel messaggio di benvenuto (“Non sono uno strumento diagnostico: per qualsiasi dubbio consulta un medico”) e dopo ogni previsione (“Questo è solo un supporto informativo basato su un modello statistico, non una diagnosi medica. Consulta sempre un professionista sanitario”) — una buona pratica di ingegneria responsabile, presente fin dalla prima versione del codice, non aggiunta come ripensamento.

[**Interpretazione**] Restano comunque rischi specifici di questo scenario d'uso, distinti da quelli già discussi per la pipeline offline (Parte XI): primo, l'input dell'utente in un chatbot conversazionale non è mai mancante per costruzione (ogni domanda richiede una risposta valida, capitolo 54.1) — un contrasto netto con il training set, dove il 66%/53% dei valori di `ca/thal` sono

imputati (capitolo 29.3, capitolo 46.2); il classificatore ha quindi imparato in gran parte da un profilo “generico imputato” per queste due feature, e potrebbe comportarsi in modo diverso di fronte a un valore realmente osservato e specifico, fornito da un utente che lo conosce davvero. Secondo, un utente non esperto potrebbe rispondere in modo inaccurato a domande cliniche tecniche (per esempio, “depressione del tratto ST indotta dall’esercizio” — capitolo 54.1, un valore che tipicamente richiede un elettrocardiogramma da sforzo, non qualcosa che un paziente conosce a memoria) — un rischio di qualità dell’input specifico dell’interazione diretta con una persona non clinica, che il disegno della pipeline offline (dati già raccolti da professionisti) non doveva affrontare.

ATTENZIONE — nessuno di questi due rischi invalida l’utilità potenziale di un prototipo come questo per scopi educativi o di ricerca — ma renderlo uno strumento realmente usato da pazienti richiederebbe una validazione specifica per questo scenario, distinta dalla validazione (già limitata, Parte XI) della pipeline offline da cui eredita il modello.

Riepilogo

Il chatbot, esistente solo su un branch non unito, chiude il ciclo di inferenza rimasto aperto sul resto del progetto, riuscendo elegantemente le funzioni di conversione testo già lette al capitolo 22. Addestra però un classificatore nuovo sull’intero dataset, riuscendo una soglia calibrata su classificatori diversi (quelli della cross-validation), e affronta un’asimmetria non banale fra un training set con valori spesso imputati e un’inferenza con input sempre genuini. I disclaimer già presenti nel codice sono una buona pratica, ma non sostituiscono una validazione specifica per lo scenario conversazionale.

Domande di autoverifica

1. Il chatbot riusa uno dei sette classificatori già addestrati durante la cross-validation di `classification.py`? No: `_load_deployed_bundle()` addestra un `LogisticRegression` nuovo sull’intero insieme di embedding disponibile, riuscendo solo la soglia τ già calcolata dalla cross-validation offline — non uno dei classificatori specifici di un singolo fold.

2. Perché l’assenza di valori mancanti nell’input del chatbot è un’asimmetria rispetto al training set, non solo un dettaglio positivo? Perché il classificatore è stato addestrato su un training set dove `ca` e `thal` sono imputati nel 66% e 53% dei casi (capitolo 29.3): ha quindi imparato in gran parte da un profilo “generico imputato” per queste due feature, mentre nell’uso conversazionale riceverà sempre un valore realmente fornito dall’utente — un contesto diverso da quello dominante nei dati di addestramento.

3. Quali due disclaimer sono già presenti nel codice del chatbot, prima ancora di qualunque discussione su validazione aggiuntiva? Il messaggio di benvenuto (“Non sono uno strumento diagnostico...”) e il messaggio dopo ogni previsione (“...non una diagnosi medica. Consulta sempre un professionista sanitario”) — entrambi scritti direttamente nel codice sorgente del chatbot, non aggiunti come ripensamento esterno.

MATERIALE PER LA TESI 1. Il diagramma di sequenza del flusso di inferenza (Figura 54.1), con il punto esatto di riuso del codice della pipeline offline — riusabile in “Lavori futuri” per descrivere un’estensione già prototipata. 2. L’analisi del disallineamento fra il classificatore riaddestrato e la soglia ereditata dalla cross-validation — riusabile come punto di discussione tecnica specifico, distinto dai limiti

già discussi per la pipeline offline. 3. L'asimmetria fra dati di addestramento imputati e input conversazionali sempre genuini — riusabile come argomento originale sui rischi specifici del passaggio da un sistema di validazione offline a uno di inferenza interattiva.

Capitolo 55 — Direzioni di sviluppo difendibili in sede di discussione

Obiettivi del capitolo - Avere tre proposte di estensione concrete, ciascuna motivata da un limite già documentato in questo libro, non generiche. - Sapere, per ciascuna, quale sforzo di implementazione richiederebbe e cosa dimostrerebbe se realizzata. - Poter rispondere con sicurezza se un membro della commissione chiede “cosa faresti dopo?”.

Ogni proposta di questo capitolo risponde a un limite specifico già identificato nel libro, con un rimando esatto — non è un elenco generico di “possibili miglioramenti futuri”.

55.1 Aggiungere un test set finale indipendente

[Fatto] Risponde direttamente al limite più grave individuato al capitolo 51.1: `X_test/y_test`, già calcolati da `preprocessing.py:43` e mai usati (capitolo 21.1). **Implementazione concreta:** modificare `preprocessing_data()` per restituire anche `X_test/y_test` (imputati con le statistiche calcolate sul training set, mai su quelle del test — un’attenzione necessaria per non introdurre la contaminazione già evitata altrove, capitolo 33.3); aggiungere una fase 8 (capitolo 28.3 ha già mostrato come) che generi embedding per il test set con gli stessi sette modelli, applichi i classificatori già addestrati (senza riaddestrarli), e riporti le metriche finali su questo insieme mai toccato durante lo sviluppo.

[Interpretazione] Cosa dimostrerebbe: se le metriche sul test set indipendente fossero vicine a quelle di cross-validation già riportate (capitoli 44-45), rafforzerebbe sostanzialmente la fiducia nei risultati esistenti. Se fossero sensibilmente più basse, confermerebbe che la validazione incrociata aveva un margine di ottimismo non trascurabile — un risultato negativo altrettanto interessante, e più onesto di non verificarlo affatto.

55.2 Un classificatore non lineare come confronto

[Fatto] Risponde al suggerimento già presente, ma mai realizzato, nel testo statico di `generatereport.py:221` (“Introduce a non-linear classifier”) e al posizionamento metodologico del capitolo 53.1 (linear probing come scelta specifica, non necessariamente ottimale in assoluto). **Implementazione concreta:** sostituire, in una copia di `classification.py`, `LogisticRegression` con un `GradientBoostingClassifier` o `RandomForestClassifier` di `scikit-learn` (nessuna nuova dipendenza da aggiungere a `requirements.txt`, già presente nella stessa libreria) — a parità di ogni altro passaggio della pipeline (stessi embedding, stessa `StratifiedKfold`, stesso protocollo di soglia).

[Interpretazione] Cosa dimostrerebbe: se un classificatore non lineare non migliorasse sostanzialmente le prestazioni rispetto alla regressione logistica, sarebbe una prova indiretta che il segnale utile negli embedding è già ben rappresentabile linearmente — coerente con l’idea di

linear probing del capitolo 53.1, e un argomento a favore della scelta del progetto originale. Se lo migliorasse sostanzialmente, indicherebbe che il progetto ha lasciato del segnale non sfruttato sul tavolo, particolarmente rilevante per Heart Disease dove il capitolo 32.3 ha già mostrato un regime $p > n$ sfavorevole a un modello semplice quanto a uno complesso, in modi diversi.

ATTENZIONE — questo confronto avrebbe senso solo se accompagnato dallo stesso rigore statistico già applicato in questo libro (bootstrap, test di significatività, capitolo 36-37): una singola cifra di accuratezza più alta per il classificatore non lineare, senza un intervallo di confidenza o un test di significatività, ripeterebbe esattamente l'errore di lettura “solo descrittiva” già criticato ai capitoli 44.3-45.2.

55.3 Calibrazione delle probabilità e reportistica generata dai dati

[Fatto] Risponde a due limiti distinti trattati in dettaglio altrove: la soglia F1-ottima ottimisticamente calibrata (capitolo 35.3, capitolo 51.2), e il testo statico del report (capitolo 52). **Implementazione concreta, parte 1 (calibrazione):** `sklearn.calibration.CalibratedClassifierCV` avvolgerebbe il classificatore esistente per produrre probabilità meglio calibrate (nel senso tecnico: una probabilità stimata dello 0.7 dovrebbe corrispondere, su molti casi, a una frequenza osservata di positivi vicina al 70%) — una proprietà distinta dall'accuratezza o dall'AUC, mai verificata in questo progetto. **Implementazione concreta, parte 2 (report):** la correzione minima già proposta al capitolo 27.3 e al capitolo 52.3, con l'aggiunta di un test di coerenza automatico che verifichi che ogni nome di modello citato nel testo narrativo corrisponda davvero alla sua posizione in classifica.

[Interpretazione] Cosa dimostrerebbe: la calibrazione risponde a una domanda diversa da quella già misurata (discriminazione fra classi) — se le probabilità stimate sono utilizzabili direttamente come stime di rischio (rilevante per un contesto clinico, capitolo 1.2), non solo come un punteggio da sogliare. La correzione del report eliminerebbe, con lo sforzo minimo possibile, il singolo difetto più concreto e meglio documentato di tutto il progetto (capitolo 52).

Riepilogo

Tre direzioni di sviluppo, ciascuna ancorata a un limite specifico già documentato: un test set finale indipendente per validare la stima di generalizzazione oltre la cross-validation; un classificatore non lineare come termine di paragone per la scelta di linear probing; calibrazione delle probabilità e un report generato dai dati, invece che scritto a priori. Nessuna richiede nuove dipendenze esterne al progetto; tutte sono realizzabili con le competenze già trattate in questo libro.

Domande di autoverifica

1. Perché aggiungere un test set finale indipendente richiederebbe imputare il test set con le statistiche del training set, non le proprie? Perché altrimenti si introdurrebbe esattamente il tipo di contaminazione fra training e test già evitato altrove nel progetto (capitolo 33.3): usare informazione del test set per calcolare i parametri di imputazione violerebbe l'indipendenza che rende il test set utile in primo luogo.

2. Se un classificatore non lineare non migliorasse sostanzialmente le prestazioni rispetto alla regressione logistica, cosa suggerirebbe questo risultato? Che il segnale utile presente negli embedding è già ben rappresentabile con un confine di decisione lineare — un argomento indiretto a favore della scelta di linear probing fatta dal progetto originale, non solo un risultato “negativo”.

3. Perché la calibrazione delle probabilità misura qualcosa di diverso dall'accuratezza o dall'AUC già riportate nei capitoli 44-45? Perché riguarda se il valore numerico della probabilità stimata corrisponde davvero a una frequenza osservata coerente (una probabilità dello 0.7 dovrebbe verificarsi circa il 70% delle volte), una proprietà indipendente dalla capacità del modello di ordinare correttamente i casi (AUC) o di classificarli correttamente a una soglia data (accuratezza, F1).

MATERIALE PER LA TESI 1. Le tre proposte con implementazione concreta e collegamento esplicito al limite che risolvono — riusabili integralmente come sezione “Lavori futuri” della tesi, già argomentate e non generiche. 2. L'osservazione sul rigore statistico necessario per un confronto onesto con un classificatore non lineare — riusabile come promemoria metodologico per chi realizzasse effettivamente questo confronto. 3. La distinzione fra calibrazione e discriminazione come proprietà distinte di un classificatore — riusabile come precisazione tecnica utile in una sezione che discuta l'utilità clinica reale del sistema.

Capitolo 56 — Mappa capitolo per capitolo verso le sezioni tipiche di una tesi triennale

Obiettivi del capitolo - Avere, sezione per sezione di una tesi tipica, l'elenco esatto dei capitoli di questo libro da cui attingere. - Non dover rileggere l'intero libro per trovare il materiale pertinente a una singola sezione della tesi. - Capire quali capitoli servono a più di una sezione, e perché.

Questo capitolo prende sul serio l'anteprima già data al capitolo 0.3.2, ed entra nel dettaglio capitolo per capitolo.

56.1 Introduzione e motivazione

Attingi da: capitolo 1 (il problema clinico reale, i due task), capitolo 6.3 (le due domande di ricerca, con il rimando esatto al codice), capitolo 53.1 (la riformulazione della domanda in termini di linear probing — utile per non sovrastimare l'ambizione dell'introduzione).

56.2 Stato dell'arte

Attingi da: capitolo 3.3 (le famiglie di approccio alternative), capitolo 5 (embedding, modelli generalisti e biomedici), capitolo 53 (posizionamento e minacce alla validità), Appendice D (bibliografia annotata — **verificare ogni citazione prima dell'uso**, capitolo 57.3).

56.3 Materiali e metodi

La sezione con più materiale disponibile. Attingi da: Parte IV (architettura, capitoli 17-19), Parte V (i nove moduli, capitoli 20-28, con interfaccia pubblica ed effetti collaterali già documentati per ciascuno), Parte VI (i due dataset, capitoli 29-31, con i numeri reali di qualità dei dati), Parte VII (le formule del modello, capitoli 32-39, con ogni simbolo tradotto e il rimando al codice), capitolo 43 (protocollo sperimentale, con la dichiarazione esplicita di provenienza di ogni numero).

ATTENZIONE — per questa sezione in particolare, non limitarti a riassumere i capitoli: la tesi deve descrivere solo ciò che è rilevante per gli obiettivi specifici della tesi stessa, non ripetere l'intero libro. Usa questo capitolo per navigare, non come sostituto della selezione critica che solo tu puoi fare conoscendo lo scopo esatto della tua tesi.

56.4 Risultati

Attingi da: Parte VIII (i tre casi d’uso, capitoli 40-42, con le tabelle reali già numerate), Parte IX per intero (capitoli 43-47: tabelle con intervalli di confidenza, confronto per famiglia, test statistici, analisi degli errori con la scoperta sui casi sintetici, baseline banale). Le Tabelle 44.1 e 45.1 sono probabilmente le due tabelle singole più riutilizzabili di tutto il libro per questa sezione.

56.5 Discussione e limiti

Attingi da: Parte XI per intero (capitoli 51-53: sintesi dei limiti metodologici con gravità e correggibilità valutate, il caso del report statico come esempio paradigmatico, posizionamento e minacce alla validità), più i singoli punti critici sparsi nei capitoli precedenti (capitolo 4.2 e 29.3 sulla discrepanza 297/920, capitolo 32.3 sul rapporto parametri/campioni, capitolo 46.2 sui casi sintetici).

56.6 Conclusioni e lavori futuri

Attingi da: Parte XII per intero (capitolo 54, il chatbot come estensione già prototipata; capitolo 55, le tre direzioni di sviluppo motivate), capitolo 57.2 (misure ancora da produrre, se la tesi include un contributo sperimentale originale).

Tabella di sintesi

Sezione di tesi	Parti/capitoli principali	Capitoli di supporto trasversale
Introduzione e motivazione	1, 6.3	53.1
Stato dell’arte	3.3, 5, 53	Appendice D
Materiali e metodi	IV, V, VI, VII, 43	2 (fondamenti ML), 33 (validazione)
Risultati	VIII, IX	—
Discussione e limiti	XI	4.2, 29.3, 32.3, 46.2
Conclusioni e lavori futuri	XII	57.2

Riepilogo

Ogni sezione tipica di una tesi triennale ha un insieme preciso di capitoli di riferimento in questo libro, con Materiali e metodi che attinge dal maggior numero di parti (IV-VII) e Discussione e limiti che raccoglie sia la sintesi organizzata della Parte XI sia i punti critici sparsi lungo tutto il libro. Nessuna sezione della tesi dovrebbe richiedere di rileggere l’intero manuale da capo.

Domande di autoverifica

1. Quale parte del libro fornisce il maggior volume di materiale per la sezione “Materiali e metodi”? Le Parti IV-VII insieme (architettura, codice modulo per modulo, dati, matematica del modello), più il capitolo 43 per la dichiarazione di provenienza dei dati.

2. Perché la sezione “Discussione e limiti” attinge sia dalla Parte XI sia da singoli capitoli sparsi altrove nel libro? Perché la Parte XI raccoglie e pesa i limiti principali in modo organizzato, ma alcune scoperte critiche specifiche (la discrepanza 297/920, il rapporto

parametri/campioni, i casi sintetici negli errori) sono nate e sono documentate nei capitoli dove il tema di dominio le rendeva pertinenti, non tutte ripetute per esteso nella Parte XI.

3. Perché il capitolo consiglia di non limitarsi a riassumere i capitoli di “Materiali e metodi” nella tesi? Perché la tesi deve riflettere una selezione critica specifica per i suoi obiettivi, non l'intero contenuto di un manuale pensato per l'apprendimento completo: usare questo capitolo come mappa di navigazione, non come sostituto del giudizio editoriale necessario per scrivere la tesi stessa.

MATERIALE PER LA TESI 1. La tabella di sintesi completa — riusabile come indice di lavoro personale durante la stesura della tesi, da tenere aperto capitolo per capitolo. 2. L'elenco dei capitoli di supporto trasversale per “Discussione e limiti” — riusabile per assicurarsi di non dimenticare nessuna delle scoperte critiche sparse nel libro. 3. Il riquadro Attenzione sulla selezione critica — riusabile come promemoria metodologico su come usare questo libro correttamente, non come sostituto del proprio giudizio.

Capitolo 57 — Cosa è già pronto e cosa manca ancora

Obiettivi del capitolo - Avere un inventario preciso di ogni figura, tabella e formula numerata già pronta in questo libro. - Sapere esattamente quali misure richiederebbero un nuovo esperimento, con la procedura per ottenerle. - Avere l'elenco completo dei riferimenti bibliografici ancora da verificare prima di poterli citare in tesi.

57.1 Figure e tabelle riutilizzabili, capitolo per capitolo

[Fatto] Elenco delle figure Mermaid numerate: Figura 2.1 (ciclo addestramento/validazione/inferenza), Figura 3.1 (catena tabellare→testo→embedding→classificazione), Figura 12.1 (thread pool e semaforo Ollama), Figura 17.1 (architettura completa), Figura 18.1 (sequenza di un'esecuzione), Figura 54.1 (sequenza di inferenza del chatbot).

[Fatto] Elenco delle tabelle principali con dati reali: la tabella di corrispondenza Java→Python (capitolo 13.1), lo schema delle 14 feature di Heart Disease (capitolo 29.2), lo schema delle 19 feature di Diabetes130 (capitolo 30.2), la tabella completa degli iperparametri (capitolo 39.1), le Tabelle 44.1 e 45.1 (risultati completi con IC per entrambi i dataset), la Tabella 46.1 (tassi di errore), la tabella dei limiti con gravità e correggibilità (capitolo 51).

[Fatto] Formule numerate: 25.1 (deviazione di feature), 32.1-32.2 (regressione logistica), 33.1 (k-fold), 34.1-34.3 (le tre metriche), 35.1-35.2 (soglia F1-ottima, confronto rigoroso), 36.1-36.4 (bootstrap), 37.1-37.3 (i tre test statistici) — l'elenco completo, con la spiegazione di ogni simbolo, è raccolto in Appendice C.

57.2 Misure ancora da produrre, con procedura

[Fatto] Tre misure, già proposte nei capitoli precedenti, richiederebbero un nuovo esperimento non condotto in questo libro:

1. **Prestazioni sul test set finale indipendente** (capitolo 55.1). Procedura: modificare `preprocessing_data()` per restituire anche `X_test/y_test`; imputare il test set con le statistiche del training set; generare embedding per il test set con i sette modelli; applicare i classificatori già addestrati (senza riaddestrarli) e calcolare le tre metriche.
2. **Confronto con un classificatore non lineare** (capitolo 55.2). Procedura: sostituire `LogisticRegression` con `GradientBoostingClassifier` in una copia di `classification.py`, a parità di ogni altro passaggio; ripetere bootstrap e test statistici sui nuovi risultati; confrontare con le Tabelle 44.1-45.1 esistenti.
3. **Entità precisa dell'ottimismo da soglia** (esercizio del capitolo 35.3). Procedura: per ciascun modello già presente in `datas/`, ricalcolare F1 con $\tau = 0.5$ fisso usando i file `{model}_y_true.npy/{model}_y_score.npy` già disponibili, e confrontare con l'F1 ripor-

tato nelle tabelle esistenti — non richiede nemmeno riaddestrare nulla, solo ricalcolare una metrica su dati già presenti.

PROVA TU — la terza misura è realizzabile in pochi minuti con i dati già nel repository, senza Ollama né Hugging Face. È il modo più rapido di trasformare uno dei limiti metodologici di questo libro (capitolo 35.3, capitolo 51.2) in un numero concreto per la tua tesi.

57.3 Riferimenti bibliografici da recuperare e verificare

[Fatto] Quattro riferimenti sono già verificati con identificativo concreto (DOI), citati direttamente da `docs/DATASET.md` e già usati nei capitoli 29.1 e 30.1: Janosi et al. (1988, Heart Disease, DOI 10.24432/C52P4W), Detrano et al. (1989, American Journal of Cardiology), Strack et al. (2014, Diabetes130, DOI 10.24432/C5230J), Strack et al. (2014, BioMed Research International).

[Fatto] Marcati [DA VERIFICARE] in questo libro, da cercare e confermare prima di citarli in tesi (Appendice D ne riporta lo stato dopo la verifica con WebSearch/WebFetch, capitolo di questo libro): il paper originale di E5 (Wang et al., capitolo 5.2), il paper originale di GTE (Li et al., capitolo 5.2), Hastie/Tibshirani/Friedman su bias-variance (capitolo 2.3), DeLong/DeLong/Clarke-Pearson (1988, capitolo 37.3), McInnes/Healy/Melville su UMAP (capitolo 38.2).

Riepilogo

Questo libro fornisce sei figure Mermaid numerate, oltre dieci tabelle con dati reali, e diciassette formule numerate, tutte pronte per l'uso diretto in tesi. Tre misure aggiuntive richiederebbero un nuovo esperimento, con procedura già delineata per ciascuna — una realizzabile in pochi minuti senza nuovi calcoli pesanti. Cinque riferimenti bibliografici restano da verificare con una fonte esterna prima di poter essere citati con sicurezza.

Domande di autoverifica

1. **Quale delle tre misure ancora da produrre è realizzabile senza generare nuovi embedding o riaddestrare alcun modello?** Il ricalcolo dell'F1 con soglia fissa $\tau = 0.5$ sui dati di predizione già salvati (`y_true.npy/y_score.npy`): richiede solo di ricalcolare una metrica su file già presenti nel repository.
2. **Quanti riferimenti bibliografici sono già verificati con un identificativo concreto in questo libro, e quanti restano da verificare?** Quattro sono già verificati con DOI (i due dataset e i loro paper originali); cinque restano marcati [DA VERIFICARE] e vanno confermati con una ricerca esterna prima di essere citati.
3. **Perché nessuna delle tabelle o figure di questo libro dovrebbe essere incollata nella tesi senza adattamento?** Perché questo libro è materiale di riferimento, non testo da copiare: ogni tabella o figura va integrata nel contesto specifico della tesi, con la propria numerazione, didascalia e discussione — l'elenco di questo capitolo serve a sapere dove trovarle, non a sostituire la scrittura della tesi stessa.

MATERIALE PER LA TESI 1. L'inventario completo di figure, tabelle e formule — riusabile come checklist per assicurarsi di non dimenticare materiale già pronto durante la stesura. 2. Le tre misure ancora da produrre con procedura dettagliata — riusabile direttamente come piano di lavoro per un contributo sperimentale originale

nella tesi. 3. L'elenco dei riferimenti da verificare, con quelli già confermati distinti chiaramente — riusabile come lista di controllo prima della consegna finale della bibliografia.

Capitolo 58 — Venti domande della commissione, con traccia di risposta

Obiettivi del capitolo - Arrivare alla discussione con una risposta già pensata per le domande più prevedibili. - Avere, per ciascuna domanda, il capitolo esatto del libro da cui la risposta completa proviene. - Distinguere le domande a cui puoi rispondere con un fatto verificato da quelle che richiedono un’opinione motivata.

Ogni traccia di risposta è un punto di partenza, non un testo da recitare a memoria: la commissione nota la differenza fra una risposta capita e una imparata.

58.1 Dominio e motivazione (5 domande)

1. “Perché avete convertito dati tabellari in testo invece di classificarli direttamente?” Traccia: per testare se le rappresentazioni di modelli linguistici pre-addestrati catturano segnale clinico da dati strutturati — una domanda di ricerca specifica (capitolo 3.2, capitolo 6.3), non un’affermazione che questo approccio sia superiore alla classificazione diretta, mai testata in questo progetto (capitolo 1.3, capitolo 53.2).

2. “Cosa prevede esattamente il sistema, in termini clinici concreti?” Traccia: due cose diverse per i due dataset — presenza di malattia coronarica (diagnosi) per Heart Disease, riammissione ospedaliera entro 30 giorni (non “riammissione in generale”) per Diabetes130 (capitolo 1.1, capitolo 30.3).

3. “Perché confrontare modelli generalisti e biomedici, e non solo usare il migliore in assoluto?” Traccia: è la seconda domanda di ricerca esplicita del progetto (capitolo 6.3, README.md:60–61) — l’obiettivo è isolare l’effetto del dominio di pre-addestramento, non solo massimizzare una metrica.

4. “Il nome del progetto è ‘Forecast’: fate previsioni di serie temporali?” Traccia: no — nonostante il nome storico, il codice non tratta alcuna dimensione temporale: è classificazione binaria su dati clinici in un singolo momento (capitolo 0.3.3, capitolo 1).

5. “Che tipo di utente finale trarrebbe beneficio da questo sistema?” Traccia: nella forma attuale, nessuno direttamente — la pipeline produce report di ricerca, non un’interfaccia utilizzabile in produzione; solo il prototipo di chatbot su un branch separato (capitolo 54) si avvicina a un’interfaccia utente, e non è stato integrato né validato per questo scopo.

58.2 Metodo e implementazione (8 domande)

6. “Perché regressione logistica e non una rete neurale più complessa?” Traccia: linear probing su un embedding congelato è una scelta metodologica specifica per isolare il segnale già

presente nella rappresentazione (capitolo 32.3, capitolo 53.1); per Heart Disease è anche prudente dato il regime $p > n$ per i modelli a 1024 dimensioni.

7. “Come avete gestito lo sbilanciamento delle classi?” Traccia: SMOTENC sulle feature grezze, prima della codifica, per mantenere i record sintetici convertibili in testo (capitolo 21.2) — con la precisazione che Heart Disease è in realtà quasi bilanciato (55.3%/44.7%), mentre Diabetes130 è marcatamente sbilanciato (11.16%/88.84%, capitolo 4.2).

8. “Come scegliete la soglia di decisione?” Traccia: massimizzando F1 su ciascun fold di validazione (Formula 35.1) — con la precisazione onesta che questo introduce un ottimismo statistico contenuto, distinto e più lieve di un vero data leakage (capitolo 35.3, capitolo 33.3).

9. “Quali metriche avete usato, e perché non altre?” Traccia: accuratezza, F1 macro, ROC-AUC (capitolo 34) — non metriche di regressione (MAE, RMSE), perché il problema è di classificazione, non di previsione di un valore continuo.

10. “Come avete stimato l’incertezza dei risultati?” Traccia: bootstrap non parametrico a 10.000 iterazioni sulle predizioni già ottenute (capitolo 36), con intervalli di confidenza al 95% con il metodo percentile.

11. “Perché tre test statistici diversi invece di uno solo?” Traccia: triangolazione metodologica — Wilcoxon (non parametrico), t-test appaiato (più potente se la normalità regge), DeLong (specifico per l’AUC, tiene conto della correlazione fra modelli sugli stessi casi) — la concordanza fra i tre rafforza la fiducia nelle conclusioni (capitolo 26.3, capitolo 37).

12. “Avete applicato una correzione per confronti multipli?” Traccia: no — con 21 confronti a coppie per metrica, questo è un limite dichiarato esplicitamente (capitolo 39.2), non scoperto dalla commissione: menzionarlo prima che venga chiesto è più forte che doverlo ammettere in risposta.

13. “Come garantite la riproducibilità dei risultati?” Traccia: sei semi casuali fissati a 42 in punti indipendenti del codice (capitolo 39.3) — con la precisazione onesta che non esiste una costante centralizzata, e che la riproducibilità copre le componenti seedate del progetto, non necessariamente ogni sorgente di variabilità esterna (per esempio versioni dei modelli su Ollama).

58.3 Limiti e validità (7 domande)

14. “Qual è, secondo voi, il limite più serio di questo lavoro?” Traccia: l’assenza di un test set finale indipendente (capitolo 51.1) — un limite strutturale, reso concreto da un numero specifico (il baseline sulla popolazione reale di Diabetes130 raggiungerebbe 88.84%, capitolo 47.3), non solo un principio metodologico astratto.

15. “I risultati generalizzano oltre questi due dataset?” Traccia: no, esplicitamente — due dataset storici, di provenienza occidentale, uno campionato parzialmente; nessuna base per generalizzare a testo clinico autentico o ad altre popolazioni (capitolo 53.2).

16. “Come giustificate l’affermazione che i modelli biomedici sono superiori?” Traccia: con cautela differenziata per dataset — su Diabetes130 il test di DeLong conferma una superiorità netta e quasi universale (20 coppie su 21 significative, capitolo 45.2); su Heart Disease il quadro è più sfumato, con i modelli generalisti più grandi statisticamente indistinguibili da alcuni biomedici (capitolo 44.3).

17. “Avete trovato bug o inconsistenze nel codice originale?” Traccia: sì, diverse, verificate e documentate — la discrepanza fra la documentazione (297 record) e il codice reale (920 righe) per Heart Disease (capitolo 29.1), il testo narrativo statico del report che contraddice le

proprie tabelle (capitolo 52), il comando di installazione errato per un modello Ollama (capitolo 15.2).

18. “Come avete verificato la qualità dei dati?” Traccia: scomponendo la mancanza dei dati per centro clinico e per feature, non fermandosi a un controllo aggregato — la scoperta che `ca` e `thal` sono dati reali quasi solo per Cleveland (capitolo 29.3) è un esempio diretto di questo livello di verifica.

19. “Gli ‘hardest cases’ del vostro sistema sono casi clinicamente interessanti?” Traccia: in parte no — verificato che alcuni sono record sintetici (età non intera) e che la maggioranza condivide esattamente i valori di imputazione più comuni per `ca/thal`, non necessariamente un profilo clinico raro (capitolo 46.2).

20. “Che uso responsabile fareste di un sistema come questo in un contesto reale?” Traccia: nessuno diretto senza ulteriore lavoro — servirebbe come minimo un test set indipendente sulla popolazione reale (capitolo 55.1), una calibrazione esplicita delle probabilità (capitolo 55.3), e una validazione specifica per qualunque interfaccia utente (capitolo 54.3) prima di considerare un uso anche solo di supporto informativo.

Riepilogo

Venti domande organizzate in tre aree — dominio e motivazione, metodo e implementazione, limiti e validità — ciascuna con una traccia di risposta ancorata a un capitolo specifico di questo libro. Le domande sui limiti (58.3) sono probabilmente le più probabili in una discussione seria, ed è per questo che il libro le tratta con lo stesso rigore delle domande sui risultati positivi.

Domande di autoverifica

1. Quale domanda di questo capitolo riguarda direttamente la scoperta più forte e meglio verificata di tutto il libro? La domanda 19, sugli “hardest cases”: la risposta si basa sulla verifica diretta (età non intera, valori di imputazione confermati con mediana e moda calcolate) del capitolo 46.2.

2. Perché menzionare da soli, senza aspettare la domanda, l’assenza di correzione per confronti multipli (domanda 12) è una strategia migliore che aspettare di doverlo ammettere? Perché dichiarare un limite prima che venga scoperto mostra padronanza critica del proprio lavoro, mentre doverlo ammettere solo dopo una domanda diretta può apparire come una lacuna nascosta o non notata.

3. La domanda 16 richiede una risposta uniforme per entrambi i dataset, o differenziata? Differenziata: la superiorità dei modelli biomedici è statisticamente netta su Diabetes130 ma più sfumata su Heart Disease, dove alcuni modelli generalisti grandi non sono distinguibili statisticamente da alcuni modelli biomedici — una risposta uniforme sarebbe imprecisa.

MATERIALE PER LA TESI 1. Le venti domande con traccia di risposta — riusabili direttamente come materiale di preparazione alla discussione, capitolo per capitolo. 2. La distinzione fra domande con risposta fattuale diretta e domande che richiedono un giudizio motivato — riusabile per calibrare il proprio livello di sicurezza nella risposta durante la discussione. 3. L’insieme delle sette domande sui limiti (58.3) — riusabile come base per la propria autovalutazione critica prima della consegna finale della tesi, non solo come preparazione a una domanda altrui.

Appendice A — Glossario generale

Solo termini di dominio e di statistica/machine learning. I termini di programmazione generale (variabile, ciclo, eccezione...) non compaiono per costruzione: il libro presuppone che tu li conosca già da Java (capitolo 0).

Accuratezza (accuracy) — Proporzione di previsioni corrette sul totale (Formula 34.1). Ingannevole su classi sbilanciate. → capitolo 34.1

Addestramento (training) — Fase in cui i parametri di un modello vengono adattati a dati etichettati. → capitolo 2.2

Baseline — Modello di riferimento banale usato per stabilire un pavimento minimo di prestazione. → capitolo 47.1

Bootstrap — Tecnica di ricampionamento con reinserimento per stimare l'incertezza di una metrica senza assunzioni distributive. → capitolo 36

Classificazione binaria — Compito di assegnare un'osservazione a una fra due categorie. → capitolo 4.1

Curva ROC — Grafico del tasso di veri positivi contro il tasso di falsi positivi al variare della soglia di decisione. → capitolo 34.3

Data leakage — Fuga di informazione che non dovrebbe essere disponibile al momento della previsione, che gonfia artificialmente le prestazioni misurate. Tassonomia: diretto, contaminazione train/test, selezione di soglie/ipерparametri, temporale. → capitolo 33.3

DeLong, test di — Test statistico per confrontare due AUC correlate, calcolate sugli stessi casi. → capitolo 37.3

Deviazione standard bootstrap — Dispersione della distribuzione empirica bootstrap di una metrica (Formula 36.4). → capitolo 36.2

Duck typing — (Nota: è un idiom Python, non un termine di dominio, ma incluso perché ricorrente) Verificare la presenza di un comportamento (un metodo) invece del tipo dichiarato di un oggetto. → capitolo 7.2

Embedding — Rappresentazione di un testo come vettore numerico di lunghezza fissa, costruita in modo che testi semanticamente simili producano vettori vicini. → capitolo 5.1

F1 (macro) — Media aritmetica dell'F1 calcolato separatamente per ciascuna classe (Formula 34.2). → capitolo 34.2

Falso positivo / falso negativo — Un caso negativo classificato come positivo / un caso positivo classificato come negativo. → capitolo 4.3

Famiglia di modelli — Categoria di un modello di embedding: generalista, biomedico, biomedico per frasi. → capitolo 5.3

Generalizzazione — Capacità di un modello di comportarsi bene su dati mai visti durante l'addestramento. → capitolo 2.3

Imputazione — Sostituzione di un valore mancante con una stima (mediana per variabili numeriche, moda per categoriali in questo progetto). → capitolo 21.2

Inferenza — Applicazione di un modello già addestrato e validato a un input nuovo, di etichetta ignota. → capitolo 2.2

Intervallo di confidenza (percentile) — Intervallo costruito dai percentili di una distribuzione bootstrap (Formula 36.3). → capitolo 36.2

K-fold / k-fold stratificato — Procedura di validazione che suddivide i dati in k parti, usando ciascuna a turno come validazione; la variante stratificata preserva la proporzione di classe in ogni parte. → capitolo 33.2

Linear probing — Addestrare solo un classificatore lineare sopra un embedding pre-addestrato e mai modificato. → capitolo 32.3

Overfitting / underfitting — Sovradattamento (basso errore in addestramento, alto su dati nuovi) / sottoadattamento (alto errore su entrambi). → capitolo 2.3

Pooling (mean pooling) — Combinazione dei vettori di più token in un unico vettore di frase, tipicamente per media. → capitolo 5.2

Regolarizzazione L2 — Penalizzazione sulla norma dei pesi di un modello, che scoraggia valori estremi (Formula 32.2). → capitolo 32.2

ROC-AUC — Area sotto la curva ROC; probabilità che un caso positivo scelto a caso riceva un punteggio più alto di un caso negativo scelto a caso. → capitolo 34.3

SMOTE / SMOTENC — Tecnica di bilanciamento delle classi che genera record sintetici della classe minoritaria; SMOTENC ne è la variante per feature miste numeriche/categoriali. → capitolo 21.2

Soglia di decisione (τ) — Valore che separa un punteggio di probabilità in etichetta positiva o negativa. → capitolo 35.1

Sovrapposizione (overlap) delle famiglie / minacce alla validità — Interna, esterna, di costruito, statistica: quattro categorie classiche di limite di uno studio empirico. → capitolo 53.3

Tokenizzazione — Suddivisione di un testo in unità più piccole (sotto-parole) prima dell'elaborazione da parte di un modello linguistico. → capitolo 5.2

Triangolazione metodologica — Uso di più metodi/test con assunzioni diverse per rafforzare la fiducia in una conclusione quando concordano. → capitolo 26.3

UMAP — Tecnica non supervisionata di riduzione di dimensionalità, usata in questo progetto solo per visualizzazione. → capitolo 38

Validazione — Fase in cui le prestazioni di un modello già addestrato vengono misurate su dati non usati per l'addestramento, per guidare decisioni di sviluppo. → capitolo 2.2, capitolo 33.1

Wilcoxon signed-rank, test di — Test non parametrico per confrontare due campioni accoppiati basato sui ranghi delle differenze. → capitolo 37.1

Appendice B — Riferimento completo delle funzioni pubbliche

Firma, parametri, ritorno ed effetti collaterali per ogni funzione dei nove file di `master`. Le funzioni con prefisso underscore sono interne per convenzione (capitolo 10.3) ma elencate ugualmente per completezza. Nessuna annotazione di tipo compare nel codice originale (capitolo 7.2): i tipi indicati qui sono dedotti dalla lettura del corpo delle funzioni, non dichiarati dal codice.

`main.py`

- `parse_args()` → `Namespace` con attributo `dataset` (`str`). Legge `sys.argv`.
- `main()` → `None`. Effetto: esegue l'intera pipeline per il dataset scelto (capitolo 28).

`function.py`

- `get_output_dirs(dataset: str) → dict[str, str]`. Effetto: crea le cartelle se non esistono. Solleva `ValueError` se `dataset` non è valido.
- `load_heart_disease()` → `pd.DataFrame` (920 righe × 14 colonne).
- `load_diabetes130(sample_size=20000, random_state=42) → pd.DataFrame`.
- `save_figure(fig, path_no_ext: str) → None`. Effetto: salva `.png` e `.pdf`.
- `get_model_palette(model_names: list) → dict[str, tuple]` (colore RGB per modello).
- `delete_files_embeddings/preprocessing/results/graphics(folder: str) → None`. Effetto: cancella file su disco per pattern di sottostringa.
- `plot_data_heatmap(X, num_cols=None, graphics_dir=...) → None`. Effetto: salva grafico.
- `plot_umap(X, y, title, graphics_dir=...) → None`. Effetto: salva grafico.
- `plot_boxplots(results_dict, results_dir=...) → None`. Effetto: salva grafico.
- `plot_roc_comparison(roc_data, filename="ROC_comparison", graphics_dir=...) → None`. Effetto: salva grafico.
- `plot_confusion(y_true, y_pred, name, graphics_dir=...) → None`. Effetto: salva grafico.
- `plot_metric_comparison(df_summary, results_dir=...) → None`. Effetto: salva grafico.
- `plot_mean_ci(df_summary, results_dir=...) → None`. Effetto: salva grafico.
- `plot_family_comparison(bootstrap_results, results_dir=...) → None`. Effetto: salva grafico.
- `plot_error_rates(df_error_summary, results_dir=...) → None`. Effetto: salva grafico.

- `plot_feature_deviation(df_deviation, results_dir=...)` → `None`. Effetto: salva grafico.
- `_configure_plot_style()` → `None`. Eseguita automaticamente all'import del modulo (capitolo 10.2).

preprocessing.py

- `preprocessing_data(dataset="heart_disease")` → (`X_train_bal`: `pd.DataFrame`, `y_train_bal`: `pd.Series`). Effetto: salva `.numpy/.csv`, genera 2 grafici.
- `impute_raw(X, num_cols, cat_cols)` → `pd.DataFrame` (copia, nessun side-effect sull'originale).
- `balance_classes(X, y, cat_cols)` → (`X_res`, `y_res`).
- `build_encoder(X, y, num_cols, cat_cols)` → `ColumnTransformer` addestrato.
- `data_processed(X_train, y_train, preprocessor, num_cols, cat_cols)` → `pd.DataFrame` con colonna `target`.
- `save_data_processed(X_train_emb_df, preprocessing_dir)` → `None`. Effetto: salva 2 file `.numpy`.

embedding.py

- `embeddings(X, y, dataset="heart_disease")` → `None`. Effetto: genera e salva `embedding` per 7 modelli.
- `record_to_text_heart_disease(row)` → `str`.
- `record_to_text_diabetes130(row)` → `str`.
- `_fmt_num/_fmt_cat/_fmt_bool/_fmt_raw(value, ...)` → `str`.
- `save_embeddings_to_numpy(embeddings, filename)` / `save_labels_to_numpy(labels, filename)` → `None`. Effetto: `np.save`.
- `generate_embeddings_batch(model_name, texts, batch_size=16, max_retries=5, retry_delay=2.0, inter_batch_delay=0.3)` → `list`. Solleva `RuntimeError` dopo `max_retries` fallimenti.
- `generate_embeddings_hf(texts, model_name)` → `np.ndarray`. Effetto: scarica un modello se non in cache.
- `process_model(model: dict, texts, labels, embeddings_dir)` → `None`. Effetto: salva `embedding` e etichette. Solleva `RuntimeError` con causa concatenata (`from e`).
- `generate_all_embeddings(texts, labels, embeddings_dir, max_workers=3)` → `None`. Effetto: orchestra 7 chiamate a `process_model` in un pool di thread.

classification.py

- `training_classifier(dataset="heart_disease")` → `None`. Effetto: popola `function.results` (side-effect globale); salva `.numpy/.csv` per 7 modelli; genera 1 grafico.

evaluation.py

- `evaluate_results(dataset="heart_disease")` → `None`. Effetto: salva `bootstrap .numpy`, CSV riassuntivo, 6 grafici.
- `bootstrap_metrics(y_true, y_score, y_pred, n_iter=10000, seed=42)` → `dict[str, np.ndarray]`.
- `ci(a, alpha=0.95)` → (`float`, (`float`, `float`)).

error_analysis.py

- `analyze_errors(dataset="heart_disease")` → `None`. Effetto: salva `error_summary.csv`, `hardest_cases.csv`, `feature_deviation.csv`, 14 CSV di FP/FN, 2 grafici.

statisticaltest.py

- `test_statistical_tests(dataset="heart_disease")` → `None`. Effetto: salva `wilcoxon_comparison.csv`, `ttest_comparison.csv`; chiama `test_delong`.
- `test_delong(dirs: dict)` → `None`. Effetto: salva `delong_comparison.csv`; stampa un avviso se le etichette vere di due modelli non coincidono.

generatereport.py

- `generate_report(dataset="heart_disease")` → `None`. Effetto: salva `report.md`. Solleva `FileNotFoundError` se `encoder_comparison_summary.csv` non esiste.
- `load_summary(summary_path)` → `pd.DataFrame`. Solleva `FileNotFoundError` esplicito.
- `load_statistical_results(wilcoxon_path, ttest_path, delong_path)` → `dict[str, pd.DataFrame | None]`.
- `generate_markdown(summary, dirs)` → `str`. Contiene testo narrativo statico (capitolo 27.2, 52) per le sezioni Discussion/Conclusions/Improvements.

Appendice C — Formulario

Tutte le formule numerate del libro, in ordine, con il rimando al capitolo dove ogni simbolo è spiegato per esteso e alla riga di codice che la implementa.

25.1 — Deviazione di feature (capitolo 25.3, `error_analysis.py:65-75`)

$$d_{\text{feature}} = \frac{\bar{x}_{\text{errore}} - \bar{x}_{\text{corretto}}}{s_{\text{pooled}}}$$

32.1 — Regressione logistica (capitolo 32.1, `classification.py:16,28`)

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

32.2 — Funzione obiettivo regolarizzata (capitolo 32.2, `classification.py:16`, default scikit-learn)

$$\min_{\mathbf{w}, b} \frac{1}{2} \mathbf{w}^\top \mathbf{w} + C \sum_{i=1}^n \log \left(1 + e^{-y_i(\mathbf{w}^\top \mathbf{x}_i + b)} \right)$$

33.1 — K-fold (capitolo 33.2, `classification.py:15`)

$$\text{per } i = 1, \dots, k: \quad \text{addestra su } \bigcup_{j \neq i} D_j, \quad \text{valuta su } D_i$$

34.1 — Accuracy (capitolo 34.1, `classification.py:39`)

$$\text{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN}$$

34.2 — F1 (capitolo 34.2, `classification.py:40`)

$$F1 = 2 \cdot \frac{\text{precisione} \cdot \text{recall}}{\text{precisione} + \text{recall}}, \quad \text{precisione} = \frac{VP}{VP + FP}, \quad \text{recall} = \frac{VP}{VP + FN}$$

34.3 — TPR/FPR per la curva ROC (capitolo 34.3, `function.py:264`)

$$\text{TPR}(\tau) = \frac{VP(\tau)}{VP(\tau) + FN(\tau)}, \quad \text{FPR}(\tau) = \frac{FP(\tau)}{FP(\tau) + VN(\tau)}$$

35.1 — Soglia F1-ottima (capitolo 35.2, `classification.py:30-35`)

$$\tau^* = \arg \max_{\tau \in T} F1(y_{\text{val}}, \mathbb{1}[y_{\text{score}} \geq \tau])$$

35.2 — Soglia con calibrazione separata, non implementata (capitolo 35.3)

$$\tau_{\text{rigoroso}}^* = \arg \max_{\tau \in T} F1(y_{\text{calib}}, \mathbb{1}[y_{\text{score,calib}} \geq \tau]), \text{ valutato su } y_{\text{val}} \neq y_{\text{calib}}$$

36.1 — Ricampionamento bootstrap (capitolo 36.1, `evaluation.py:66`)

$$I^{(b)} = \{i_1, \dots, i_n\}, \quad i_j \sim \mathcal{U}\{1, \dots, n\} \text{ indipendenti}, \quad b = 1, \dots, B$$

36.2 — Stima bootstrap di una metrica (capitolo 36.1, `evaluation.py:68-70`)

$$\hat{M}^{(b)} = M(\{y_{\text{true},i}\}_{i \in I^{(b)}}, \{y_{\text{pred},i}\}_{i \in I^{(b)}})$$

36.3 — Intervallo di confidenza percentile (capitolo 36.2, `evaluation.py:77-81`)

$$\text{IC}_\alpha = [\hat{M}_{(0.025)}, \hat{M}_{(0.975)}]$$

36.4 — Deviazione standard bootstrap (capitolo 36.2, `evaluation.py:37`)

$$\widehat{SE}_{\text{boot}}(M) = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (\hat{M}^{(b)} - \bar{M})^2}$$

37.1 — Wilcoxon signed-rank (capitolo 37.1, `statisticaltest.py:35`)

$$W = \min(W^+, W^-), \quad W^+ = \sum_{i: d_i > 0} R_i, \quad W^- = \sum_{i: d_i < 0} R_i$$

37.2 — t-test appaiato (capitolo 37.2, `statisticaltest.py:48`)

$$t = \frac{\bar{d}}{s_d / \sqrt{n}}$$

37.3 — Test di DeLong (capitolo 37.3, `statisticaltest.py:95`, delegato a `MLstatkit`)

$$Z = \frac{\widehat{\text{AUC}}_A - \widehat{\text{AUC}}_B}{\sqrt{\widehat{\text{Var}}(\widehat{\text{AUC}}_A) + \widehat{\text{Var}}(\widehat{\text{AUC}}_B) - 2 \widehat{\text{Cov}}(\widehat{\text{AUC}}_A, \widehat{\text{AUC}}_B)}}$$

Appendice D — Bibliografia annotata

Ogni voce riporta l'affidabilità della fonte e come è stata verificata. Nessuna voce è stata lasciata come “sembra plausibile”: le quattro già citate nella documentazione del progetto sono state controllate contro `docs/DATASET.md`; le otto aggiuntive sono state cercate e confermate con WebSearch in questa sessione (2026-09-05/06), con DOI, arXiv ID o ISBN verificato. L'esportazione in formato BibTeX è in `bibliografia.bib`, nella stessa cartella.

Dataset (citati direttamente nel progetto)

[Verificato — citato in `docs/DATASET.md:67` del progetto] Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1988). *Heart Disease*. UCI Machine Learning Repository. <https://doi.org/10.24432/C52P4W>

[Verificato — citato in `docs/DATASET.md:68`] Detrano, R., Janosi, A., Steinbrunn, W., Pfisterer, M., Schmid, K., Sandhu, S., Guppy, K. H., Lee, S., & Froelicher, V. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. *American Journal of Cardiology*, 64(5), 304–310.

[Verificato — citato in `docs/DATASET.md:113`] Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L., Ventura, S., Cios, K. J., & Clore, J. (2014). *Diabetes 130-US Hospitals for Years 1999-2008*. UCI Machine Learning Repository. <https://doi.org/10.24432/C5230J>

[Verificato — citato in `docs/DATASET.md:114`] Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L., Ventura, S., Cios, K. J., & Clore, J. (2014). Impact of HbA1c measurement on hospital readmission rates: analysis of 70,000 clinical database patient records. *BioMed Research International*, 2014, Article 781670.

Metodo statistico (verificati con WebSearch in questa sessione)

[Verificato — WebSearch, 2026-09-05] DeLong, E. R., DeLong, D. M., & Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: a nonparametric approach. *Biometrics*, 44(3), 837–845. <https://doi.org/10.2307/2531595> — fonda il test usato in `statisticaltest.py` (capitolo 37.3) tramite la libreria `MLstatkit`.

[Verificato — WebSearch, 2026-09-05] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16. <https://doi.org/10.1613/jair.953> — introduce sia SMOTE sia la sua estensione a feature miste numeriche/categoriali (SMOTENC), usata in `preprocessing.py:84-92` (capitolo 21.2).

[Verificato — WebSearch, 2026-09-05] McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. *arXiv preprint* arXiv:1802.03426. <https://arxiv.org/abs/1802.03426> — algoritmo usato in `function.py:224` (capitolo 38).

[Approfondimento facoltativo, verificato — WebSearch, 2026-09-05] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. ISBN 978-0-387-84857-0. — riferimento generale su bias-variance tradeoff (capitolo 2.3), non specifico di questo progetto.

Modelli di embedding (verificati con WebSearch in questa sessione)

[Verificato — WebSearch, 2026-09-05] Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., & Wei, F. (2022). Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv preprint* arXiv:2212.03533. <https://arxiv.org/abs/2212.03533> — famiglia E5, usata via Ollama in `function.py:39,42` (capitolo 5.2). [Da verificare] l'elenco completo degli autori qui riportato proviene da una fonte secondaria (scispace/semanticscholar); verificarlo contro il PDF originale prima di una citazione in tesi che richieda precisione assoluta sull'ordine degli autori.

[Verificato — WebSearch, 2026-09-05] Li, Z., Zhang, X., Zhang, Y., Long, D., Xie, P., & Zhang, M. (2023). Towards General Text Embeddings with Multi-stage Contrastive Learning. *arXiv preprint* arXiv:2308.03281. <https://arxiv.org/abs/2308.03281> — famiglia GTE, usata via Ollama in `function.py:40-41` (capitolo 5.2).

[Verificato — WebSearch, 2026-09-05] Alsentzer, E., Murphy, J., Boag, W., Weng, W.-H., Jin, D., Naumann, T., & McDermott, M. (2019). Publicly Available Clinical BERT Embeddings. In *Proceedings of the 2nd Clinical Natural Language Processing Workshop* (pp. 72–78). Association for Computational Linguistics. arXiv:1904.03323. — modello emilyalsentzer/Bio_ClinicalBERT, usato in `function.py:46` (capitolo 5.3, capitolo 22.3).

[Verificato — WebSearch, 2026-09-05] Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C. H., & Kang, J. (2020). BioBERT: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 36(4), 1234–1240. <https://doi.org/10.1093/bioinformatics/btz682> — famiglia biomedica a cui appartiene il modello `sentence-biobert` (pritamdeka/BioBERT-mnli-snli-scinli `function.py:48`, capitolo 5.3). [Da verificare] il modello specifico usato in questo progetto è una variante fine-tuned di BioBERT per sentence-embedding: questo riferimento copre il modello di base (BioBERT), non necessariamente il fine-tuning specifico applicato dall'autore del modello su Hugging Face, non identificato con un proprio paper in questa sessione.

[Da verificare — non citare prima del controllo] Il modello NeuML/pubmedbert-base-embeddings (`function.py:47`, capitolo 5.3, capitolo 22.3) non è stato ricercato in questa sessione: è presumibilmente basato su PubMedBERT (Gu et al., “Domain-Specific Language Model Pretraining for Biomedical Natural Language Processing”), ma questa attribuzione non è stata verificata con una ricerca dedicata e non va citata come tale senza controllo.

Riepilogo affidabilità

12 riferimenti verificati con identificativo concreto (DOI, arXiv ID o ISBN); 1 riferimento (NeuML/pubmedbert-base-embeddings) esplicitamente non cercato, marcato da non citare prima di una ricerca dedicata; 2 precisazioni minori marcate “da verificare” su dettagli specifici

(ordine autori E5, attribuzione esatta del fine-tuning di sentence-biobert) che non inficiano l'identificazione del lavoro principale ma meritano un controllo finale prima della consegna della tesi.

Appendice E — Zone d’ombra

Domande aperte che questo libro non ha potuto chiudere leggendo solo il codice e i dati disponibili. Da girare a chi ha scritto il progetto, o da trattare esplicitamente come limite dichiarato in tesi. Ordinate per capitolo di prima comparsa.

1. (**capitolo 6, tech sheet §8.1**) A cosa serve `ucimlrepo` in `requirements.txt` se non risulta importata in nessuno dei nove file di `master`? Possibile residuo di uno script di download dati precedente, mai rimosso dal file delle dipendenze.
2. (**capitolo 15.2, capitolo 16.1**) Il comando `ollama pull yxchia/multilingual-e5-base` in `README.md:104` è un refuso di battitura o il residuo di una versione precedente del progetto che usava davvero quel modello, prima di passare a `jeffh/intfloat-e5-base-v2:q8_0` in `function.py:39`?
3. (**capitolo 27.2, capitolo 52**) Il testo narrativo statico di `generatereport.py:192-225` è una funzionalità incompleta (mai finita di implementare) o una scelta consapevole per un primo prototipo di report, poi mai rivista?
4. (**capitolo 29.1, tech sheet §7bis**) La cifra “297” in `docs/DATASET.md:17/README.md:78` è un refuso ereditato dalla letteratura, il residuo di una versione precedente del progetto che usava solo il sottoinsieme Cleveland, o un’altra causa? Il capitolo 29.1 fornisce un’inferenza motivata (4+2=6 righe scartate spiegano esattamente 303-6=297) ma non una conferma diretta da chi ha scritto la documentazione.
5. (**capitolo 29.3, capitolo 31.1, capitolo 46.2**) L’imputazione di `ca/thal` (Heart Disease) e `max_glu_serum/A1Cresult` (Diabetes130) su una maggioranza di valori mancanti è una scelta consapevole (per esempio, per mantenere il numero di feature costante fra i due dataset) o un’assunzione non esaminata dagli autori originali? Un’alternativa — escludere queste feature, o aggiungere un indicatore esplicito di “valore imputato” — non risulta mai considerata nel codice.
6. (**capitolo 31.3**) Il margine di sicurezza per valori NaN residui in `_fmt_num()`, `_fmt_cat()`, `_fmt_bool()` (`embedding.py:28-41`) si è mai attivato in una vera esecuzione della pipeline, o è codice difensivo per un caso che l’attuale sequenza imputazione→SMOTENC→conversione a testo non produce mai?
7. (**capitolo 54.1, tech sheet §8.2**) Quale dei tre modelli biomedici (o, meno probabilmente, uno dei quattro generalisti) è effettivamente designato da `EMBEDDING_MODEL` nella copia divergente di `function.py` sul branch `chatbot`? Il capitolo 54.1 offre un’inferenza motivata (l’uso di `SentenceTransformer` suggerisce un modello biomedico, non uno via Ollama) ma non ha letto quella versione specifica del file.
8. (**Appendice D**) L’elenco completo degli autori del paper E5 (Wang et al., 2022) riportato in questo libro proviene da una fonte secondaria (aggregatori di paper), non dal PDF originale: va confermato prima di una citazione che richieda precisione assoluta sull’ordine degli autori.

9. (**Appendice D**) Il modello `pritamdeka/BioBERT-mnli-snli-scinli-scitail-mednli-stsb` (`sentence-biobert`, `function.py:48`) è una variante fine-tuned di BioBERT: quale pubblicazione, se esiste, descrive esattamente questo fine-tuning specifico (oltre al BioBERT di base, già citato)? Non cercata in questa sessione.
10. (**Appendice D**) L'attribuzione del modello `NeuML/pubmedbert-base-embeddings` (`function.py:47`) alla linea di ricerca PubMedBERT non è stata verificata con una ricerca dedicata in questa sessione: non citare l'origine accademica di questo modello specifico senza prima confermarla.

Come usare questa lista

Ogni voce è formulata come domanda diretta, pronta per essere girata al relatore o a chi ha scritto il codice originale. Le voci 1-7 riguardano il codice e i dati del progetto; le voci 8-10 riguardano solo la precisione bibliografica e non intaccano la comprensione tecnica del progetto.

Appendice F — Indice analitico

A accuratezza, 34.1, 44.1, 45.1, 47.3 · argparse, 28.2 · ambiente virtuale, 14 · AUC, vedi ROC-AUC

B baseline, 47 · bibliografia, App. D · bootstrap, 24.1, 36 · branch chatbot, 0, 19.3, 22 (nota), 54

C **ca** (feature Heart Disease), 29.2, 29.3, 31.2, 46.2 · calibrazione, 55.3 · casi limite, 31.3 · chatbot, 54 · classi sbilanciate, 4.2, 21.2 · classificazione binaria, 4 · comprehension, 8.2 · concorrenza, 12 · context manager, 11.2

D data leakage, 33.3, 35.3, 51.2 · dataset Diabetes130, 30 · dataset Heart Disease, 29 · debugging, 50 · decoratori, 11.3 · DeLong, test di, 26.2, 37.3, 44.3, 45.2 · dizionario globale (**results**), 8.3, 19.2 · duck typing, 7.2

E **e5-base/e5-large**, 5.3, 32.3, 39.1 · embedding, 5, 22 · eccezioni, 11.1 · errori (analisi), 25, 46 · estensioni, 42, 55

F F1 (macro), 34.2 · falso positivo/negativo, 4.3, 46.1 · famiglia di modelli, 5.3, 6.3, 44.2, 45.2

G generatereport.py, 27, 52 · GIL, 12.1 · **gte-base/gte-large**, 5.3, 32.3 · glossario, App. A

H **hardest_cases**, 25.2, 46.2

I imputazione, 21.2, 29.3, 30.2, 31.1, 46.2 · installazione, 15 · intervalli di confidenza, 36.2, 44.1, 45.1 · iperparametri, 39

K k-fold (stratificato), 23.1, 33.2

L linear probing, 32.3, 53.1 · **logging** (assenza), 50.2 · LogisticRegression, 32

M **main.py**, 28 · **max_glu_serum/A1Cresult**, 30.2, 46.2 (analogo) · minacce alla validità, 53.3 · mocking, 49.3 · moduli (Python), 10.1 · modello di riferimento banale, vedi baseline

N **num** (target Heart Disease), 1.1, 6.2

O overfitting/underfitting, 2.3, 32.3 · overloading (assenza), 9.3

P **pubmedbert**, 5.3 · **pytest**, 48.3, 49

Q **questionario/venti domande**, 58

R **readmitted** (target Diabetes130), 1.1, 30.3 · regolarizzazione (C), 32.2, 39.1 · regressione logistica, 32 · report statico (bug), 27.2, 52 · riproducibilità (semi casuali), 39.3 · ROC-AUC, 34.3, 44.1, 45.1

S **self**, 7.3 · **sentence-biobert**, 5.3 · sistema di validazione (train/val/test), 33.1 · SMO-TENC, 21.2, 31.2 · soglia di decisione (τ), 23.2, 35 · sovracampionamento sintetico, vedi SMOTENC

T test automatici (assenza), 48 · test set (mai usato), 21.1, 33.1, 47.3, 51.1 · **thal** (feature Heart Disease), 29.2, 29.3, 46.2 · thread pool, 12.2 · tokenizzazione, 5.2 · tipizzazione dinamica, 7.2

U UMAP, 38 · underfitting, vedi overfitting

V validazione (concetti generali), 2.2, 33 · valori mancanti, 29.3, 30.2, 31.1

W Wilcoxon, test di, 26.1, 37.1

Z zone d'ombra, App. E